System-Level Mapping, Analysis, and Management of Real-Time Applications in Many-Core Systems

Methoden zur Abbildung, Analyse und Verwaltung von Echtzeitanwendungen in Mehrkernsystemen

Der Technischen Fakultät
der Friedrich-Alexander-Universität
Erlangen-Nürnberg
zur
Erlangung des Doktorgrades Dr.-Ing.

vorgelegt von

Behnaz Pourmohseni

Als Dissertation genehmigt
von der Technischen Fakultät
der Friedrich-Alexander-Universität Erlangen-Nürnberg
Tag der mündlichen Prüfung: 28.05.2021

Vorsitzender des Promotionsorgans: Prof. Dr.-Ing. habil. Andreas Paul Fröba

Gutachter: Prof. Dr.-Ing. Jürgen Teich
Prof. Dr. Petru Eles

Dedicated with respect and gratitude to all the beautiful souls who strive to make the world a better place; Those who do not hesitate to lend a helping hand, to offer a listening ear, to share a smile, to make a difference for the better. Your acts of kindness and compassion echo in eternity.

Abstract

Heterogeneous many-core platforms are becoming the de facto standard architectures for embedded systems, fulfilling their high demand for computational power consequent to the rapid increase in the number and the workload of the applications in such systems. Contrarily to historically being statically one-time programmed and the mix of running applications being fixed, modern embedded systems are often subject to a highly dynamic environment in which the mix of concurrently executed applications, the availability of resources, and the non-functional requirements of each application may change at run time on-demand. This necessitates adaptive application mapping and system management schemes to effectively cope with the system dynamism. In recent years, applications with hard real-time requirements are being increasingly observed in emerging embedded systems in various time-critical areas, e.g., automotive, telecommunications, and avionics. In this context, novel design methodologies are required to enable an adaptive mapping and management of hard real-time applications on many-core platforms with verified worst-case timing guarantees.

Recently, so-called Hybrid Application Mapping (HAM) methodologies have been proposed that split the design process into an offline application mapping phase and an online system management phase and, thereby, can enable a dynamic mapping and management of hard real-time applications on many-core platforms. In HAM, for each application, a set of high-quality mapping alternatives with diverse resource requirements and verified timing guarantees are computed offline in a Design Space Exploration (DSE). These mappings are then used online by a Run-time Platform Manager (RPM) to launch the application on-demand by selecting a precomputed mapping that adheres to the current timing constraints of the application and the resource availability in the system. In line with this hybrid scheme, the work at hand proposes a collection of methodologies, analyses, and techniques that enable an automated mapping and adaptive management of hard real-time applications on heterogeneous many-core platforms with verified timing guarantees.

Foremost, a formal timing analysis technique is proposed which can derive worst-case timing guarantees for hard real-time applications in (heterogeneous) many-core systems. In light of this analysis, a novel mapping optimization approach is proposed in which the amount of resources allocated per mapping is fine-tuned according

to the timing requirements of the application to obtain high-quality mappings with satisfactory timing guarantees and enhanced resource efficiency.

In many-core systems, hard real-time applications are particularly susceptible to built-in autonomous Dynamic Thermal Management (DTM) mechanisms which are responsible for preserving the thermal safety of the system. At the sight of thermal violations (overheating), DTM mechanisms suspend or decelerate the execution of the applications that are running in the thermally affected regions which may lead to the violation of their real-time constraints. To eliminate such fatal interferences, this work proposes a thermally composable HAM methodology that preserves the thermal safety of the system proactively at all times and, thereby, eliminates the exposure of real-time applications to DTM without jeopardizing the thermal integrity of the system.

The offline DSE in HAM typically delivers a large set of Pareto-optimal mappings which imposes a considerable management overhead on the RPM and may degrade its responsiveness. This can be particularly fatal for hard real-time applications due to their strict dependence on prompt reactions from the RPM, e.g., upon a change in their timing constraints. As a remedy, this work proposes an automatic mapping-set distillation methodology that distills a promising subset of the mappings for online use to alleviate the RPM's overhead while retaining a diverse blend of quality- and resource-demand trade-offs to address various run-time scenarios.

For a seamless satisfaction of the timing constraints of hard real-time applications in many-core systems, on-the-fly adaptation of their mappings becomes inevitable in view of drastic changes in their timing constraints or the failure of resources they use, e.g., due to faults or overheating. To that end, this work proposes two mapping adaptation methodologies for hard real-time applications in many-core systems. In the first approach, adaptations are realized in the form of a reconfiguration between precomputed mappings, while the second approach empowers fine-grained mapping adaptations in the form of the migration of any subset of the application's tasks to any desired locations. Both methodologies are supplied with formal timing analysis techniques and lightweight admission checks to verify the real-time conformity of each adaptation option online according to the current timing constraints of the application.

Acknowledgments

First and foremost, I would like to express my sincere gratitude to my Ph.D. advisor, Prof. Dr.-Ing. Jürgen Teich, for his continuous support throughout the years. The invaluable insights and vision he shared with me, the outstanding and dynamic research environment he welcomed me into, our fruitful discussions, and the trust he invested in me to approach research problems freely and pursue my ideas have shaped this dissertation to a large extent, for which I am grateful to him. I would also like to sincerely thank Prof. Dr. Petru Eles for co-examining this work and investing time and effort into reviewing it. Likewise, I would like to thank also the other members of the examination committee, Prof. Dr.-Ing. habil. Wolfgang Schröder-Preikschat and Prof. Dr.-Ing. Dr.-Ing. habil. Robert Weigel, for their time and support.

I would also like to dearly thank my colleagues at the Chair for Hardware/Software Co-Design for their relentless support. It is my great pleasure to have been part of this incredible team. In particular, I would like to thank Prof. Dr.-Ing. Michael Glaß, Dr.-Ing. Stefan Wildermann, and Dr.-Ing. Fedor Smirnov for all their support, their valuable feedback on my work (including this dissertation), and the engaging discussions we had over the years. I truly enjoyed working with them.

Last but not least, I would like to thank my beloved family and friends. My most special thanks go to my wonderful parents and to my best friend, my dear husband, Mehrdad, for their love and support during these years and beyond.

Contents

1 Introduction **1**
- 1.1 Embedded Many-Core Systems . . . 1
  - 1.1.1 Application Mapping Challenges and Trends . . . 2
  - 1.1.2 Prospect of Real-Time Applications . . . 3
- 1.2 Dissertation Contributions . . . 5
  - 1.2.1 Mapping Optimization and Timing Analysis . . . 5
  - 1.2.2 Thermal Safety and Thermal Composability . . . 7
  - 1.2.3 Automatic Mapping-Set Distillation . . . 8
  - 1.2.4 Hard Real-Time Online Mapping Adaptation . . . 9
- 1.3 Dissertation Organization . . . 10

2 Fundamentals **11**
- 2.1 Heterogeneous Many-Core Platforms . . . 11
  - 2.1.1 Many-Core Evolution . . . 11
  - 2.1.2 Platform Architecture . . . 12
  - 2.1.3 Memory Scheme . . . 15
  - 2.1.4 Communication Scheme . . . 15
- 2.2 Real-Time Execution on Many-Core Platforms . . . 17
  - 2.2.1 Real-Time Applications and Systems . . . 18
  - 2.2.2 Worst-Case Timing Verification . . . 19
  - 2.2.3 Timing Predictability . . . 23
  - 2.2.4 Timing Composability . . . 23
  - 2.2.5 Invasive Computing . . . 27
- 2.3 System Model . . . 29
  - 2.3.1 Application Model . . . 29
  - 2.3.2 Architecture Model . . . 31
  - 2.3.3 Mapping Edges . . . 32
  - 2.3.4 Execution and Operation Model . . . 32
- 2.4 Application Mapping in Many-Core Systems . . . 33
  - 2.4.1 Application Mapping Procedure . . . 34
  - 2.4.2 Mapping Schemes for Multi-Application Systems . . . 37

2.4.3 Application Mapping Methodologies 39
2.4.4 Hybrid Application Mapping (HAM) 40

3 Isolation-Aware Mapping Optimization and Timing Analysis 45
3.1 Inter-Application Isolation 47
3.1.1 Inter-Application Isolation Schemes 47
3.1.2 Isolation-Aware Application Mapping: Motivation 51
3.2 Isolation-Aware Mapping Optimization 54
3.2.1 Isolation-Scheme Encoding and Resource Allocation 55
3.2.2 Isolation-Aware Scheduling 56
3.3 Worst-Case Timing Analysis 62
3.3.1 Worst-Case Response Time Analysis 62
3.3.2 Worst-Case Traversal Time Analysis 65
3.3.3 Application Timing Guarantees 67
3.4 Experimental Results 68
3.4.1 Experiments Setup 68
3.4.2 Results Discussion 70
3.5 Related Work 74
3.6 Summary 76

4 Thermal Composability in Hybrid Application Mapping 79
4.1 Thermal Safety and Thermal Composability 80
4.1.1 Many-Core Thermal Problem: Cause and Control 80
4.1.2 Thermal Composability 81
4.2 Thermally Composable Hybrid Application Mapping 83
4.2.1 Thermal-Safety Analysis of Mappings 85
4.2.2 Thermal-Safety Admission Check 88
4.2.3 Thermal-Aware Design Space Exploration (DSE) 89
4.3 Experimental Results 91
4.3.1 Experiments Setup 91
4.3.2 Results Discussion 94
4.4 Related Work 101
4.5 Summary 103

5 Automatic Mapping-Set Distillation 105
5.1 Distillation in Hybrid Application Mapping 106
5.1.1 Importance of Distillation 107
5.1.2 Distillation Challenges 109
5.2 Mapping-Set Distillation Methodology 110
5.2.1 Quality-Space Partitioning 111
5.2.2 Resource-Space Pareto Ranking 112
5.2.3 Automatic Distillation Algorithm 113

5.3 Experimental Results . . . 117
5.3.1 Experiments Setup . . . 117
5.3.2 Results Discussion . . . 119
5.4 Related Work . . . 124
5.5 Summary . . . 125

6 Hard Real-Time Online Mapping Adaptation 127
6.1 Mapping Adaptation in Many-Core Systems . . . 129
6.1.1 Importance of Mapping Adaptation . . . 129
6.1.2 Challenges of Mapping Adaptation . . . 130
6.2 Hard Real-Time Mapping Reconfiguration . . . 134
6.2.1 Mapping Reconfiguration Mechanism . . . 136
6.2.2 Mapping Reconfiguration Analysis . . . 138
6.2.3 Mapping Reconfiguration Admission Check . . . 146
6.2.4 Experimental Results . . . 147
6.3 Hard Real-Time Task Migration . . . 155
6.3.1 Task Migration Mechanism . . . 157
6.3.2 Task Migration Timing Analysis . . . 158
6.3.3 Task Migration Admission Check . . . 161
6.3.4 Experimental Results . . . 162
6.4 Related Work . . . 167
6.5 Summary . . . 169

7 Conclusion and Future Directions 171
7.1 Summary . . . 171
7.2 Future Work . . . 173

German Part 175

Bibliography 181

Author's Own Publications 199

List of Symbols 203

Acronyms 207

Index 209

1 Introduction

For decades, the microprocessor industry has been able to deliver an exponential performance increase across semiconductor technology generations commensurate with Moore's Law [Moo65]. Since hitting the power wall and the end of Dennard Scaling [DGR+74] in the year 2005, this trend has been maintained by a shift from frequency scaling to multi-core integration. Over the past decade, advances in the semiconductor industry coupled with novel on-chip communication and storage schemes have enabled a massive integration of processing cores on a single chip with a scalable performance yield, marking the beginning of the many-core era.

1.1 Embedded Many-Core Systems

Embedded systems have today become an indispensable part of our daily lives and are extensively used in various areas, e.g., automotive and land transportation, aerospace and avionics, telecommunications, medical support and healthcare, industrial automation, and consumer electronics. Recent trends in the embedded industry indicate an unprecedented increase in the processing-power requirements of emerging embedded systems which partly roots in the rapid increase in the number of applications that are hosted in a modern embedded system. The large set of processing cores integrated in a many-core chip delivers an immense processing power to meet the ever-increasing performance requirements of emerging and future embedded applications and systems. This potential has given rise to a widespread shift from single/multi-core platforms towards many-core platforms in the embedded domains of computing in recent years.

The computational power of a many-core chip can, however, be exploited only partially due to the increased on-chip density of power consumption and the limitations of the current chip packaging and cooling technologies for dissipating the heat generated thereby. In fact, to preserve the thermal integrity of the chip, a considerable share of cores must remain idle or heavily underutilized, a phenomenon termed *Dark Silicon* [EBA+11]. As a remedy for this utilization penalty, architectural heterogeneity has proven particularly effective where various types of processing resources with diverse performance and power characteristics are integrated on a single chip to cater to the specific performance requirements of individual applications. Consequently,

heterogeneous many-core architectures that encompass a variety of low-power, high-performance, and specialized processing resources on a single chip are becoming the de facto standard platforms for emerging embedded systems.

Heterogeneous many-core platforms provide a promising solution to meet the performance requirements of modern embedded systems. Putting this potential into practice, however, entails addressing several design challenges. In the following, an overview of the major design challenges of modern embedded systems on (heterogeneous) many-core platforms is provided.

1.1.1 Application Mapping Challenges and Trends

Embedded applications typically come with a set of *non-functional requirements* in terms of timing, safety, security, reliability, etc. A satisfactory execution of such applications, thus, not only necessitates their functionally correct execution, but also the satisfaction of their non-functional requirements. Therefore, the analysis and verification of non-functional properties of execution is a crucial aspect of design which can render the design process particularly challenging. *Real-time requirements*, i.e., constraints on the worst-case timing properties of execution, are predominant examples of non-functional requirements in embedded systems. The verification of these requirements often relies on complex and compute-intensive timing analyses which are also difficult to develop, especially in the case of heterogeneous many-core system targets.

The large number of applications hosted within one system is another factor which contributes significantly to the design complexity of modern embedded systems. To exploit the potential computational power of many-core platforms, parallel and multi-threaded computing models are adopted in the development of embedded applications. In such models, the functionality of each application is partitioned into a set of (processing) tasks that can be executed in parallel on different resources [Mar06; Moy13]. In this context, the design complexity of optimally mapping applications onto the resources of a many-core platform grows commensurate with the space of possible task-to-resource assignment options. This space scales exponentially with the number of tasks in the system and can quickly become intractable even for automated Design Space Exploration (DSE) schemes. The resulting immense design space necessitates *scalable* design methodologies for emerging embedded systems. This calls for reconsidering the methodologies in practice and introducing novel design schemes suited for large-scale embedded systems.

The *workload dynamism* in emerging embedded applications and systems is another major factor which contributes significantly to their design complexity on many-core platforms. In a dynamic system, individual applications may be launched and terminated on-demand, oftentimes independently from each other and in reaction to unforeseeable external events, e.g., user requests. In fact, in many embedded systems, only a fraction of all supported applications are simultaneously active at each point

in time. Moreover, both the workload and the non-functional requirements of an application may also vary over time, resulting in a dynamic demand of that application for resources throughout its lifespan. In this context, novel application mapping and system design schemes are required which can handle the relentless fluctuations in the mixes of concurrently executed applications and the dynamism in the workload and non-functional requirements of each application. This renders an *adaptive* mapping and management of applications in modern embedded systems inevitable. In fact, only by using a proper adaptive system management scheme, it will be possible to achieve a high utilization of the underlying resources while meeting the non-functional requirements of individual applications in view of the system dynamics.

In recent years, *composability* has been established as a system property crucial for mitigating the design complexity of embedded systems [AMH+11; Kop11]. System composability enables the complex process of mapping a set of applications onto platform resources to be partitioned into a per-application mapping phase, followed by a system integration phase based on the results of the former. This practice substitutes the traditional *integrated* mapping scheme with a *constructive* (per-application) alternative which enables a significant reduction in the complexity of the mapping process. Moreover, in light of composability, the dynamism in the mixes of concurrently executed applications and the workload of each application can be addressed by performing the system integration at run time. This empowers dynamic and adaptive application mapping and system management schemes. In such schemes, applications are launched on-demand according to the current system state and availability of resources, and once finished, they are removed from the platform to release their occupied resources to be used by other running or incoming applications.

1.1.2 Prospect of Real-Time Applications

Online mapping of applications in many-core systems is subject to stringent time and processing-power limitations: Not only are the system management decisions conducted on the resources of the embedded platform itself which have limited computational power, but also, the overhead of the decision-making processes directly affects the responsiveness of the system to incoming application-launch requests. Thus, dynamic mapping schemes must inevitably resort to *fast heuristics* to find, in due time, a mapping of an incoming application onto the resources of the platform such that it (i) merely relies on the available (free) resources of the platform and (ii) satisfies the non-functional requirements of the application. Verifying the satisfaction of *real-time requirements*, however, involves computationally intensive and thus time-consuming timing analyses that cannot be practically performed online. Therefore, an online computation of mappings for applications with real-time requirements is generally deemed impractical.

In recent years, so-called *Hybrid Application Mapping (HAM)* methodologies have emerged which offer a practical solution to this problem, see [SSK+13; WWG+18b;

PGH+20] for an overview. HAM methodologies employ a compute-intensive offline application characterization phase followed by a lightweight online management phase. In the offline phase, a DSE is performed for each application to compute a set of Pareto-optimal[1] mapping alternatives with diverse resource requirements and worst-case timing guarantees which address various online resource-availability and timing-requirement scenarios. These mappings are then used online by a lightweight *Run-time Platform Manager (RPM)* which launches each application on-demand using one of its precomputed mappings according to the current availability of resources and timing requirements of the application. By enforcing temporal and/or spatial isolation between concurrently executed applications, *timing composability* is established, meaning that concurrent applications are decoupled from each other w.r.t. their (worst-case) timing behavior [AMH+11]. This ensures that the statically obtained timing guarantees for each mapping also hold seamlessly at run time in the presence of other applications in the system.

Major Open Challenges

Empowered by composability, HAM outlines a viable solution for mitigating the complexity of mapping and management of real-time applications in emerging embedded systems targeting heterogeneous many-core platforms. In the case of *hard real-time* applications, however, several crucial design and verification hurdles must still be addressed before this hybrid design scheme can be put into practice: For the timing verification of hard real-time applications, novel *worst-case timing analysis techniques* are required that comply with the tiled architecture of emerging many-core platforms, their network-on-chip communication infrastructure, and their distributed memory scheme. Moreover, achieving a high system utilization necessitates *adaptive resource-reservation policies* to enable fine-tuning the reserved resources of each application according to its compute-power demand and timing constraints and, thereby, avoiding an over-reservation or under-utilization of resources.

Thermal aspects are particularly crucial for real-time applications. In emerging many-core platforms, the increased on-chip temperature serves as an indirect source of inter-application interference since the unsafe thermal behavior of one application can trigger autonomous thermal-control mechanisms in the system which may interfere with the execution of several running applications. This annihilates system composability and may lead to the violation of real-time constraints, rendering *thermal safety* and *thermal composability* crucial factors to be regarded and enforced alongside timing requirements to ensure a deadline-adhering execution of hard real-time applications.

In a many-core platform, running applications are occasionally also subject to unexpected critical situations, e.g., resource failures due to hardware faults or overheating.

[1] Given a set of mappings optimized w.r.t. a set of design objectives, a mapping is called Pareto optimal if it is not dominated by any other mappings w.r.t. the objectives [BTT98].

Such events may affect the execution behavior of applications if not counteracted accordingly by adapting their mapping during their execution, e.g., to evacuate an overheated region. In the context of hard real-time applications, novel mechanisms are, thus, required to enable the *on-the-fly adaptation of mappings* with *verified worst-case timing guarantees* to enforce a seamless satisfaction of hard timing constraints before, during, and after the adaptation process. Real-time mapping adaptation mechanisms can also be employed in a non-critical setting, e.g., to improve the system's performance through load balancing or to release resources that are required for an incoming application without an adverse impact on currently running applications.

In response to the challenges outlined above, the dissertation at hand proposes a collection of methodologies, analyses, and techniques in line with the principles of HAM to enable an automated mapping and adaptive management of hard real-time applications on heterogeneous many-core platforms. In the following, the key contributions of this work are briefly introduced.

1.2 Dissertation Contributions

Figure 1.1 provides an overview of the major contributions of this dissertation, integrated into the flow of Hybrid Application Mapping (HAM) as the promising scheme for an adaptive mapping and management of hard real-time applications in many-core systems. The proposed methodologies and techniques are highlighted in color and span across the areas of (i) mapping optimization and worst-case timing analysis (green), (ii) thermal safety and thermal composability (red), (iii) automatic mapping-set distillation (orange), (iv) real-time mapping reconfiguration (blue), and (v) real-time task migration (cyan). When integrated into the offline application characterization and online management phases of HAM as depicted in Figure 1.1, these contributions enable a thermally safe and composable, scalable, and adaptable mapping and management of hard real-time applications on heterogeneous many-core platforms with guaranteed worst-case timing properties. These contributions are briefly introduced in the following.

1.2.1 Mapping Optimization and Timing Analysis

The ability to reason about the timing properties of applications is a fundamental and indispensable trait of any application mapping methodology used in the context of real-time applications and systems. In the case of *hard real-time* applications, *worst-case timing analyses* are required to provide timing *guarantees* for each mapping, accounting for the worst-case timing behavior of the application's tasks and the worst-case timing interference imposed on them when accessing shared resources. In a composable system, the maximum degree of such timing interferences is regulated through temporal/spatial isolation among concurrently executed applications, enforced

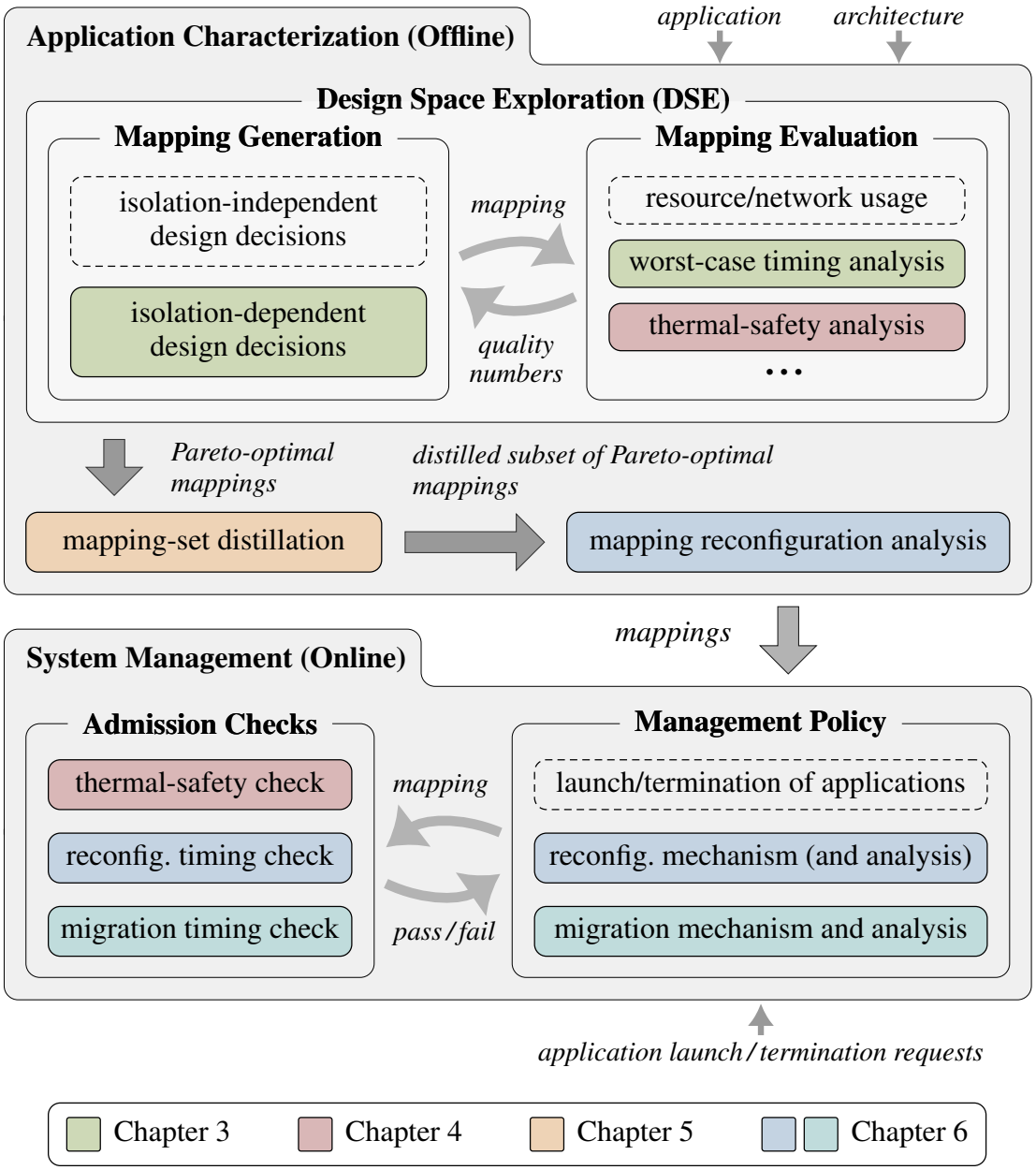


Figure 1.1: An overview of the dissertation contributions (highlighted in color) integrated into the offline and online phases of Hybrid Application Mapping (HAM). Dashed blocks denote some of the existing components of HAM.

by an exclusive reservation of resources (or resource budgets) per application at its launch time. In the current practice of application mapping in many-core systems, due to the limitations of existing timing analyses, a *fixed* inter-application isolation scheme is enforced throughout the system. This imposes a *fixed* resource-reservation policy

on every mapping of every application in the system, regardless of their actual timing and compute-power requirements. Such a fixed isolation scheme often results in sub-optimal mappings and leads to a significant under-utilization of system resources.

As a remedy, this work proposes an *isolation-aware mapping optimization and analysis* methodology consisting of an (i) *isolation-aware mapping optimization* approach and a (ii) *worst-case timing analysis* which can be integrated into the offline characterization phase of HAM (see the green blocks in Figure 1.1). These components enable exploring different choices of isolation schemes (temporal or spatial) for each resource and in each mapping of each application during the mapping optimization process and provide worst-case timing guarantees for each mapping obtained thereby, see [PSW+19]. As a result, the proposed methodology empowers the offline DSE in HAM to find high-quality mappings in which the amount of reserved resources is well-adjusted to the timing and compute-power requirements of the application. The key enabler for this methodology is the proposed timing analysis. Contrarily to existing techniques, the proposed analysis is applicable to mappings with *arbitrary combinations* of isolation schemes on their allocated resources and delivers safe yet tight worst-case timing guarantees per mapping according to the choices of isolation schemes applied to the resources used by it. This methodology is presented in detail in Chapter 3.

1.2.2 Thermal Safety and Thermal Composability

The massive integration of resources in emerging many-core platforms entails a significant increase in the density of power consumption on the chip, leading to an increased on-chip temperature which can engender overheated regions and even jeopardize the thermal integrity of the chip. To ensure thermal safety, many-core platforms employ autonomous thermal-control mechanisms that continuously monitor the thermal state of the chip and take countermeasures such as power gating at the sight of thermal violations. These countermeasures, however, interfere with the execution of the applications that are running in the thermally affected regions and may lead to the violation of their real-time constraints. In the current practice of HAM, thermal safety is disregarded altogether. Hence, existing HAM methodologies not only fail in ensuring a seamless satisfaction of real-time constraints but also cannot preserve the system composability, since the unsafe thermal behavior of one application may indeed trigger thermal-control countermeasures that affect also the other (possibly real-time) applications in their close vicinity.

As a remedy, this work proposes a *thermally safe and thermally composable* methodology which can be incorporated into the flow of HAM to proactively enforce thermal safety throughout the system at all times, see [PSK+19]. As a result, thermal-control countermeasures of the chip are never triggered and, thereby, composability is preserved. The proposed methodology is composed of two components: a (i) *thermal-safety analysis* and a lightweight (ii) *thermal-safety admission check* (see the red

blocks in Figure 1.1). The thermal-safety analysis is integrated into the offline application characterization phase of HAM and computes an abstract thermal-safety index for each generated mapping. The thermal-safety admission check is used online at the launch time of each application and examines the thermal safety of each mapping candidate for the current system utilization based on its precomputed thermal-safety index. Using this approach, mapping candidates which may induce thermal violations for the current system load are proactively identified and dismissed. Thereby, the exposure of real-time applications to thermal-control countermeasures is eliminated *without* compromising the thermal integrity of the chip. The proposed methodology is presented in detail in Chapter 4.

1.2.3 Automatic Mapping-Set Distillation

During the offline DSE in HAM, mappings are often optimized w.r.t. multiple quality objectives, e.g., latency, throughput, energy consumption, and reliability. Moreover, in the case of heterogeneous many-core platforms, the number of allocated cores from each core type is also optimized which introduces one design objective per core type[2]. As a result, the offline DSE in HAM often embodies a many-objective optimization setting and typically delivers a large set of Pareto-optimal mappings. Such a high volume of mappings imposes an unacceptable management and storage overhead for online use. Selecting a promising subset of Pareto-optimal mappings for online use is a crucial but challenging task due to the incomparability of the mappings w.r.t. the design objectives.

As a remedy, this work proposes a *mapping-set distillation* methodology (see the orange block in Figure 1.1). The proposed methodology selects a subset of Pareto-optimal mappings for online use such that the retained set of mappings features two properties: (i) diverse quality trade-offs to meet various online quality requirements and (ii) diverse yet efficient resource combinations to enhance the embeddability of the application in view of a dynamic availability of resources at run time, see [PGT17]. To identify the promising mappings, the proposed methodology distinguishes quality objectives from resource objectives and treats each set accordingly: First, the space of quality objectives is partitioned into regions of comparable quality trade-offs. Subsequently, resource-efficient mappings are distilled from different quality regions using Pareto ranking of mappings w.r.t. resource objectives. Mappings distilled using this approach are demonstrated to outperform those retained using multi-objective truncation techniques w.r.t. both embeddability and diversity of quality trade-offs. This methodology is presented in detail in Chapter 5.

[2] Optimizing the number of allocated cores per core type is a common practice in HAM to enable addressing various resource availability scenarios by diversifying the resource demand of mappings.

1.2.4 Hard Real-Time Online Mapping Adaptation

In many-core systems, running applications are occasionally subject to unexpected critical situations, e.g., resource failures due to hardware faults or the emergence of hot spots. Such events might impair the expected behavior of applications and, therefore, must be counteracted accordingly to preserve the expected execution behavior. In this context, adapting the mapping of running applications *during* their execution becomes inevitable. Mapping adaptation is predominantly realized by means of *task migration* which may cause changes in the timing behavior of applications during and after their adaptation process. For hard real-time applications, the worst-case temporal overhead of each adaptation process must be analyzed to guarantee a seamless satisfaction of the timing constraints during and after the adaptation process. Such a timing verification, however, necessitates compute-intensive worst-case timing analyses that are not suited for online use. In this context, to enable on-the-fly mapping adaptations with hard real-time guarantees and affordable run-time compute overhead, this work proposes two contributions which are outlined in the following and presented in detail in Chapter 6.

Hard Real-Time Mapping Reconfiguration

The work at hand proposes a *hard real-time mapping reconfiguration* methodology that enables on-demand mapping adaptations in the form of a transition between the statically computed mappings of an application with hard timing guarantees, see [PWG+17; PWG+19]. A reconfiguration between precomputed mappings, on the one hand, eliminates the need for an online timing verification of the post-adaptation mapping since worst-case timing guarantees are already provided for each mapping during the offline DSE. On the other hand, as the mappings are known offline, the worst-case latency of the reconfiguration process for each pair of source and destination mappings can also be analyzed offline and, then, provided to the RPM for online use.

The proposed mapping reconfiguration methodology is composed of three components (see the blue blocks in Figure 1.1): a (i) *timing-predictable reconfiguration mechanism* which is used online by the RPM to perform mapping reconfigurations timing-predictably, a (ii) *reconfiguration analysis* which is performed offline [PWG+17] or partially offline and partially online [PWG+19] to derive the necessary information (including worst-case timing guarantees) for each reconfiguration to be used by the RPM, and a lightweight (iii) *reconfiguration admission check* which is used online by the RPM to examine the admissibility of a reconfiguration w.r.t. the given timing constraints of the application. To the best of our knowledge, this methodology is the first solution to enable on-the-fly adaptation of individual applications with hard real-time guarantees in the context of (heterogeneous) many-core systems.

Hard Real-Time Task Migration

Mapping reconfiguration empowers the dynamic adaptation of real-time applications only to a limited extent since (i) it is restricted to a set of statically computed options which may not be able to address all possible online situations. Moreover, (ii) it often involves rearranging the deployment of most or even all tasks of the application which typically entails a significant temporal overhead that might not be affordable subject to the current timing constraints of the application.

To lift these restrictions, this work proposes a *hard real-time task migration* methodology which enables an on-demand migration of *any* subset of an application's tasks to *any* destination cores with worst-case timing guarantees. Hence, it empowers *fine-grained* mapping adaptations *without* relying on a static timing analysis or a restricted set of adaptation options, see [PSW+20]. The proposed migration methodology is composed of three components which are used online by the RPM (see the cyan blocks in Figure 1.1): a (i) *timing-predictable migration mechanism* which enables the RPM to perform task migrations timing-predictably, a lightweight (ii) *migration analysis* to formally bound the worst-case latency of the migration processes, and a lightweight (iii) *migration admission check* which examines the admissibility of the migration latency w.r.t. the given timing constraints of the application and the fluctuations in its timing behavior during and after the migration processes.

1.3 Dissertation Organization

The remainder of this work is organized as follows: In Chapter 2, the fundamental concepts required for this work are presented, including an overview of many-core architectures, the fundamentals of real-time execution in many-core systems, the system model used in this work, the procedure of application mapping in many-core systems, and an overview of different classes of many-core application mapping methodologies in practice. Chapter 3 focuses on resource reservation and timing analysis in the scope of many-core systems and presents the proposed isolation-aware mapping optimization and worst-case timing analysis methodology in detail. In Chapter 4, the thermal aspects associated with the mapping and management of applications in many-core systems are discussed, and the proposed thermally composable HAM methodology is presented. Chapter 5 delineates the impact of mapping-set distillation on the viability of HAM and presents the proposed automatic mapping-set distillation methodology. Chapter 6 focuses on the topic of online mapping adaptation for hard real-time applications in many-core systems and presents the proposed real-time mapping reconfiguration and task migration methodologies. Finally, Chapter 7 concludes this dissertation by providing a summary of this work and outlining possible future directions.

2 Fundamentals

This chapter introduces the fundamental concepts which form the basis of this work. Given that the work at hand aims at empowering a satisfactory execution of hard real-time applications in many-core systems, first, an overview of heterogeneous many-core platforms is provided in Section 2.1. Then, the fundamentals of real-time execution on many-core platforms are presented in Section 2.2. Following that, the system model considered in this work is detailed in Section 2.3. Finally, the principles of application mapping in many-core systems are discussed in Section 2.4 which elaborates on the procedure of system-level application mapping, provides an overview of design methodologies for mapping and management of applications in many-core systems, and presents the design methodology considered in this work.

2.1 Heterogeneous Many-Core Platforms

This section provides an overview of heterogeneous many-core platforms. First, the many-core evolution path and key characteristics are presented in Section 2.1.1. Thereafter, the architectural properties, memory scheme, and communication scheme of many-core platforms are introduced in Sections 2.1.2 to 2.1.4, respectively.

2.1.1 Many-Core Evolution

The introduction of Multi-Processor System-on-Chip (MPSoC) platforms in the embedded areas dates back to the early 2000s. The first MPSoCs designed for embedded applications include Lucent Daytona [AAB+00] for wireless base stations (2000), C-5 Network Processor [Cor01] for network packet processing (2001), Philips Viper Nexperia [DJR01] for multimedia processing (2001), and STMicroelectronics Nomadik [AAC+03] for mobile computing (2003) [WJM08]. In each of these so-called *multi-core* platforms, a set of cores, memories, caches (and, optionally, other peripherals, e.g., application-specific hardware accelerators) are integrated on a chip with a bus-based interconnection scheme and a shared-memory communication scheme.

For years, the multi-core integration technology invested in MPSoC platforms has been able to address the ever-growing performance requirements of embedded

applications and systems. The computational capacity of a multi-core platform scales proportionally to its number of cores and can be exploited by multi-threaded and parallel applications where the functionality of each application is partitioned into a set of (processing) *tasks* that can be executed concurrently on different resources [Moy13; Mar06]. In this context, a higher number of cores on a chip leads to a higher number of concurrent processes which brings about a higher volume of communication traffic between cores and memories. This results in a higher demand for communication bandwidth and memory throughput to meet the performance requirements of the applications. Nonetheless, on the one hand, the service latency of a shared memory grows with the number of its concurrent access requests. On the other hand, due to its poor bandwidth scalability, a bus-based interconnection infrastructure can fulfill the bandwidth requirements of a system only for a limited volume of communication traffic. Consequently, the bus-based interconnection and shared-memory communication scheme of multi-core platforms eventually became the bottleneck for the bandwidth scalability of MPSoCs. This renders a multi-core integration beyond a handful of cores effectively futile due to the stagnation of the deliverable system performance.

To address this bandwidth scalability restriction, a series of fundamental modifications have been applied in the architecture, organization, and operation of MPSoC platforms, leading to the introduction of the so-called *many-core* platforms. Foremost, the shared-memory communication scheme and the bus-based interconnection infrastructure have been substituted with scalable alternatives: The globally shared memory scheme has been replaced by a scalable *distributed memory scheme* in which each memory is shared among only a small collection of its neighboring resources, eliminating remote memory-access privileges. Such a distributed memory scheme mitigates the volume of requests per memory, resulting in a reduced service latency per request. In the same line, the centralized bus-based interconnection infrastructure has been substituted by *Network-on-Chip (NoC)* structures with distributed scheduling and routing. This enables the concurrent operation of multiple communication channels and, thereby, offers a scalable bandwidth capacity [NFM+12; BD02].

The outlined combination of the distributed memory scheme and the NoC-based interconnection infrastructure offers an unprecedented bandwidth scalability. It has enabled the microprocessor industry to integrate exceedingly large numbers of cores on a single chip with a scalable performance capacity w.r.t. both computation and communication. This has led to the introduction of many-core platforms into the embedded market. Following this line of evolution, the main distinctions between multi- and many-core platforms lie in their on-chip interconnection infrastructure and memory scheme.

2.1.2 Platform Architecture

A many-core platform typically integrates a large set of processing, communication, and memory resources on a single chip. Examples of many-core platforms include

the Tilera TILE-Gx many-core family [Til12], Kalray MPPA-256 (Massively Parallel Processor Array) [dDAB+13], Intel SCC (Single-chip Cloud Computer) [HDH+10], Intel Xeon Phi [Sod15], and the KiloCore [BSP+17]. A many-core platform typically follows a modular and regular organization of resources where resources are clustered into a set of so-called *(compute) tiles* that are interconnected via a NoC. Furthermore, a distributed memory scheme is applied across the chip such that the memory space on each tile is accessible only to the resources located on that tile.

In a *homogeneous* many-core platform, all processing resources are instances of the same type of core. A *heterogeneous* many-core platform, on the other hand, encompasses various types of processing resources to tailor the computational power provided to each application according to its computational requirements and, thereby, enhance the energy-performance balance of the whole system. In a heterogeneous many-core platform, the cores comprised within one tile are typically homogeneous, while different tiles contain different types of cores. Figure 2.1 illustrates an exemplary heterogeneous tiled many-core platform composed of 16 tiles with a 4×4 mesh NoC interconnection where different core types are distinguished by color. Note that, in addition to *compute tiles*, a many-core platform may also be provisioned with a set of *memory tiles* that offer mass storage on the chip and a set of *I/O tiles* for connectivity with off-chip peripherals, e.g., sensors and actuators.

Compute-Tile Organization

Figure 2.2 provides a detailed view of the internal organization of different tiles of the exemplary many-core platform illustrated in Figure 2.1. In a many-core platform, the internal architecture of each tile resembles the architecture of a multi-core platform: Each tile consists of (i) a set of cores, (ii) a set of memories, and (iii) a Network Adapter (NA). The NA is a part of the NoC, acting as an interface for the exchange of data between tiles via the NoC. Additional peripherals, such as hardware accelerators, may also be incorporated on a tile. Within each tile, a bus-based interconnection infrastructure is employed which enables the local cores and the NA to access the on-tile memories via a set of (memory) buses. In addition to the memories, each tile can be equipped with shared L2 caches and each core can have private data/instruction L1 caches. Different tiles on the platform may have different numbers of cores, memories, and buses, as exemplified in Figure 2.2.

Network-on-Chip (NoC) Architecture

Tiles in a many-core platform are interconnected via a NoC which consists of a set of Network Adapters (NAs) (one NA located on each tile), a set of routers (one router per tile), and a set of links that connect routers to each other and their local NAs. The arrangement of tiles in a many-core platform depends on its choice of NoC topology, e.g., mesh, tree, ring, torus, and star [BDY06; DT04]. A regular two-dimensional

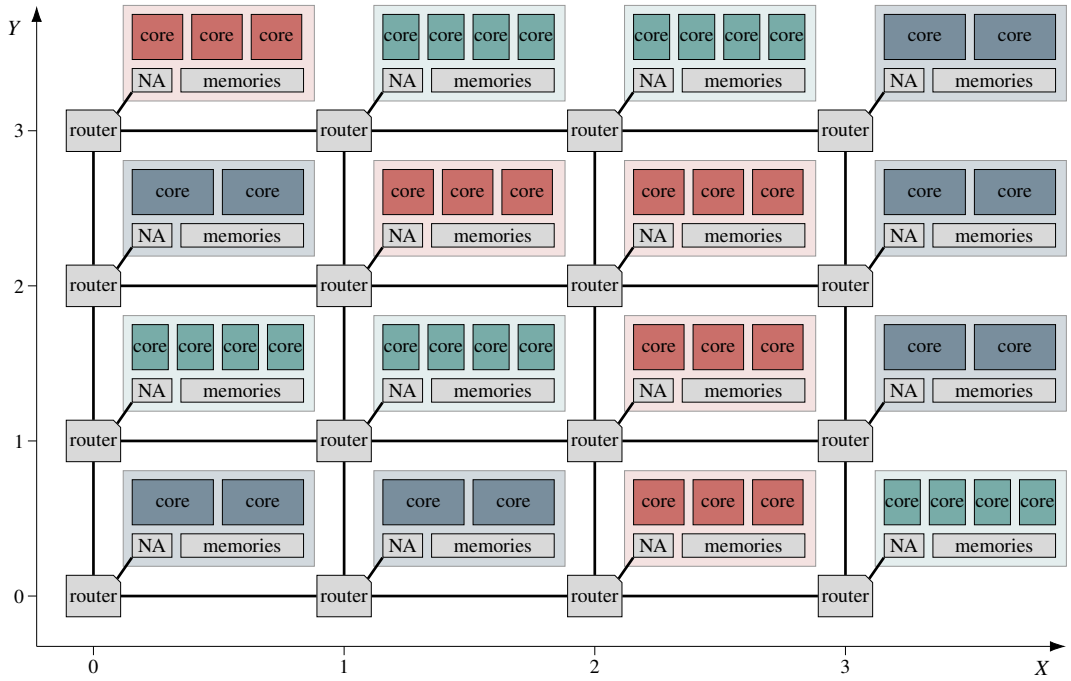


Figure 2.1: Example of a heterogeneous tiled many-core architecture consisting of 16 tiles with a 4×4 mesh Network-on-Chip (NoC) interconnection. The XY-plane provides the NoC location of each router and its local tile. Each tile comprises a set of cores, a set of memories, and a Network Adapter (NA). Different types of cores are distinguished by color.

placement of tiles with a mesh NoC topology is the predominant architectural structure employed in many-core platforms, see Figure 2.1.

In a two-dimensional mesh NoC, each router is connected to the NA on its local tile and each of its adjacent routers in the cardinal directions so that it is connected to 2–4 other routers subject to its location on the chip, see Figure 2.1. Each NoC connection (connecting two adjacent routers or a router to its local NA) is realized via two uni-directional links, one per direction of data transfer. Each NA has one input and one output port. It comprises a Transmitter (TX) unit and a Receiver (RX) unit which are responsible for the coordination of the outbound and incoming traffic of the tile, respectively. Each TX unit (RX unit) is equipped with (i) a buffer for the intermediate storage of the outgoing (incoming) traffic of the tile and (ii) the control logic necessary for the coordination and transfer of the outgoing (incoming) traffic between the local router and the on-tile memories. Accordingly, each router can have up to five input and up to five output ports and is equipped with (i) an input buffer per input port for intermediate storage of arriving data, (ii) a 5×5 crossbar connecting each input buffer to each output port, and (iii) the control logic necessary for the coordination and routing of traffic to adjacent routers or the local NA.

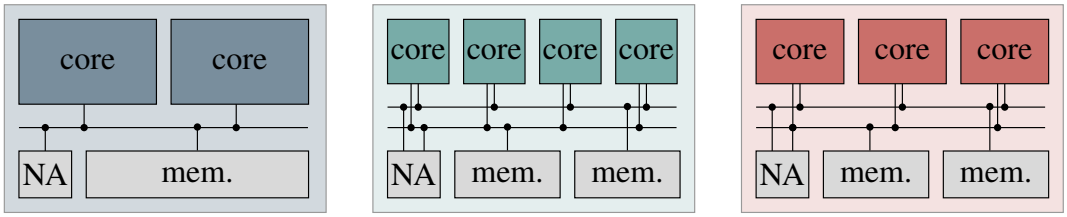


Figure 2.2: A detailed view of the internal architecture of the three tile types from Figure 2.1. In each tile, the cores and the NA access the local memories via a set of (memory) buses. Core types are distinguished by color.

2.1.3 Memory Scheme

Scalability is a central goal in the design of many-core systems, and the memory scheme in a many-core platform is developed such that system scalability is preserved. To that end, many-core systems employ a *distributed* memory scheme without remote-access privileges between tiles, while a *shared* memory scheme is used within each tile. The distributed memory scheme between tiles, also referred to as a *No Remote Memory Access (NORMA)* architecture [MDA+16], restricts the accessibility of the memories on each tile to the resources located on that tile, namely, the local cores and the NA. The memory space on each tile is generally shared among the local cores and NA, while it can also be partitioned into regions with additional access restrictions. For instance, a private memory space can be dedicated to each core. Likewise, the dedicated memory space of each core can be further partitioned among the processes executed on that core. Similarly, a dedicated memory space can be provisioned for the storage of each outgoing (incoming) communication traffic of the tile, accessible to the TX unit (RX unit) of the NA and the respective producing (consuming) core or process. In this context, the operating system typically offers dynamic memory allocation services for a dynamic partitioning of the memory space among the on-tile resources and/or the processes running on the local cores.

2.1.4 Communication Scheme

Many-core systems employ two communication schemes for data transfers within one tile (intra-tile communication) and between tiles (inter-tile communication) in accordance with their shared intra-tile and distributed inter-tile memory schemes.

Intra-Tile Communications

The exchange of data within one tile, namely, an *intra-tile communication*, is typically realized through the shared memories of that tile. To that end, the communication source (also known as the producer) writes the data into an assigned memory space within one of the on-tile memories, and the communication destination (also known as

the consumer) reads the data from that memory space. This communication scheme enables data transfers between two cores located on the same tile and can also be employed for the exchange of data between two processes executed on the same core.

Inter-Tile Communications

The transfer of data between cores located on two different tiles, namely, an *inter-tile communication*, is realized over the NoC. Due to the distributed memory scheme between tiles, inter-tile communications are realized by means of *explicit message passing* between the source and destination tiles. To that end, first (i) the communication source (producer) writes the data into an assigned space within one of the on-tile memories. Then, (ii) the local NA reads the data from the memory, prepares it for the transfer via the NoC, and dispatches the data to its local router. Once available at the router, (iii) the data is forwarded through a chain of routers over the NoC in a hop-by-hop manner towards the destination tile. At the destination tile, (iv) the local router forwards the data to the RX unit of the NA which prepares the data and writes it into an assigned space within one of the on-tile memories from where (v) the communication destination (consumer) reads the data afterwards.

Network-on-Chip Operation

The realization of an inter-tile communication (steps (ii)–(iv) detailed above) strongly depends on the switching strategy, routing algorithm, and the flow-control mechanism of the NoC [BM06; DT04]. In the following, a summary of the current practices in many-core NoC design w.r.t. these aspects is provided.

Switching Strategy The switching strategy of a NoC specifies how data is transferred through the NoC. Most many-core NoCs employ a *packet-switching strategy* in which data is partitioned into a set of packets that are transferred—in-order, in a pipelined manner, and independently from each other—towards the destination tile. In this line, *wormhole switching* [NM93] is a widely used packet-switching strategy in which data packets are subdivided into fixed-length flow-control digits (flits) which are transferred over the NoC. The flits associated with a data packet consist of (i) a *head flit* carrying routing and sequencing information, (ii) a series of *payload flits* (also called body flits) containing the actual data, and (iii) a *tail flit* signaling the end of the packet and carrying, e.g., an error-checking code. The subdivision of packets into flits (called *packet flitization*) before the transfer and the reconstruction of the packet from flits (called *packet assembly*) after the transfer are performed by the TX unit of the NA on the source tile and the RX unit of the NA on the receiver tile, respectively.

Routing Algorithm The routing algorithm of a NoC determines the path (sequence of routers) through which data is transferred between two tiles. NoC routing al-

gorithms are classified w.r.t. their routing decision procedure as either *adaptive* or *oblivious* [WLC16]. An adaptive routing algorithm considers the network traffic in its routing decisions to balance the NoC load at the cost of an increased router complexity. Alternatively, oblivious routing algorithms disregard the network traffic in their routing decisions to simplify the router design. In the scope of oblivious algorithms, *deterministic* routing algorithms simplify the network design considerably by using a fixed route for each pair of source and destination tiles and, thereby, eliminating the routing decision process altogether. *XY-routing* is a widely-used deterministic routing algorithm in which data is first routed in the X-direction and then in the Y-direction towards the destination router [NM93]. XY-routing is also a *minimal* routing algorithm, as it realizes the transfer of data with the shortest possible route length quantified by the number of hops in the route.

Flow-Control Mechanism The flow-control mechanism of a NoC coordinates the forwarding of traffic flows in the network by steering the allocation of router buffers and links to individual traffic participants. *Credit-based* flow control is predominantly used in NoCs to steer the flow of traffic by means of so-called *credits* which signal the availability of input buffer space in each router to its respective predecessor router [DT04]. In this scheme, data forwarding is orchestrated by the credits communicated between adjacent routers which also inherently realizes a back-pressure mechanism. *Virtual-channel* flow control [Dal92] is another widely-used mechanism in which the physical space of each buffer is logically partitioned into several so-called *virtual channels* which can be exclusively assigned to different traffic participants. Using virtual channels, quality of service guarantees can be provided per communication flow since they prevent concurrent flows which pass through the same router from blocking each other, see, e.g., [HKK+13].

2.2 Real-Time Execution on Many-Core Platforms

Embedded applications often have certain requirements w.r.t. their non-functional properties of execution that must be satisfied throughout their lifespan. Examples of these non-functional properties include timing properties (e.g., latency and throughput), power and energy consumption, and dependability properties (e.g., safety, security, and reliability). The choice and the number of non-functional requirements of an application typically depend on the application and system domain. Timing requirements are among the most common non-functional requirements in embedded systems.

In this section, first, real-time applications and systems are introduced in Section 2.2.1. Afterwards, an overview of timing verification procedure of many-core systems is provided, and the timing analyses involved in timing verification processes are described in Section 2.2.2. Thereafter, two key system properties crucial for the design of real-time systems, namely, timing predictability and timing composability, are

introduced in Sections 2.2.3 and 2.2.4, respectively, and it is shown how they can be established in a many-core system. Finally, Section 2.2.5 introduces the paradigm of *invasive computing* as a promising solution for the design of hard real-time embedded systems based on heterogeneous many-core platforms.

2.2.1 Real-Time Applications and Systems

A timing requirement generally specifies an upper and/or lower bound on the timing properties of a process or an application, e.g., an upper bound on its execution latency or a lower bound on its throughput. Timing requirements are relevant for systems in which the correctness of their behavior depends not only on the logical results of their computation but also on the physical instant (time) at which these results are ready, e.g., the airbag control system in a car. By definition, an application with at least one timing requirement is called a *real-time application*, and a system that contains at least one real-time application is referred to as a *real-time system* [Kop11].

Timing requirements of real-time applications can be classified as either *soft* or *hard* subject to their *strictness*: In the case of a *soft real-time* requirement, *occasional* timing violations are considered tolerable. Such violations, however, degrade the system's performance or the quality/usefulness of the obtained results. For instance, real-time applications from the areas of multimedia, e.g., audio/video streaming and image processing, typically have soft real-time requirements [BLA+05]. In the case of a *hard real-time* requirement, on the other hand, any timing violation may lead to a system failure, and therefore, timing violations are not tolerable at all.

Recent years give evidence of a rapid increase in the number and diversity of hard real-time applications in embedded systems. For instance, the introduction of embedded systems in time-critical areas, e.g., automotive electronics, medical imaging, avionics, telecommunications, industrial plant automation, and consumer electronics, has given rise to a high volume of applications with hard timing constraints [DB11; SEU+15; SDM+17]. In this line, a hard real-time system is defined as:

> **Hard Real-Time System** [Kop11]: A real-time computer system that must meet at least one hard deadline is called a hard real-time computer system or a safety-critical computer system.

Here, the term *safety-critical* is often associated with systems in which a system failure, e.g., consequent to the violation of a hard timing requirement, can put human lives at risk [Kni02]. A real-time application may also have non-timing requirements, e.g., in terms of safety or security. The methodologies presented in this work focus on the timing aspects of execution for hard real-time applications in many-core systems. These methodologies can be used orthogonally to or in combination with techniques that address other non-functional requirements of applications.

2.2.2 Worst-Case Timing Verification

Hard real-time applications rely on *timing verification techniques* that examine the satisfaction of their timing requirements. Verifying a seamless satisfaction of hard real-time requirements is a challenging task since the timing behavior of an application often strongly depends on (i) the input of the application and (ii) the state of its execution environment (also called execution context). The latter denotes the state of the system resources used by the application and the presence and behavior of other applications in the system. The uncertainty in the input or execution environment of an application typically results in a huge variability in the non-functional properties of its execution, e.g., its timing behavior. In this context, Teich et al. introduce the concept of **-predictability* defined as follows:

> ***-predictability** [TGR+16]: Let o denote a non-functional property of a program (implementation) p and the uncertainty of its input (space) given by I and environment by Q. The predictability (marker) of objective o for program p is described by the interval
>
> $$o(p,Q,I) = [INF_o(p,Q,I), SUP_o(p,Q,I)] \tag{2.1}$$
>
> where INF_o and SUP_o denote the infimum and supremum [of property o for program p] under variation of state $q \in Q$ and input $i \in I$, respectively.

In line with this definition, techniques aiming at the verification of timing properties of hard real-time applications mainly focus on deriving *worst-case timing guarantees* for each application that hold under any variation of its input and execution environment. The main goal of such a technique is to derive *safe* and *tight* timing guarantees. Here, *safeness* denotes that the timing property of interest does not violate the derived timing guarantee under any variation of the input or environment state. Thus, a safe timing guarantee specifies an *upper/lower bound* on the actual worst-case timing behavior of the application. *Tightness*, on the other hand, denotes the closeness of the derived guarantee to the actual worst-case timing behavior without jeopardizing safeness. Timing verification of hard real-time applications is typically performed in two steps, namely, (i) a *context-independent timing analysis* followed by (ii) a *context-aware timing analysis*, which are detailed in the following.

Context-Independent Timing Analysis

The timing verification of a hard real-time application begins with a *context-independent timing analysis*. In such an analysis, each task of the application is considered individually, and its *Worst-Case Execution Time (WCET)* is derived in full isolation, i.e., without interruption and in the absence of any other concurrent processes in the

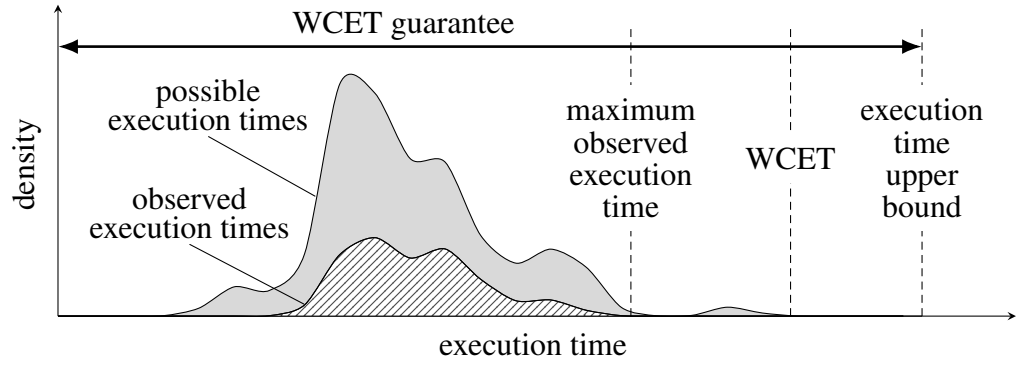


Figure 2.3: Worst-Case Execution Time (WCET) and the distribution of possible and observable execution times of an illustrative task according to [WEE+08].

system [MRR+19]. Since the execution time of a task often varies with changes in (i) its input(s), (ii) its internal state, and (iii) the timing behavior of the underlying hardware resources, the goal of WCET analysis is to identify a combination of these factors that results in the highest possible execution time for that task.

For the vast majority of applications, an enumeration of all execution scenarios of a task is computationally impractical due to the intractable number of possible scenarios [KP08]. Therefore, WCET analyses aim at finding a so-called *WCET guarantee* which specifies a safe and tight *upper bound* on the actual WCET of a task, see Figure 2.3. To that end, compute-heavy analyses and processes are employed including (i) *control flow analysis* to characterize the internal execution paths of the task, (ii) *hardware subsystem analysis* to provide an abstract model of the timing behavior of the underlying hardware resources, and (iii) *estimate calculation* which uses the results of (i) and (ii) to bound the WCET of the task [WEE+08; CRR+14]. To tighten the obtained WCET guarantee, these analyses often employ and combine measurement-based, simulation-based, statistical, and/or formal techniques.

Context-Aware Timing Analysis

The timing behavior of a task strongly depends also on its execution context, i.e., the presence and the behavior of other processes in the system. Sharing resources among concurrently executed tasks can lead to considerable variations in their timing behavior since the accesses of one task to a shared resource may be blocked or delayed by concurrent accesses from other tasks. Due to the imposition of such *timing interference* scenarios, e.g., preemption delay on a shared core or delays imposed when accessing memory via a shared bus, the maximum interval between the arrival time and the completion time of a task, referred to as its *Worst-Case Response Time (WCRT)*, often exceeds the WCET guarantee derived for it in isolation [DB11].

The second phase of the timing verification of a hard real-time application aims at

deriving an upper bound on the WCRT of each task in its execution context. Such an upper bound accounts for both the execution time of the task and the maximum possible amount of timing interference that may be imposed on its execution due to the presence of concurrent processes in the system. To that end, *WCRT analyses* (also called *schedulability analyses*) are used to derive a safe and tight *WCRT guarantee* for each task based on (i) its WCET guarantee, (ii) the arbitration/scheduling policy of the shared resources it uses, and (iii) the timing behavior of other users of the shared resources [MRR+19]. In the scope of a many-core system, WCRT analysis mainly involves bounding the preemption delays in the case of shared cores and the delays imposed on the transfer of data to/from tile memories via shared buses.

Communication Timing Analysis

Different tasks of an application are often subject to *data dependencies* such that the results produced by one task serve as input for another task. The exchange of data between data-dependent tasks imposes a temporal overhead that must be taken into account when deriving worst-case timing guarantees for the whole application. Note that the WCRT of each task also accounts for reading/writing its input/output data from/to the respective memories on the tile where the task is executed.

As detailed in Section 2.1.4, the shared-memory scheme within each tile of a many-core platform enables the intra-tile data exchanges between data-dependent tasks executed on the same tile to happen *implicitly* through the shared memories of that tile. Thus, the temporal overhead of intra-tile transfers is restricted to the time taken for writing and reading the data to/from memory which is already accounted for in the WCRT guarantee of the producer and consumer tasks.

Due to the distributed (NORMA) memory scheme between different tiles, however, inter-tile data exchanges necessarily depend on an *explicit* transfer of data via the NoC between the source and destination tiles. The worst-case temporal overhead of such transfer processes, namely, the *Worst-Case Traversal Time (WCTT)* of inter-tile communications, is not incorporated in the WCRT of the producer and consumer tasks and, therefore, must be bounded separately. Similarly to the worst-case timing behavior of tasks, the WCTT of a data transfer consists of (i) context-independent and (ii) context-dependent latency components that can be analyzed separately or in a single step, introduced in the following.

Basic Transfer Time The context-independent latency component of an inter-tile data transfer is referred to as the *basic transfer time* which denotes the worst-case transfer time of the data in full isolation, i.e., in the absence of any other concurrent communication flows in the network. To bound the basic transfer time, context-independent timing analysis is performed, taking into account (i) the maximum size of the data to be transferred and (ii) the timing behavior of the resources involved in the transfer process, namely, memories, buses, NAs, and routers [DT04].

Worst-Case Network Jitter Concurrent communication flows in a network can interfere with the timing behavior of one another directly or indirectly if resources are shared among multiple communication flows. The maximum amount of timing interference imposed on the transfer of a data set is referred to as the *network jitter*. The network jitter is analyzed using a context-aware timing analysis based on (i) the previously derived basic transfer time of the data, (ii) the arbitration/scheduling policy of the resources involved in its transfer, (iii) the timing behavior of concurrent communication flows which use these resources, and (iv) the operational characteristics of the NoC, e.g., switching strategy and flow-control mechanism [HRG+16].

Compositional Timing Analysis

In a many-core system, several resources are typically involved in the execution of a task or the inter-tile transfer of data, each contributing a latency component to the overall latency of the respective computation or communication process. For instance, the execution time of a task consists of the time taken to read/write its data from/to the memory over a bus and the time taken to perform the computation on the core. Similarly, the inter-tile transfer latency of a communication flow consists of the time taken to read the data from the memory over a bus, the time taken to transfer the data over the network, and the time taken to write the data into the destination memory. Any change in the context of each step, e.g., interfering processes, can lead to a change in the latency of that step and, hence, the overall latency of the whole process. To derive a safe worst-case timing guarantee during the WCRT analysis of a task or the WCTT analysis of an inter-tile data transfer, all possible combinations of timing interference scenarios imposed in the steps above must be taken into account to identify the scenario that leads to the highest possible latency for the overall process.

Due to the intractable number of possible combinations of interference scenarios in different steps, an integrated consideration of them typically becomes computationally infeasible [MTT18]. As a remedy, the complexity of the timing verification process can be mitigated by means of *compositional timing analyses*. In such analyses, the worst-case timing interference imposed on a process is *decomposed* into a set of timing interference *contributions*, each incurred in one step of the process and/or on one or a subset of involved resources [HRW15; HAE+17]. Timing decomposition is realized in such a way that (i) each interference contribution can be analyzed and bounded individually, and then, (ii) the derived bounds for different contributions can be combined according to a so-called *combination function* of the timing decomposition to obtain a globally safe timing guarantee for the whole process [HRW15]. Such a timing decomposition is often applied during different steps of worst-case timing analysis as a complexity-reducing mechanism. The timing analyses presented in this work are developed in a compositional manner.

2.2.3 Timing Predictability

In many-core systems, performance-boosting features such as branch prediction, out-of-order execution, and pipelining, have proven effective in enhancing the average-case performance of applications. Such features, however, often manifest an *unpredictable timing behavior* which can be fatal for hard real-time applications relying on worst-case timing guarantees: The timing unpredictability of these features contributes significantly to the uncertainty of the execution environment of applications, i.e., leads to an expansion in the state space of the environment Q in Eq. (2.1). The inflated uncertainty of the environment state often results in a significant increase in the worst-case timing properties of execution and may lead to situations where derivable timing guarantees are too large to be of any practical use [But11; TGR+16]. Some uncertain resource behaviors, e.g., random resource arbitration or cache replacement policies, even render an exact analysis of worst-case timing guarantees impossible. Moreover, such features often give rise to *timing anomalies*, defined as follows:

> **Timing Anomaly** [LS99; RWT+06]: A timing anomaly refers to a situation in which the local worst-case timing scenario, e.g., a cache miss or a branch misprediction, does not entail the global worst-case timing scenario, e.g., maximum possible execution time.

Timing anomalies render compositional timing analyses difficult (if not impossible) and complicate the *formal* analysis of worst-case timing guarantees [MTT18]. Therefore, *timing predictability* is an indispensable property of any platform used in a hard real-time context so that practical bounds on the worst-case timing properties of applications can be derived analytically: In a timing-predictable system (i) the range of observable values for timing properties of applications is narrowed down such that practical (useful) worst-case timing guarantees can be derived for them. Moreover, the timing-predictable behavior of resources prevents timing anomalies and, thereby, (ii) enables the use of *formal* and *compositional* timing analyses to derive timing guarantees at an affordable computational overhead [AMH+11]. This work considers timing predictable many-core systems in which each and every resource has a predictable timing behavior devoid of randomness or timing anomalies. In the rest of this work, the term predictability refers to timing predictability unless stated otherwise.

2.2.4 Timing Composability

Composability is another key complexity-reducing system property which is of crucial importance for the design of real-time embedded systems based on many-core platforms. In general, composability outlines the ability to develop a system through the integration of individually designed and verified components such that a property

verified for a component individually remains unaffected in the presence of other components after the system integration. Kopetz defines composability as follows:

> **Composability** [Kop11]: An architecture [system] is said to be composable w.r.t. a specified property if the system integration will not invalidate this property once the property has been established at the subsystem level. Examples of such properties are timeliness or testability. In a composable architecture [system], the system properties follow from the subsystem properties.

Timing composability is a fundamental system property for the design of modern real-time systems which host a large number of applications. In a timing-composable system, concurrent applications are decoupled from each other w.r.t. their worst-case timing behavior so that the timing verification of each application can be performed individually, irrespective of the presence and/or behavior of the other applications in the system [AMH+11]. By extension, timing composability also enables independent timing analysis of individual tasks of an application where timing guarantees can be derived for different tasks separately and then combined to bound the timing properties of the whole application. In the rest of this work, the term composability refers to timing composability unless stated otherwise.

Composability may be established by means of *spatial* and/or *temporal* isolation of applications from each other. While spatial isolation eliminates resource sharing by assigning each resource exclusively to one application, temporal isolation allows resources to be shared among applications in such a way that their worst-case timing behavior remains independent from each other. Establishing temporal isolation between concurrent applications necessitates the practice of so-called *composable arbitration policies* on shared resources which are discussed in the following.

Composable Resource Sharing

Temporal isolation between concurrent applications entails two essential cornerstones: The practice of (i) *composable arbitration policies* on shared resources and (ii) *exclusive reservation of resource budgets* per requestor of each shared resource. A composable arbitration policy erects a temporal firewall between concurrent requestors of a shared resource. This enables the worst-case timing behavior of each requestor to be analyzed based on its reserved resource budget and the arbitration policy of the shared resource *regardless* of the presence or the behavior of other requestors. In fact, a composable arbitration policy ensures that temporal control resides within the shared resource itself and not in its requestors [Kop11]. This renders *event-driven* and/or *priority-based* arbitration policies, e.g., Round-Robin (RR) or First-Come-First-Serve (FCFS), non-composable [AMH+11]. Contrarily, *contention-less* and *time-triggered* arbitration policies are composable. In particular, *Time-Division Multiple Access*

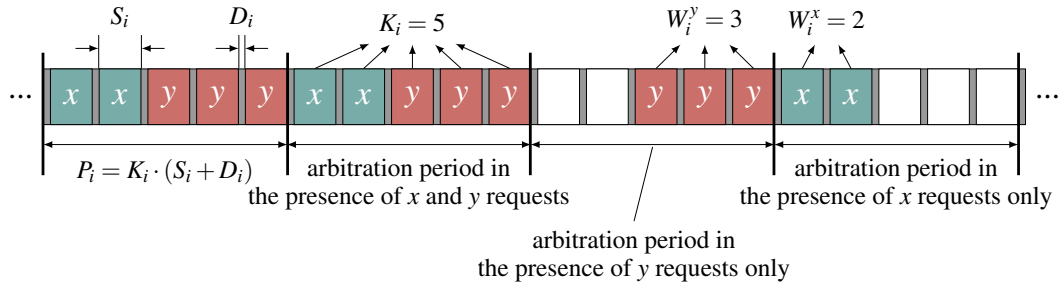


Figure 2.4: Example of Time-Division Multiple Access (TDMA) arbitration policy of wheel size $K_i = 5$ for a resource i with two requestors, x and y, which have $W_i^x = 2$ and $W_i^y = 3$ reserved time slots per arbitration period, respectively.

(TDMA) and *Weighted Round-Robin (WRR)* policies are well-established and widely used composable arbitration policies [GAC+13; HGB+09; HKK+13].

Note that composability and timing predictability are two orthogonal system properties where the presence or the lack of one does not necessarily confirm or invalidate the other [AMH+11]. For instance, a shared resource with RR arbitration annihilates composability, while leaving timing predictability intact. Likewise, caches with random replacement policy annihilate timing predictability, while they leave system composability unharmed as long as they are not shared. In this work, each shared resource of a given many-core architecture is assumed to have a composable arbitration policy, e.g., TDMA or WRR, to establish timing composability. These policies are outlined in the following.

Time-Division Multiple Access (TDMA) Arbitration

TDMA is a composable arbitration policy that is widely used in real-time systems. In TDMA, physical time is divided into a series of *time slots* of equal length which are periodically assigned to resource requestors. Each arbitration period, also called *TDMA round*, consists of a fixed number of consecutive time slots referred to as *wheel size*, see [JW04; HKK+13]. A requestor may reserve one or more periodic time slots. Each pair of consecutive time slots may be separated by a fixed-length (relatively short) control time slot used by the resource arbiter to switch between different requestors. The length of each control slot is equal to the *arbitration delay* of the arbiter. For instance, the arbitration delay of a shared core reflects the context-switch latency of the operating system for switching between tasks.

Figure 2.4 depicts an example of TDMA arbitration. The TDMA arbitration policy of a resource i can be specified by three parameters: (i) the length S_i of each arbitration time slot, (ii) the length D_i of each control slot, and (iii) the arbitration wheel size K_i. Based on these parameters, the length of an arbitration period P_i of resource i is calculated as $P_i = K_i \cdot (S_i + D_i)$. Here, a requestor x which has reserved $W_i^x \in \mathbb{N}^+_{\leq K_i}$

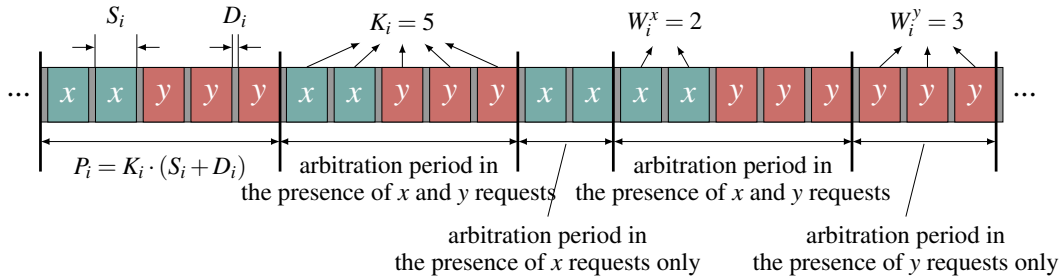


Figure 2.5: Example of Weighted Round-Robin (WRR) arbitration policy of wheel size $K_i = 5$ for a resource i with two requestors, x and y, which have $W_i^x = 2$ and $W_i^y = 3$ reserved time slots per arbitration period, respectively.

periodic slots on i receives a dedicated periodic resource time budget of $(W_i^x \cdot S_i)$ and, hence, experiences an aggregate wait time of $(P_i - W_i^x \cdot S_i)$ per arbitration period P_i. In this work, W_i^x is referred to as the *arbitration weight* of requestor x on resource i.

TDMA is a *non-work-conserving* arbitration policy [JW04]. That means, a shared resource under TDMA arbitration remains idle during the time slots that are not used by their assigned requestors even if pending requests are present from other requestors. This is visualized in Figure 2.4 where the white slots represent unused arbitration slots in which i remains idle. Since resource requestors typically do not utilize each and every one of their reserved slots, TDMA arbitration often results in a considerable under-utilization of resources and a poor average-case performance. This renders TDMA a rather unattractive arbitration policy for composable systems in which resources may be shared among real-time and best-effort applications [KHM+13].

Weighted Round Robin (WRR) Arbitration

WRR has fundamental similarities to TDMA in that it divides time into a series of equally long and periodically arbitrated time slots. However, unlike TDMA, WRR is a *work-conserving* arbitration policy in which unused time slots (regardless of them being reserved or not) are skipped to minimize the wait time of requestors and the idle time of the resource, see [QLD10; HKK+13]. This scheme results in a dynamic adjustment of the length of each arbitration period according to the number of active requests in that period. Compared to TDMA, this adaptability leaves the worst-case timing guarantees derived for real-time applications intact while improving the average-case performance considerably, e.g., in favor of best-effort applications [HKK+13].

Figure 2.5 illustrates an example of WRR arbitration. The WRR arbitration policy of a resource i can be described by three parameters: (i) the length S_i of each arbitration slot, (ii) the length D_i of each control slot, and (iii) the *maximum* arbitration wheel size K_i. Based on these parameters, the *maximum* length of an arbitration period P_i

is calculated as $P_i = K_i \cdot (S_i + D_i)$. However, depending on the number of present requests, the actual length of an arbitration period may be smaller than P_i.

Note that under WRR, each requestor of resource i is guaranteed to receive its reserved number of periodic time slots in *any* sequence of K_i consecutive time slots. Therefore, worst-case timing guarantees can be derived for each requestor based on its reserved number of periodic time slots and the arbitration parameters (S_i, D_i, and K_i) similarly to TDMA. Here, a requestor x which has reserved $W_i^x \in \mathbb{N}^+_{\leq K_i}$ periodic arbitration slots on resource i attains (upon request) an aggregate dedicated resource time of *at least* $(W_i^x \cdot S_i)$ in each time window of length P_i. This results in a *maximum* aggregate wait time of $(P_i - W_i^x \cdot S_i)$ per time window P_i for x on resource i.

2.2.5 Invasive Computing

Heterogeneous many-core platforms are widely recognized as viable solutions for the design of emerging and future embedded systems. These systems typically encompass a large number of applications with various non-functional requirements, including hard timing constraints, and oftentimes exhibit dynamic execution and workload profiles throughout their lifespan. A successful adoption of many-core platforms for the design of such systems, however, necessitates a reconsideration of existing design and programming practices and the introduction of novel design methodologies to address the major design challenges of emerging embedded systems targeting many-core platforms. In particular, programmability, adaptivity, scalability, power and thermal constraints, real-time capability, reliability, and fault tolerance are envisioned to constitute the major design challenges of contemporary and future embedded systems targeting large-scale heterogeneous many-core platforms [THH+11].

In response to these challenges, Teich [Tei08] has proposed a novel programming paradigm called *invasive computing* which addresses the major aspects of the design challenges outlined above by introducing the concept of *resource-aware programming*. In invasive computing, applications are empowered to (i) negotiate their resource requirements with the system manager online to attain their required resources in view of a dynamic availability of resources, (ii) distribute their workload on the resources provided to them, and (iii) release these resources upon the completion of their execution. In this scope, Teich et al. define invasive programming as follows:

> **Invasive Programming** [THH+11]: Invasive programming denotes the capability of a program running on a parallel computer to request and temporarily claim processor, communication, and memory resources in the neighborhood of its actual computing environment, to then execute in parallel the given program using these claimed resources, and to be capable to subsequently free these resources again.

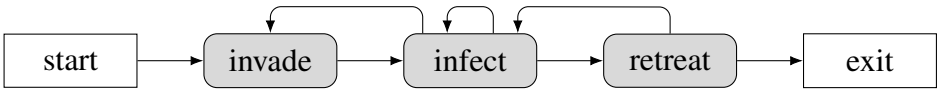


Figure 2.6: Execution phases of an invasive program according to [HRS+11].

Based on this definition, an invasive computing scheme can be established by providing language and system support for invasive programming through three identified basic programming constructs, namely, *invade*, *infect*, and *retreat*, to enable resource-aware application programming. The execution phases of an invasive program in terms of these three primitives are shown in Figure 2.6, adopted from [HRS+11]. As illustrated, the execution of an invasive program begins with issuing an *invade* call to request an initial *claim* of resources denoting a set of processing, communication, and memory resources required for the execution of the program. Following a successful invasion (allocation) of resources, the program can deploy and execute its code on the claimed resources using an *infect* call. During its execution, it can infect its claimed resources several times (also called *re-infect*), e.g., to perform various processes on the claimed resources. Once the execution of the program is completed, it can release the claimed resources using the *retreat* call to terminate and exit. A program may also request an adaptation of its claimed resources during its execution to shrink (using retreat) or expand (using invade) its set of claimed resources, e.g., in reaction to a change in its workload.

Invasive primitives, namely, invade, infect, and retreat, can be introduced into many programming languages to attain resource awareness. For instance, the library InvadeX10 [HRS+11] introduces these primitives into the X10 programming language [CGS+05] from IBM. Provided with these language constructs, the application programmer can specify the set of resources required for the satisfactory execution of the application w.r.t. its non-functional requirements. Alternatively, invasive computing also provides X10 language support for the programmer to specify the desired range for each non-functional property of interest (e.g., timing, power consumption, reliability, and security) in terms of so-called *requirements* [TGR+16] which denote soft/hard upper- and lower bound constraints on the non-functional property in question. Based on these user-specified requirements, a suitable claim of resources can be derived, e.g., using system-level Design Space Exploration (DSE) techniques, and the program can be subsequently augmented with the derived claim of resources.

The description above elucidates the capability of invasive computing in establishing an adaptive execution of applications on many-core platforms by means of resource-aware programming. The exclusive reservation of resources (or resource budgets) per application in invasive computing can be seen as an enabler of timing composability in a system by enforcing spatial/temporal isolation between concurrent applications. This, in turn, enables a scalable (per-application) design and a non-functional verification scheme which is particularly crucial in the context of hard real-time applications and

systems. In the scope of this work, invasive computing serves as a promising solution to provide the features and fundamentals based on which the methodologies, analyses, and techniques proposed in this work can be incorporated into the design practice of emerging embedded systems to empower an adaptive and satisfactory execution of hard real-time applications on heterogeneous many-core platforms.

2.3 System Model

This section introduces the system model used in this work. The system-level design of an embedded system—consisting of a set of applications to be executed on a given many-core platform—mainly involves the *mapping* of the applications in the system onto the resources of the platform. The description of the applications and the platform architecture are initially provided in a Systems-Level Design Language (SLDL), e.g., SysteMoC [FHT06]. This description is then transformed into a *symbolic* system model that is used for the mapping of applications onto platform resources.

In this work, the graph-based system model as introduced first in [BTT98] is used which is referred to as the *specification*, see [LSG+09] for a SLDL-based transformation. The specification consists of graph-based representations of each application and the platform architecture as well as a set of *attributes* reflecting certain non-functional properties of application tasks and platform resources. This representation encapsulates all necessary information for the mapping of applications onto a platform and the evaluation of the quality of obtained mapping solutions w.r.t. a set of non-functional properties of interest, e.g., timeliness and energy consumption.

In the following, the building blocks of the specification used in this work are presented as follows: In Sections 2.3.1 and 2.3.2, the application model and the architecture model are presented, respectively, and their graph-based representations used in this work are introduced. Afterwards, Section 2.3.3 introduces mapping edges that link the application graph to the architecture graph and reflect the design space for the application mapping process.

2.3.1 Application Model

In the context of multi/many-core systems, applications are typically partitioned into a set of processing tasks that can be executed in parallel on different resources—and exchange data with each other, if necessary—to exploit the parallel processing potential of the underlying platform [Moy13; Mar06]. In the system model used in this work, an application is specified by a so-called *application graph* $G_{\text{app}}(V_{\text{app}}, E_{\text{app}})$, also referred to as *problem graph* [BTT98] or *task graph* [DRW98]. Here, $V_{\text{app}} = T \cup M$ is a bipartite set of vertices where $T = \{t_0, ..., t_{|T|-1}\}$ denotes the set of (processing) *tasks* of the application, and $M = \{m_0, ..., m_{|M|-1}\}$ denotes the set of *messages* exchanged among the tasks. The notations $|T|$ and $|M|$ denote the cardinality of sets T and M,

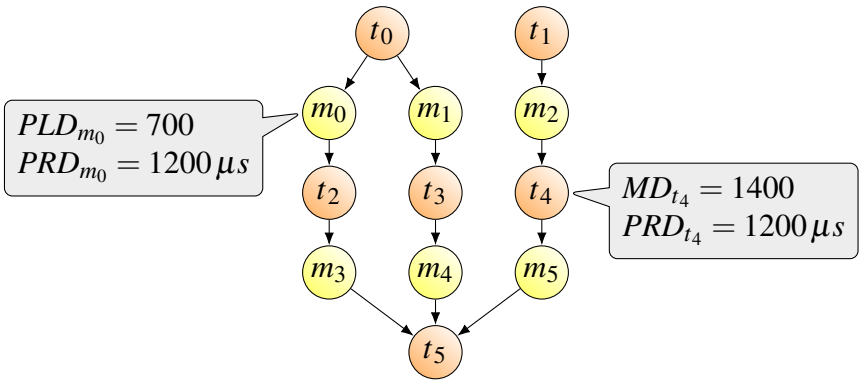


Figure 2.7: Example of an application graph $G_{\text{app}}(V_{\text{app}}, E_{\text{app}})$ where $V_{\text{app}} = T \cup M$ contains tasks $T = \{t_0, ..., t_5\}$ and messages $M = \{m_0, ..., m_m\}$, and edges $E_{\text{app}} \subseteq (T \times M) \cup (M \times T)$ denote data dependencies among tasks and messages. Attributes are exemplified here for task t_4 and message m_0.

respectively. The directed edges $E_{\text{app}} \subseteq (T \times M) \cup (M \times T)$ in the application graph specify the *data dependencies* (partial order) between tasks and messages. Each data dependency $e \in E_{\text{app}}$ connects a task to a message or vice versa. An example of an application graph is shown in Figure 2.7.

In the applications considered in this work, each message $m \in M$ has exactly one predecessor (producing task) and one successor (consuming task). Here, convenience functions $\text{IN}(t) \subseteq M$ and $\text{OUT}(t) \subseteq M$ are used to provide the set of input and output messages of task t, respectively. The union set of input and output messages of task t is provided by function $\text{IO}(t) = \text{IN}(t) \cup \text{OUT}(t) \subseteq M$.

This work considers applications with *hard real-time* constraints on their end-to-end latency (makespan) referred to as the *application deadline*. A mapping of such an application is considered *useful* only if it is guaranteed to meet the deadline of the application when executing it on a given platform. In the application graph, tasks and messages are annotated with a set of *attributes* presented in the following. The attributes are used during the application mapping process and/or for the evaluation of the non-functional properties of mappings, e.g., worst-case timing guarantees.

Task and Message Attributes

Each task $t \in T$ in the application graph is annotated with its *maximum memory demand*, i.e., the maximum number of single-word memory accesses it may perform during its execution, denoted by MD_t. Each task is also annotated with an *execution period* PRD_t which specifies the (minimum) inter-arrival time of its input message(s). Similarly, each message $m \in M$ is annotated with its *maximum payload size* PLD_m (in number of words) as well as its *transfer period* PRD_m which denotes the (minimum) inter-production time of that message.

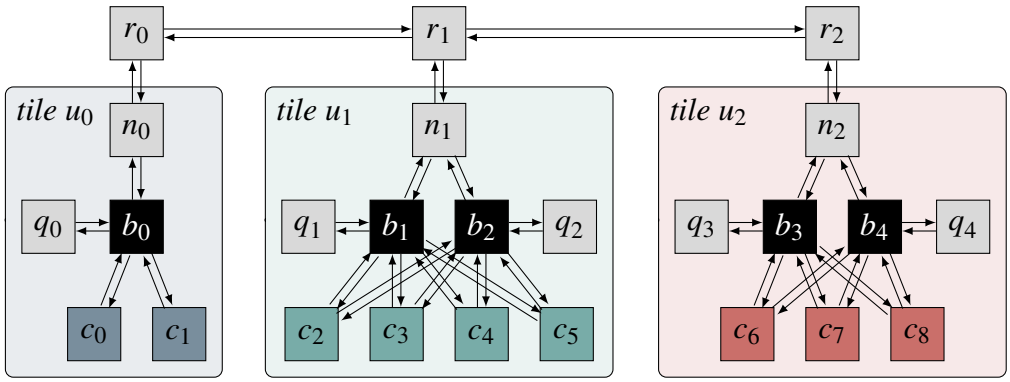


Figure 2.8: Example of an architecture graph for a heterogeneous many-core platform composed of three tiles of different types (from Figure 2.2) with a 3×1 NoC organization. Core types are denoted by color (blue, green, and red).

2.3.2 Architecture Model

In the system model of this work, the architecture of a many-core platform is specified by a so-called *architecture graph* $G_{\text{arch}}(V_{\text{arch}}, E_{\text{arch}})$. Here, the set of vertices $V_{\text{arch}} = C \cup Q \cup B \cup N \cup R$ represents the resources of the platform where each $c \in C$ denotes a processing core, each $q \in Q$ a memory, each $b \in B$ a bus, each $n \in N$ a Network Adapter (NA), and each $r \in R$ a router of the NoC. The edges $E_{\text{arch}} \subseteq V_{\text{arch}} \times V_{\text{arch}}$ represent physical connections (links) between the resources. Non-router resources of the architecture, i.e., cores, memories, buses, and NAs, are organized as a set U of tiles where each tile $u \in U$ consists of a NA, the buses connected to it, and all cores and memories connected to these buses. Each non-router resource belongs to exactly one tile, hence, tiles are (i) *disjoint*, i.e., $u_i \cap u_j = \emptyset$ for all $u_i, u_j \in U$ where $i \neq j$, and (ii) *covering*, i.e., $\cup_{u \in U} u = C \cup Q \cup B \cup N$. In this work, convenience function $\text{TILE}(i) \in U$ is used to provide the tile $u \in U$ on which the non-router resource $i \in C \cup Q \cup B \cup N$ is located. Figure 2.8 illustrates an example of an architecture graph for a heterogeneous many-core platform consisting of three tiles with a 3×1 NoC organization. The graph-based representation of the tiles in Figure 2.8 corresponds to the three tile architectures depicted in Figure 2.2.

Resource Attributes

Each resource in the architecture graph is annotated with a set of attributes that reflect the necessary non-functional properties of that resource. In the system model of this work, each memory $q \in Q$ in the architecture graph is annotated with its basic *service time* ST_q which denotes the time required to perform a single-word memory operation (read/write) in isolation. The service time ST_q of a memory $q \in Q$ comprises the latency of address/data-exchange cycles and the intrinsic delay of the memory.

Since this work considers composable systems, any shared resource must have a

composable arbitration policy, e.g., TDMA or WRR. As detailed in Section 2.2.4, such an arbitration policy can be described by (i) the length S of each arbitration slot, (ii) the maximum delay D of the arbiter, and (iii) the arbitration wheel size K. In this line, the arbitration parameters of resources are provided as follows:

- Each bus $b \in B$ is annotated with its (maximum) arbitration wheel size K_b, slot length S_b, and arbitration delay D_b.
- Each core $c \in C$ is annotated with its scheduling wheel size K_c, scheduling slot length S_c, and scheduling delay D_c. The latter corresponds to the context-switch overhead of the operating system. Each core $c \in C$ is also annotated with its arbitration weight W_b^c on each local bus $b \in B$ over which c accesses memories.
- Each NA $n \in N$ is annotated with the arbitration parameters of its TX unit tx, namely, K_{tx}, S_{tx}, and D_{tx}, and those of its RX unit rx, namely, K_{rx}, S_{rx}, and D_{rx}. It is also annotated with the arbitration weight of its TX and RX units on each local bus $b \in B$ used to access memories, denoted by W_b^{tx} and W_b^{rx}, respectively.
- Each router $r \in R$ is annotated with its routing delay D_r as well as the arbitration capacity K_r and slot length S_r of each of its output ports.

2.3.3 Mapping Edges

In the specification, the application and architecture graphs are connected by a set of so-called *mapping edges* $V \subseteq T \times C$ that reflect the design space of possible task-to-core assignments. A mapping edge $v = (t,c) \in V$ denotes that task $t \in T$ can be executed on core $c \in C$. Each mapping edge $v = (t,c) \in V$ may also be annotated with a set of attributes which reflect non-functional properties of task t when executed on core c. In the system model of this work, each mapping edge $v = (t,c) \in V$ is annotated with the WCET of task t on core c, denoted by $WCET_{t,c}$.

Contrarily to tasks, messages are not subject to predetermined restrictions regarding their choices of storage and communication resources. That means, each message $m \in M$ may be stored in any memory $q \in Q$ in the architecture and, in the case of an inter-tile message, it may be transferred via any NoC route between its source and destination tiles. Therefore, it is not required to explicitly denote the mapping possibilities of messages in the specification. As will be detailed in Section 2.4.1, the choice of memory and (in the case of inter-tile communications) NoC route for each message depends on the mapping of its producing and consuming tasks to cores and the routing algorithm of the NoC, respectively.

2.3.4 Execution and Operation Model

In an application specified by application graph $G_{\text{app}}(T \cup M, E_{\text{app}})$, the execution of tasks T and the transfer of messages M between them must follow the order reflected

by the data dependencies E_{app}. Each iteration of execution for a task $t \in T$ can be initiated only after one instance of each of its input messages $m \in \text{IN}(t)$ is available in the memories of the tile on which t is executed. For instance, the execution of task t_5 in Figure 2.7 can be initiated only if one instance of each input message m_3, m_4, and m_5 is locally available on the tile where t is to be executed. During its execution, a task t produces one instance of each of its output messages $m \in \text{OUT}(t)$ and stores them in the memories of the tile on which it is executed. The maximum number of memory operations performed for reading/writing input/output messages from/to the memory is incorporated in the memory demand MD_t attribute of task t. Memory operations are considered to be *blocking*, thus, the execution of a task is stalled during its memory operations. Each memory operation corresponds to a single-word memory access with a service time of ST_q for each memory $q \in Q$ which is given as an attribute of q.

In general, resources of a many-core platform may be shared among multiple requestors unless exclusively reserved. Recall from Section 2.2.4 that a composable arbitration of shared resources necessitates a time-triggered arbitration scheme. Hence, to enable a composable arbitration of memory buses between the resources located on each tile, *burst* memory operations are excluded which enables a semi-instantaneous switching between different requestors. Here, for accessing a block of data, a requestor may perform several consecutive single-word memory operations during its reserved time slot(s) on the bus. Likewise, to preserve composability also when sharing cores between multiple tasks, a *preemptive execution* model [DB11] is considered. This enables the operating system to halt the execution of each task preemptively to have its processing time confined within its reserved scheduling time slot(s). Similarly, to preserve composability when sharing NoC routes between concurrent communication flows, wormhole-switched NoCs with credit-based virtual-channel flow control are considered. This allows per-link bandwidth reservation and time-triggered preemption of each communication flow at each router [HKK+13]. In such NoCs, the arbitration slot length S_r of each router $r \in R$ is typically equal to the duration of one clock cycle of the NoC which suffices for the transfer of one flit of data over a link.

2.4 Application Mapping in Many-Core Systems

During the design process of an embedded system based on a given many-core platform, the applications in the system are mapped onto the resources of the platform. This section provides an overview of the current practice of application mapping in many-core systems as follows: First, Section 2.4.1 presents the standard procedure of mapping an application on a many-core platform. Thereafter, Sections 2.4.2 and 2.4.3 provide an overview and classification of existing mapping schemes and methodologies for multi/many-core systems consisting of several applications. Finally, Section 2.4.4 introduces the mapping methodology used in this work, namely, *Hybrid Application Mapping (HAM)*, and elucidates its key characteristics which are of

principal significance for the design of adaptive hard real-time embedded systems.

2.4.1 Application Mapping Procedure

The process of mapping an application onto a many-core platform is realized using the *system-level synthesis* procedure introduced in [BTT98]. In line with this procedure, the *specification* presented in Section 2.3 is used to compute a so-called *mapping*[1] of the application on the platform by conducting four sets of design decisions, namely, *binding*, *routing*, *allocation*, and *scheduling*, as detailed in the following.

Binding

Binding design decisions determine the assignment of tasks T to cores C and the assignment of messages M to memories Q. These assignments are provided by a so-called *binding* $\beta = \beta^{\text{task}} \cup \beta^{\text{msg}}$ and are derived as follows: First, the binding β^{task} of tasks to cores, formally defined in Eq. (2.2), is determined by selecting exactly one mapping edge[2] $v = (t,c) \in V$ for each task $t \in T$, denoting that task t is mapped to core $c \in C$. In this work, convenience function $\text{CORE}(t,\beta)$ provides the core $c \in C$ to which task $t \in T$ is mapped in binding β. After the binding β^{task} of tasks to cores has been determined, the binding β^{msg} of messages to memories, formally defined in Eq. (2.3), is determined in such a way that each message $m \in M$ is mapped (i) to exactly one memory on the tile where it is produced and (ii) to exactly one memory on the tile where it is consumed. Here, $(m,q) \in \beta^{\text{msg}}$ denotes that message $m \in M$ is to be stored in memory $q \in Q$.

$$\beta^{\text{task}} = \big\{(t,c) \mid t \in T \wedge c \in C\big\} \tag{2.2}$$

$$\beta^{\text{msg}} = \big\{(m,q) \mid m \in M \wedge q \in Q\big\} \tag{2.3}$$

Routing

Routing design decisions determine a NoC *route* for each inter-tile message, i.e., each message that is communicated between tasks which are mapped to different tiles. The *route* of an inter-tile message $m \in M$ establishes a direct path between the source and destination tiles of m over the NoC and is denoted by a non-repeating sequence or tuple $(\hat{r}) \subseteq R \cup N$ of connected routers and NAs ($\hat{r} \in R \cup N$). The route of each inter-tile message is determined in accordance with the routing algorithm of the NoC, e.g., XY-routing [NM93]. The routes of messages are provided as a so-called *routing* ρ, formally defined in Eq. (2.4). In this work, convenience function $\text{ROUTE}(m,\rho)$ provides the NoC route of message $m \in M$ in routing ρ. Moreover, the sequence of routers located on the NoC route of a message $m \in M$ (excluding the

[1] The term *mapping* in this work corresponds to what is defined as *implementation* in [BTT98].
[2] This work does not consider multi-binding.

NAs on the source and destination tile) is denoted by $\rho_m = \text{ROUTE}(m,\rho) \setminus N \subseteq R$. For any message $m \in M$ produced and consumed on the same tile, $\text{ROUTE}(m,\rho) = \emptyset$ as m does not require a NoC transfer.

$$\rho = \left\{ (m,(\hat{r})) \mid m \in M \wedge \hat{r} \in R \cup N \right\} \tag{2.4}$$

Allocation

Allocation design decisions determine the set of resources used during the execution of tasks T and/or transfer of messages M as a so-called *allocation* $\alpha \subseteq C \cup Q \cup B \cup N \cup R$. Based on the binding and routing decisions, the allocation α contains (i) the cores to which tasks are mapped (specified by β^{task}), (ii) the memories to which messages are mapped (specified by β^{msg}), and (iii) the NoC routers and NAs over which inter-tile messages are routed (specified by ρ). In addition to that, the allocation α also includes (iv) the buses connecting the allocated cores and NAs to the allocated memories on their tile as these buses are used to access memories.

Scheduling

Scheduling design decisions specify the budget of each task/message on the (possibly shared) resources it uses. Given the arbitration parameters of resources (provided as resource attributes in the architecture graph), a so-called *schedule* $\sigma = \sigma^{\text{task}} \cup \sigma^{\text{msg}}$ is specified: Here, the schedule σ^{task} of tasks, formally defined by Eq. (2.5), denotes for each task $t \in T$, the *arbitration weight* W_c^t, i.e., the number of periodic time slots, to be reserved for task t on the core to which t is mapped, i.e., $c = \text{CORE}(t,\beta)$. Likewise, the schedule of messages σ^{msg}, formally defined by Eq. (2.6), denotes for each inter-tile message $m \in M$, the arbitration weight $W^m_{(\hat{r})}$ of m on each NoC resource (router or NA) $\hat{r} \in R \cup N$ located on m's NoC route $(\hat{r}) = \text{ROUTE}(m,\rho)$.

$$\sigma^{\text{task}} = \left\{ (t, W_c^t) \mid t \in T \wedge c = \text{CORE}(t,\beta) \wedge W_c^t \in \mathbb{N}^+_{\leq K_c} \right\} \tag{2.5}$$

$$\sigma^{\text{msg}} = \left\{ \left(m, W^m_{(\hat{r})}\right) \mid m \in M \wedge (\hat{r}) = \text{ROUTE}(m,\rho) \wedge (\forall \hat{r} \in (\hat{r}) : W^m_{(\hat{r})} \in \mathbb{N}^+_{\leq K_{\hat{r}}}) \right\} \tag{2.6}$$

Mapping Feasibility

The result of the binding, routing, allocation, and scheduling design decisions is a *mapping* μ of the application on the given many-core architecture, specified as $\mu = (\beta_\mu, \rho_\mu, \alpha_\mu, \sigma_\mu)$. A mapping μ is *feasible* if its constituting components, namely, β_μ, ρ_μ, α_μ, and σ_μ, satisfy the following conditions:

- **Binding (β_μ):** Each task $t \in T$ must be mapped to exactly one core $c \in C$ according to t's mapping possibilities denoted by the mapping edges in the

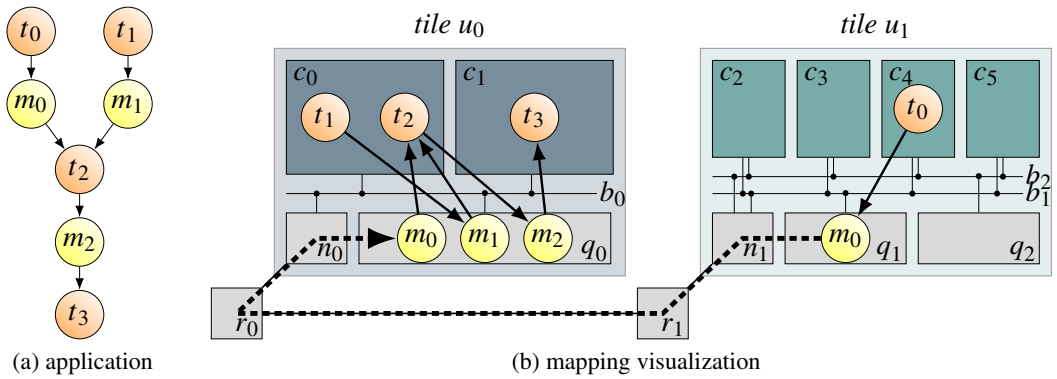
specification. Each message $m \in M$ must be mapped to exactly one memory $q \in Q$ on each tile where m is produced or consumed.

- **Routing (ρ_μ):** Each inter-tile message $m \in M$ must have an acyclic NoC route that connects m's source tile to its destination tile.
- **Allocation (α_μ):** The following resources must be allocated: Every core $c \in C$ with at least one task mapped to it, every memory $q \in Q$ with at least one message mapped to it, every NoC router $r \in R$ and every NA $n \in N$ with at least one message routed over it, and every bus $b \in B$ that connects the allocated cores and NAs to the allocated memories on the respective tile.
- **Schedule (σ_μ):** Each task $t \in T$ must have a positive arbitration weight W_c^t on the core $c \in C$ to which it is mapped. The overall arbitration weight assigned to the tasks that are mapped to a core $c \in C$ shall not exceed c's arbitration capacity K_c. Each inter-tile message $m \in M$ must have a positive arbitration weight $W_{(\hat{r})}^m$ on each router and each NA (TX/RX unit) located on its route $(\hat{r}) \subseteq R \cup N$. The overall arbitration weight assigned to messages routed over a router or NA (TX/RX unit) $i \in R \cup N$ shall not exceed i's arbitration capacity K_i.

Mapping Example

Figure 2.9 illustrates an exemplary mapping of an application on a 2×1 many-core platform. Here, tasks t_1–t_3 are mapped to cores located on tile u_0, while task t_0 is mapped to a core on tile u_1. Thus, messages m_1 and m_2 are intra-tile messages communicated internally within u_0, while message m_0 is an inter-tile message transferred via the NoC route (n_1, r_1, r_0, n_0) between memories q_1 and q_0 on the two tiles u_1 and u_0, respectively. The NoC route of m_0 is denoted by a dashed line in Figure 2.9. For simplicity, assume an arbitration capacity of $K_i = 10$ for each resource $i \in C \cup N \cup R$ where tasks t_0, t_1, t_2, and t_3 have arbitration weights of 2, 4, 3, and 8 on their respective cores, and m_0 has an arbitration weight of 5 on each resource on its NoC route. The mapping can be formally described as $\mu = (\beta_\mu, \rho_\mu, \alpha_\mu, \sigma_\mu)$ where:

$$
\begin{aligned}
\beta_\mu^{\text{task}} &= \Big\{(t_0, c_4), (t_1, c_0), (t_2, c_0), (t_3, c_1)\Big\} \\
\beta_\mu^{\text{msg}} &= \Big\{(m_0, q_0), (m_0, q_1), (m_1, q_0), (m_2, q_0)\Big\} \\
\rho_\mu &= \Big\{\big(m_0, (n_1, r_1, r_0, n_0)\big)\Big\} \\
\alpha_\mu &= \Big\{c_0, c_1, c_4, q_0, q_1, b_0, b_1, n_0, n_1, r_0, r_1\Big\} \\
\sigma_\mu^{\text{task}} &= \Big\{(t_0, 2), (t_1, 4), (t_2, 3), (t_3, 8)\Big\} \\
\sigma_\mu^{\text{msg}} &= \Big\{(m_0, 5)\Big\}
\end{aligned}
$$



(a) application (b) mapping visualization

Figure 2.9: Example of an application (a) and the visualization of a mapping μ of the application on a 2×1 many-core platform (b).

2.4.2 Mapping Schemes for Multi-Application Systems

Emerging embedded systems typically comprise several applications to be executed on a many-core platform. In this context, the goal of the design process is to find an *optimal* mapping of all applications on the platform. This, however, is a challenging task: As shown in [BTT98], even finding one *feasible* mapping of a single application on a many-core platform is an NP-complete problem. Consequently, finding an *optimal* mapping of an application on a many-core platform embodies an NP-hard optimization problem [GJ79]. The progressive increase in the number of applications integrated into a modern embedded system contributes further to the complexity of the design process as the space of possible mapping solutions grows exponentially with the overall number of tasks in the system.

Recent years also give evidence of a rapid increase in the dynamism of application workloads in embedded systems which also contributes considerably to the complexity of the system design process: In a dynamic system, each application may be launched and terminated on-demand, independently from other applications, and oftentimes in reaction to unforeseeable external events, e.g., user requests. In such systems, often only a fraction of all applications are simultaneously active at each point in time. Finding an optimal design in this context entails the consideration of all possible *system use-cases*—each denoting one combination of concurrently executed applications. The number of possible system use-cases, however, grows exponentially with the number of independent applications in the system and can become intractable in modern embedded systems which comprise a large number of applications [AMH+11].

Over the past decade, various methodologies have been introduced for the design of multi-application embedded systems based on many-core platforms. These methodologies can be classified into two general categories: (i) *integrated* mapping approaches which are inherited from the multi-core mapping paradigm and (ii) *constructive* mapping approaches which have emerged to alleviate the shortcomings of the former in

addressing the design complexity encountered in the context of mapping on many-core platforms. In the following, these classes are introduced.

Integrated Mapping Schemes

The process of mapping the applications of an embedded system onto a given target platform is traditionally realized based on an *integrated* mapping scheme. In such a scheme, all applications are considered at the same time within one mapping process, and mapping decisions are conducted for all applications simultaneously. This integrated scheme performs well for small systems consisting of one or only a few small applications with a relatively static workload targeting a platform with a relatively small number of resources, e.g., a multi-core platform. However, due to its poor scalability w.r.t. the number of applications in the system, the number of resources in the platform, and the number of system use-cases, this scheme is not considered viable for the design of emerging embedded systems comprising a large number of applications with a dynamic workload profile targeting many-core platforms.

Constructive Mapping Schemes

With the introduction of many-core platforms in embedded domains, the traditional integrated mapping scheme has been gradually replaced by a *constructive* scheme to alleviate the design complexity [AMH+11]. In a constructive mapping approach, the mapping process of the system is partitioned into two phases: First, (i) a *per-application mapping computation* phase is performed in which one or a set of high-quality mappings are computed for each application individually. Thereafter, in (ii) a *system integration* phase, the precomputed mappings of the applications are used to assemble the whole system. In systems designed based on a constructive approach, *system composability* ensures that the non-functional properties of each mapping evaluated during the per-application mapping computation phase also hold in the presence of the other applications in the system after the integration [Kop11; AMH+11], see Section 2.2.4.

A constructive mapping scheme, on the one hand, reduces the design complexity of a multi-application system to the complexity of individual application mapping processes. On the other hand, this simplification allows parts of the resulting less-demanding design steps to be conducted at run time. Such a shift enables an *adaptive* mapping scheme in which run-time situations, e.g., workload fluctuations, are taken into consideration during the mapping process of applications [SSK+13]. This potential has led to the emergence of different classes of many-core application mapping methodologies which are outlined in the following.

2.4.3 Application Mapping Methodologies

Application mapping is one of the most crucial aspects of system design based on many-core platforms [MOP+09]. As mentioned before, optimally mapping an application onto a given many-core platform has been shown to be an NP-hard optimization problem. Using an exact optimization approach, e.g., Integer Linear Programming (ILP), to find optimal mappings is, thus, impractical due to the intractable design space of possible mappings. In this context, Design Space Exploration (DSE) schemes based on *meta-heuristic* optimization techniques, e.g., evolutionary algorithms [Dav91; FF95], simulated annealing [KGV83], or particle swarm optimization [Ken10], have become the de facto standard approaches for many-core mapping optimization as they have proven effective in delivering high-quality mappings at the expense of an affordable computational effort, see [BTT98].

In addition to their choice of the optimization technique, application mapping methodologies employed for the constructive design of embedded systems can be classified based on *when* they perform the mapping optimization and/or the system integration. In this scope, [SSK+13] classifies multi/many-core application mapping approaches into two categories: (i) *static* (also called *offline* or *design-time*) mapping methodologies and (ii) *dynamic* (also called *online* or *run-time*) mapping methodologies. In the following, an overview of these methodologies is provided before the application mapping methodology used in this work is presented in Section 2.4.4.

Static Application Mapping

In a static application mapping approach, all mapping decisions are conducted at design time: First, DSE is performed to find an optimal mapping for each application, and then, system integration is performed using the mappings computed for the applications. This design scheme results in a fixed (static) mapping of applications onto the platform resources once the offline design process is finished [SSK+13]. Since they conduct all design decisions offline, static mapping approaches require global knowledge about all applications in the system and can exploit this holistic system view to find an optimal mapping for each application.

The offline design scheme of static mapping approaches enables the use of (i) computationally intensive mapping optimization procedures, see, e.g., [BBL+10; HM05; MBS+05; LWH+05], to find high-quality mappings as well as (ii) complex non-functional verification techniques to evaluate the quality of each mapping w.r.t. the non-functional properties of interest. In an embedded system, such a verification process is often crucial for the quality assessment of mappings, e.g., timing verification for hard real-time applications. Due to the inflexibility of their design solutions, however, static mapping approaches perform well only in the case of systems with a relatively static workload. Therefore, they lose practicality in the case of dynamic systems or for systems where the whole set of applications is not known statically.

Dynamic Application Mapping

Dynamic application mapping approaches alleviate the inflexibility of static mapping approaches by conducting mapping decisions at run time when an application must be launched. To that end, a so-called *Run-time Platform Manager (RPM)* is present in the system (e.g., as part of the operating system) which supervises the system by (i) launching each application on-demand according to the availability of resources, (ii) removing a running application upon a termination request, and (iii) adapting the mapping of running applications (if necessary), see, e.g., [KBL+11]. A dynamic mapping scheme reduces the complexity of application mapping by eliminating the need for a prior consideration of all possible system use-case and resource-availability scenarios in the mapping procedure since the system state is known when mapping decisions take place. By taking into account the current system state in their mapping decisions, dynamic mapping approaches enable an *adaptive* mapping scheme for the design of dynamic many-core systems [SSK+13].

This advantage, however, comes at the expense of increased time pressure for the application mapping process since the overhead of this process directly affects the system's performance. Due to the limited time and computational capacity at run time, dynamic mapping methodologies cannot afford compute-intensive mapping optimization procedures. Instead, they must resort to *lightweight heuristics* to compute mappings in an iterative and/or incremental (task-by-task) manner which typically results in sub-optimal mappings, see, e.g., [COM08]. Furthermore, they also cannot afford compute-heavy procedures for the non-functional verification of their obtained mappings and, therefore, lose applicability for applications that depend on such verification processes, e.g., hard real-time applications.

2.4.4 Hybrid Application Mapping (HAM)

In recent years, a new class of application mapping methodologies has emerged which combines static and dynamic mapping methodologies in a hybrid (offline/online) mapping scheme to exploit the advantages of each, while disposing of their shortcomings, see [SSK+13; WWG+18b; PGH+20] for an overview. This so-called *Hybrid Application Mapping (HAM)* scheme employs (i) *DSE* at design time to find a set of (Pareto-)optimal mappings for each application, followed by (ii) *system management* at run time where a suitable precomputed mapping is selected dynamically when an application must be launched. The design flow of HAM is illustrated in Figure 2.10 and elucidated in the following.

Design-Time Design Space Exploration (DSE)

In HAM, mapping optimization is performed for each application separately at design time using DSE, see Figure 2.10 (top). For each application, the DSE takes as input the

specification (consisting of the application graph, the architecture graph, and the mapping edges[3] as detailed in Section 2.3. The DSE embodies a multi-objective mapping optimization process in which mappings are (i) *generated* using the application mapping procedure detailed in Section 2.4.1 (by conducting binding, routing, allocation, and scheduling decisions). The quality of each generated mapping is then (ii) *evaluated* w.r.t. multiple design objectives such as timing (see, e.g., [WGW+14; WWG+18a]), reliability (see, e.g., [HWZ+15; WWG+18b]), security (see, e.g., [WWG+16]), energy consumption, and resource usage. Non-functional *requirements* of each application, e.g., real-time constraints, are expressed as constrained objectives whereby mappings with unsatisfactory behavior w.r.t. the application requirements are marked as infeasible and dismissed during the optimization process. Subject to the choice of non-functional qualities of interest, the evaluation can be realized using measurement-based, simulation-based, analytical, or statistical techniques.

During the optimization process, the DSE collects a set of mappings with Pareto-optimal qualities w.r.t. the design objectives. These mappings will be provided to the RPM to be used at run time, e.g., to launch applications on-demand. An efficient DSE of mappings can be achieved using the so-called *constraint graphs* introduced in [WGW+14]. A constraint graph represents a class of mappings abstractly by specifying (i) a partitioning of the tasks into a set of task clusters, (ii) an assignment of each task cluster to a core type, (iii) the maximum distance (NoC hops) allowed between each pair of tasks clusters containing data-dependent tasks, and (iv) the required NoC budget between data-dependent task clusters. Each constraint graph may have several concrete mapping realizations on a given many-core architecture. In [SWG+17], based on constraint graphs, a reformulation of the mapping problem into a problem of task clustering and resource-type mapping is introduced which enables an efficient DSE of applications on many-core architectures by eliminating redundant, symmetric mappings from the search space.

Run-Time System Management

In HAM, the set of Pareto-optimal mappings computed for each application during the offline DSE are provided to a Run-time Platform Manager (RPM) which launches and terminates applications dynamically on-demand and coordinates the deployment of applications in the system, see Figure 2.10 (bottom). To that end, the RPM employs a (i) *management policy* that steers its decisions based on the current state of the systems, e.g., the availability of resources, and non-functional requirements of running and incoming applications, e.g., real-time constraints. Whenever an application shall be launched, the RPM considers the precomputed mappings of that application to select (or, optionally, rank and filter) mapping candidates w.r.t. the criteria above and

[3]For clarity of visualization, in Figure 2.10, all mapping edges of one task to any core of the architecture are summarized as an abstract mapping edge marked with the types (colors) of target cores. For instance, task t_1 can be mapped onto green or blue cores.

in line with its management policy. The admissibility of each mapping candidate in view of the current system state and application requirements is examined using a set of (ii) *admission checks* in terms of, e.g., the availability of required resources for each mapping, its energy/power consumption, timeliness, etc. Note that in the case of a constraint-graph-based representation of mapping classes, the embedding feasibility of each constraint graph can be checked using exact constraint-solving approaches, e.g., SAT [SWG+17], or by means of heuristics, e.g., backtracking algorithms [WGW+14; WWG+18a]. Once a suitable mapping is identified which passes all admission checks, the RPM uses that mapping to launch the application. It may also change the mapping of a running application during its execution in a similar procedure, e.g., to evacuate an overheated core. Once the execution of an application is finished, the RPM removes it from the platform and releases its resources to be used later by other running or incoming applications.

HAM for Hard Real-Time Applications

The shortcomings of static and dynamic application mapping methodologies render these approaches unfit for the design of dynamic embedded systems with hard real-time constraints targeting (heterogeneous) many-core platforms: Static mapping approaches fail in offering the necessary flexibility and mapping adaptability for systems with dynamic workload profiles, and dynamic mapping approaches fail in providing the necessary timing guarantees for hard real-time applications.

The hybrid design flow of HAM methodologies combines the advantages of both static and dynamic methodologies while disposing of their major shortcomings: Similarly to static mapping approaches, the offline DSE in HAM can benefit from computationally intensive optimization techniques and non-functional verification procedures. While the former enables finding high-quality mappings, the latter is crucial for applications that rely on complex verification processes to obtain non-functional guarantees, e.g., hard real-time guarantees. On the other hand, similarly to dynamic mapping approaches, HAM approaches determine (select) the mapping to be used for each application at run time which empowers them to incorporate the available information about the current system state, e.g., resource availability, into their mapping selection procedure. This enables an adaptive application mapping and resource management scheme which is crucial for systems with dynamic workload profiles.

The features and characteristics of HAM render it a particularly promising scheme for the design of emerging embedded systems with hard real-time applications and dynamic workload profiles targeting heterogeneous many-core platforms. On this account, the methodologies, techniques, and analyses proposed in this work are developed based on this hybrid mapping scheme. The contributions of this work enable a timing-predictable, resource-efficient, thermally safe and composable, and adaptable execution of hard real-time applications with strict timing guarantees in heterogeneous many-core systems.

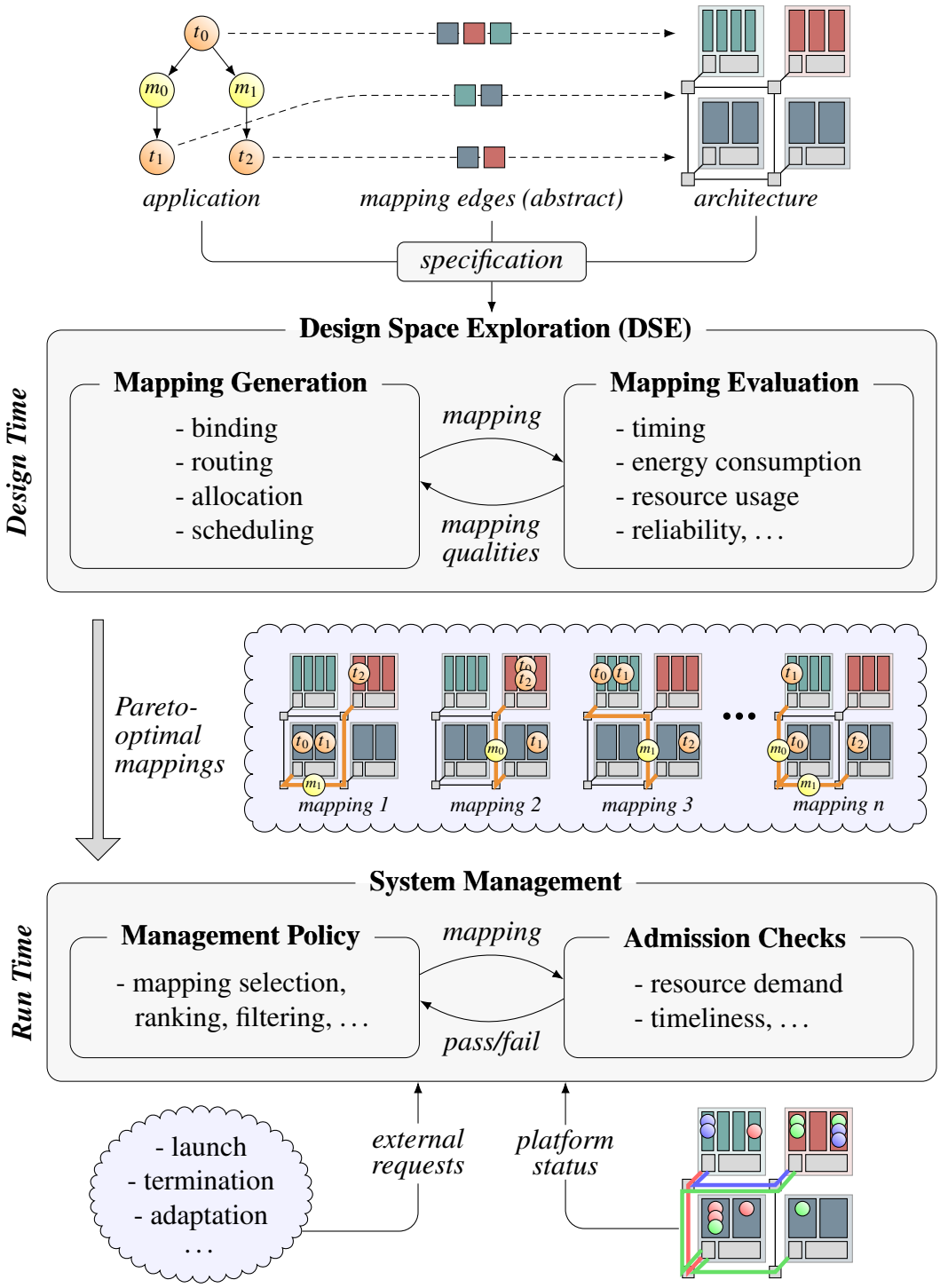


Figure 2.10: Flow of Hybrid Application Mapping (HAM). At design time, a Design Space Exploration (DSE) is performed per application to compute a set of Pareto-optimal mappings. At run time, applications are launched on-demand using their precomputed mappings.

3
Isolation-Aware Mapping Optimization and Timing Analysis

Timing composability and timing predictability are crucial system properties necessary for the adoption of many-core platforms in embedded systems with real-time constraints. As detailed in Chapter 2, in a *timing-composable* system, concurrent applications are isolated from each other by means of exclusive resource (or resource-budget) reservation per application. In a *timing-predictable* system, every resource has a predictable behavior which enables the use of formal analyses for the timing verification of real-time applications. Providing timing guarantees for hard real-time applications in such systems necessitates a proper choice of an (i) *inter-application isolation scheme* which specifies rules as to how resources must be reserved per application so that concurrent applications are temporally and/or spatially isolated from each other. In light of that, a (ii) *worst-case timing analysis* is also required to formally derive timing guarantees for each application based on its reserved resources (or resource budgets).

In the context of application mapping on many-core platforms, state-of-the-art design practices address the two aspects of inter-application isolation and worst-case timing analysis only to a limited extent: To provide worst-case timing guarantees for an application, a timing analysis technique must be able to safely bound the delay components and timing interferences that contribute to the worst-case timing properties of that application. The timing analyses used in the scope of many-core systems, however, often cannot analyze all possible timing interferences that can arise in a many-core system in the general case. Thus, for providing safe worst-case timing guarantees, these analyses necessarily rely on some *restrictive assumptions*, e.g., about the platform architecture and/or the deployment of applications on it, to ensure that those timing interferences that they cannot analyze also never occur[1]. Such assumptions result in a *fixed* inter-application isolation scheme (i.e., a fixed choice of temporal/spatial isolation scheme for each resource) that is applied throughout the

[1]For instance, some analyses rely on the assumption that the architecture consists of *single-core tiles* only (due to their lack of bus and NA interference analysis) or the assumption of *single-tile mappings* only (due to their lack of NoC latency analysis). See Section 3.5 for a detailed discussion.

system and, thus, imposes a fixed resource-reservation policy on each mapping of each application. Such a fixed policy often results in a mismatch between the amount of resources reserved per application and its actual performance and timing requirements. In turn, this mismatch manifests itself either in unsatisfactory timing guarantees (in case reserved resources are insufficient) or in an under-utilization of resources (in case reserved resources surpass the application's demand).

As a remedy, this chapter presents two contributions. Foremost, a (i) *worst-case timing analysis* is proposed for (heterogeneous) many-core systems. It can formally bound the worst-case timing properties of real-time applications in many-core systems *without* being subject to the restrictive assumptions of the existing analyses. In light of this timing analysis, the system is no longer restricted to a fixed inter-application isolation scheme and resource-reservation policy. Instead, a *flexible* resource-reservation policy can be employed in which the choice of temporal/spatial isolation scheme applied to each resource in each mapping of each application can be adjusted flexibly. This enables adjusting the amount of resources reserved per application according to its performance and timing requirements. To that end, an (ii) *isolation-aware mapping optimization* approach is presented which enables the mapping optimizer to explore the choices of temporal/spatial isolation scheme for each resource used in each mapping of each application. For mappings which employ *a combination of* different isolation schemes on their allocated resources, the proposed timing analysis is used to provide worst-case timing guarantees. By incorporating (i) and (ii) into the Design Space Exploration (DSE) in Hybrid Application Mapping (HAM), the mapping optimizer will automatically find mappings in which the amount of reserved resources is optimized according to the performance and timing requirements of the application so that its real-time constraints are satisfied in a resource-efficient manner.

The remainder of this chapter is organized as follows[2]. In Section 3.1, an overview of the current practice of many-core application mapping w.r.t. inter-application isolation schemes is provided, and the advantages of an isolation-aware application mapping approach are outlined using an illustrative example. Then, in Section 3.2, the proposed isolation-aware mapping optimization approach is presented[2], followed by the proposed worst-case timing analysis presented in Section 3.3. The effectiveness of mapping optimization based on the proposed approaches is demonstrated through a series of experimental results presented[2] in Section 3.4. Finally, an overview of related work is provided in Section 3.5, followed by a summary of the chapter in Section 3.6.

[2]The contents of this chapter are based on and partly published in [PSW+19] which has appeared in the Proceedings of the 31st Euromicro Conference on Real-Time Systems (ECRTS). In particular, major parts of Sections 3.3 and 3.4 are published in [PSW+19].

3.1 Inter-Application Isolation

Isolation between concurrent applications is a key principle in the design of real-time systems as it enables establishing system composability, see Section 2.2.4. A separation between concurrent applications w.r.t. the resources they use can be realized by means of (i) *spatial isolation* where each resource is exclusively reserved for one application. Alternatively, (ii) *temporal isolation* can be exerted which allows resources to be shared among applications. The latter, however, necessitates a *composable arbitration policy*, e.g., Time-Division Multiple Access (TDMA) or Weighted Round-Robin (WRR), to regulate the maximum amount of temporal interference that can be experienced by each requestor of a shared resource due to the presence of other requestors. Through this regulation, a composable arbitration policy renders the worst-case timing behavior of each requestor of a shared resource independent from the presence and the behavior of other requestors, see Section 2.2.4.

The choices of temporal and/or spatial isolation exerted between applications on different resources within a system manifest the so-called *(inter-application) isolation scheme* of that system. The isolation scheme of a system specifies a resource-reservation policy which must be followed by each application in that system. It regulates the amount of resources that must be reserved per running application and, therefore, has a strong impact on the worst-case timing behavior of each application, the resource utilization of the system, and other qualities that depend on the timing and/or resource usage of each application, e.g., energy consumption. This impact renders the choice of the inter-application isolation scheme a crucial design decision.

3.1.1 Inter-Application Isolation Schemes

The inter-application isolation schemes typically employed in many-core systems can be categorized into three classes: *Core-Sharing (CS)*, *Core-Reservation (CR)*, and *Tile-Reservation (TR)* isolation schemes. These schemes differ from each other w.r.t. the choice of temporal/spatial isolation exerted on different resources located on each tile of the underlying many-core platform. Noteworthy, sharing the Network-on-Chip (NoC) between concurrent applications can hardly be avoided as it establishes the necessary connectivity between different tiles of the platform [MTT18]. Therefore, concurrent applications are typically *temporally* isolated from each other on the NoC.

Figure 3.1 illustrates the three classes of many-core isolation schemes for an exemplary application with three tasks (t_0, t_1, and t_2) mapped to two cores (c_0 and c_1) of a three-core tile on a many-core platform. For all cores $c \in C$, the arbitration capacity as defined in Section 2.2.4 is assumed to be $K_c = 5$ which is visualized by the five periodic scheduling time slots depicted per core. For a timely execution, t_0 requires a minimum of three slots on c_0 while t_1 and t_2 each require a minimum of two slots on c_1. This resource demand can be derived by means of timing analysis. The reservation state of each scheduling slot of each core is highlighted by color according to the color

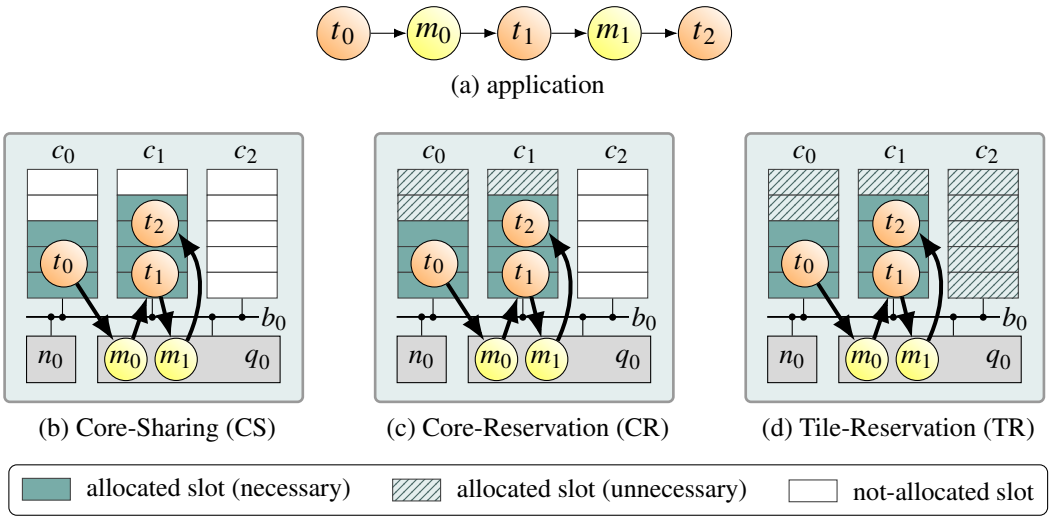


Figure 3.1: Inter-application isolation schemes in many-core systems, depicted for an exemplary application with tasks t_0, t_1, and t_2 which are mapped to cores c_0 and c_1 of a three-core tile in a many-core platform. The allocation state of each time slot of each core is denoted by the given color code.

code in Figure 3.1 (bottom). In the following, the three classes of isolation schemes are explained using this example.

Core-Sharing (CS)

In systems designed based on the so-called *Core-Sharing (CS)* isolation scheme, each core may be shared among multiple applications, see Figure 3.1b. Here, the core scheduling slots required for each task are reserved for the application at its launch time while the remaining slots of each core can be used by other applications. In Figure 3.1b, the application in question reserves three slots on core c_0 for task t_0 and four slots on core c_1 for tasks t_1 and t_2. Hence, the remaining two slots on c_0, one slot on c_1, and all five slots on c_2 can be used by other applications.

Note that the buses located on each tile enable memory access and, therefore, are shared between the cores and the Network Adapter (NA) located on that tile. Moreover, the NA on each tile is shared between all incoming and outgoing messages of that tile. Hence, under the CS isolation scheme where multiple applications can use the cores on a tile, they are automatically entitled to use the buses and the NA of that tile in a shared manner. The CS isolation scheme, therefore, establishes *temporal isolation* between concurrent applications on all resources. This renders CS the least restrictive isolation scheme in many-core systems which results in a high degree of inter-application timing interference, but also enables a high utilization of resources.

Core-Reservation (CR)

In systems designed based on the so-called *Core-Reservation (CR)* isolation scheme, each core is exclusively reserved for one application, see Figure 3.1c. Compared to CS, this reservation scheme eliminates inter-application timing interferences which arise due to execution preemption on cores. Hence, it leads to tighter Worst-Case Response Time (WCRT) bounds for tasks and, thereby, enhanced timing properties for applications. Nonetheless, as it disregards the actual compute-power demand of tasks, the CR isolation scheme often leads to an over-reservation and, thereby, under-utilization of cores. In Figure 3.1c, cores c_0 and c_1 will be exclusively reserved for the application in question at its launch time while core c_2 can be used by other applications. Here, c_0 and c_1 are utilized up to 60% and 80%, respectively.

The CR isolation scheme still allows different cores of a tile to be allocated by different applications. Hence, while each core is exclusively assigned to one application, the buses and the NA located on each tile are shared among the applications which use the cores located on that tile. Therefore, CR establishes inter-application *spatial isolation* on cores and *temporal isolation* on other resources.

Tile-Reservation (TR)

In systems designed based on the so-called *Tile-Reservation (TR)* isolation scheme, each tile of the many-core platform is reserved for one running application exclusively, see Figure 3.1d. TR is, therefore, the most restrictive isolation scheme in practice which eliminates all inter-application timing interferences on intra-tile resources altogether. This advantage, however, comes at the expense of a typically significant over-reservation of resources. The TR isolation scheme establishes *spatial isolation* between concurrent applications at the tile level.

Inter-Application Isolation in Practice

In the state-of-the-art practices of application mapping in composable many-core systems, e.g., [SKS13; WGW+14; WWG+18a], a *fixed* choice of inter-application isolation scheme (CS, CR, or TR) is exerted onto every mapping of every application in the system[3]. The inter-application isolation scheme of a system specifies the resource-reservation policy followed by applications in that system and, therefore, has a strong impact on its utilization of resources and the (worst-case) timing properties of applications. This impact is demonstrated in Figure 3.2 for a mapping of an application deployed on two adjacent tiles of an exemplary many-core platform that uses WRR for the arbitration of shared resources. The mapping is visualized under the three isolation schemes CS, CR, and TR in Figures 3.2b to 3.2d, respectively. For each isolation scheme, the resource cost and the worst-case end-to-end latency (makespan) of the mapping are reported below the respective sub-figure. Resource cost is calculated as

[3] See Section 3.5 for an overview of the state-of-the-art practices w.r.t. inter-application isolation.

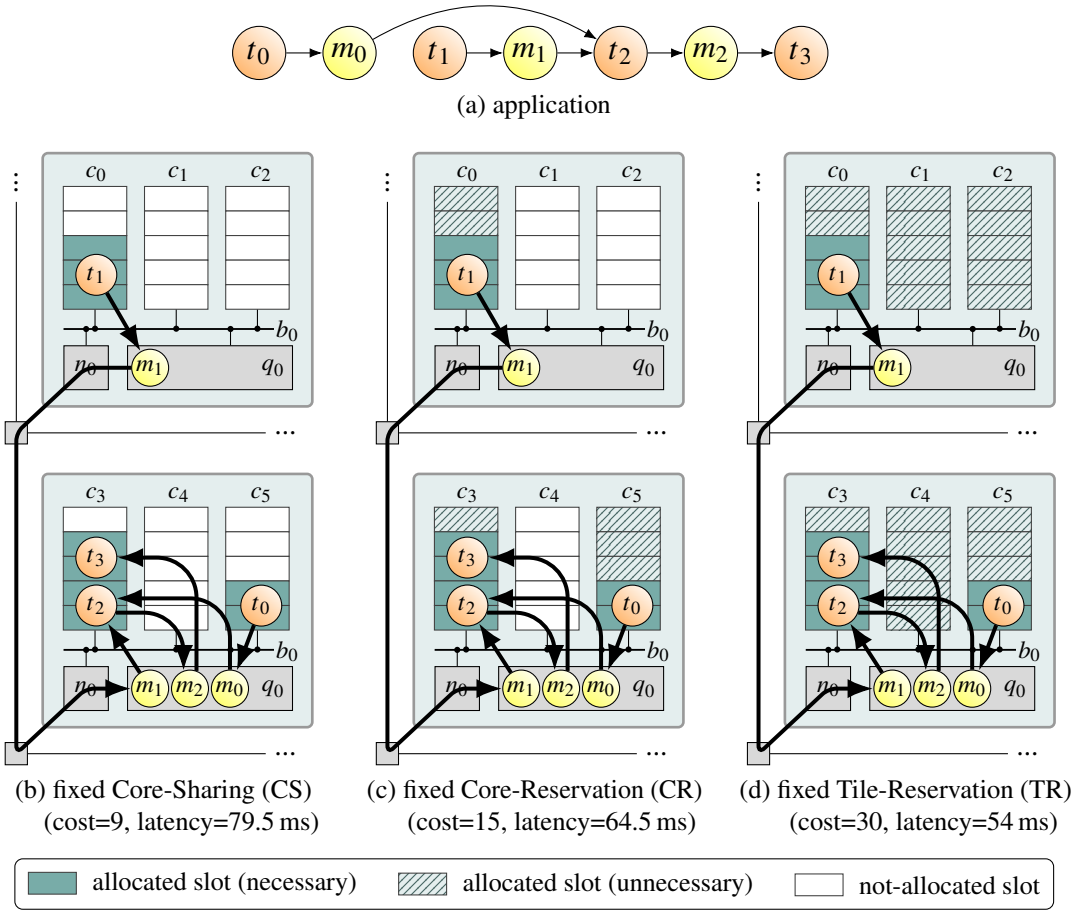


Figure 3.2: Example of fixed inter-application isolation schemes, i.e., CS (b), CR (c), and TR (d), for an exemplary mapping of an application. For each scheme, the resource cost and worst-case latency of the mapping are given as well.

the total number of core scheduling slots allocated by the mapping. End-to-end latency (makespan) denotes the latency of the longest path in the application graph which is calculated using Eq. (3.1) for this example. The Worst-Case Response Time (WCRT) of tasks t_0–t_3 and the Worst-Case Traversal Time (WCTT) of the inter-tile message m_1 are provided in Table 3.1 for different choices of isolation scheme. Note that intra-tile messages m_0 and m_2 are implicitly communicated via the shared memories of the lower tile, and their write/read latency is already included in the WCRT of their respective producing/consuming tasks.

$$\text{latency} = \max\Big\{WCRT(t_0), \big(WCRT(t_1) + WCTT(m_1)\big)\Big\} + WCRT(t_2) + WCRT(t_3) \quad (3.1)$$

Due to its exclusively temporal isolation between concurrent applications, the CS isolation scheme in Figure 3.2b results in the lowest resource cost of 9 scheduling slots

which, however, comes at the expense of a notably high end-to-end latency of 79.5 ms. The TR isolation scheme in Figure 3.2d, on the other hand, exclusively employs spatial isolation between concurrent applications, resulting in the lowest end-to-end latency (54 ms) at the expense of the highest resource cost (30 slots). Finally, CR offers a moderate trade-off between worst-case latency (64.5 ms) and resource cost (15 slots) by restricting the exclusive reservation of resources to cores only, see Figure 3.2c.

3.1.2 Isolation-Aware Application Mapping: Motivation

A single and fixed isolation scheme strongly restricts the design space of possible mappings of an application on a given platform as it imposes a constant resource-reservation policy on each and every mapping of each application in the system. This practice often leads to mappings where the amount of reserved resources is not properly adjusted to the actual performance and timing requirements of the application. The resulting mismatch between the reserved and required computational power, in turn, harms resource utilization if the reserved resources for an application exceed its demand (over-reservation) which is often observed under the TR isolation scheme. Less restrictive isolation schemes, namely, CS and CR, mitigate this utilization penalty by allowing all or a subset of resources to be shared among multiple applications. Nonetheless, this often results in a high degree of inter-application timing interference which could lead to mappings with degraded or even unacceptable timing guarantees.

These shortcomings can be addressed by applying *a combination of* different inter-application isolation schemes on different resources used by a mapping. Figure 3.3 illustrates the advantage of applying isolation schemes in combination for the exemplary mapping previously studied under fixed isolation schemes in Figure 3.2. Applying isolation schemes in combination enables 12 additional isolation scenarios for this mapping—in addition to the fixed scenarios depicted in Figure 3.2—three of which are shown in Figure 3.3. In the first combined scheme shown in Figure 3.3a, CS

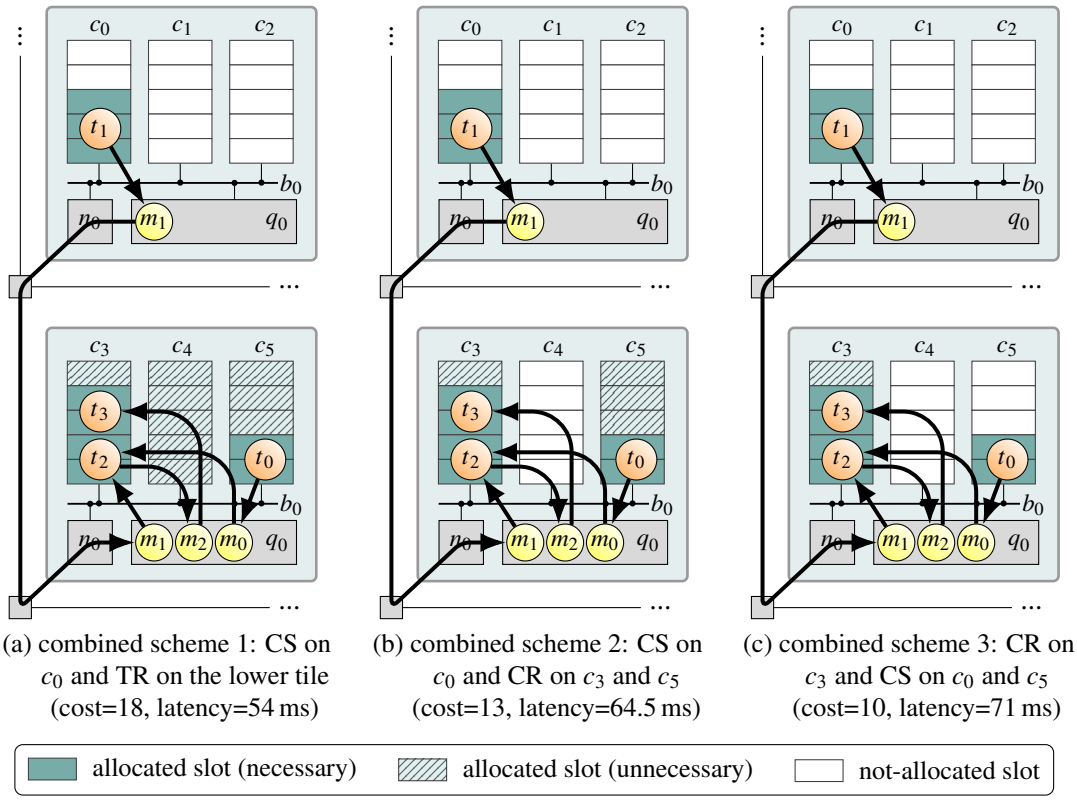
Table 3.1: WCRT of tasks t_0–t_3 and WCTT of inter-tile message m_1 in the mapping shown in Figures 3.2 and 3.3, provided for different isolation schemes.

(a) WCRT of tasks t_0–t_3

isolation scheme	WCRT [ms]			
	t_0	t_1	t_2	t_3
TR	29.0	7.5	10.5	16.5
CR	31.5	11.5	12.0	21.0
CS	38.0	16.5	14.5	27.0

(b) WCTT of message m_1

isolation scheme		WCTT [ms]
source	destination	
TR	TR	1.7
TR	CR/CS	2.1
CR/CS	TR	2.6
CR/CS	CR/CS	3.2



(a) combined scheme 1: CS on c_0 and TR on the lower tile (cost=18, latency=54 ms)

(b) combined scheme 2: CS on c_0 and CR on c_3 and c_5 (cost=13, latency=64.5 ms)

(c) combined scheme 3: CR on c_3 and CS on c_0 and c_5 (cost=10, latency=71 ms)

Figure 3.3: Examples of applying inter-application isolation schemes in combination for the mapping from Figure 3.2. Here, three combinations of isolation schemes are depicted. For each combination, the resource cost and worst-case latency of the mapping are reported.

is employed on core c_0 while TR is used for the lower tile. This scheme outperforms the fixed TR scheme from Figure 3.2d by offering the same timing guarantee of 54 ms with a 40% lower resource cost. The second combined scheme in Figure 3.3b employs CS on core c_0 and CR on c_3 and c_5 and, thereby, outperforms the fixed CR scheme from Figure 3.2c by offering the same timing guarantee of 64.5 ms with 13% lower resource cost. Finally, the third combined scheme in Figure 3.3c employs CR on core c_3 and CS on c_0 and c_5 which enables a new Pareto-optimal latency-cost trade-off that was not achievable previously under a fixed choice of isolation scheme.

Figure 3.4 illustrates the distribution of all 15 possible isolation scenarios of the mapping in question in the space of worst-case latency and resource cost. The trade-offs achieved under the three fixed schemes from Figure 3.2 and under the three combined schemes from Figure 3.3 are labeled accordingly. The scenarios with Pareto-optimal trade-offs are marked with red squares which dominate the trade-offs located in the shaded region. As illustrated, nearly all Pareto-optimal trade-offs are obtained

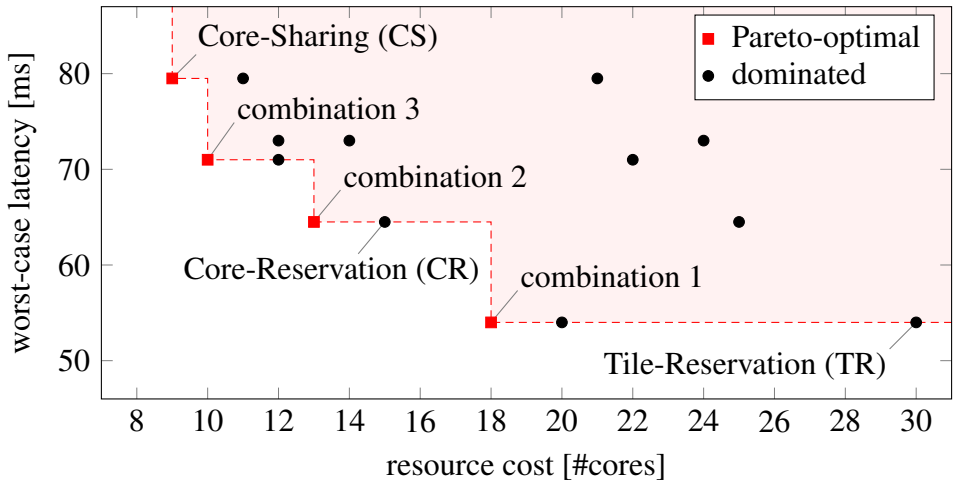


Figure 3.4: Latency-cost trade-offs achieved by the 15 possible inter-application isolation scenarios for the mapping from Figures 3.2 and 3.3 (reference of the labels). The highlighted region is dominated by Pareto-optimal scenarios.

only when different isolation schemes are applied in combination within one mapping.

Contributions Overview

The practice of fixed inter-application isolation schemes in real-time many-core systems mainly roots in the lack of timing analyses that can capture all possible timing components contributing to the execution latency of applications: Existing timing analyses employed in a many-core context often cannot capture all timing contributions that may arise during the execution of applications, in the general case. To be able to provide safe worst-case timing guarantees, these analyses necessarily rely on fixed inter-application isolation schemes to ensure that those timing contributions that they cannot analyze will also never occur[4]. As a remedy, this chapter presents two contributions which enable flexible combinations of the above introduced inter-application isolation schemes (CS, CR, and TR): (i) a *worst-case timing analysis* for (heterogeneous) many-core architectures which neither depends on a fixed choice of inter-application isolation scheme nor is subject to the restrictive assumptions[5] of existing timing analyses, (ii) an *isolation-aware mapping optimization* approach which enables exploring combinations of isolation schemes on the resources used by each mapping generated during the mapping optimization process.

Together, the proposed mapping optimization approach and worst-case timing

[4]Note that applying a combination of different timing analyses to cover all timing contributions is generally not practical due to the different (and possibly conflicting) assumptions of these analyses regarding the platform architecture and/or the arbitration scheme of resources.

[5]See Footnote 1 on Page 45 for some examples.

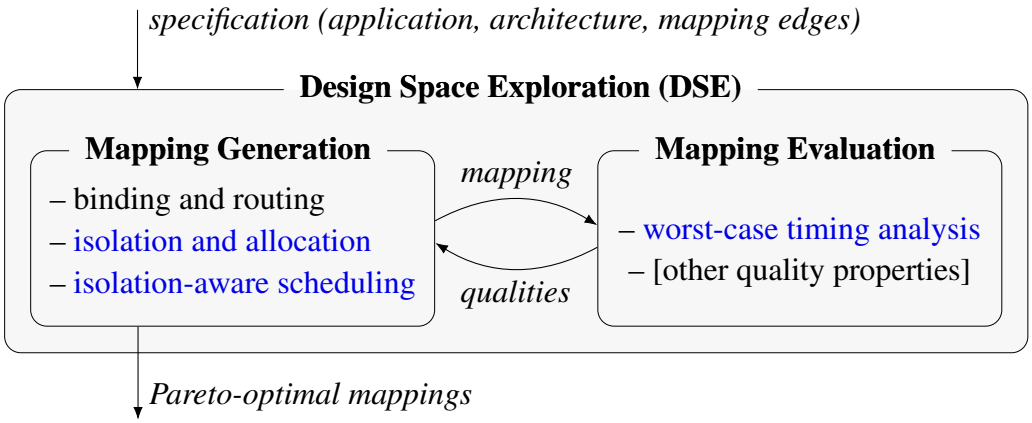


Figure 3.5: An overview of the proposed isolation-aware mapping optimization approach (left) and worst-case timing analysis (right) incorporated into the design-time DSE in HAM. The contributions are highlighted in blue.

analysis enable an *adaptive* resource-reservation policy in which the amount of resources reserved per application is adjusted to its actual performance and timing requirements to enhance the timing and resource utilization trade-off. In the resulting isolation-aware DSE scheme, the mapping optimizer automatically finds mappings with superior quality trade-offs (including verified worst-case timing guarantees) compared to mappings derived in a fixed-isolation-scheme setting. An overview of the proposed contributions incorporated into the DSE of HAM is illustrated in Figure 3.5. These contributions will be presented in detail in Sections 3.2 and 3.3. Note that any combination of CS, CR, and TR isolation schemes can be expressed in terms of the number of core scheduling slots to be reserved on a set of cores. For instance, TR corresponds to the reservation of all scheduling slots of all cores on the respective tile. Therefore, the imposition of isolation schemes in combination can be realized using the existing resource-reservation mechanisms in composable many-core systems.

3.2 Isolation-Aware Mapping Optimization

This section presents the proposed isolation-aware mapping optimization approach. In this approach, for each mapping generated during the DSE in HAM, the choices of isolation scheme for each allocated resource are explored. As detailed in Section 2.4.1, the generation of a mapping involves four sets of design decisions, namely, binding, routing, allocation, and scheduling. For the exploration of the design space of mappings, the mapping optimization problem is typically *encoded* using (i) a set of decision variables whose assignment is decided by the optimizer and (ii) a set of constraints formulated based on the decision variables to restrict the solution space of the optimizer to valid (feasible) mappings, see, e.g., [BTT98]. In the proposed mapping

optimization approach, a set of new design decisions are introduced and the encoding of (some) design decisions is adapted to enable the optimizer to explore the choices of isolation scheme for each resource allocated in each mapping, see Figure 3.5 (left).

The set of resources used by a mapping are determined by the binding and routing decisions. The former specify the assignment of tasks to cores and messages to memories, and the latter specify a NoC route for each inter-tile message, see Section 2.4.1. While the encoding of the binding and routing decisions remains agnostic to the exploration of isolation schemes, the allocation of resources and the scheduling of tasks and messages strongly depend on the choices of isolation scheme made by the optimizer and, therefore, must be adapted.

For the optimization of mappings, this work uses the SAT-Decoding [LGH+07] optimization approach which employs an evolutionary algorithm coupled with a Pseudo-Boolean (PB) solver for an efficient DSE of discrete constrained optimization problems. In SAT-Decoding, a set of PB constraints are used to specify the design space of valid solutions, see also [GTL+17]. For the mapping optimization problem at hand, the PB constraints from [LSF14] are used to explore the binding of tasks to cores. Without loss of generality, the encoding of routing decisions is realized based on the XY-routing algorithm [NM93]. To explore isolation schemes, we introduce additional decision variables and constraints. In the following, first, Section 3.2.1 presents the encoding of isolation schemes and the corresponding adaptation of the allocation. Thereafter, the isolation-aware scheduling of mappings is presented in Section 3.2.2.

3.2.1 Isolation-Scheme Encoding and Resource Allocation

To explore the space of possible combinations of isolation schemes (CS, CR, and TR) for the resources used by each mapping, we provide the mapping optimizer with two binary design variables for each core $c \in C$ with at least one task $t \in T$ mapped to it in the mapping under generation. By adjusting the assigned values of these binary variables, the optimizer can realize various combinations of isolation schemes within the mapping. The assigned values of these variables for each core denote the isolation-scheme preferences of that core in the mapping in question which we refer to as the *preferred* isolation scheme of that core. Once the preferred isolation scheme of all cores has been determined, the so-called *activated* isolation scheme of each core is derived by resolving the possible conflicts between the preferred isolation schemes of the cores located on the same tile. This procedure is detailed in the following.

Each core $c \in C$ with at least one task mapped to it is associated with two binary design variables v_c^{TR} and v_c^{CR}. The first variable $v_c^{\mathrm{TR}} \in \{0,1\}$ determines whether the tile on which core c is located is to be allocated exclusively. Hence, $v_c^{\mathrm{TR}} = 1$ denotes the preference of the TR isolation scheme for core c. The second variable $v_c^{\mathrm{CR}} \in \{0,1\}$ determines whether c is to be allocated exclusively and comes into effect if TR is not preferred for c, i.e., $v_c^{\mathrm{TR}} = 0$. Thus, CR is preferred for core c if condition $\neg v_c^{\mathrm{TR}} \wedge v_c^{\mathrm{CR}} = 1$ holds. Finally, if neither TR nor CR is preferred for core c, i.e.,

$\neg v_c^{\mathrm{TR}} \wedge \neg v_c^{\mathrm{CR}} = 1$, the preferred isolation scheme of c is taken to be CS.

After the preferred isolation scheme for each core is decided, the *activated* isolation schemes in the mapping are decoded as follows: TR is activated on a tile $u \in U$ iff (i) u has at least one core $c \in u$ with a TR preferred isolation scheme or (ii) all cores on u have a CR preferred scheme. Once TR is activated on a tile u, it is also activated on each and every core of that tile, regardless of their preferred isolation scheme. CR is activated on a core c iff (i) CR is preferred for c, and (ii) c is not located on a tile with an activated TR scheme. Finally, CS is activated on a core c iff (i) CS is preferred for c and (ii) c is not located on a tile with an activated TR scheme.

Once the activated isolation schemes have been derived, the resources to be allocated are specified. To that end, all resources located on tiles with an activated TR isolation scheme are allocated in addition to the other resources used by the mapping which are outlined in the description of the allocation decisions in Section 2.4.1.

3.2.2 Isolation-Aware Scheduling

In the scheduling step, the so-called *schedule* of tasks and messages on the resources used by them is specified. That is, for each task/message $x \in T \cup M$ that uses resource i, an arbitration weight $W_i^x \in [1, K_i]$ is determined, denoting the number of periodic time slots to be reserved for x on resource i. Here, K_i represents the arbitration capacity (total number of periodic time slots) of i, see the definition of schedule in Section 2.4.1.

In the proposed isolation-aware mapping optimization approach, the schedule of tasks and messages on their respective resources is determined based on (i) the activated isolation scheme of the resources in the mapping (decided by the optimizer as detailed in Section 3.2.1) and (ii) the arbitration policy of each resource (provided in the architecture graph). To that end, first, the arbitration weight W_i^x of each task/message $x \in T \cup M$ on each respective resource i used by x is calculated. Subsequently, a set of so-called *arbitration tuples* are constructed based on them which summarize all necessary scheduling information for the worst-case timing analysis of the mapping. In the following, first, the scheduling procedure for tasks and messages, i.e., the calculation of their arbitration weights, is presented. Following that, the construction of arbitration tuples based on calculated arbitration weights is detailed.

Scheduling of Tasks and Messages

The schedule of a task/message $x \in T \cup M$ on each resource i used by x is derived by determining the arbitration weight W_i^x of x on i. To that end, we present an iterative scheduling approach based on the proposed timing analysis. Details of the timing analysis will be presented in Section 3.3. At this point, suppose that the timing analysis can be used to safely bound the WCRT of each task and the WCTT of each inter-tile message based on its arbitration tuples which summarize all necessary inputs for the timing analysis. In general, the arbitration tuple of each requestor x of resource i

is constructed as (S_i, W_i^x, P_i^x) where $S_i \in \mathbb{R}^+$ denotes the length of each arbitration time slot of i, $W_i^x \in \mathbb{N}^+_{\leq K_i}$ the arbitration weight of x on i, and $P_i^x \in \mathbb{R}^+$ the worst-case arbitration period perceived by x on i. Here, $K_i \in \mathbb{N}^+$ denotes the arbitration capacity of resource i. The scheduling of tasks and messages is performed as follows.

Task Scheduling For each task $t \in T$ with execution period (implicit deadline) PRD_t (specified by the application) mapped to core $c \in C$, the arbitration weight W_c^t of t on c is calculated using the following iterative task scheduling procedure: Starting with the lowest arbitration weight $W_c^t = 1$:

1. Arbitration tuple (S_c, W_c^t, P_c) is constructed and used as input to the timing analysis to calculate the WCRT of task t on core c. The WCRT analysis is performed using Eqs. (3.2) to (3.5) which will be presented later in Section 3.3.

2. If the WCRT of t does not exceed its given deadline PRD_t, the arbitration weight W_c^t will be used for t on c as the minimum arbitration weight which satisfies the timing constraint of t. Otherwise, W_c^t is incremented by one, and the procedure returns to step 1.

Message Scheduling For each inter-tile message $m \in M$ with production period (implicit deadline) PRD_m (specified by the application) routed over NoC route $(\hat{r}) \subseteq R \cup N$, the arbitration weight $W^m_{(\hat{r})}$ of m on resources $\hat{r} \in (\hat{r})$ is calculated using the following iterative message scheduling procedure: Starting with the lowest arbitration weight $W^m_{(\hat{r})} = 1$ on each resource $\hat{r} \in (\hat{r})$:

1. Arbitration tuples $(S_{\hat{r}}, W^m_{(\hat{r})}, P_{\hat{r}})$ for m on each resource $\hat{r} \in (\hat{r})$ are constructed and used as input to the timing analysis to calculate the WCTT of message m on its NoC route $(\hat{r})$. The WCTT analysis is performed using Eqs. (3.6) to (3.10) which will be presented later in Section 3.3.

2. If the WCTT of m does not exceed its deadline PRD_m, the arbitration weight $W^m_{(\hat{r})}$ will be used for m on each resource $\hat{r} \in (\hat{r})$ as the minimum arbitration weight which satisfies m's timing constraint. Otherwise, $W^m_{(\hat{r})}$ is incremented by one on each resource $\hat{r} \in (\hat{r})$, and the procedure returns to step 1.

Schedulability Check After the arbitration weights have been derived for all tasks and messages on each resource they use, the schedulability of the mapping is evaluated as follows. If the overall arbitration weight demanded by all tasks $\hat{T} \subseteq T$ mapped to a core $c \in C$ exceeds the arbitration capacity of c, i.e., $\sum_{t \in \hat{T}} W_c^t > K_c$, the mapping is considered unschedulable and is discarded. Likewise, if the overall arbitration weight demanded on a NoC router or the Transmitter (TX) or Receiver (RX) unit of a NA exceeds its arbitration capacity, the mapping is considered unschedulable

and is discarded. If no capacity violations are observed, the mapping is considered schedulable, and the derived arbitration weights will be used to construct the arbitration tuples as presented in the following.

Calculation of Arbitration Tuples

As mentioned before, for each requestor (user) x of each resource i, the arbitration tuple of x on i is constructed as (S_i, W_i^x, P_i^x) where S_i denotes the length of each arbitration time slot of i, W_i^x the arbitration weight for x on i, and P_i^x the worst-case arbitration period perceived by x on i. The arbitration tuples of a mapping summarize all the necessary scheduling information for the worst-case timing analysis of the mapping and serve as input for the proposed timing analysis. While S_i is provided as an attribute of i in the architecture graph and W_i^x is calculated as detailed above regardless of the choices of isolation schemes, P_i^x is the key parameter that captures the impact of the choices of isolation schemes and provides isolation awareness to the proposed timing analysis: As detailed previously in Section 2.2.4, the worst-case arbitration period P_i of a resource i under a composable arbitration policy, e.g., TDMA or WRR, can be calculated as $P_i = K_i \cdot (S_i + D_i)$ where K_i denotes the arbitration capacity of i, S_i the arbitration slot length, and D_i the arbiter delay. Here, P_i provides an *upper bound* on the worst-case arbitration period P_i^x perceived by any requestor x of i. However, subject to the choice of the isolation scheme applied on i, a tighter but still safe upper bound on P_i^x can be derived as follows.

Under a *work-conserving* arbitration scheme, e.g., WRR, the arbitration period $P_i = K_i \cdot (S_i + D_i)$ is observed only if all K_i time slots of resource i are assigned to requestors and are used. An imposition of CR or TR isolation scheme on i can, however, result in scenarios in which some slots of i are reserved by the mapping under scheduling but not used by any of its tasks or messages. Such a scenario is illustrated in Figure 3.6 in which resource i with an arbitration capacity of $K_i = 5$ is used by requestors x and y with arbitration weights of $W_i^x = 2$ and $W_i^y = 1$ on i, respectively. If the choice of isolation scheme activated on i allows the remaining two time slots of i to be used by other applications, then, in the worst case, x and y can perceive an arbitration period of $P_i^y = P_i^x = P_i = 5 \cdot (S_i + D_i)$ when accessing i. However, if the applied isolation scheme results in an exclusive reservation of i for the mapping under scheduling, then it can be safely assumed that the two remaining time slots of i which are not used by x or y will not be used at all and, hence, are always skipped by the arbiter. Thus, the worst-case arbitration period perceived by x and y when accessing i can be safely tightened as $P_i^y = P_i^x = 3 \cdot (S_i + D_i) < P_i$.

The refinement of the worst-case arbitration period P_i^x perceived by requestor x on resource i enables eliminating a considerable degree of pessimism from the worst-case timing analysis of the mapping and, thus, deriving tighter but still safe worst-case timing guarantees. For each requestor x of an exclusively allocated resource i with a work-conserving arbitration policy, P_i^x can be calculated by disregarding the reserved

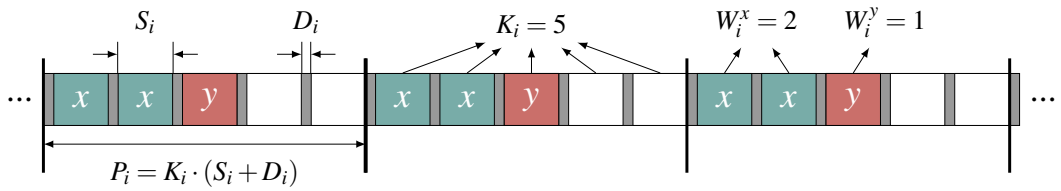


Figure 3.6: Arbitration of a resource i with arbitration capacity $K_i = 5$. Requestors x and y are assigned to i with arbitration weights of $W_i^x = 2$ and $W_i^y = 1$ per arbitration period P_i, respectively.

slots of i which are not assigned to any requestors in the mapping under scheduling. For instance, for an exclusively allocated core $c \in C$ with tasks $\hat{T} \subseteq T$ mapped to it, the worst-case arbitration period perceived by each task $t \in \hat{T}$ can be calculated as $P_c^t = \sum_{t \in \hat{T}} W_c^t \cdot (S_i + D_i)$. Similarly, for each bus $b \in B$ in an exclusively allocated tile, the idle bus masters will be excluded when calculating the worst-case bus arbitration period P_b^i perceived by each non-idle bus master i. Idle bus masters are idle cores (i.e., cores that do not host any tasks) and idle TX/RX unit of the NA (in case no messages are injected from/to the tile to/from the NoC).

Note that, for each resource i with the set X of requestors, it is guaranteed that $P_i^x \leq P_i$ for each requestor $x \in X$ since it has already been verified in the schedulability test that $\sum_{x \in X} W_i^x \leq K_i$. Also note that the refinement of the worst-case arbitration period as discussed above can be applied only in the case of work-conserving arbitration policies, e.g., WRR. Otherwise, no period reductions may take place. After the worst-case arbitration period perceived by each requestor of each resource in the mapping is calculated, the set of arbitration tuples are constructed which serve as input for the timing analysis of the mapping. To that end, the following arbitration tuples are constructed per mapping which are also exemplified for a mapping in Figure 3.7.

Task ▶ Core For each task $t \in T$ mapped to core $c \in C$, the arbitration tuple (S_c, W_c^t, P_c^t) of t on c is constructed. Here, S_c is provided by the architecture, W_c^t is calculated as part of t's schedule, and P_c^t is calculated as detailed above.

Message ▶ TX/RX For each inter-tile message $m \in M$ routed through TX unit tx on m's source tile and RX unit rx on m's destination tile, the arbitration tuples $(S_{tx}, W_{tx}^m, P_{tx}^m)$ and $(S_{rx}, W_{rx}^m, P_{rx}^m)$ are constructed. Here, S_{tx} and S_{rx} are provided by the architecture, W_{tx}^m and W_{rx}^m are calculated as part of m's schedule where $W_{tx}^m = W_{rx}^m = W_{(\hat{r})}^m$, and P_{tx}^m and P_{rx}^m are calculated as detailed above.

Message ▶ NoC Routers Let $\rho_m \subseteq R$ denote the sequence of routers on the NoC route of an inter-tile message m. Since NoC routers are generally not allocated exclusively, (i) the worst-case arbitration period P_r^m perceived by m on each router

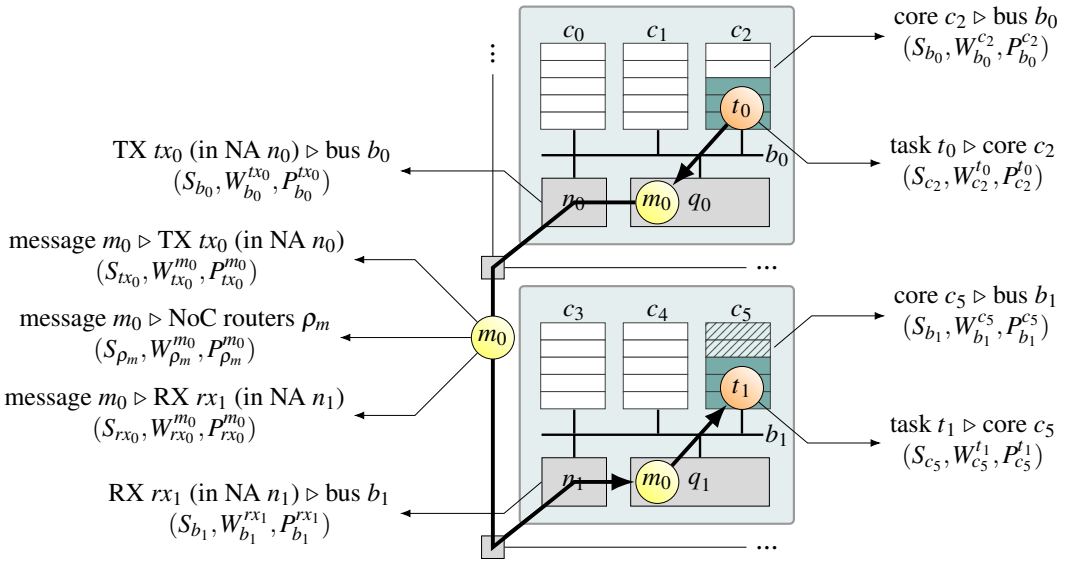


Figure 3.7: Example of a mapping and the arbitration tuples calculated for it which serve as input for its isolation-aware worst-case timing analysis.

$r \in \rho_m$ is equal to r's arbitration period. Moreover, (ii) m also has the same arbitration weight $W^m_{\rho_m}$ on each router $r \in \rho_m$, calculated as part of its schedule. Given (i) and (ii), and knowing that different routers of a NoC also have the same arbitration slot length[6], m will have the same arbitration tuple on each router $r \in \rho_m$ located on its NoC route which is denoted as $(S_{\rho_m}, W^m_{\rho_m}, P^m_{\rho_m})$. Here, $S_{\rho_m} = S_r$ and $P^m_{\rho_m} = P_r = K_r \cdot (S_r + D_r)$ for any router $r \in \rho_m$ where K_r, S_r, and D_r are provided by the architecture. $W^m_{\rho_m}$ is calculated as part of m's schedule.

Core ▶ Bus For each core $c \in C$ with at least one mapped task, the arbitration tuple of c on each bus $b \in B$ located on the same tile is constructed as (S_b, W^c_b, P^c_b). Here, S_b and W^c_b are provided by the architecture, and P^c_b is calculated as detailed above.

TX/RX ▶ Bus For the TX unit tx of each NA $n \in N$ that routes at least one outbound message out of its tile, the arbitration tuple on each on-tile bus $b \in B$ is calculated and denoted as $(S_b, W^{tx}_b, P^{tx}_b)$. Likewise, for the RX rx unit of each NA $n \in N$ that routes at least one incoming message into its tile, the arbitration tuple on each on-tile bus $b \in B$ is calculated and denoted as $(S_b, W^{rx}_b, P^{rx}_b)$. Here, W^{tx}_b, W^{rx}_b, and S_b are provided by the architecture, and P^{tx}_b and P^{rx}_b are calculated as detailed above.

[6] In NoCs with a wormhole switching strategy (as considered in this work), the arbitration slot length S_r of a router $r \in R$ is typically equal to the duration of one clock cycle of the NoC which suffices for the transfer of one flow-control digit (flit) of data over a link.

Computational Complexity

The isolation-aware scheduling approach presented above involves (i) the scheduling of tasks and messages, (ii) the schedulability check of the mapping, and (iii) the calculation of the arbitration tuples. In the following, the computational complexity of each step (i)–(iii) and the overall computational complexity of the isolation-aware scheduling approach is analyzed.

Scheduling of Tasks and Messages The scheduling of tasks and messages is performed based on the WCRT and WCTT analyses which will be presented in Section 3.3. As will be shown later, both analyses are constant-time non-iterative analyses each with a computational complexity of $O(1)$. For each task $t \in T$ mapped to core $c \in C$ with arbitration capacity K_c, the scheduling procedure (i.e., the calculation of the arbitration weight W_c^t of t on c) may involve a maximum of K_c iterations of WCRT analysis, thus, resulting in a computational complexity of $O(K_c)$. Therefore, the scheduling of *all* tasks introduces a computational complexity of $O(K_{\text{core}}^{\max}|T|)$ where $K_{\text{core}}^{\max} = \max_{c \in C} K_c$ denotes the maximum arbitration capacity among all cores $c \in C$ in the architecture, and $|T|$ denotes the total number of tasks in the application. Similarly, the scheduling procedure of each inter-tile message $m \in M$ may involve a maximum of $K_{\text{NoC}}^{\max}$ iterations of WCTT analysis where $K_{\text{NoC}}^{\max} = \max_{i \in N \cup R} K_i$ denotes the maximum arbitration capacity among (TX/RX units of) all NAs $n \in N$ and all routers $r \in R$ in the architecture. Thus, the scheduling of all inter-tile messages has a computational complexity of $O(K_{\text{NoC}}^{\max}|M|)$ where $|M|$ denotes the total number of messages in the application. Therefore, the overall computational complexity of the scheduling of tasks and messages will be $O(K_{\text{core}}^{\max}|T| + K_{\text{NoC}}^{\max}|M|)$.

Schedulability Check The schedulability check for each mapping (after the scheduling of tasks and messages) involves an enumeration of the tasks mapped to each allocated core $c \in C$ as well as an enumeration of the inter-tile messages routed over each allocated NA $n \in N$ and router $r \in R$. In the general case, a nested loop over the allocated cores $c \in C$ (outer loop) and the set of tasks $t \in T$ mapped to each core (inner loop) introduces a computational complexity of $O(|C||T|)$. However, since each task is necessarily mapped to exactly one core, the total number of iterations for the inner loop will never exceed $|T|$, which reduces the computational complexity of the nested loop to $O(|C| + |T|)$. The computational complexity of the enumeration of the inter-tile messages $m \in M$ routed over each allocated NA $n \in N$ and router $r \in R$ can be derived as $O(|N||M|)$ and $O(|R||M|)$, respectively. Therefore, the schedulability check introduces an overall computational complexity of $O((|C| + |T|) + (|N||M|) + (|R||M|))$ which can be simplified to $O(|C| + |T| + |R||M|)$ given that $|N| = |R|$.

Calculation of Arbitration Tuples The calculation of the arbitration tuples involves the three following enumerations: (i) an enumeration of the tasks mapped to each

allocated core $c \in C$ which has a computational complexity of $O(|C| + |T|)$ as shown above, (ii) an enumeration of the inter-tile messages routed over each allocated NA $n \in N$ and router $r \in R$ which has a computational complexity of $O(|R||M|)$ as shown above given that $|N| = |R|$, and (iii) an enumeration of the bus masters (i.e., the on-tile cores and the single NA) connected to each allocated bus $b \in B$ which has a computational complexity of $O(|B||C|)$. Thus, the calculation of the arbitration tuples introduces an overall computational complexity of $O(|C| + |T| + |R||M| + |B||C|)$ which can be simplified to $O(|T| + |R||M| + |B||C|)$.

Overall Computational Complexity The overall computational complexity of the proposed isolation-aware scheduling approach can be derived by combining the computational complexity of its individual steps analyzed above. Overall, the proposed isolation-aware scheduling approach introduces a computational complexity of $O(K_{\text{core}}^{\max}|T| + K_{\text{NoC}}^{\max}|M| + |R||M| + |B||C|)$ which grows linearly with the number $|T|$ of tasks and the number $|M|$ of messages in the application.

3.3 Worst-Case Timing Analysis[7]

This section presents the proposed worst-case timing analysis for (heterogeneous) many-core systems. Given the arbitration tuples of a mapping, the proposed timing analysis formally derives an upper bound on the Worst-Case Response Time (WCRT) of each task $t \in T$ and on the Worst-Case Traversal Time (WCTT) of each inter-tile message $m \in M$. The obtained WCRT and WCTT bounds are then used to bound the worst-case end-to-end latency (makespan) and throughput of the mapping. The proposed timing analysis is performed compositionally, i.e., the worst-case timing behavior of each task or message is decomposed into several timing contributions on different resources which are analyzed separately and, then, are combined to obtain globally safe timing guarantees. For further details on compositional timing analyses, see Section 2.2.2.

3.3.1 Worst-Case Response Time Analysis

The Worst-Case Response Time (WCRT) of a task t denotes the worst-case time interval between t's arrival time and completion time on the core to which it is mapped. The WCRT of t accounts for the context-independent worst-case latency of t and the worst-case timing interferences that may be imposed on t in its execution context, i.e., by concurrent processes in the system when t accesses shared resources. In each iteration of its execution, task t reads its required data and input messages from the on-tile memory associated with it, performs its processing, and writes its output

[7] Major parts of this section have been published in [PSW+19].

messages into its associated memory. In general, t may be exposed to two sources of timing interference in each iteration of its execution: (i) interference on the bus when accessing memory and (ii) preemption delays on the core. Therefore, the WCRT of a task $t \in T$ can be formally bounded as:

$$WCRT(t,c,q,b) = WCET_{t,c} + MD_t \cdot ST_q + I^{\text{bus}}(t,c,q,b) + I^{\text{core}}(t,c,q,b) \quad (3.2)$$

where $WCET_{t,c}$ denotes the worst-case execution time of t on core c in isolation, assuming zero delays for bus and memory accesses and an uninterrupted execution of t on c. MD_t denotes the maximum memory demand (total number of single-word memory accesses) of t. ST_q denotes the service time of memory q, i.e., the turnaround delay of a single-word memory access in the absence of interferences on the bus. Thus, $(MD_t \cdot ST_q)$ provides the overall latency of t's memory accesses when performed in isolation. $I^{\text{bus}}(t,c,q,b)$ denotes the worst-case delay imposed on t's execution due to interferences on bus b over which memory q is accessed. Finally, $I^{\text{core}}(t,c,q,b)$ denotes the worst-case preemption delay imposed on t's execution on core c. Here, $WCET_{t,c}$ is provided as an attribute of the mapping edge (t,c) in the specification, MD_t is an attribute of t in the application graph, and ST_q is an attribute of q in the architecture graph, see Section 2.3. Interference delays $I^{\text{bus}}(t,c,q,b)$ and $I^{\text{core}}(t,c,q,b)$ can be formally bounded as follows.

Worst-Case Bus Interference

Suppose that task t mapped to core c accesses memory q over bus b. The worst-case bus interference $I^{\text{bus}}(t,c,q,b)$ imposed on t's execution for its memory accesses can be formally bounded based on the memory demand MD_t of t, the service time ST_q of memory q, and the arbitration tuple (S_b, W_b^c, P_b^c) of core c on bus b. To bound $I^{\text{bus}}(t,c,q,b)$, first, we derive the maximum number of bus arbitration slots that t may require for performing MD_t memory operations.

The maximum number of bus arbitration slots required by t is denoted as $\Phi(t,c,q,b)$ and is subject to two upper bounds: On the one hand, $\Phi(t,c,q,b)$ cannot exceed MD_t, since each bus slot is necessarily long enough to allow performing at least one single-word memory access. Thus, it is guaranteed that $\Phi(t,c,q,b) \leq MD_t$. On the other hand, $\Phi(t,c,q,b)$ cannot exceed the maximum number of bus slots that may pass during t's execution in isolation, i.e., $\lceil (WCET_{t,c} + MD_t \cdot ST_q)/S_b \rceil$. Thus, $\Phi(t,c,q,b)$ can be bounded using Eq. (3.3) where the two arguments of the min{} operation correspond to the two upper bounds outlined above.

$$\Phi(t,c,q,b) = \min\left\{ MD_t, \left\lceil \frac{WCET_{t,c} + MD_t \cdot ST_q}{S_b} \right\rceil \right\} \quad (3.3)$$

In the worst case, the memory accesses of t are distributed across its execution time in such a way that the $\Phi(t,c,q,b)$ required bus slots for t fall into separate bus

arbitration periods. As a result, t may experience the maximum wait time for attaining each of its required bus slots. In each bus arbitration period perceived by t on b, a total of W_b^c bus slots are dedicated to t executed on core c. Given the length S_b of each bus slot of b and the worst-case bus period P_b^c perceived by t on b, a worst-case wait time of $(P_b^c - W_b^c \cdot S_b)$ may be imposed on t's execution per bus arbitration period before it acquires the bus ownership for each of its required bus slots. Thus, the overall bus interference imposed on t's execution can be bounded as:

$$I^{\text{bus}}(t,c,q,b) = \Phi(t,c,q,b) \cdot (P_b^c - W_b^c \cdot S_b) \tag{3.4}$$

where the first factor $\Phi(t,c,q,b)$ denotes the maximum number of bus slots required by t, and the second factor $(P_b^c - W_b^c \cdot S_b)$ denotes the maximum wait time imposed on t for attaining a dedicated bus slot per bus arbitration period P_b^c.

In some cases, a memory access is initiated so late that it cannot be completed before the end of the current bus arbitration slot. To compensate for this misalignment, the arbitration delay D_b of each bus $b \in B$ is provisioned with a guard time equal to the service time ST_q of its associated memory q. This practice (i) enables memory accesses that are initiated too late to be completed before the next bus slot is assigned to its respective requestor and, thereby, (ii) eliminates memory interferences due to overlapping memory accesses from different bus masters in consecutive bus slots. Note that if task t accesses multiple memories, the bus-interference analysis above must be applied for each respective memory bus b_i over which t performs MD_t^i single-word accesses to memory q_i with a service time of ST_{q_i}.

Worst-Case Core Preemption Latency

Supposing that task t mapped to core c accesses memory q over bus b, the worst-case delay imposed on t's execution due to the preemption of its execution on c can be formally bounded based on the arbitration tuple (S_c, W_c^t, P_c^t) of t on c and the arbitration tuple (S_b, W_b^c, P_b^c) of c on the bus b over which memory q is accessed. In each iteration of its execution, task t requires an overall dedicated processor time of $(WCET_{t,c} + MD_t \cdot ST_q + I^{\text{bus}}(t,c,q,b))$ on core c in order to complete its processing and perform its memory accesses to q over the shared bus b. Given that t has a total of W_c^t dedicated core scheduling slots per scheduling period of c, it receives an overall dedicated processor time of $(W_c^t \cdot S_c)$ per perceived core scheduling period P_c^t. Hence, a worst-case periodic wait time of $(P_c^t - W_c^t \cdot S_c)$ can be imposed on t's execution due to preemption. Therefore, the overall preemption delay imposed on t's execution can be formally bounded as:

$$I^{\text{core}}(t,c,q,b) = \left\lceil \frac{WCET_{t,c} + MD_t \cdot ST_q + I^{\text{bus}}(t,c,q,b)}{W_c^t \cdot S_c} \right\rceil \cdot \left(P_c^t - W_c^t \cdot S_c\right) \tag{3.5}$$

where the first factor bounds the number of core scheduling periods over which the execution of t may span, and the second factor gives the worst-case wait time imposed

on t's execution per core scheduling period. Note that also here, a memory access may be initiated too late to be completed before the end of the current core scheduling slot. To compensate for this misalignment, the context-switch latency D_c of each core $c \in C$ is provisioned with a guard time equal to the maximum service time among all memories accessible to c. As a result, any memory access started in the current core scheduling slot is guaranteed to be completed before the next context switch of c.

3.3.2 Worst-Case Traversal Time Analysis

The Worst-Case Traversal Time (WCTT) of an inter-tile message m denotes the worst-case latency of the transfer process of m from the memory on the source tile where m is produced to the target memory on the destination tile where m will be consumed. As elaborated in Section 2.1.4, the transfer process of an inter-tile message m is realized in three steps: Once m is written by its producer task in the source memory q, (i) the TX unit tx of the local NA reads m from memory q via q's associated bus b, decomposes m into flow-control digits (flits), and injects the flits into the NoC. On the NoC, (ii) the flits of m are routed over the sequence of routers $\rho_m \subseteq R$ on m's NoC route towards the destination tile. Once the flits arrive at the destination tile, (iii) the RX unit rx of the local NA reconstructs m from its flits and writes it into m's dedicated space in the target memory $\hat{q}$ over $\hat{q}$'s associated bus $\hat{b}$. In line with the three steps above, the WCTT of an inter-tile message m can be formally bounded as:

$$WCTT(m,tx,q,b,\rho_m,rx,\hat{q},\hat{b}) = D^{\mathrm{TX}}(m,tx,q,b) + D^{\mathrm{NoC}}(m,\rho_m) + D^{\mathrm{RX}}(m,rx,\hat{q},\hat{b}) \tag{3.6}$$

where $D^{\mathrm{TX}}(m,tx,q,b)$, $D^{\mathrm{NoC}}(m,\rho_m)$, and $D^{\mathrm{RX}}(m,rx,\hat{q},\hat{b})$ denote the worst-case latency of the transfer steps (i)–(iii) above, respectively. These latency contributions can be bounded formally using the following analysis.

Worst-Case Network Adapter Latency

Suppose that TX unit tx shall read inter-tile message m from memory q over bus b for injection into the NoC. This process is subject to two sources of interference: (i) bus interference since reading m from memory is performed over a shared bus and (ii) TX interference since tx may be shared among multiple outbound messages transmitted concurrently from the tile. In the following, we bound the worst-case latency of this process, denoted by $D^{\mathrm{TX}}(m,tx,q,b)$, using the arbitration tuple $(S_b, W_b^{tx}, P_b^{tx})$ of tx on b and the arbitration tuple $(S_{tx}, W_{tx}^m, P_{tx}^m)$ of m on tx. The maximum payload size PLD_m of m is also known which is provided by the application graph.

To bound $D^{\mathrm{TX}}(m,tx,q,b)$, first, we formally bound the maximum number of bus arbitration slots that can be required for reading m from memory q, denoted as $\Omega(m,q,b)$: Generally, in each bus arbitration slot, a total of $\lceil S_b/ST_q \rceil$ consecutive accesses to memory q can be performed via its associated bus b. Therefore, given

the maximum payload size PLD_m of m in number of single-word memory accesses, $\Omega(m,q,b)$ can be bounded as:

$$\Omega(m,q,b) = \left\lceil PLD_m \Big/ \left\lceil \frac{S_b}{ST_q} \right\rceil \right\rceil \tag{3.7}$$

Given $\Omega(m,q,b)$ and provided that tx has a reserved arbitration weight of W_b^{tx} on bus b, the overall bus delay imposed on tx for reading m from memory q can be bounded using Eq. (3.8). There, $\lceil \Omega(m,q,b)/W_b^{tx} \rceil$ bounds the number of bus arbitration periods that may pass before tx attains all $\Omega(m,q,b)$ dedicated bus slots required for reading m, and $(P_b^{tx} - W_b^{tx} \cdot S_b)$ bounds the worst-case wait time imposed on tx per bus arbitration period.

$$I^{\text{bus}}(m,tx,q,b) = \left\lceil \frac{\Omega(m,q,b)}{W_b^{tx}} \right\rceil \cdot \left(P_b^{tx} - W_b^{tx} \cdot S_b\right) \tag{3.8}$$

Finally, $D^{\text{TX}}(m,tx,q,b)$ can be formally bounded using Eq. (3.9). There, the first summand $(PLD_m \cdot ST_q)$ denotes the latency of reading m from memory q in isolation, the second summand $I^{\text{bus}}(m,tx,q,b)$ derived by Eq. (3.8) denotes the worst-case bus interference imposed on tx when reading message m from memory q via shared bus b, and the third summand bounds the worst-case TX interference imposed when reading m from the memory. Here, $\lceil (PLD_m \cdot ST_q + I^{\text{bus}}(m,tx,q,b))/(W_{tx}^m \cdot S_{tx}) \rceil$ bounds the maximum number of TX arbitration periods involved for reading m and $(P_{tx}^m - W_{tx}^m \cdot S_{tx})$ bounds the worst-case wait time imposed per TX arbitration period P_{tx}^m on the process of reading m. The RX latency, i.e., $D^{\text{RX}}(m,rx,\hat{q},\hat{b})$ in Eq. (3.6) can be bounded similarly to the analysis of the TX latency in Eqs. (3.7) to (3.9).

$$\begin{aligned} D^{\text{TX}}(m,tx,q,b) = {} & PLD_m \cdot ST_q + I^{\text{bus}}(m,tx,q,b) \\ & + \left\lceil \frac{PLD_m \cdot ST_q + I^{\text{bus}}(m,tx,q,b)}{W_{tx}^m \cdot S_{tx}} \right\rceil \cdot (P_{tx}^m - W_{tx}^m \cdot S_{tx}) \end{aligned} \tag{3.9}$$

Worst-Case Network-on-Chip Latency

Suppose that message m is transferred over the sequence of NoC routers $\rho_m \subseteq R$ connecting its source and destination tiles. The worst-case latency of this transfer process, denoted by $D^{\text{NoC}}(m,\rho_m)$, can be formally bounded based on the arbitration tuple $(S_{\rho_m}, W_{\rho_m}^m, P_{\rho_m}^m)$ of m on the routers in ρ_m. Note that, in a wormhole-switched NoC (as considered in this work), the length S_{ρ_m} of one arbitration slot of a router is equal to the NoC clock cycle denoted by τ^{NoC}, hence, $S_{\rho_m} = \tau^{\text{NoC}}$. The worst-case latency of transferring m over router sequence ρ_m can be formally bounded as:

$$\begin{aligned} D^{\text{NoC}}(m,\rho_m) = {} & (F_m - 1 + |\rho_m| \cdot D^{\text{router}}) \cdot S_{\rho_m} \\ & + \left(\left\lceil \frac{F_m}{W_{\rho_m}^m} \right\rceil - 1 + |\rho_m| \right) \cdot \left(P_{\rho_m}^m - W_{\rho_m}^m \cdot S_{\rho_m} \right) \end{aligned} \tag{3.10}$$

which is adopted from [WGW+14]. Here, F_m denotes the number of flits associated with m which is calculated based on m's payload size PLD_m and the NoC bit width. Moreover, $|\rho_m|$ denotes the length of the router sequence ρ_m in number of hops, τ^{NoC} denotes the duration of one NoC clock cycle, and D^{router} denotes the latency of a router in number of clock cycles. The first summand in Eq. (3.10) derives the latency of transferring F_m flits over the router sequence ρ_m in isolation, i.e., in the absence of any interferences on ρ_m. The second summand in Eq. (3.10) formally bounds the worst-case interference imposed on the transfer of m over ρ_m in the presence of other communication flows on the NoC. Here, the first factor $(\lceil F_m / W^m_{\rho_m} \rceil - 1 + |\rho_m|)$ bounds the maximum number of router arbitration periods in which the flits of m may be stalled due to the transfer of concurrent flows. The second factor $(P^m_{\rho_m} - W^m_{\rho_m} \cdot \tau^{\text{NoC}})$ bounds the worst-case wait time imposed per router arbitration period $P^m_{\rho_m}$, cf. [WGW+14].

3.3.3 Application Timing Guarantees

Let $WCRT(t)$ denote the WCRT of task $t \in T$ and $WCTT(m)$ the WCTT of inter-tile message $m \in M$ analyzed as presented in Sections 3.3.1 and 3.3.2, respectively. Based on the derived bounds on the WCRT of tasks and WCTT of messages, the worst-case throughput TH_{app} of the mapping can be formally bounded using Eq. (3.11). Likewise, the worst-case end-to-end application latency (makespan) L_{app} can be bounded using Eq. (3.12) where Π denotes the set of all end-to-end paths in the application graph.

$$TH_{\text{app}} = \max \left\{ \max_{t \in T} \left\{ WCRT(t) \right\}, \max_{m \in M} \left\{ WCTT(m) \right\} \right\}^{-1} \tag{3.11}$$

$$L_{\text{app}} = \max_{\pi \in \Pi} \left\{ \sum\nolimits_{t \in (T \cap \pi)} WCRT(t) + \sum\nolimits_{m \in (M \cap \pi)} WCTT(m) \right\} \tag{3.12}$$

Computational Complexity

The WCRT analysis of each task $t \in T$ realized using Eqs. (3.2) to (3.5) and the WCTT analysis of each inter-tile message $m \in M$ realized using Eqs. (3.6) to (3.10) are constant-time non-iterative analyses with a computational complexity of $O(1)$. Hence, the throughput analysis of an application using Eq. (3.11) introduces a computation complexity of $O(|T| + |M|)$ where $|T|$ and $|M|$ denote the number of tasks and messages in the application graph, respectively. Having the application graph provided as an adjacency list, the calculation of the worst-case end-to-end application latency in Eq. (3.12) can be realized using a Depth-First Search (DFS) algorithm with a computational complexity of $O(|T| + |M|)$.

3.4 Experimental Results[8]

This section presents the results of a series of experiments conducted to evaluate the impact of the proposed isolation-aware mapping optimization and timing analysis approach on the quality of the obtained mappings. To that end, the effectiveness of this approach is evaluated in comparison with existing mapping optimization approaches based on a fixed isolation scheme.

3.4.1 Experiments Setup

Architectures The experiments are performed based on three heterogeneous many-core architectures with 4×4, 5×5, and 6×6 tiles, respectively. Each architecture is composed of three core types (IBM PowerPC, AMD K6-2, and AMD K6-III) provided by the Embedded System Synthesis Benchmarks Suite (E3S) [Dic10]. Each tile of each architecture is composed of four homogeneous cores such that the three architectures encompass 64, 100, and 144 cores, respectively, with diverse numbers of instances from the core types above. In addition to four cores, each tile also contains one shared memory, one NA with separate TX and RX units, and a shared bus providing memory connectivity for cores, the TX, and the RX. Every shared resource has a WRR arbitration policy configured as follows: For each core c, an arbitration capacity of $K_c = 10$ slots, a scheduling slot length of $S_c = 50$ us, and a context-switch overhead of $D_c = 10$ us is considered. On each bus b, each master i (TX, RX, and the four cores) has an arbitration weight of $W_b^i = 1$ slot, resulting in a bus arbitration capacity of $K_b = 6$ slots. The length of each bus slot S_b is set equal to the memory service time which is 7 clock cycles. For each NoC router r, an arbitration capacity of $K_r = 10$ slots and an arbitration slot length of $S_r = \tau^{\text{NoC}} = 10$ ns are considered. For each TX tx and RX rx, an arbitration capacity of $K_{tx} = K_{rx} = 10$ slots and a slot length equal to the bus arbitration period is considered. The NoC employs the XY-routing algorithm [NM93].

Applications For the experiments, four real-time applications are used from the embedded domains of networking (7 tasks, 9 messages), consumer (11 tasks, 12 messages), telecommunications (14 tasks, 20 messages), and automotive (18 tasks, 21 messages). The applications are provided as benchmarks by E3S [Dic10]. For each application, E3S [Dic10] also provides the Worst-Case Execution Time (WCET) of each task on each core type, the memory demand of each task, the payload size of each message, and the execution/transfer period of each task/message which are required for the timing analysis of mappings.

[8] Major parts of this section have been published in [PSW+19].

Design Objectives The optimization of mappings for each application on each architecture is performed w.r.t. three design objectives that are commonly considered in the DSE of real-time applications in composable many-core systems: (i) *worst-case application latency (makespan)* derived using the proposed timing analysis, (ii) *resource cost* calculated as the sum of the core scheduling slots reserved on each allocated core divided by its arbitration capacity which is set to $K_c = 10$ for all cores $c \in C$, and (iii) *energy consumption*, calculated based on the WCRT of tasks, the core power-consumption parameters provided by [Dic10] for each task of each investigated benchmark application, the payload size of each message provided by [Dic10], and the NoC/bus energy model from [WSK+05] considering a link length of 2 mm for NoC links and 5 mm for buses.

Design Space Exploration (DSE) For the optimization of mappings, the OpenDSE framework [RLG+18] and the NSGA-II [DPA+02] multi-objective evolutionary algorithm provided by the optimization framework OPT4J [LGR+11] are used. During DSE, the mapping optimizer follows the course of several optimization iterations to generate new mappings and evaluate them w.r.t. the given design objectives. It collects a population of mappings which is updated per iteration and is used for the generation of mappings in the next iteration. The optimizer also retains the set of Pareto-optimal mappings found so far. The DSE is configured to perform 4,000 optimization iterations based on a population of 100 mappings where 25 mappings are generated per iteration. The experimental results reported in this section represent an average over 20 runs of the DSE for each application on each architecture.

Investigated Approaches In the experiments, the proposed isolation-aware mapping optimization approach is compared with mapping optimization approaches based on one of the three fixed isolation schemes, namely, Core-Sharing (CS), Core-Reservation (CR), and Tile-Reservation (TR), introduced in Section 3.1.1. For each optimization approach and each given pair of application and architecture, the DSE delivers a set of Pareto-optimal mappings w.r.t. the design objectives introduced above. The investigated approaches are compared with each other in terms of the quality of mappings obtained using each.

Quality Metric For the quality comparison of the mappings obtained by the investigated approaches, the well-established ε-*dominance* metric [LTD+02] from the domain of multi-objective optimization is used. ε-dominance is a scalar value reflecting the quality of solutions in the space of multiple objectives and is defined as follows: Let Γ represent a set of Pareto-optimal mappings obtained by an optimization approach in the space of a set of design objectives O to be minimized. Also, let Γ^{ref} represent a reference set of mappings containing the true Pareto-optimal mappings for the optimization problem at hand. To assess the quality of mappings in Γ w.r.t. Γ^{ref},

ε-dominance provides a scalar indicator $\varepsilon_\Gamma \in [0,1)$ calculated as:

$$\varepsilon_\Gamma = \min\left\{\varepsilon \in [0,1) \mid \forall \mu' \in \Gamma^{\text{ref}}, \exists \mu \in \Gamma \ni \left(\forall o \in O:\ (1-\varepsilon)\cdot \mu_o \leq \mu'_o\right)\right\} \quad (3.13)$$

where μ_o and μ'_o denote the quality of mappings $\mu \in \Gamma$ and $\mu' \in \Gamma^{\text{ref}}$, respectively, w.r.t. design objective $o \in O$. A *smaller* value for ε_Γ indicates a smaller relative distance between the mappings in Γ and those in the reference set Γ^{ref} in the space of design objectives which, in turn, denotes a *higher quality* of mappings in μ. Due to the exponential complexity of the many-core mapping problem, finding the set Γ^{ref} of true Pareto-optimal mappings is computationally infeasible. Therefore, similarly to the common practice in the multi-objective optimization domain, the reference set Γ^{ref} in our experiments is constructed as the union set of the Pareto-optimal mappings obtained by the four optimization approaches under investigation, denoting the collection of the best reachable mappings.

3.4.2 Results Discussion

Figures 3.8 and 3.9 illustrate the ε-dominance indicator of the proposed isolation-aware approach and the three fixed-isolation-scheme approaches, namely, CS, CR and TR, across their exploration (optimization) run time. The results in Figure 3.8 correspond to the smaller applications, namely, networking and consumer, (plot columns) on the three architectures (plot rows) while the results in Figure 3.9 correspond to the bigger applications, namely, telecom and automotive, (plot columns) on the three architectures (plot rows).

In general, the quality of mappings obtained by each approach improves as the exploration progresses, resulting in a decrease in the ε-dominance indicator of each approach throughout the course of its optimization iterations. Also, the overall exploration time of each approach grows with the application size (compare the plots in the same row in Figures 3.8 and 3.9) and architecture size (compare the plots in the same column in Figures 3.8 and 3.9). The results consistently demonstrate that, regardless of the choice of application and architecture, (i) the proposed isolation-aware approach always outperforms the fixed-isolation-scheme approaches. This is indicated by its lower ε-dominance indicator at the end of its exploration in all plots in Figures 3.8 and 3.9. Moreover, (ii) the proposed isolation-aware approach exhibits an exploration run time within the same range as the other approaches and, hence, does not impact the scalability of the mapping optimization process adversely. Among the fixed-isolation-scheme approaches, CR generally performs better than CS and TR for most cases, while CS exhibits the worst quality of mappings for all applications and architectures. In summary, over all 12 combinations of applications and architectures, the proposed isolation-aware approach achieves an ε-dominance improvement of up to 67% (compared to CS) with an average improvement of 26% compared to the three fixed-isolation-scheme approaches.

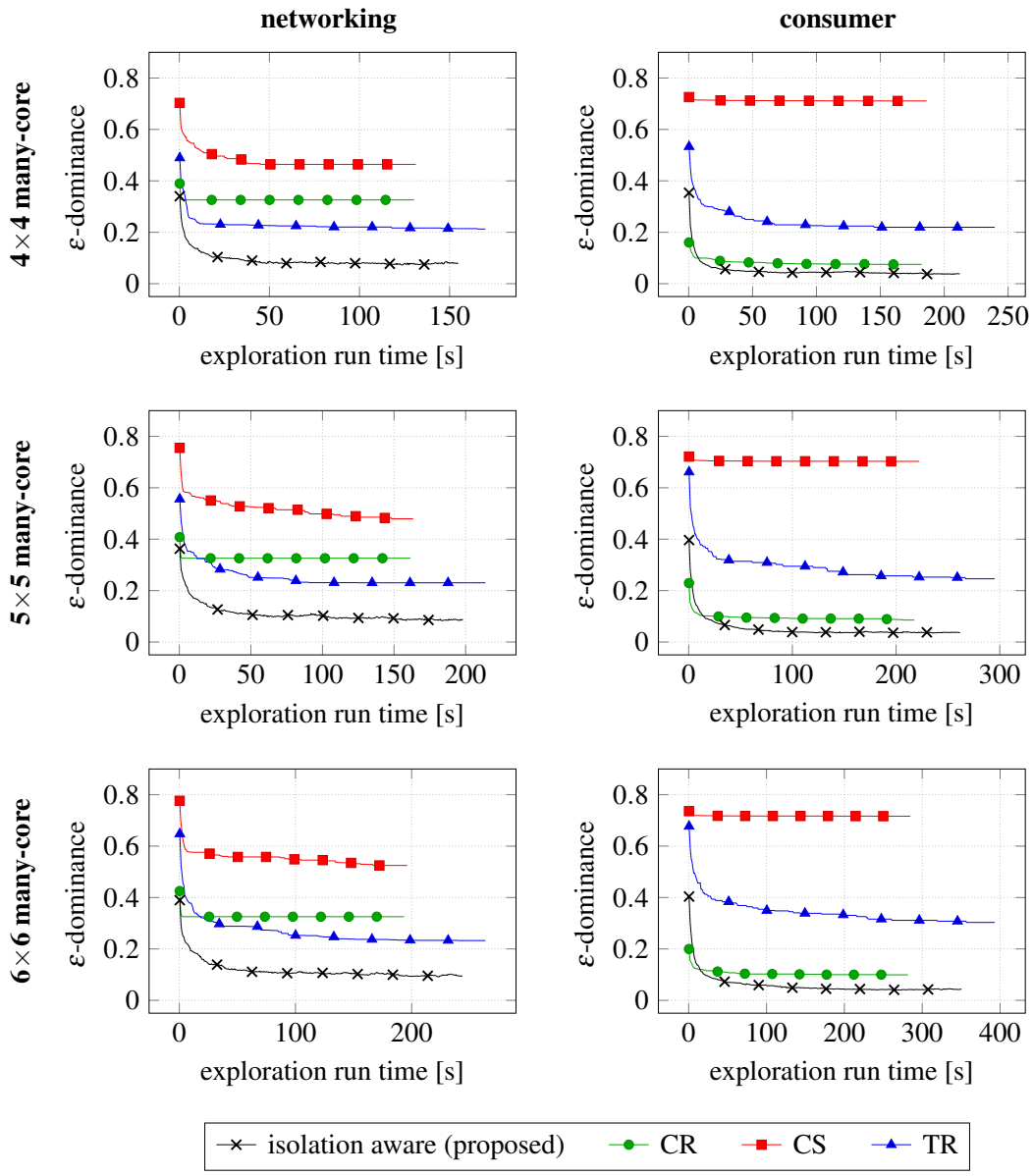


Figure 3.8: ε-dominance of the investigated mapping optimization approaches versus the exploration run time for the networking and consumer benchmarks (plot columns) on the three many-core architecture (plot rows) [PSW+19].

To further inspect the quality differences between the mappings obtained using each of the compared approaches, the distribution of the mappings delivered by each approach is investigated in the three-dimensional objective space. Figure 3.10 illustrates the spread of the Pareto-optimal mappings obtained for each application on the 6×6 architecture in the objective space. Similar distributions are also obtained for the 4×4

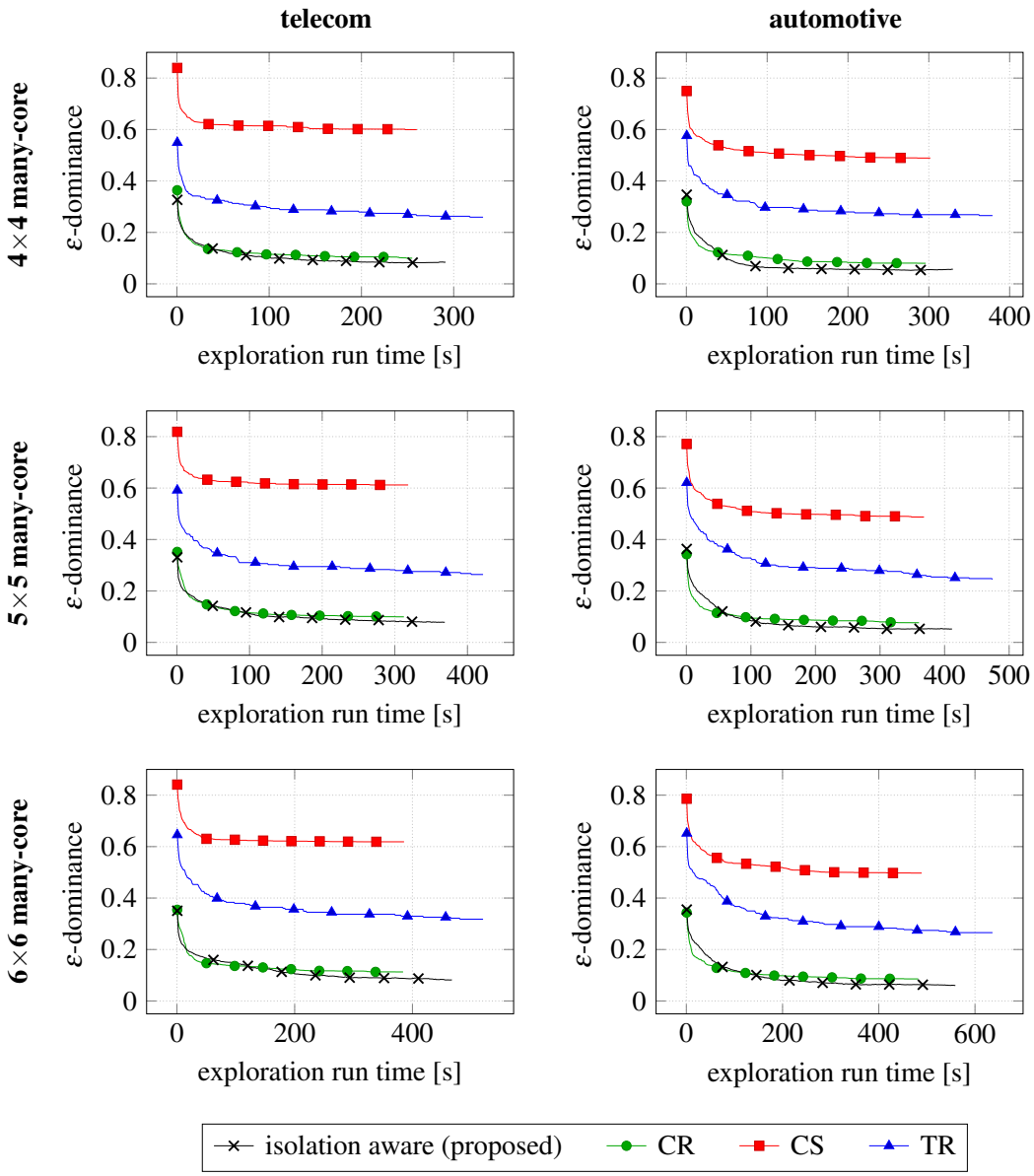


Figure 3.9: ε-dominance of the investigated mapping optimization approaches versus the exploration run time for the telecom and automotive benchmarks (plot columns) on the three many-core architecture (plot rows) [PSW+19].

and 5×5 architectures.

As illustrated in Figure 3.10, the mappings delivered by the CS approach exhibit relatively low resource costs since CS allocates only a minimal amount of resource budget, just enough to meet the deadline of each task and message. This, however, entails high worst-case inter-application interferences and, thus, considerably high worst-case latency and energy consumption as shown in Figure 3.10. The TR approach,

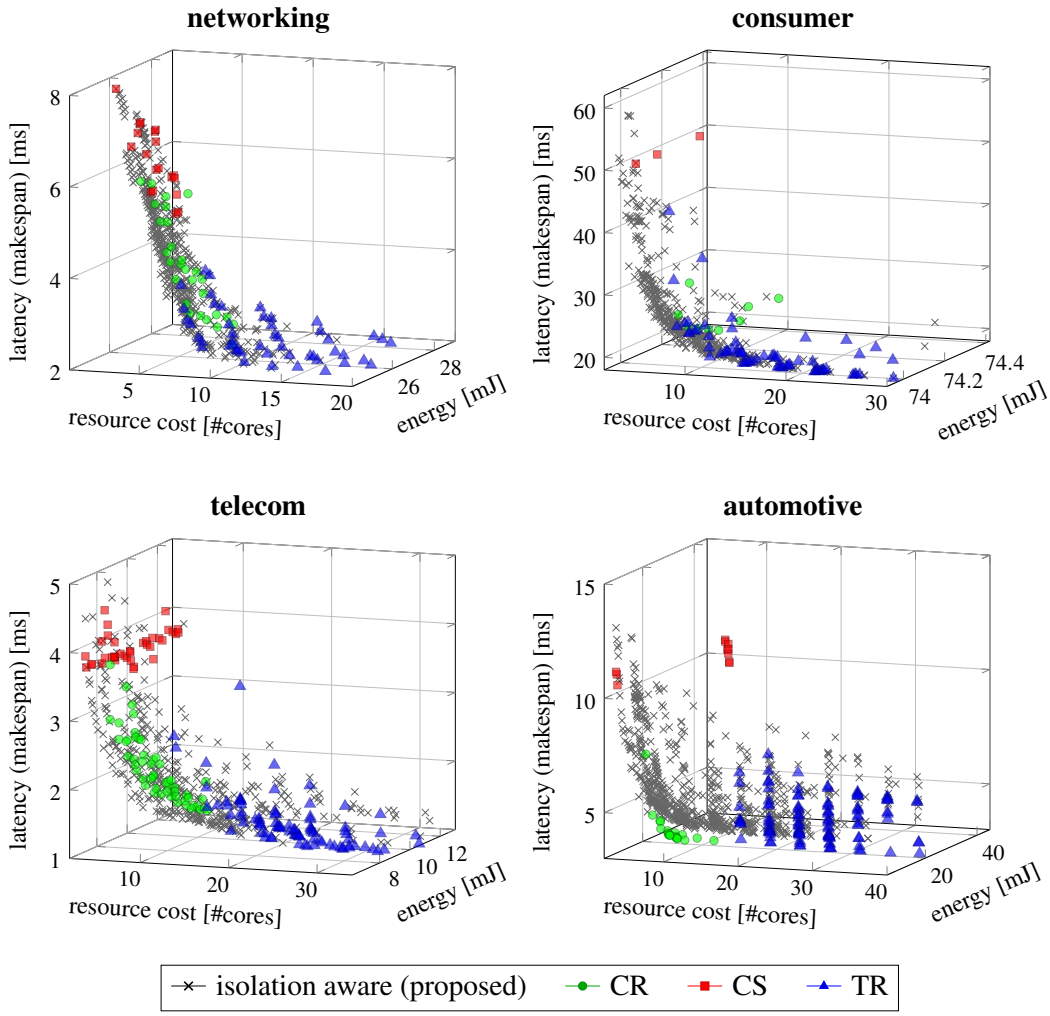


Figure 3.10: Distribution of the Pareto-optimal mappings obtained by each investigated mapping optimization approach in the 3D objective space. Each plot corresponds to one application on the 6×6 architecture [PSW+19].

on the other hand, allocates tiles exclusively (resulting in a relatively high resource cost) which eliminates all on-tile inter-application interferences (resulting in relatively low latency and energy consumption). When considering all design objectives collectively for calculating their ε-dominance indicators according to Eq. (3.13), both CS and TR need a large scaling factor ε to compensate for their respective inferior objective values, explaining their poor (high) ε-dominance indicators in Figures 3.8 and 3.9. A compromise between CS and TR is achieved by CR which limits the exclusive reservation of resources to cores only. This establishes a moderate trade-off between resource cost, worst-case latency, and energy consumption which results in a better (lower) ε-dominance indicator.

The fixed isolation schemes of the approaches above (CS, CR, and TR) significantly restrict the mapping design space available to the optimizer to only a small part of the actual design space. This, in turn, restricts their coverage of the solution and objective space to considerably smaller sub-spaces. Contrarily, the proposed isolation-aware approach provides the mapping optimizer with a design space that not only covers the design space of each of the three fixed-isolation-scheme approaches but also extends beyond that by enabling the optimizer to also explore the choices of isolation schemes in combination within each mapping. This enables the wide spread of the solutions found under the isolation-aware approach in Figure 3.10. As a result, using the proposed approach, the optimizer is empowered to find mappings of high quality (w.r.t. the design objectives) which are inaccessible to the other approaches. Thereby, it outperforms the other approaches as confirmed by its lower ε-dominance indicator in Figures 3.8 and 3.9 for each application on each architecture.

3.5 Related Work

The worst-case timing analysis of applications in a many-core system is typically performed in two steps as detailed in Section 2.2.2. First, context-independent timing analysis is performed in which the WCET of each task is bounded for its execution in isolation, i.e., in the absence of any timing interferences. Authors of [WEE+08] provide an extensive overview of the tools, methods, and techniques used for context-independent WCET analysis of tasks. After WCET analysis, schedulability analysis is performed in the execution context of the application to bound the additional delays that may be imposed on shared resources due to external interferences.

The majority of timing analyses for multi/many-core systems mainly focus on worst-case interference (schedulability) analysis based on the context-independent timing properties of tasks and messages. For instance, the timing analyses presented in [ADI+15; DNA16; DAI+17; GLS+12; GSH+16; RMM+16; RDP17; SS16] derive upper bounds on the WCRT of tasks in a multi-core context in which several cores are connected to a set of memories over a set of shared buses, similarly to the organization of resources within a tile of a many-core platform. Authors of [GLS+12; GSH+16] analyze WCRT based on a non-preemptive Fixed-Priority (FP) core scheduling policy and a Round-Robin (RR) memory bus arbitration. Authors of [ADI+15; DAI+17] present a framework for multi-core WCRT analysis based on a preemptive FP core scheduling policy in conjunction with multiple bus arbitration policies, e.g., FP, RR, and TDMA. In [RMM+16; SS16], a WCRT analysis is presented based on a non-preemptive FP core scheduling policy and a multi-level bus arbitration scheme which combines FP and RR policies. In [RDP17], bus interference is analyzed by means of an Integer Linear Programming (ILP)-based timing analysis for a RR arbitration policy. Authors in [DNA16] present an analysis of bus interference under various arbitration policies, e.g., FP and TDMA. Regarding the interference on the NoC,

[ZSO+13] and [ZB08] analyze the NoC transfer delays under RR and FP arbitration policies, respectively.

The analyses introduced above predominantly consider *contention-oriented* and/or non-preemptive arbitration/scheduling policies, e.g., priority-based policies. For such policies, timing guarantees can be derived only if the complete set of applications that access the shared resources is known to the analysis. The advantage of a composable system, however, is that mapping optimization and timing evaluation of each application can be performed independently from other applications. Thus, the mix of concurrently executed applications that share resources with an application is not required to be known at the time of its timing analysis, see Section 2.2.2. Hence, contention-oriented policies can be used in a composable system only as long as the respective resource is exclusively used by one application. That means, contention-oriented policies can be used in a composable system only if each mapping of each application (i) employs a TR isolation scheme only, (ii) is confined within one tile only, and (iii) does not depend on any inter-tile communications, e.g., I/O transfers. In practice, however, due to the small number of cores in each tile [Til12; HDH+10], the high performance demand of modern applications necessitates the incorporation of several tiles in the mapping of some applications to meet their real-time requirements. For such applications, the involved parts of the NoC could be (in theory) exclusively allocated so that an independent timing analysis of multi-tile mappings becomes possible, e.g., using the NoC analyses in [ZSO+13; ZB08]. In practice, however, sharing the NoC among applications can hardly be avoided [MTT18].

Contention-free arbitration policies that operate preemptively based on time slicing, e.g., TDMA and WRR, enable inter-application resource sharing without violating system composability. For instance, provided a NoC [HKK+13] with WRR arbitration policy, authors of [WGW+14; WWG+18a] analyze NoC delays based on the reserved budget for each message, independently of the other communication flows that may share the same NoC links. For WCRT analysis, a preemptive, priority-based, RR core scheduling policy is considered in [WGW+14] which, however, restricts the applicability of its analysis to mappings in which cores are allocated exclusively. Authors of [WWG+18a], consider a similar scheduling policy but restrict the maximum number of tasks that can be mapped on one core. Thereby, their timing analysis can also be adopted (with slight modifications) to bound the WCRT of tasks under a WRR scheduling policy to cover preemption delays that may be imposed by other (currently unknown) applications. Both analyses, [WGW+14] and [WWG+18a], however, do not consider the NA and bus interference delays that arise in multi-core tiles, e.g., in [dDAB+13; HDH+10]. The timing analysis presented in this work is developed in compliance with composable arbitration policies, e.g., TDMA and WRR, and captures timing interferences that may arise on cores, buses, NAs, and the NoC in a many-core system. This renders it applicable to many-core systems with multi-core tiles and mappings that may span across multiple tiles *without* relying on restrictive assumption about the platform architecture or the deployment of applications on it.

Existing timing analyses used in the context of many-core systems are tailored to a fixed choice of inter-application isolation scheme and, hence, cannot provide worst-case timing guarantees if multiple isolation schemes are applied in combination. For instance, the analyses in [ADI+15; DNA16; DAI+17; GLS+12; GSH+16; RMM+16; RDP17; SS16] are applicable only under the TR isolation scheme. The analyses presented in [WGW+14; WWG+18a] are applicable to many-core systems with single-core tiles under a TR/CR isolation scheme only. However, the analysis in [WWG+18a] can also be adapted to be used for systems with single-core tiles under a CS isolation scheme. Contrarily to these analyses, the timing analysis proposed in this work does not depend on a fixed inter-application isolation scheme and, hence, can provide worst-case timing guarantees for mappings which employ a combination of different isolation schemes on their allocated resources.

To the best of our knowledge, existing DSE and mapping optimization approaches used in the context of HAM, e.g., [KSK+13; MAV+10; PP12; SS17; SKS13; WGW+14; WWG+18a; YAM+11], consider (or, implicitly assume) a fixed inter-application isolation scheme that is applied exclusively to every explored mapping for every application in the system. For instance, the resource-reservation scheme in the DSE approaches in [PP12; MAV+10; YAM+11] manifest the CR isolation scheme, [KSK+13; SKS13; WGW+14; WWG+18a] follow the TR isolation scheme, while [WWG+18a] covers CS as well. Contrarily to the approaches above, the isolation-aware mapping optimization and timing analysis proposed in this work enable arbitrary combinations of isolation schemes to be employed within one mapping by controlling the choice of the isolation scheme applied per allocated resource for each generated mapping. This enables the optimizer to automatically find high-quality mappings in which the amount of reserved resources is fine-tuned according to the performance and timing requirements of the application.

3.6 Summary

The state-of-the-art practice of application mapping for real-time composable systems targeting many-core platforms is restricted to a fixed inter-application isolation scheme that is applied exclusively on each and every mapping of every application in the system, excluding numerous promising mappings from the mapping solution space as elaborated in Section 3.1. This restriction mainly roots in the inability of the existing timing analyses applied in a many-core context in capturing all timing components contributing to the execution latency of an application in a many-core system, in the general case.

As a remedy, this chapter presented a (i) *worst-case timing analysis* specifically developed for many-core systems and, enabled by it, an (ii) *isolation-aware mapping optimization* approach for the offline DSE in HAM. The isolation-aware mapping optimization approach presented in Section 3.2 enables the mapping optimizer to

explore combinations of inter-application isolation schemes applied on different resources within each mapping generated during the mapping optimization process. For each thereby obtained mapping, the timing analysis presented in Section 3.3 formally bounds the worst-case timing properties based on the choices of isolation schemes in that mapping. Together, the isolation-aware mapping optimization approach and the timing analysis enable the optimizer to find high-quality mappings in which the amount of reserved resources is well-adjusted to the performance and timing requirements of the application. A series of experimental results presented in Section 3.4 demonstrated the effectiveness of the proposed approach in enhancing the quality of the obtained mappings in comparison to the current practices of inter-application isolation in many-core systems.

4

Thermal Composability in Hybrid Application Mapping

The ongoing advancements in the semiconductor industry have enabled a continuous process technology downsizing which empowers the massive integration of resources in many-core chips. Although many-core chips address the high computational-power demand of modern embedded systems and applications, their dense integration of resources gives rise to high temperatures on the chip which can jeopardize the system's thermal integrity and lead to permanent transistor failures or even chip burn-down. To ensure thermal safety, many-core systems employ Dynamic Thermal Management (DTM) mechanisms that monitor the thermal state of the chip steadily and counteract hot spots autonomously using power-control mechanisms, e.g., power gating, to reduce the temperature of the overheated regions into a safe range.

Although they preserve the thermal integrity of the systems, DTM countermeasures interfere with the execution of the applications which are running in the overheated regions and, hence, may lead to deadline violations in a real-time context. In the case of hard real-time applications, the deadline violations consequent to the activation of DTM countermeasures can jeopardize the functional integrity of the whole system and even put human lives at risk in safety-critical systems, e.g., in automotive or medical domains. To prevent such unforeseeable interferences, the occurrence of thermal violations must be prevented such that DTM countermeasures are never triggered in the first place. State-of-the-art Hybrid Application Mapping (HAM) methodologies do not consider this critical aspect in their mapping and management processes for real-time applications and, thereby, indeed fail in providing safe real-time guarantees.

As a remedy, this chapter presents a *thermally composable HAM methodology* which introduces a (i) *thermal-safety analysis* into the design-time Design Space Exploration (DSE) in HAM and provides the Run-time Platform Manager (RPM) with a lightweight (ii) *thermal-safety admission check* to be used when launching applications. The thermal-safety analysis derives an abstract thermal-safety index for each mapping offline which is then used for the online thermal-safety admission check on-the-fly. Using (i) and (ii), the proposed methodology enforces thermal safety at all times *proactively* (by performing (ii) at the launch time of applications) so that DTM

countermeasures are never triggered and real-time guarantees are preserved for all running applications.

The remainder of this chapter is organized as follows[1]. In Section 4.1, the concepts of thermal safety and thermal composability are elucidated, and the contribution of this chapter is motivated. Section 4.2 presents the proposed thermally composable HAM methodology. In Section 4.3, the effectiveness of the proposed approach is demonstrated through a series of experimental results. Finally, an overview of related work is provided in Section 4.4, followed by a summary of the chapter in Section 4.5.

4.1 Thermal Safety and Thermal Composability

4.1.1 Many-Core Thermal Problem: Cause and Control

The consumption of power on a chip generates heat which, in turn, leads to a gradual increase in temperature. Figure 4.1 illustrates the thermal impact (left) of application execution in a many-core chip (right). As illustrated, the execution of the application leads to a temperature increase in the region used by the application (due to power consumption) as well as in the regions nearby the region used by the application (due to heat transfer between neighboring regions). The magnitude and the spread of temperature observed on a chip are directly related to the *density* of power consumption and its *distribution* on the chip [PCS+18].

The massive density of transistors integrated within many-core chips (enabled by the continuous technology downsizing) results in a noticeably high density of power consumption and, therefore, may also lead to a significant increase in the range of observable temperatures on the chip. In this scope, the cooling solution of the chip, namely, the combination of the thermal compound, heat spreader, heat sink, etc., is designed so as to dissipate the generated heat rapidly and, thereby, prevent overheating. However, due to the technological limitations of the cooling subsystem and chip packaging, the generated heat in a many-core chip cannot be dissipated fast enough to prevent excessively high temperatures that may harm the thermal integrity of the chip by causing permanent transistor failures [SGH+14].

In this context, many-core systems rely on DTM mechanisms. Such mechanisms steadily monitor the temperature of different regions of the chip and use power-control mechanisms such as power gating, clock gating, or Dynamic Voltage and Frequency Scaling (DVFS) to autonomously counteract overheating in regions where the observed temperature exceeds a predefined critical temperature threshold [HC17; KPS+17]. Although they preserve the thermal integrity of the chip, such countermeasures interfere with the execution of the applications that are running in the thermally affected regions and may lead to deadline violations for real-time applications which is not tolerable

[1] The contents of this chapter are based on and partly published in [PSK+19] which has appeared in the Proceedings of the 40th IEEE Real-Time Systems Symposium (RTSS).

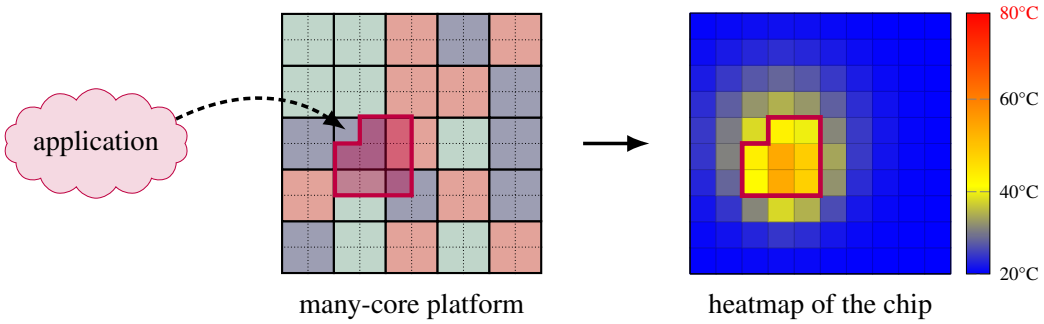


Figure 4.1: Thermal impact of application execution on a many-core chip.

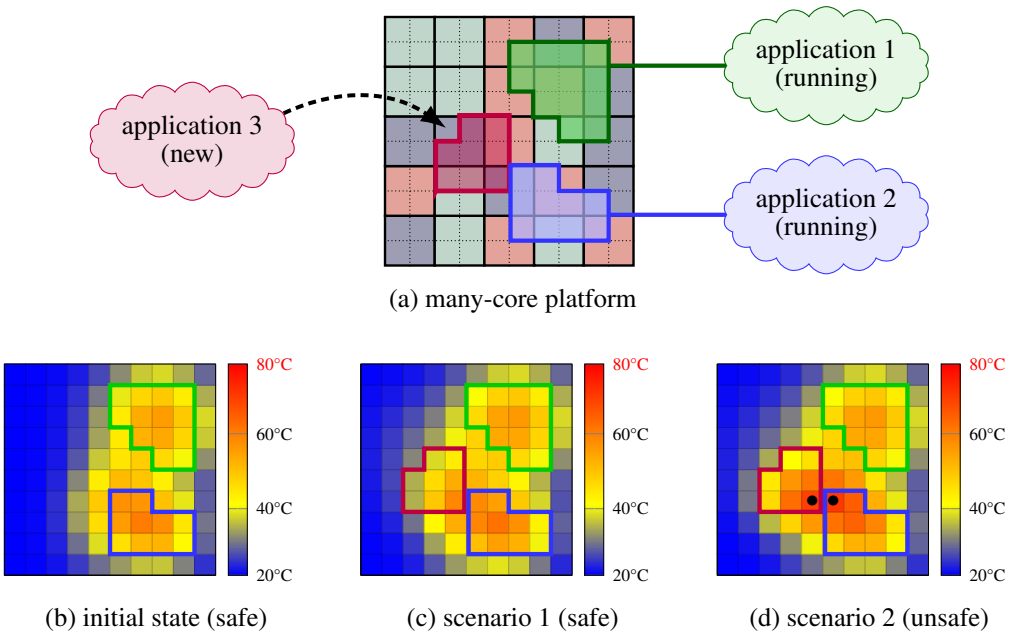
in a hard real-time context. To prevent such unforeseeable interferences without jeopardizing the thermal integrity of the chip, it must be *proactively* ensured that the applications in the system have a thermally safe behavior such that thermal violations are eliminated altogether and, hence, DTM countermeasures are never triggered.

To verify the thermal safety of different mappings of an application, *worst-case thermal analysis* can be used. Due to their relatively high computational overhead, thermal analyses are not suited for online use and, therefore, must be performed offline. This restriction can, however, be properly addressed in the context of HAM since, in HAM, the set of mappings for each application is computed offline and, therefore, the thermal analysis of each mapping may also be performed offline to identify and discard mappings with unsafe thermal behavior.

4.1.2 Thermal Composability

Using thermal analysis, unsafe mappings can be identified in which the density of power consumption results in the violation of the critical temperature threshold of the chip. However, the sheer exclusion of such mappings does not guarantee thermal safety: When applications are executed concurrently, the *heat transfer* between the neighboring regions of the chip used by different applications can change the observed temperature distribution within the region used by each application. As a result, thermal violations may occur within the regions used by one or multiple running applications, even though the mapping used by each of the applications has been verified to have a thermally safe behavior in solitude.

Such a situation is exemplified in Figure 4.2 where a new application is to be launched in a system where two other applications are already running, see Figure 4.2a. The initial thermal state of the chip prior to the launch of the new application, illustrated in Figure 4.2b, gives evidence that the two deployed applications are running thermally safe. Note here also the temperature induced by the running applications on the region to be used by the new application. Now, subject to the thermal profile of the mapping considered for launching the new application, two possible thermal scenarios may



(a) many-core platform

(b) initial state (safe) (c) scenario 1 (safe) (d) scenario 2 (unsafe)

Figure 4.2: Example of possible thermal scenarios consequent to the concurrent execution of applications in a many-core system. Due to heat transfer between neighboring regions of the chip, launching a new application may indeed cause thermal violations (marked cores in Figure 4.2d) even though each of the involved mappings is verified to be thermally safe in solitude.

arise: (i) The system may remain in a thermally safe state after launching the new application, see Figure 4.2c. Here, the thermal behavior of the new mapping leaves a temperature headroom large enough for the heat induced by the other applications on the regions used by the new application. Likewise, the already running applications also have a large enough temperature headroom for the heat induced by the new application on their respective regions such that the system remains in a thermally safe state after launching the new application. However, (ii) launching the new application may also lead to thermal violations in the system if the heat induced by the concurrent applications on the region used by an application exceeds the temperature headroom of the respective region, see Figure 4.2d. Hence, even though each mapping is verified to be thermally safe in solitude, the combination of active mappings in the system may still cause thermal violations, see the overheated (marked) cores in Figure 4.2d.

In the context of HAM, to prevent such thermal violation scenarios, not only (i) each and every mapping must be verified to have a thermally safe behavior but also (ii) *thermal composability* must be established in a many-core system. Thermal composability ensures that none of the applications in the system is exposed to thermal violations induced due to the presence or the thermal activity of other applications in the system. Only through thermal composability, it can be ensured that the timing

guarantees of real-time applications are not at risk due to unforeseeable interferences from DTM countermeasures triggered in reaction to thermal violations caused by other running applications. State-of-the-art HAM methodologies do not consider thermal composability in their mapping and management processes for real-time applications and, hence, indeed fail in providing safe real-time guarantees.

Establishing thermal composability in a system, however, is challenging since it is generally not feasible to control the heat transfer between neighboring regions on a chip. In this scope, *admission control mechanisms* are required to preserve thermal composability by verifying the post-launch thermal safety of the system prior to launching new applications. Such mechanisms rely on a worst-case thermal analysis which takes the actual distribution of the applications in the system and their thermal profile into account to provide worst-case thermal-safety guarantees, similarly to the thermal-safety analysis of a mapping. This, however, necessitates the thermal analysis to be performed online (at the launch time of applications) when the set of running applications and their deployment of the chip are known. However, performing thermal analysis online is not practical due to its high computational overhead.

Contribution Overview

This work presents a *thermally composable HAM methodology* which enforces thermal safety proactively at the launch time of an application *without* relying on an online worst-case thermal analysis of the system. The proposed methodology introduces a (i) *thermal-safety analysis* into the DSE in HAM and provides the RPM with a lightweight (ii) *thermal-safety admission check* to be used when launching applications. The thermal-safety analysis computes an abstract thermal-safety index for each mapping which expresses its thermal safety as a function of the number of simultaneously active cores in the system. At run time, the thermal-safety admission check uses the precomputed thermal-safety indices of mappings for a lightweight examination of the system's thermal safety when launching applications without involving a time-consuming worst-case thermal analysis of the system. Using the admission check, the RPM can identify and avoid mapping candidates which may lead to thermal violations for the given system load and, thereby, preserves thermal safety proactively and establishes thermal composability. As a result, *without* jeopardizing the system's thermal integrity, the exposure of real-time applications to DTM countermeasures is eliminated and, hence, real-time guarantees remain intact.

4.2 Thermally Composable Hybrid Application Mapping

This section presents the proposed thermally composable HAM methodology. Figure 4.3 provides an overview of the proposed methodology where the novel compo-

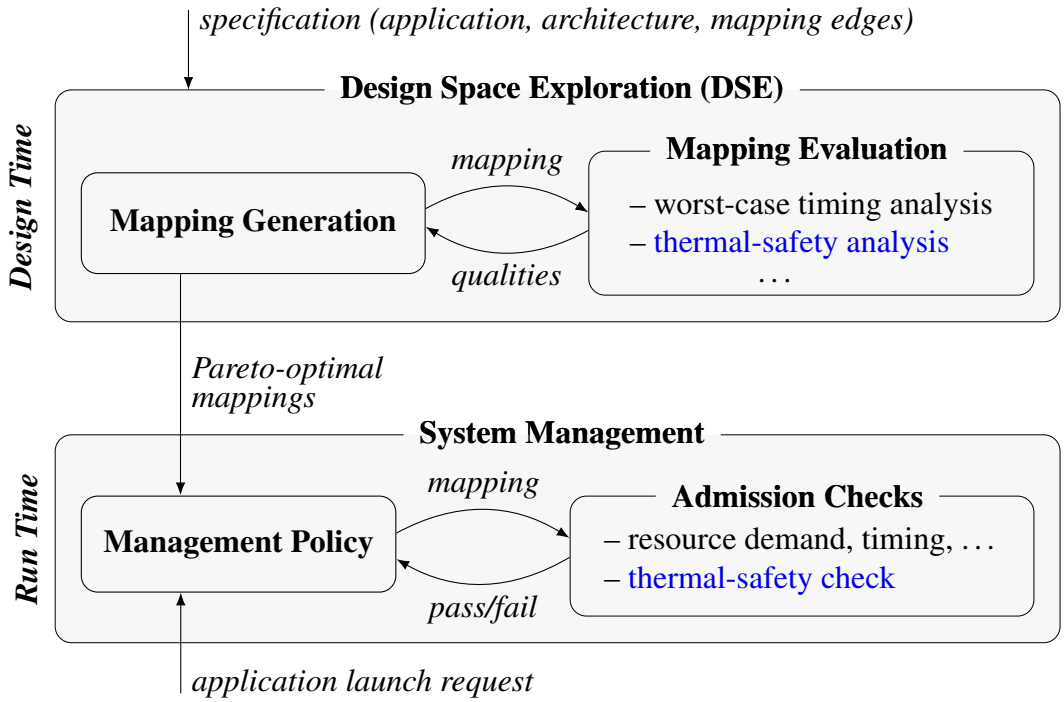


Figure 4.3: An overview of the proposed thermally composable Hybrid Application Mapping (HAM) methodology. The contributions are highlighted in blue.

nents and steps are highlighted in blue and outlined in the following. The proposed approach introduces an offline (i) *thermal-safety analysis* and an online (ii) *thermal-safety admission check* into the standard flow of HAM. The offline thermal-safety analysis investigates the thermal behavior of each generated mapping. It (i) verifies the thermal safety of each mapping in solitude and (ii) derives an abstract thermal-safety index for each mapping, termed *Thermally Safe Utilization (TSU)*. The TSU of a mapping expresses the thermal safety of that mapping as a function of the number of simultaneously active cores in the system. By incorporating the thermal-safety analysis as an evaluator into the offline DSE (in addition to worst-case timing analysis), a *thermal-aware DSE* scheme is established in which not only the satisfaction of real-time requirements is verified, but also mappings are optimized w.r.t. their thermal safety so that Pareto-optimal mappings with desirable thermal behavior are obtained. At run time, when launching an application, the RPM uses the proposed thermal-safety admission check to examine the thermal safety of each mapping candidate for the current system load based on the precomputed TSU of mappings without involving a time-consuming worst-case thermal analysis.

In the following, first, the proposed offline thermal-safety analysis is presented in Section 4.2.1. Then, the online thermal-safety admission check is presented in Section 4.2.2. Finally, the proposed thermally composable HAM methodology is pre-

sented in Section 4.2.3 which incorporates the thermal-safety analysis and admission check into the offline and online phases of HAM, respectively, to guarantee the thermal safety of the system proactively and, thereby, establish thermal composability.

4.2.1 Thermal-Safety Analysis of Mappings

The proposed offline thermal-safety analysis derives an abstract thermal-safety index, namely, *Thermally Safe Utilization (TSU)*, for each mapping by means of worst-case thermal analysis. The TSU of each mapping not only can be used to examine the thermal safety of that mapping in solitude but also facilitates the online thermal-safety admission check. In fact, it enables the RPM to examine the thermal safety of the system based on the TSU of active mappings and the number of cores that are simultaneously active in the system.

Thermal analysis of mappings and systems can be performed based on temperature itself or based on power consumption and power density as the main cause of heat generation in the system. In the context of HAM, the worst-case thermal-safety analysis of mappings based on a constraint on temperature itself is not practical. That is because, in a thermal analysis based on a temperature constraint, the temperature of each core can be derived only if global knowledge about the thermal activity of all cores in the system is available to evaluate the heat transfer between cores [HGV+06]. In HAM, however, the information about the system load will be known only at run time when all applications running in the system are known and, hence, is not available offline at the time of thermal analysis of individual mappings.

In contrast to temperature, the density of power consumption in each mapping can be derived offline since it merely depends on the execution profile and activity of the application on the allocated resources in that mapping. Therefore, the work at hand uses the thermal analysis approach presented in [KPS+17; PKC+17] which derives a constraint on the *density of power consumption* in the chip. This so-called *thermally safe power-density* constraint is guaranteed to enforce thermal safety w.r.t. a given critical temperature threshold. It is derived based on (i) the thermal model of the chip (including its floorplan, RC thermal network, cooling subsystem, etc.) and (ii) the *system utilization* denoting the number of simultaneously active cores in the system. In its model, the thermal analysis from [KPS+17; PKC+17] also takes into account the heat transfer between the cores on the chip, and its power-density constraint is verified to guarantee thermal safety as demonstrated in [PKC+17].

In the proposed thermal-safety analysis of mappings, the worst-case thermal analysis from [KPS+17; PKC+17] is used to derive a thermally safe power-density constraint for each system utilization level (number of active cores). Using the power-density constraints derived for different system utilization levels, a so-called *Thermally Safe Power-density Dictionary (TSPD)* is constructed as a thermal-safety reference for the chip. The TSPD is then used for the thermal-safety analysis of each mapping at design time to derive an abstract thermal-safety index (TSU) for that mapping based on the

peak power density observed on the cores allocated by that mapping. In the following, first, the computation of TSPD is elucidated. Then, the calculation of the TSU of mappings based on TSPD is detailed.

Thermally Safe Power-Density Dictionary (TSPD)

For the thermal-safety analysis of mappings, the worst-case thermal analysis presented in [KPS+17; PKC+17] is used to calculate a thermal-safety reference for the chip based on power density, referred to as the *Thermally Safe Power-density Dictionary (TSPD)* which is defined as:

$$TSPD = \{\langle n, PD\rangle \mid n = 1,2,\ldots,|C|\ ,\ PD \geq 0\} \tag{4.1}$$

Each entry $\langle n, PD\rangle$ of the TSPD corresponds to the case of having n cores simultaneously active in the system and provides a power-density constraint PD that guarantees thermal safety of the system as long as the density of power consumption on each and every one of the n active cores does not exceed PD. In Eq. (4.1), $|C|$ denotes the total number of cores on the platform. The TSPD is computed once for a many-core chip based on its thermal model (including the floorplan, RC thermal network, etc.) and is then used for the thermal-safety analysis of any mapping generated for that platform.

For the calculation of the $\langle n, PD\rangle$ entries of the TSPD, the thermal-safe power-density analysis from [KPS+17; PKC+17] is used. For each $n = 1,2,\ldots,|C|$, this analysis can identify the *worst possible* selection of n simultaneously active cores which results in the highest possible temperature in the system. For this selection of n active cores, the analysis derives a power-density constraint PD that ensures thermal safety. Here, $\langle n, PD\rangle$ constitutes one entry of the TSPD. Since PD is derived based on the *worst possible* selection of n simultaneously active cores, it is guaranteed to ensure thermal safety for *any* selection of n simultaneously active cores in the system, i.e., n active cores from *any* core type and at *any* location on the chip. This enables abstracting away from the type and the location of active cores on the chip during the calculation of the TSPD which considerably facilitates the TSU analysis of mappings (at design time) and the thermal-safety admission check (at run time).

To calculate the TSPD, first, we derive the thermal model of the chip and its cooling subsystem using the HotSpot simulator [HGV+06]. This information serves as input for the analysis from [KPS+17; PKC+17]. Using this analysis, we derive PD for each possible number $n = 1,2,\ldots,|C|$ of simultaneously active cores in the system and, thereby, construct all $\langle n, PD\rangle$ entries of the TSPD. Figure 4.4 visualizes the TSPD calculated for a 52-core many-core chip (which will be used later in this chapter for the experimental evaluations). The illustrated TSPD denotes that, for instance, for a total of $n = 16$ simultaneously active cores in the system, the thermal safety of the system is guaranteed as long as the density of power consumption per active core does not exceed $PD = 1.49\,\mathrm{W/mm^2}$. After all entries of the TSPD are calculated, it will be used for the thermal-safety analysis of mappings as presented in the following.

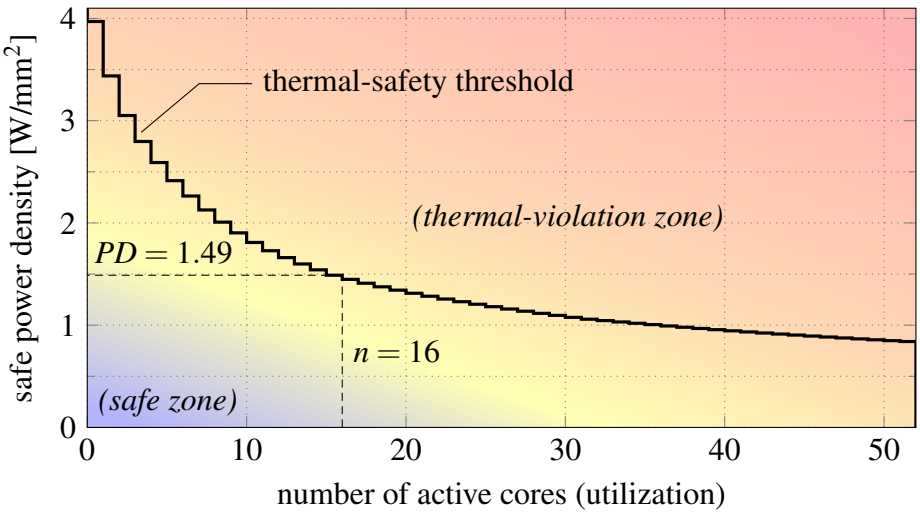


Figure 4.4: Visualization of the Thermally Safe Power-density Dictionary (TSPD) derived for a 52-core many-core chip. For each given number n of simultaneously active cores in the system, TSPD provides a power-density constraint PD that guarantees thermal safety as long as the density of power consumption by each active cores does not exceed PD.

Thermally Safe Utilization (TSU) Analysis

The TSPD is used in the offline thermal-safety analysis of mappings to determine, for each mapping μ, the maximum system utilization for which the presence of μ in the system is guaranteed not to cause any thermal violations. This utilization constraint is referred to as the *Thermally Safe Utilization (TSU)* of that mapping, denoted as $TSU(\mu)$ for mapping μ. Here, the system utilization denotes the total number of cores (regardless of their type and on-chip locations) which are simultaneously active in the system, i.e., are in use by the mapping in question or any other currently running applications in the system. The TSU of mappings provides all the necessary information for a lightweight thermal-safety evaluation of the system at run time.

For the calculation of $TSU(\mu)$, first, the peak density of power consumption among all cores allocated by μ is determined as:

$$PD_{\max}(\mu) = \max\left\{PD(c) \mid c \in C_\mu\right\} \tag{4.2}$$

where, $PD(c)$ denotes the peak density of power consumption on core c, and $C_\mu \subseteq C$ denotes the set of cores allocated by mapping μ. To calculate $PD(c)$ for each allocated core $c \in C_\mu$, first, Eq. (4.3) is used to determine the peak power consumption $p(c)$ of core c. The first summand in Eq. (4.3) denotes the dynamic power consumption of c calculated based on c's switching activity $a_c \in [0,1]$, effective capacitance $\hat{c}_c$, supply voltage V_c^{dd}, and operating frequency f_c. We consider $a_c = 1$ to account for

the worst-case switching activity. The second summand in Eq. (4.3) gives the static power consumption of c calculated based on c's leakage current I_{q_c} and supply voltage V_c^{dd}. Once $p(c)$ is determined, Eq. (4.4) is used to derive c's peak density of power consumption $PD(c)$ where A_c denotes the area of core c.

$$p(c) = a_c \cdot \hat{c}_c \cdot {V_c^{\mathrm{dd}}}^2 \cdot f_c \; + \; I_{q_c} \cdot V_c^{\mathrm{dd}} \tag{4.3}$$

$$PD(c) = p(c) / A_c \tag{4.4}$$

After $PD_{\max}(\mu)$ is derived, we iterate through the entries $\langle n, PD \rangle$ of the TSPD to identify the largest number n of simultaneously active cores for which $PD_{\max}(\mu)$ is guaranteed to be thermally safe, i.e., it does not exceed the thermally safe power-density constraint PD of that entry ($PD_{\max}(\mu) \leq PD$). This number n denotes the Thermally Safe Utilization (TSU) of mapping μ, derived using Eq. (4.5). For a mapping μ, $TSU(\mu)$ denotes the maximum number of cores (from any type and at any location on the chip) that may be simultaneously active in the system for which the presence of mapping μ—with a peak power density of $PD_{\max}(\mu)$—is guaranteed not to cause any thermal violations.

$$TSU(\mu) = \max \{ n \mid \langle n, PD \rangle \in TSPD \wedge PD_{\max}(\mu) \leq PD \} \tag{4.5}$$

Note that, to guarantee thermal safety, a mapping μ shall never be used if the total number of active cores in the system exceeds $TSU(\mu)$. Thus, a mapping μ with $TSU(\mu) < |C_\mu|$ shall *never* be used: Since the number of cores allocated by it ($|C_\mu|$) exceeds the maximum number of simultaneously active cores for which it is guaranteed to be thermally safe ($TSU(\mu)$), mapping μ may cause thermal violations even in isolation, i.e., in the absence of any other applications in the system. Thus, any mapping μ with $TSU(\mu) < |C_\mu|$ may be discarded already at design time as it has no practical use. In fact, the proposed online thermal-safety admission check ensures that such mappings are never admitted into the system as presented in the following.

4.2.2 Thermal-Safety Admission Check

The thermal-safety analysis presented above is used offline to derive the TSU of each mapping. The TSU of mappings provides all the information that is necessary to realize a lightweight online thermal-safety evaluation of the system. To preserve thermal composability in the system, the thermal safety of the system must be enforced *proactively* at all times such that DTM countermeasures are never triggered. To that end, when launching an application at run time, each mapping candidate must be examined to ensure that its introduction into the system is guaranteed not to cause any thermal violations. That means, it must be ensured that, after the introduction of the mapping candidate in question into the system, the resulting system load is thermally tolerable for each and every running application in the system.

We present a lightweight thermal-safety admission check that can be used by the RPM to evaluate the thermal safety of a mapping candidate for a given system load based on the precomputed TSU of mappings and the number of active cores in the system. Consider a situation in which a new application is to be launched in a system where a set of applications is already deployed and running in a thermally safe state. Let Γ denote the set of mappings that are in use by already running applications. For a mapping candidate μ of the new application, the thermal safety of the system is guaranteed to be preserved after the introduction of μ into the system if the following inequality holds:

$$|C_\mu| + \sum_{\mu' \in \Gamma} |C_{\mu'}| \leq \min \left\{ TSU(\mu), \min_{\mu' \in \Gamma} TSU(\mu') \right\} \tag{4.6}$$

Eq. (4.6) holds only if the total number of active cores after the introduction of the new mapping into the system (i.e., $|C_\mu| + \sum_{\mu' \in \Gamma} |C_{\mu'}|$) does not exceed the TSU of any of the mappings in the system. Any mapping candidate of the new application which does not pass the check above will be discarded by the RPM, as it is not guaranteed to be thermally safe for the given system load. Note that the check above necessarily guarantees *thermal composability* since it enforces thermal safety proactively by ensuring that an admitted mapping will never cause a thermal violation and, hence, will never affect the other applications in the system due to its thermal behavior.

4.2.3 Thermal-Aware Design Space Exploration (DSE)

A thermally composable HAM methodology can be established by (i) applying the proposed thermal-safety analysis for the mappings obtained from the DSE at design time to compute their TSU and (ii) incorporating the proposed thermal-safety admission check into the mapping examination process of the RPM at run time. This enables existing HAM methodologies to adopt the proposed thermal-safety analysis and admission check to achieve thermal composability *without* the need for any changes in their offline application characterization and DSE.

As will be experimentally demonstrated later in Section 4.3, subject to the design objectives applied during the DSE, the obtained mappings may exhibit considerably low TSUs. Mappings with a low TSU have a lower likelihood of passing the thermal-safety admission check at run time, especially as the system load increases. This can rapidly give rise to system scenarios in which an application cannot be launched at all because none of its mapping candidates passes the thermal-safety admission check, even though their required resources are available. In fact, the inability to launch an application using any of its mappings is a classical concern in HAM for which several remedies have been reported. For instance, one remedy which is extensively adopted by the state-of-the-art HAM methodologies is to *diversify the resource demand* of the mappings and, thereby, improve their fitness for various run-time resource availability

scenarios [WWG+18a]. Such techniques, however, improve the embeddability of applications only w.r.t. the required resources, but not thermal safety.

As a remedy, we propose to employ the thermal-safety analysis *during* the offline DSE to guide the mapping optimizer towards mappings with desirable thermal behavior. To that end, a quantitative indicator of thermal safety must be used as a design objective to render thermal safety visible to the mapping optimizer. For this purpose, the TSU of mappings can be used as the thermal-safety design objective to maximize the total number of cores that may be active in the presence of each mapping without any thermal violations. Nonetheless, a design objective in the form of $TSU(\mu) - |C_\mu|$ provides a better indication of thermal safety for a mapping μ as it aims at maximizing the total number of cores that mapping μ allows other applications to use. In the following, the advantage of $TSU(\mu) - |C_\mu|$ over $TSU(\mu)$ as the thermal-safety design objective is elucidated using an example.

Consider two mappings of an application: mapping μ_1 with $TSU(\mu_1) = 25$ and $|C_{\mu_1}| = 20$, and mapping μ_2 with $TSU(\mu_2) = 25$ and $|C_{\mu_2}| = 10$. By considering TSU as a design objective to denote thermal safety, both mappings exhibit the same quality since $TSU(\mu_1) = TSU(\mu_2) = 25$. However, an investigation of the number of cores allocated by each mapping and its comparison with the TSU of that mapping provides a notable insight: Although both mappings μ_1 and μ_2 share the same maximum system utilization of 25 active cores for which they are thermally safe, μ_2 allows a much larger number of $TSU(\mu_2) - |C_{\mu_2}| = 15$ active cores to be used by other applications as it allocates considerably fewer cores itself, compared to mapping μ_1 which allows only $TSU(\mu_1) - |C_{\mu_1}| = 5$ cores to be used by other applications. Now, consider the scenario in which the application shall be launched on the system where another application is launched beforehand using a mapping μ' with $|C_{\mu'}| = 10$ allocated cores and $TSU(\mu') = 30$. In this setting, mapping μ_1 cannot be admitted as $(|C_{\mu_1}| + |C_{\mu'}|) \not\leq \min\{TSU(\mu_1), TSU(\mu')\}$ (admission check from Eq. (4.6)), while mapping μ_2 is admissible as $(|C_{\mu_2}| + |C_{\mu'}|) \leq \min\{TSU(\mu_2), TSU(\mu')\}$. Therefore, mapping μ_2 indeed has a better thermal quality even though both μ_1 and μ_2 share the same TSU of $TSU(\mu_1) = TSU(\mu_2) = 25$.

By considering $TSU(\mu) - |C_\mu|$ as the thermal-safety design objective (to be maximized), the mapping optimizer will be able to find mappings that have a higher likelihood of passing the online thermal-safety admission check as they are optimized to tolerate the highest possible system load contributed by other applications. In the example above, $TSU(\mu) - |C_\mu|$ will evaluate to 5 and 15 for mappings μ_1 and μ_2, respectively, which denotes a higher quality for mapping μ_2 w.r.t. thermal safety as also concluded in the discussion above. Therefore, this work proposes to consider $TSU(\mu) - |C_\mu|$ as the thermal-safety design objective during the DSE. By incorporating this new design objective into the DSE, the mapping optimizer is guided towards those regions of the mapping search space that contain mapping candidates of superior thermal safety. Note that this does not compromise the other qualities/guarantees of interest, e.g., real-time guarantees, since the design objectives corresponding to those

qualities remain unaffected. After the DSE is finished, the set of collected mappings with Pareto-optimal qualities w.r.t. both the *original objectives* and the new *thermal-safety objective* is provided to the RPM for online use. As part of the following experiments, the effectiveness of this *thermal-aware DSE* approach in improving the application's embeddability w.r.t. the thermal-safety admission check is demonstrated.

4.3 Experimental Results

This section presents the results of a series of experiments that evaluate the effectiveness and the efficiency of the proposed methodology in enforcing thermal safety proactively and establishing thermal composability in support of applications with hard real-time requirements.

4.3.1 Experiments Setup

Architectures

For the experiments, two heterogeneous many-core architectures are considered, each composed of three types of processing cores, namely, IBM PowerPC, AMD K6-III, and AMD K6-2, for which the performance and circuit-level specifications for a 180 nm technology node size is provided by Embedded System Synthesis Benchmarks Suite (E3S) [Dic10]. We adapt these processing core characteristics to a 10 nm technology node using the technology scaling factors provided in [HGV+06] which are derived based on [ITR15; BBJ14]. The adapted core characteristics are provided in Table 4.1. The static power consumption of each core is set so as to contribute 20% of the total power consumption of the core under the nominal operating frequency and supply voltage [EBA+11]. The maximum power consumption of each Network-on-Chip (NoC) router and each Network Adapter (NA) is considered to be 250 mW.

Given the core specifications above, an 18 mm $\times$ 14.7 mm many-core chip floorplan is developed which comprises 52 cores entailing 16 IBM PowerPC, 16 AMD K6-III,

Table 4.1: Performance and circuit-level specifications of the processing cores in the experimental floorplan adapted to a 10 nm process technology node size.

core type	core specifications					
	area [mm^2]	frequency [GHz]	voltage [V]	capacitance [nF]	dyn. power [W]	st. power [W]
AMD K6-III	5.6	4.2	1.1	1.73	8.8	2.2
AMD K6-2	3.4	3.0	1.2	1.23	5.3	1.3
IBM PowerPC	4.4	2.0	1.4	0.29	1.1	0.3

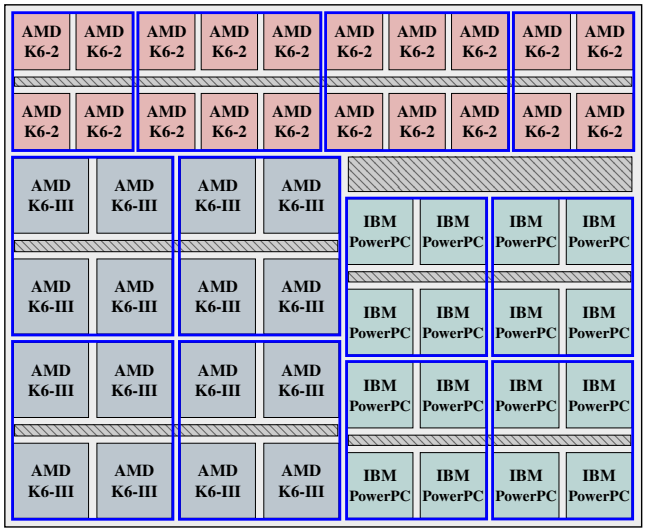


Figure 4.5: 52-core heterogeneous tiled many-core floorplan used for developing the experimental architectures. Blue box denote tiles [PSK+19].

and 20 AMD K6-2 cores, depicted in Figure 4.5. Using this floorplan, two many-core architectures are developed, representing the two main classes of tiled many-core architectures, i.e., architectures with (i) single-core and (ii) multi-core tiles. In the former, the 52-core floorplan is organized in 52 single-core tiles interconnected by a NoC. In the latter, the 52-core floorplan is organized in 12 multi-core tiles, each comprising 4–6 homogeneous cores, denoted by the blue boxes in Figure 4.5. This partitioning results in 4 multi-core tiles per core type interconnected by a 4×3 NoC.

For the floorplan, the TSPD is calculated as detailed in Section 4.2.1 using the analysis from [KPS+17; PKC+17] and based on a model of the floorplan's RC thermal network. The thermal model of the floorplan is derived by considering one thermal block per core (each of the 52 labeled rectangles in Figure 4.5) and several thermal blocks for shared memories, NAs, and routers (gray hatched regions in Figure 4.5). To derive the RC thermal network, the HotSpot simulator [HGV+06] is used with its default configuration as follows: chip thickness of 0.15 mm, silicon thermal conductivity of $100\,\mathrm{W/m \cdot K}$ and specific heat of $1.75 \cdot 10^6\,\mathrm{J/m^3 \cdot K}$, a heat sink of dimensions 6×6 cm and thickness of 6.9 mm, heat sink convection capacitance of $140.4\,\mathrm{J/K}$ and convection resistance of $0.1\,\mathrm{K/W}$, heat sink and heat spreader thermal conductivity of $400\,\mathrm{W/m \cdot K}$ and specific heat of $3.55 \cdot 10^6\,\mathrm{J/m^3 \cdot K}$, a heat spreader of dimensions 3×3 cm and thickness of 1 mm, interface material thickness of 20 um, interface material thermal conductivity of $4\,\mathrm{W/m \cdot K}$ and specific heat of $4 \cdot 10^6\,\mathrm{J/m^3 \cdot K}$.

Applications

For the experiments, four hard real-time applications are considered from the embedded domains of networking (7 tasks, 9 messages), consumer (11 tasks, 12 messages), telecommunications (14 tasks, 20 messages), and automotive (18 tasks, 21 messages). The applications are provided by E3S [Dic10]. For each application, E3S [Dic10] also provides the Worst-Case Execution Time (WCET) of each task on each core type when running on the nominal operating frequency for a 180 nm technology node size. To derive the 10 nm-technology-adapted WCET of each task $t \in T$ on each core $c \in C$, Eq. (4.7) is used in which the numerator provides the technology-independent number of core clock cycles required for the execution of task t on core c in the worst case. The denominator translates this value to the WCET for a 10 nm technology size.

$$WCET_{t,c}|_{10\text{nm}} = \frac{WCET_{t,c}|_{180\text{nm}} \cdot f_c|_{180\text{nm}}}{f_c|_{10\text{nm}}} \tag{4.7}$$

Design Objectives

For the offline DSE, in addition to thermal safety, two common types of optimization design objectives are considered to assess the quality of each generated mapping: (i) *resource cost* which is evaluated as the number of cores allocated per core type, and (ii) *real-time conformity* which is assessed w.r.t. two hard real-time constraints: a strict latency constraint for each task and message and a strict end-to-end latency (makespan) constraint for the whole application. The timing verification of each mapping is realized using the analysis presented in Section 3.3. For the optimization of (iii) *thermal safety* (for the thermal-aware DSE), the $(TSU(\mu) - |C_\mu|)$ of each mapping μ is considered as a design objective to be maximized. Here, $|C_\mu|$ denotes the number of cores allocated by mapping μ, and $TSU(\mu)$ denotes the TSU of mapping μ derived using the thermal-safety analysis presented in Section 4.2.1.

The timing analysis is also coupled with a power-minimization heuristic. For each given mapping, this heuristic determines a frequency assignment for each tile used by that mapping such that the peak power density of the mapping is minimized while the real-time requirements of the application are satisfied[2]. Hence, this heuristic already improves the thermal behavior of the mappings without adversely affecting their quality w.r.t. any of the design objectives, namely, resource demand and real-time conformity. For DVFS, five discrete frequency configurations are considered per tile, namely, $[0.2, 0.4, 0.6, 0.8, 1.0] \times f$, where f denotes the nominal frequency of the respective core type of that tile provided in Table 4.1. The voltage of each tile is scaled at the same rate as its frequency.

[2]Note that tile-level DVFS is a common scheme in many-core architectures, see, e.g., [Sam19]. It allows individual voltage-frequency assignment per tile. The cores located on one tile, however, operate on the same voltage-frequency configuration.

Design Space Exploration

For the optimization of mappings, the OpenDSE framework [RLG+18] and the NSGA-II [DPA+02] multi-objective evolutionary algorithm provided by the optimization framework OPT4J [LGR+11] are used. The DSE is configured to perform 1,000 iterations, generate 25 new mappings per iteration by applying genetic variation operators (crossover and mutation) on the mappings in the population, and retain and update a population of 100 mappings. During the optimization, a set of Pareto-optimal mappings is collected which will be the output of the optimization.

4.3.2 Results Discussion

The effectiveness and the efficiency of the proposed thermally composable HAM methodology are investigated experimentally in three aspects: (i) the degree of the pessimism borne in the TSPD to provide *worst-case* thermal guarantees, (ii) the effectiveness of the proposed thermal-safety analysis of mappings (offline) and thermal-safety admission check (online) in enforcing thermal safety proactively and establishing thermal composability, and (iii) the impact of the proposed thermal-aware DSE (in which thermal safety is considered as a design objective) on the thermal behavior of mappings in comparison to a thermal-agnostic DSE (in which thermal safety of mappings is evaluated *after* the optimization).

TSPD Pessimism

The offline thermal-safety analysis of mappings relies on the TSPD which provides, for each given number n of active cores in the system *regardless of their type and on-chip location*, a power density constraint PD that guarantees thermal safety if respected by each of the n active cores, see Section 4.2.1. To *guarantee* thermal safety, PD is calculated based on the *worst possible* selection of n simultaneously active cores which results in the highest observable temperature. Therefore, PD bears some degree of pessimism as it may be too conservative for some other selections of n active cores with a better temperature balance across the chip. This pessimism manifests itself as a system utilization penalty, i.e., a reduction in the maximum number of simultaneously active cores that are considered thermally safe for the given per-core power-density constraint.

In the first experiment, we quantify (i) the pessimism borne in the thermally safe power densities recorded in the TSPD and (ii) its manifestation as system utilization penalty. For this purpose, Monte Carlo simulation is performed in which, for each system utilization level, i.e., number $n \in \{1,\ldots,52\}$ of active cores, $5,000$ sample utilization scenarios are generated, each with n randomly selected active cores. For each sample scenario, first, the thermally safe power density pd for the *given* selection of active cores is determined using the analysis from [KPS+17; PKC+17]. Then, for

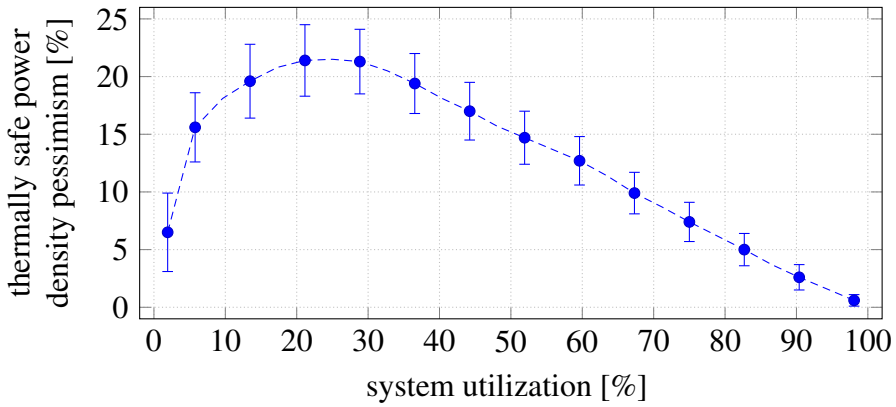


Figure 4.6: The average (circles) and standard deviation (whiskers) of the pessimism in the thermally safe power density vs. system utilization level [PSK+19].

that sample, the following values are derived to quantify the pessimism in TSPD:

- The pessimism in the thermally safe power density in the TSPD, quantified as the gap $(pd - PD)$ between the actual thermally safe power density pd for that sample scenario and the conservative thermally safe power density PD corresponding to its number n of active cores extracted from the TSPD.
- The system utilization penalty consequent to the pessimism in TSPD, quantified as the difference between the maximum number of active cores considered thermally safe (i) by the actual safe power density pd of that sample scenario and by (ii) the conservative thermally safe power density PD from the TSPD.

The distribution of the pessimism in the thermally safe power densities in the TSPD (i.e., the gap $(pd - PD)$) among the samples generated for different system utilization levels is provided in Figure 4.6. The circles in Figure 4.6 denote the average pessimism, and the whiskers denote the standard deviation of pessimism. The highest degree of pessimism is recorded at system utilization level 23% (corresponding to 12 active cores out of the 52 cores of the architecture) for which an average power density pessimism of 22% is observed. Taking all utilization levels into account, the TSPD exhibits a power density pessimism of 13% on average with an average standard deviation of 2%.

The distribution of the system utilization penalty consequent to the power density pessimism is provided in Figure 4.7 for the samples generated for different system utilization levels. Also here, the circles denote the average penalty, and the whiskers denote the standard deviation of penalty. The highest utilization penalty is assessed at a system utilization level of 46% (corresponding to 22 active cores) for which an average utilization penalty of 16% (8 cores) is observed. Taking all utilization levels into account, the pessimism in TSPD manifests itself in an average system utilization penalty of 10% (equivalent to 5 cores) with a standard deviation of 1.7% ($<$1 core).

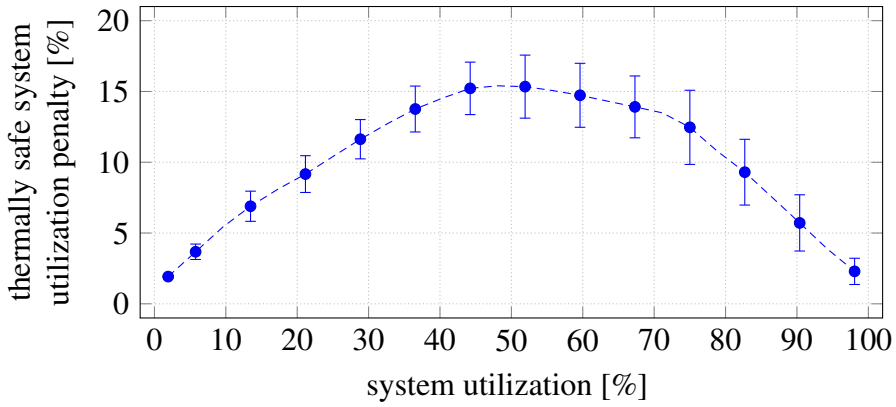


Figure 4.7: The average (circles) and the standard deviation (whiskers) of the utilization penalty consequent to the pessimism in the thermally safe power density vs. system utilization level [PSK+19].

Thermal Composability

In the next experiment, the effectiveness of the proposed offline thermal-safety analysis and online thermally-safety admission check for enforcing thermal safety and thermal composability is investigated in a comparison between two HAM methodologies:

- **SOTA-HAM** which does not consider thermal composability and, thus, represents the state-of-the-art practice of HAM where the mappings obtained from the offline DSE are directly provided to the RPM. When launching an application, the RPM checks only the availability of the required resources for each mapping candidate to determine whether that mapping candidate is embeddable w.r.t. the current system load.
- **Baseline TC-HAM** which represents a baseline thermally composable HAM approach. It extends the SOTA-HAM by (i) applying the proposed thermal-safety analysis *after* the DSE at design time to calculate the TSU for the obtained mappings before providing them to the RPM. Moreover, it (ii) incorporates the proposed thermal-safety admission check into the online mapping-examination process of the RPM to enforce thermal composability. Therefore, it represents the approach to render the SOTA-HAM approach thermally composable.

Based on the statically computed mappings, a run-time application embedding experiment is performed which compares the two HAM methodologies under investigation in terms of thermal safety and thermal composability. The run-time embedding experiment is performed separately for each benchmark application and many-core platform. Each run of the experiment starts with an empty platform (100% resource availability). During the experiment, platform tiles are occupied iteratively one by one and at random on their nominal operating frequency. The occupied resources emulate

the presence of other applications running in the system at the launch time of the application in question. At each iteration, we attempt to launch the application under investigation once (i) using the RPM in the SOTA-HAM approach which examines only the availability of the required resources for each mapping without regarding thermal safety, and once (ii) using the RPM in the baseline TC-HAM approach which examines both the availability of the required resources for each mapping and its admissibility w.r.t. thermal safety (verified using the thermal-safety admission check). For each approach, the number of embeddable mappings, i.e., mappings that pass the admission checks of each respective RPM is recorded. The iterations of preoccupying tiles and then examining the embeddability of mappings on the partially preoccupied platform are continued until all tiles are preoccupied. This experiment is repeated 50 times to incorporate diverse compositions of preoccupied resources for each level of system utilization. Moreover, the offline DSE and its subsequent 50 rounds of the run-time embedding experiment are repeated 50 times per application on each architecture to reinforce the evaluation. The results presented in the following represent an average over these runs.

Figure 4.8 illustrates the *embeddability rate* of the applications under each of the two investigated HAM methodologies for different system utilization levels (i.e., the percentage of occupied cores at the launch time of the application). The embeddability rate of an application denotes the proportion of its mappings that are considered as embeddable by the respective RPM. The plots in each row (column) correspond to one benchmark application (many-core architecture). Evidently, as the system utilization increases, the proportion of embeddable mappings for both approaches decreases until reaching 0%, denoting that the application cannot be launched using any of its mappings. While in both approaches, mappings for which the required resources are not available are rejected by the RPM, the thermal-safety admission check in the baseline TC-HAM approach also rejects those mappings for which the required resources are available but the mapping may lead to a thermal violation if it is added to the system. This results in a lower embeddability rate of the applications in the baseline TC-HAM approach compared to the SOTA-HAM approach. On average among all applications on the two architectures, the thermal-safety check in the baseline TC-HAM approach rejects up to 51% (with an average of 9%) of the mappings which are considered embeddable by the SOTA-HAM approach as their required resources are available.

To assess the capability of the baseline TC-HAM approach in establishing thermal composability, a subsequent thermal evaluation of the system is performed after launching the application using each of the mapping candidates that were admitted by either of the two HAM approaches. During this evaluation, we recorded the proportion of admitted mappings that indeed caused a thermal violation. The results uniformly verified that *none* of the mappings which passed the thermal-safety check of the baseline TC-HAM approach led to a thermal violation. This demonstrates the capability of the proposed thermal-safety analysis and thermal-safety admission

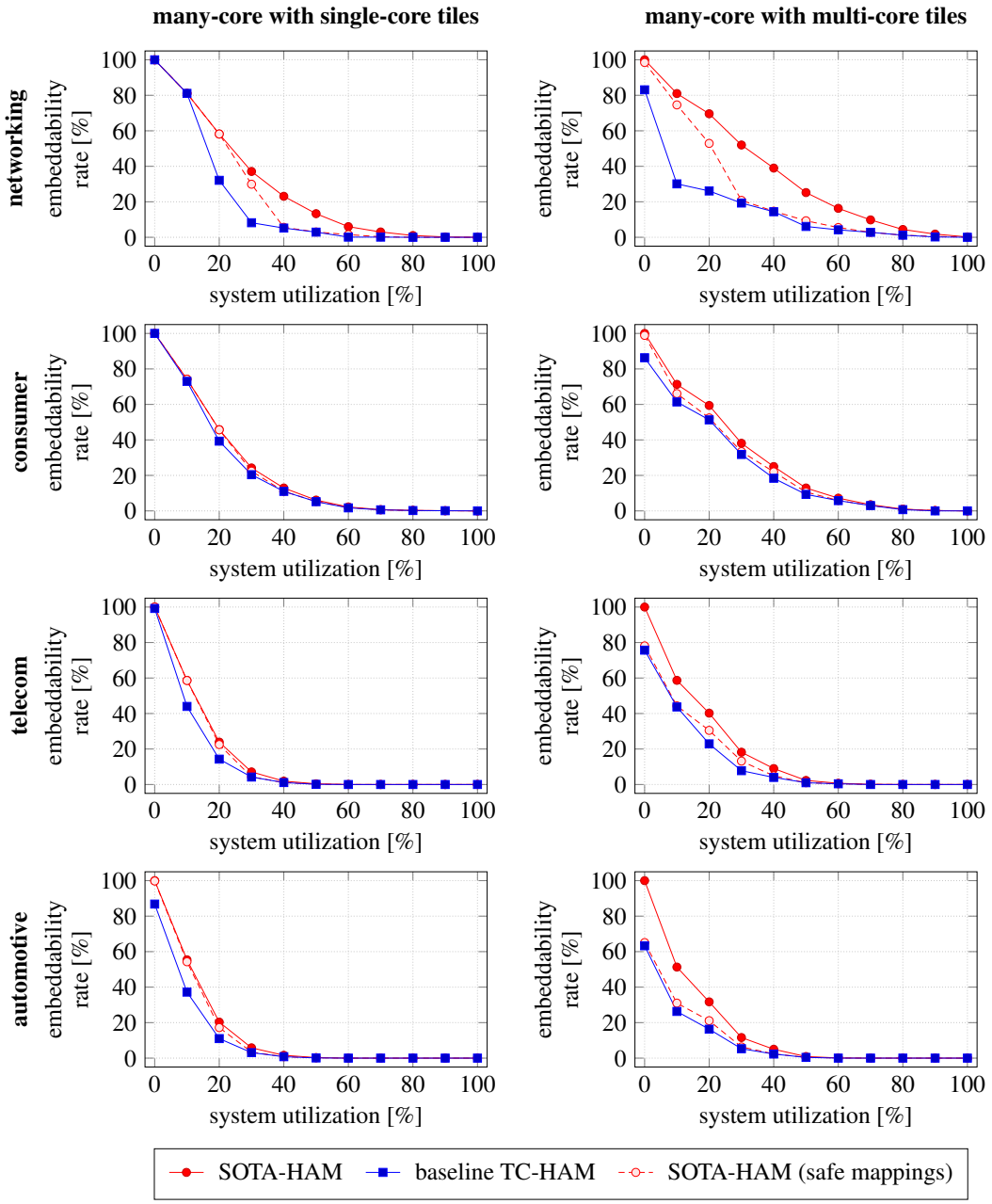


Figure 4.8: The proportion of mappings considered embeddable by the state-of-the-art HAM approaches (SOTA-HAM) which disregard thermal composability and a thermally composable HAM approach (baseline TC-HAM) which employs the proposed offline thermal-safety analysis and online thermal-safety admission checks to establish thermal composability [PSK+19]. Each row (column) of plots corresponds to one application (architecture).

check in establishing thermal composability successfully. However, in the case of the SOTA-HAM approach, a significant share of up to 100% (with an average of 30%) of the admitted mappings indeed caused a thermal violation after their introduction into the system. These violations were subsequently counteracted by the autonomous DTM countermeasures which set the frequency of the temperature-violating tile to the minimum value and, thereby, led to the violation of the real-time constraints of the application running on that tile. In Figure 4.8, the plots designated with dashed lines labeled as *SOTA-HAM (safe mappings)* provide the proportion of the mappings considered embeddable by the SOTA-HAM approach which indeed proved not to cause any thermal violations. Thus, the gap between the solid- and dashed red plots denotes the proportion of mappings admitted by the SOTA-HAM approach which caused thermal violations after their introduction into the system.

Figure 4.8 also provides another notable insight: A considerable share of mappings are not thermally safe, even for an empty platform (note the proportion of embeddable mappings at system utilization of 0%). Contrarily to the SOTA-HAM approach, the baseline TC-HAM approach identifies and rejects these mappings in its online admission check. The proportion of such mappings increases as the core count per tile increases (compare left and right plot columns) and/or as the application size grows (compare upper and lower plot rows). This occurs because the mappings generated for the larger applications or an architecture with a coarser granularity of tiles typically allocate a large number of cores—in some cases, over half of all cores in the system. This resource demand contributes significantly to the system utilization and, thus, is subject to a tighter power density constraint from the TSPD to ensure thermal safety. Many of the mappings, however, do not satisfy this constraint and, hence, are thermally unsafe even for an otherwise empty platform. Note that, for any such mapping μ, the inequality $TSU(\mu) < |C_\mu|$ holds.

Impact of Thermal-Aware DSE

The following experiment investigates the impact of the proposed thermal-aware DSE presented in Section 4.2.3 in enhancing the thermal safety of the mappings and, thereby, enhancing the likelihood of the mappings in passing the thermal-safety admission check at run time. To that end, we compare mappings obtained from (i) a *thermal-agnostic DSE* and (ii) the proposed *thermal-aware DSE* in terms of the maximum number of cores each mapping μ allows other concurrent applications to use without μ causing any thermal violations, i.e., $(TSU(\mu) - |C_\mu|)$, which we refer to as the *thermally safe background utilization* for mapping μ. In the thermal-agnostic DSE, the TSU of mappings is computed *after* the mapping optimization phase. Contrarily, in the proposed thermal-aware DSE, the TSU of mappings is evaluated *during* the optimization and the thermally safe background utilization $(TSU(\mu) - |C_\mu|)$ is regarded as an additional design objective to be maximized. The following results represent an average over 50 runs of each DSE approach per application on each many-core

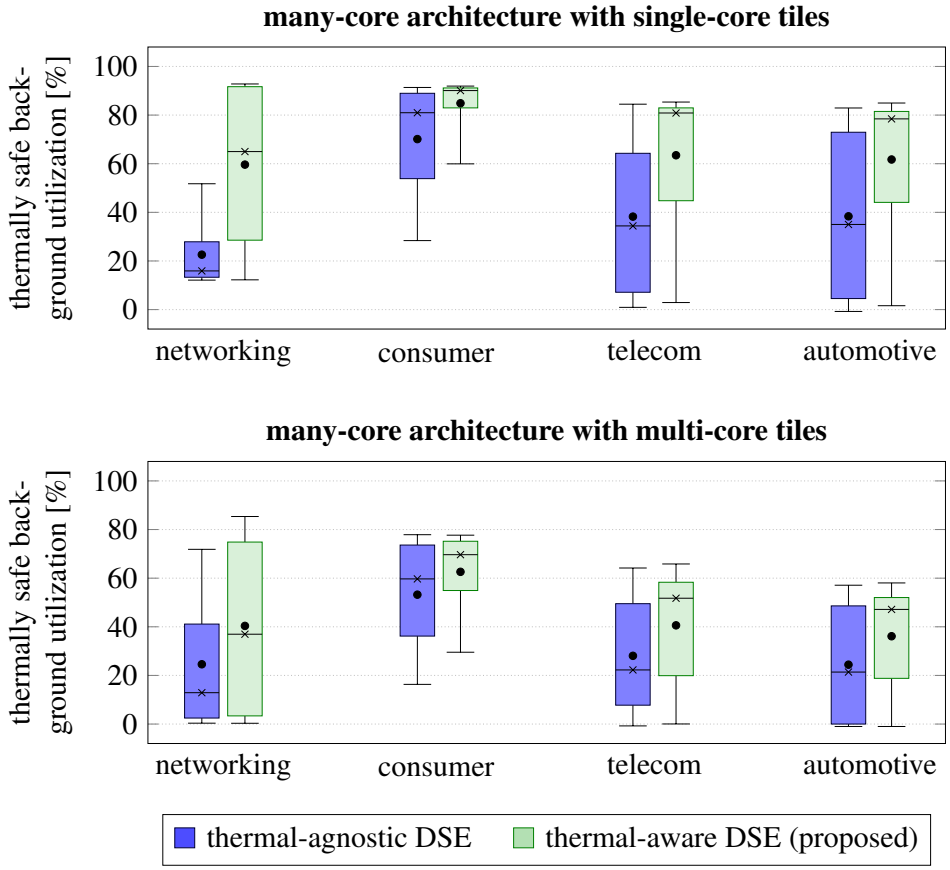


Figure 4.9: Distribution of the thermally safe background utilization $TSU(\mu) - |C_\mu|$ in the Pareto-optimal mappings μ obtained from a thermal-agnostic DSE and the proposed thermal-aware DSE approach [PSK+19].

architecture.

The distribution of thermally safe background utilization $(TSU(\mu) - |C_\mu|)$ for the Pareto-optimal mappings obtained from the two DSE approaches above are illustrated in Figure 4.9. In each box plot, the black circle denotes the average thermally safe background utilization among all mappings obtained by each DSE approach. The results exhibit that the proposed thermal-aware DSE improves the thermally safe background utilization of the mappings up to 37%, with an average improvement of 19% among all applications and architectures, compared to the thermal-agnostic DSE. This improvement enhances the likelihood of the mappings in passing the online thermal-safety admission check as discussed in Section 4.2.3. This alleviates the proportion of mappings that fail the online admission check and, in turn, improves the embeddability rate of the application.

To quantify the resulting improvement achieved in the embeddability rate of the applications, the run-time embedding experiment is repeated for two thermally com-

posable HAM approaches based on the two DSE approaches above: (i) *baseline TC-HAM* which employs the thermal-agnostic DSE, and (ii) *proposed TC-HAM* which employs the proposed thermal-aware DSE. Both approaches use the online thermal-safety admission check to establish thermal composability. For each approach, we recorded the proportion of mappings which are rejected by the thermal-safety admission check but indeed proved to be safe for the given set of active cores in the system. This measure denotes the *embeddability penalty* of each approach in establishing thermal composability. Figure 4.10 illustrates the embeddability penalty of the two investigated approaches. The results represent an average over 50 runs of the DSE per application on each architecture and 50 trials of the embedding experiment per DSE run. For the baseline TC-HAM approach, the embeddability penalty measures in Figure 4.10 corresponds to the gap between the blue and the dashed red plots in Figure 4.8. As illustrated in Figure 4.10, the enhancement in the thermal safety of mappings achieved by the proposed thermal-aware DSE considerably reduces the embeddability penalty of the proposed TC-HAM approach as it provides mappings that are guaranteed to be thermally safe for a higher background system utilization level (i.e., number of cores used by other applications). Among all applications on the two architectures, the proposed TC-HAM alleviates the embeddability penalty of the baseline TC-HAM approach up to 84% with an average of 21%.

4.4 Related Work

This section provides an overview of existing work on multi/many-core application mapping methodologies that focus on real-time and thermal aspects. In the following, the related works are categorized into the major classes of (i) static, (ii) dynamic, and (iii) hybrid mapping methodologies.

Static mapping methodologies find an optimal deployment of a given set of applications on a target architecture at design time. They benefit from a global view of the system and afford compute-intensive exploration and optimization techniques including complex verification and analyses, for instance, to ensure thermal safety as investigated by [UBE+12; DKV14; MMC+17; KAH18] or to verify the satisfaction of real-time constraints in a thermal-aware setting as investigated in [HM04; CHD10; WLC+14]. Nonetheless, all these approaches depend on a static view of the system and, hence, are not applicable in dynamic systems where the mix of concurrently executed applications may vary at run time and/or the set of all applications in the system is not known at design time [SDM+17].

Dynamic mapping methodologies can address the dynamism in the system effectively by providing an adaptive approach for many-core application mapping. Initial works in this category aimed at maximizing the system's performance irrespective of thermal constraints, e.g., [KBL+11; ATB+13; CMM+13]. In recent years, however, several thermal-aware dynamic mapping methodologies have emerged which enforce

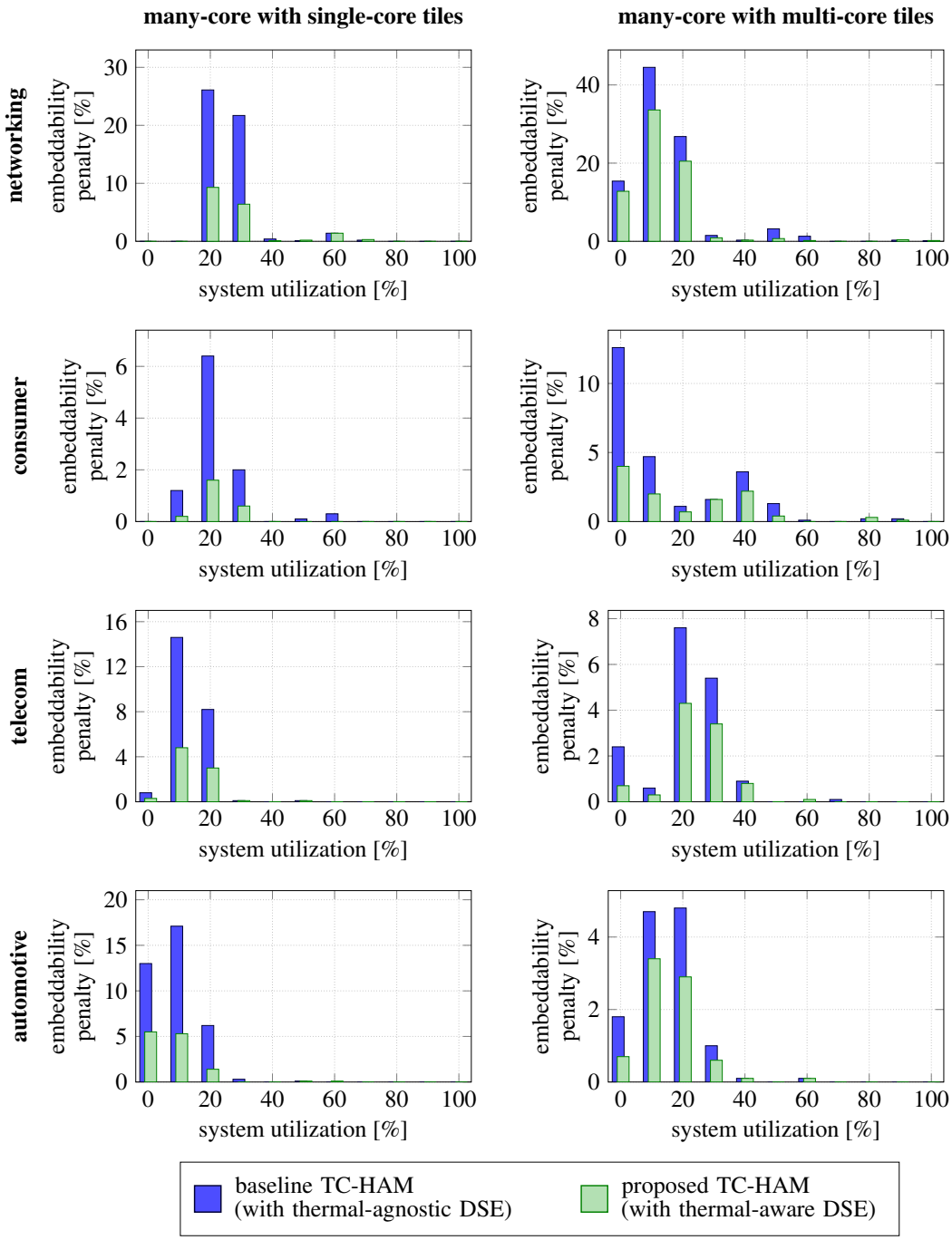


Figure 4.10: Embeddability penalty of thermally composable HAM approaches based on (i) a thermal-agnostic DSE (baseline TC-HAM) and (ii) the proposed thermal-aware DSE (proposed TC-HAM). Each row (column) of plots corresponds to one application (architecture).

chip-level power constraints to control the thermal state of the chip [HC17; WZM+14; MPV+13]. For instance, in [HC17], an online mapping approach is presented to maximize the system's performance by means of DVFS, subject to a Thermal Design Power (TDP) constraint. *Chip-level* power constraints, e.g., TDP, provide an abstraction of the underlying thermal issue without contributing as much to the complexity of the already NP-complete mapping problem [BTT98]. However, as shown in [PKC+17], chip-level power constraints such as TDP do not necessarily enforce thermal safety. This issue is addressed by a new power-budgeting concept presented in [KPS+17; PKC+17] which provides a thermally safe power density constraint per core (as a function of the number of simultaneously active cores in the system) which guarantees thermal safety. Based on this power density constraint, a dynamic application mapping approach is proposed in [KPS+17] to maximize the system's performance without surpassing the given thermally safe power density to enforce thermal safety. Although these approaches can establish thermal safety, due to the NP-complete nature of the application mapping problem, they must rely on fast (and often incremental) mapping heuristics. Hence, they cannot provide strict real-time guarantees which renders them inapplicable in a hard real-time context [SDM+17].

Hybrid Application Mapping (HAM) methodologies address this issue through a compute-intensive offline optimization and timing verification of mappings per application followed by an adaptive online application deployment based on the precomputed mappings and in line with the current resource availability [WGW+14; WWG+18a; AZ18; SKS13; BBE+11]. The majority of HAM methodologies aim at combining the advantages of static and dynamic mapping methodologies without a particular interest in thermal safety or real-time guarantees [SKS13; BBE+11]. The HAM methodology presented in [AZ18] is the only one known to us which incorporates thermal aspects into its offline analysis of mappings. It achieves thermal safety for the computed mappings by means of DVFS and idle task insertion, aiming at maximizing the system's performance. It, however, disregards thermal composability which renders the provided thermal-safety guarantees indeed unsafe. Moreover, it does not consider real-time constraints and assumes a fully connected architecture which is not realistic for many-core platforms relying on packet-switched NoCs[3]. To the best of our knowledge, the HAM methodology described in this chapter and proposed in [PSK+19] is the first work to address thermal composability in dynamic many-core systems.

4.5 Summary

Many-core systems employ autonomous DTM mechanisms to counteract overheating, e.g., using power gating. DTM countermeasures may, however, lead to the violation

[3] As shown in [WGW+14], techniques that rely on fully connected architectures cannot provide performance guarantees when applied for NoCs.

of hard real-time constraints which is not tolerable. As elaborated in Section 4.1, such interferences must be eliminated in a hard real-time context by enforcing the *thermal safety* of each application proactively to prevent the activation of DTM countermeasures. Moreover, *thermal composability* must be established in the system to guarantee that the heat transfer between the regions used by concurrent applications will never lead to thermal violations. State-of-the-art HAM methodologies disregard thermal composability in their mapping and management processes and, hence, fail in providing safe real-time guarantees.

As a remedy, this chapter presented a *thermally composable HAM methodology* in Section 4.2 which proactively enforces thermal safety in the system at all times and, thereby, establishes thermal composability. The proposed HAM methodology introduces (i) an offline *thermal-safety analysis* and (ii) an online *thermal-safety admission check* into the standard flow of HAM. The offline thermal-safety analysis derives an abstract thermal-safety index, termed TSU, for each mapping which expresses its thermal safety as a function of the number of active cores in the system. This index is used by the online thermal-safety admission check at the launch time of applications for a lightweight examination of the thermal safety of each mapping candidate for the current system load. By incorporating the thermal-safety analysis into the mapping optimization process, a thermal-aware DSE is established in which not only the satisfaction of real-time constraints is verified, but also mappings are optimized w.r.t. thermal safety. A series of experimental results presented in Section 4.3 demonstrated the effectiveness of the proposed HAM methodology in establishing thermal composability and the capability of the proposed thermal-aware DSE in delivering mappings with enhanced thermal safety.

5
Automatic Mapping-Set Distillation

In Hybrid Application Mapping (HAM) methodologies, the dynamic mapping and management of applications at run time is realized based on precomputed mapping candidates per application which are derived by an offline Design Space Exploration (DSE). During the DSE, mappings are optimized w.r.t. several design objectives that assess their quality in terms of resource demand, latency, throughput, energy consumption, etc. Given the relatively large number of design objectives, the DSE typically delivers a huge set of Pareto-optimal mappings which imposes an immense management overhead if considered in its entirety for online use. In particular, the temporal overhead imposed on the Run-time Platform Manager (RPM) for the management of a large set of mappings degrades the responsiveness of the RPM which is critical for real-time applications due to their timing constraints and, hence, strict dependence on the responsiveness of the RPM. Thus, it is of importance to reduce the number of Pareto-optimal mappings that are provided to the RPM to a limited number.

The choice of mappings provided to the RPM, however, has a strong impact on both (i) the embeddability of each application (using its retained mappings) in view of the dynamic availability of resources and (ii) the non-functional quality trade-offs (offered by its retained mappings) available to the RPM to choose from when launching the application. Selecting a promising subset of mappings for online use, termed *mapping-set distillation* in the following, is a challenging task due to the Pareto optimality and, hence, incomparability of mappings w.r.t. the design objectives. State-of-the-art HAM methodologies either disregard mapping-set distillation altogether or rely on problem-specific distillation schemes or generic multi-objective truncation techniques that suffer from limited applicability and poor distillation performance, respectively.

As a remedy, this chapter presents an automatic and configurable distillation methodology tailored to the distillation of Pareto-optimal mappings in the context of heterogeneous many-core systems. By distinguishing resource-related design objectives from quality-related design objectives and proper handling of each set, the proposed distillation methodology is able to retain a configurable number of Pareto-optimal mappings that offer diverse quality trade-offs while exhibiting a diverse yet modest resource demand and, thereby, high embeddability in various resource availability scenarios. Experimental results for a variety of applications demonstrate the effectiveness of the

proposed distillation methodology compared to multi-objective truncation techniques.

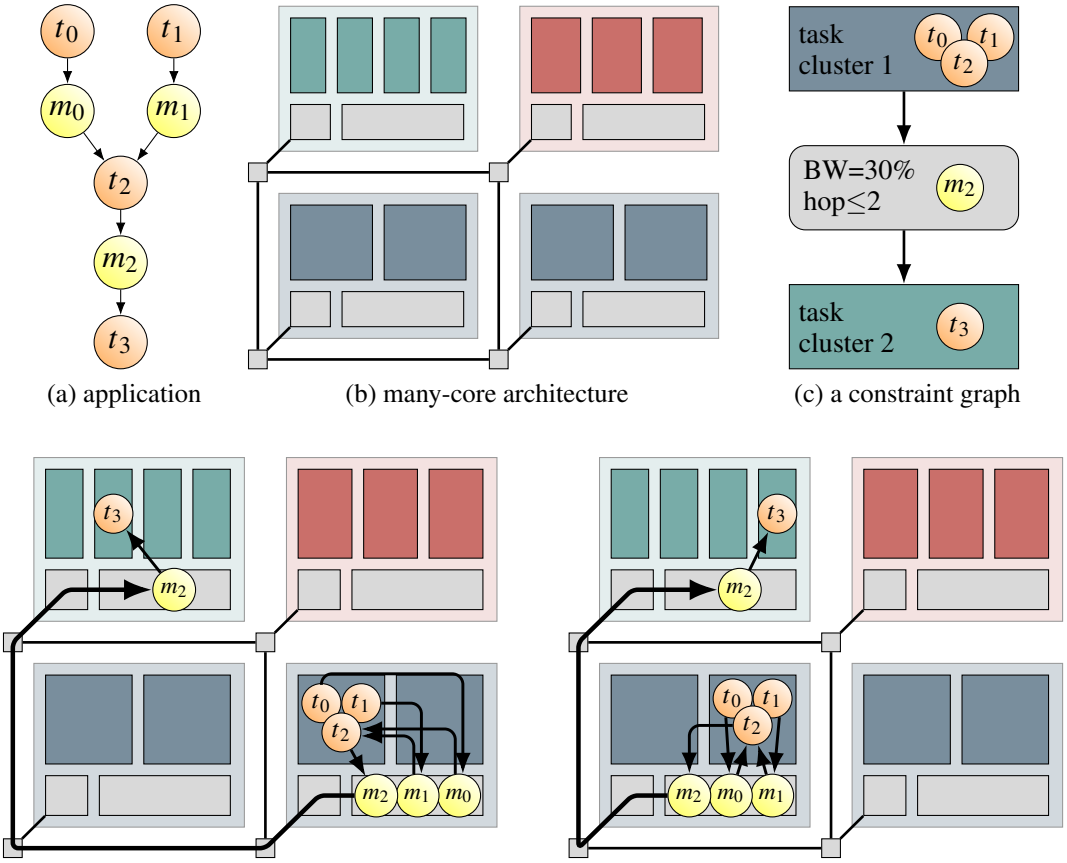
The remainder of this chapter is organized as follows[1]. Section 5.1 provides an overview of the mapping-set distillation problem and its challenges. The proposed distillation methodology is presented in Section 5.2, and its effectiveness is demonstrated through a series of experimental results in Section 5.3. Finally, an overview of related work is provided in Section 5.4, followed by a summary of the chapter in Section 5.5.

5.1 Distillation in Hybrid Application Mapping

The offline DSE in HAM typically embodies a many-objective optimization process: On the one hand, mappings are often optimized w.r.t. (i) a set of so-called *quality objectives*, e.g., latency, throughput, energy consumption, and reliability, in accordance with the non-functional properties of interest for the application in question or the system. On the other hand, in the scope of heterogeneous many-core platforms, mappings are also optimized w.r.t. (ii) a set of so-called *resource objectives*, namely, the number of allocated cores per core type, to be minimized. The resource objectives enable obtaining mappings with *diverse* resource requirements which enhances the likelihood of finding a feasible embedding at run time [WGW+14]. Given the variety of the quality properties of interest for emerging embedded applications and systems and the growing heterogeneity of processing resources in many-core platforms, the offline DSE in HAM increasingly manifests a many-objective optimization process and, hence, often delivers a substantially large set of Pareto-optimal mappings.

The statically computed Pareto-optimal mappings per application are used online by the RPM to launch applications on-demand according to the current availability of resources (and, possibly, further criteria such as thermal safety as discussed in Chapter 4). To launch an application, the RPM takes into account the precomputed set of Pareto-optimal mappings of that application and, then, selects a mapping for which the required resources are currently available. In this scope, mappings are often represented abstractly in the form of so-called *constraint graphs* [WGW+14]. A constraint graph characterizes rather a *class* of mappings by specifying (i) a partitioning of tasks into so-called *task clusters*, (ii) an assignment of each task cluster to a core type, (iii) the maximum NoC distance (hops) allowed between tasks clusters containing data-dependent tasks, and (iv) the NoC budget (e.g., bandwidth) required for the transfer of messages between data-dependent task clusters. Due to architectural symmetries, a constraint graph may have various concrete mapping realizations on a given many-core platform [WGW+14]. Figure 5.1 illustrates an example of a constraint graph and two concrete realizations corresponding to it. When launching an application, the RPM performs a so-called *embedding feasibility check* for each mapping candidate characterized by a constraint graph to find a concrete realization of

[1] The contents of this chapter are based on and partly published in [PGT17] which has appeared in the Proceedings of the 20th Conference on Design, Automation, and Test in Europe (DATE).



(a) application (b) many-core architecture (c) a constraint graph

(d) two concrete mapping realizations that correspond to the constraint graph shown in (c)

Figure 5.1: Example of a constraint graph. Shown are an application (a), a many-core architecture (b), a constraint graph (c) representing a class of mappings of the application on the architecture, and two exemplary concrete mapping realizations of the shown constraint graph (d), cf. [WGW+14].

it on the currently available resources of the platform or to deduce its unembeddability. Once a concrete mapping realization has been identified, it will be used to launch the application. In the rest of this chapter, a mapping class (i.e., a class of mappings represented by a constraint graph) is simply referred to as a mapping.

5.1.1 Importance of Distillation

The embedding feasibility check of a constraint graph is a variant of the application mapping problem which is known to be NP-complete [BTT98]. The embedding feasibility check can be realized using exact constraint-solving approaches, e.g., SAT [SWG+17], or by means of lighter heuristics, e.g., backtracking algorithms [WGW+14; WWG+18a]. However, due to the large decision space of the solver,

even lightweight feasibility check heuristics exhibit a considerable run-time overhead for the feasibility assessment of even a single constraint graph. As an example, for various application and architecture sizes, Table 5.1 provides the average run-time overhead of the backtracking-based embedding feasibility check from [WGW+14; WWG+18a] for a single constraint graph. Note that the overhead measures reported in Table 5.1 are derived on a desktop computer (Intel i7-4770 running at 3.4 GHz with 32 GiB of RAM) and may grow 1–2 orders of magnitude higher than the values reported here when measured on an embedded platform.

As reported in [SRR+18] and denoted by the values in Table 5.1, even for relatively small applications and architectures, the overhead of the embedding feasibility check of a single mapping can grow over tens or even hundreds of milliseconds when performed on an embedded platform. Thus, checking hundreds of mappings at run time may render HAM de facto impractical due to the immense temporal overhead of the RPM. Therefore, this overhead must be kept within an affordable range which heavily restricts the number of mappings that can be feasibly considered at run time. Especially in the case of real-time applications, this overhead can become critical as it degrades the responsiveness of the RPM to application requests and may lead to the violation of real-time constraints. Moreover, in addition to timing requirements, secondary factors such as storage space limitations and the complexity of the RPM's decision-making process also restrict the maximum number of mappings per application that can be feasibly considered by the RPM.

The mismatch between a typically huge number of Pareto-optimal mappings obtained from the offline DSE per application, on the one hand, and the considerably restricted number of mappings that can be feasibly considered by the RPM, on the other hand, necessitate a so-called *mapping-set distillation* step after the DSE to select a promising subset of the Pareto-optimal mapping classes to be provided to the RPM for online use. Mapping-set distillation is a crucial process in HAM as the choice of retained (distilled) mappings has a strong impact on both (i) the embeddability of the application for each given availability of resources and (ii) the diversity of the quality trade-offs (offered by the retained mappings) available to the RPM to select from.

Table 5.1: Average run-time overhead for embedding feasibility check of a constraint graph for various application and architecture sizes, performed on a desktop computer (Intel i7-4770 at 3.4 GHz with 32 GiB of RAM) [PGT17].

architecture size (NoC dimensions)	embedding feasibility check run-time [ms]			
	7 tasks	11 tasks	14 tasks	18 tasks
6×6	0.79	1.90	7.34	8.07
8×8	2.41	6.88	27.70	25.49
10×10	4.67	31.99	115.24	134.79

5.1.2 Distillation Challenges

Given a set of mappings (in terms of constraint graphs) with Pareto-optimal trade-offs in the space of a set of quality objectives and a set of resource objectives, the goal of mapping-set distillation is to retain (distill) a subset of mappings which (i) offer *diverse* trade-offs w.r.t. the quality objectives to address the—at design-time unknown—run-time quality requirements of the application and system and (ii) exhibit *efficient* resource combinations w.r.t. the resource objectives to enhance the embeddability of the application in view of a dynamic availability of resources at run time. Achieving a high diversity w.r.t. quality trade-offs and high efficiency w.r.t. resource requirements is challenging due to the inherent conflicting nature of these goals.

Another challenge of distillation roots in the huge mismatch between the number of Pareto-optimal mappings delivered by the DSE and the number of mappings that can be feasibly considered by the RPM. Oftentimes, only a *small fraction* of the mappings can be retained for online use. For instance, assuming an online management capacity of 15 mappings per application, only 1% of the mappings may be retained when the Pareto set contains over 1,500 mappings. In this context, each distilled mapping must be selected carefully to preserve the diversity of quality trade-offs and the embeddability of the original Pareto set while retaining only a fraction of its mappings.

In the multi-objective optimization paradigm, a large body of techniques exists for retaining a subset of Pareto-optimal solutions with maximal *diversity* of their objective trade-offs. These techniques rely on measures such as Crowding Distance (CD) [DPA+02], Nearest Neighbor (NN) [ZLT01], and Adaptive Grid (AG) [KC03] to maximize the *spread* of retained solutions in the objective space and, thereby, maximize the *diversity* of their trade-offs [HBK05]. However, when applied for mapping-set distillation, they retain mappings which suffer from prohibitively low embeddability. This problem roots in the fact that such techniques aim for maximizing diversity w.r.t. *all* objectives, whereas embeddability primarily depends on the *efficiency* of retained mappings w.r.t. resource objectives. In fact, to preserve embeddability, resource objectives must be considered *collectively* (as they jointly impact embeddability), and *efficient* yet *diverse* resource trade-offs must be retained. Multi-objective truncation techniques do not perceive the interdependence between the resource objectives and their impact on embeddability. Thus, by solely aiming at preserving the diversity of trade-offs w.r.t. all objectives, they retain mappings which often exhibit a relatively high demand for resources and, hence, low embeddability.

State-of-the-art HAM methodologies, either (i) disregard distillation altogether and, hence, harm the responsiveness of the RPM by providing it with too many mappings, or (ii) employ problem-specific distillation techniques with limited applicability for the problem of mapping-set distillation in general, e.g. [SKS13; KSK+13], or (iii) rely on generic truncation techniques adopted from the multi-objective optimization paradigm which perform poorly when applied for mapping-set distillation. To the best of our knowledge, the distillation methodology presented in this chapter and

proposed in [PGT17] is the first automatic and configurable distillation methodology for mapping-set distillation in the scope of HAM.

Contribution overview

This work presents an automatic and configurable mapping-set distillation methodology which is able to retain a subset of Pareto-optimal mappings with a configurable cardinality referred to as the *distillation capacity*. To maximize the *diversity* of the distilled mappings w.r.t. quality objectives and to maximize their *efficiency* and *diversity* w.r.t. resource objectives, the proposed methodology follows a two-step procedure, distinguishing these two classes of design objectives from one another: First, in the space of quality objectives, mappings are partitioned into a set of quality trade-off regions. Then, from different quality trade-off regions, representative mappings are distilled using a priority scheme based on the Pareto ranking of mappings in the space of resource objectives. This distillation scheme ensures both the diversity of the retained mappings w.r.t. quality trade-offs and the efficiency and diversity of their resource requirements. By configuring the number of quality trade-off regions, the proposed distillation methodology is able to distill any desired number of Pareto-optimal mappings. In the following, the proposed methodology is presented in detail.

5.2 Mapping-Set Distillation Methodology

This section presents the proposed mapping-set distillation methodology. The mapping-set distillation problem is described as follows: Let Γ be a set of Pareto-optimal mappings in the space of design objectives $O = O^{\text{qlt}} \cup O^{\text{res}}$ where O^{qlt} denotes the set of quality objectives and O^{res} the set of resource objectives (indicating the presence of $|O^{\text{res}}|$ cores types in the architecture). Given the set of Pareto-optimal mappings Γ, quality objectives O^{qlt}, resource objectives O^{res}, and a *distillation capacity* k, the goal of mapping-set distillation is to select a subset of mappings $\Gamma^* \subseteq \Gamma$ with cardinality $|\Gamma^*| = k$ which offer a high diversity of trade-offs in the space of quality objectives O^{qlt} and efficient yet diverse trade-offs in the space of resource objectives O^{res}.

The proposed mapping-set distillation methodology embodies a two-step distillation procedure and operates based on two *projections* of the mapping set Γ in the spaces of quality objectives and resource objectives to fulfill the distillation requirements. In the space of quality objectives O^{qlt}, mappings are partitioned into groups of similar quality trade-offs which are used to retain mappings with a *high diversity* of quality trade-offs. Here, a *configurable distillation capacity* k is realized by adjusting the number and granularity of the partitions according to k. In the space of resource objectives O^{res}, *Pareto ranking* is used to establish a priority scheme among mappings so that resource objectives are regarded *collectively*, and *efficient yet diverse* trade-offs w.r.t. resource objectives are distilled based on the Pareto rank of mappings.

In the following, first, the partitioning of the space of quality objectives and the Pareto ranking in the space of resource objectives are detailed in Sections 5.2.1 and 5.2.2, respectively. These steps are used in the proposed mapping-set distillation algorithm which is presented thereafter in Section 5.2.3.

5.2.1 Quality-Space Partitioning

For each application, the number of mappings that can be feasibly considered by the RPM strongly depends on the computational and timing budget of the RPM and the embedding feasibility check overhead per mapping which itself depends on the size of the application and the architecture. Therefore, the distillation methodology must offer a configurable distillation capacity k to be adjusted according to these parameters. For this purpose, in the proposed distillation methodology, the $|O^{\text{qlt}}|$-dimensional space of quality objectives O^{qlt} is partitioned into a set of quality-similarity regions where the mappings that are located in the same region offer *approximately* identical quality trade-offs. Then, representative mappings are distilled from different trade-off regions (partitions) so that diverse quality trade-offs are retained. By adjusting the number and size of the quality trade-off regions according to the given distillation capacity k, the number of distilled mappings is configured automatically as a function of k.

For the partitioning of the space of quality objectives, first, the quality of mappings w.r.t. each quality objective $o^{\text{qlt}} \in O^{\text{qlt}}$ is normalized to the range $[1, X]$ where $X > 1$ controls the maximum degree of quality approximation per quality region as will be elucidated in the following. After the normalization, an *exponentially spaced hyper-grid* is placed in the quality space with the grid coordinates $\sqrt[div]{X^i}$, $i \in \{0, ..., div\}$ in each objective $o^{\text{qlt}} \in O^{\text{qlt}}$, resulting in div regions (partitions) per objective. Here, div is determined based on the given distillation capacity k as will be detailed later. In this setting, each grid cell represents one quality trade-off region, encompassing mappings with similar quality trade-offs. Figure 5.2 illustrates an exemplary space of two quality objectives (o_1^{qlt} and o_2^{qlt}) normalized to the range of $[1, X]$ and partitioned using an exponentially spaced hyper-grid (dashed lines) with a grid granularity of $div = 5$ partitions per objective. Using such a setup, (i) the granularity of the regions, i.e., the degree of quality trade-off approximation, can be configured by div as a function of the distillation capacity k, while (ii) the intra-cell maximum ratio of approximation per objective remains constant (equal to $\sqrt[div]{X}$) for all cells in the grid. Thus, by distilling mappings from different cells, diverse quality trade-offs will be retained where the *spacing* between the retained trade-offs is adjusted by div (and X) which is set according to the distillation capacity k.

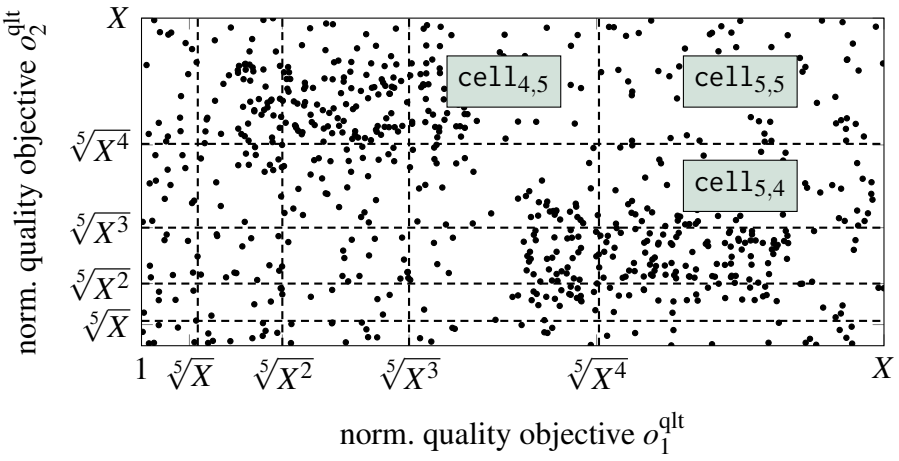


Figure 5.2: Example of a normalized space of two quality objectives, o_1^{qlt} and o_2^{qlt}, and its partitioning using an exponentially spaced hyper-grid (dashed lines) with a grid granularity of $div = 5$ partitions per objective [PGT17].

5.2.2 Resource-Space Pareto Ranking

The resources allocated by each mapping *jointly* contribute to its embeddability at run time. Thus, resource objectives (i) must be considered *collectively* during the distillation process. Furthermore, to achieve a high embeddability, distilled resource trade-offs must be (ii) *efficient* so that fewer resources are required per mapping, while also being (iii) *diverse* so that distilled mappings can satisfy diverse scenarios of resource availability at run time. For a collective investigation of resource objectives, *Pareto ranking* of mappings in the space of resource objectives is performed to reflect both diversity and efficiency of resource requirements simultaneously using the Pareto rank of each mapping. To that end, the set of mappings Γ is projected into the space of resource objectives O^{res}, and the following iterative procedure is performed to Pareto-rank all mappings w.r.t. the resource objectives. Starting with rank $i = 1$:

1. The Pareto front $\Gamma_i \subseteq \Gamma$ of mappings w.r.t. resource objectives O^{res} is determined. Here, Γ_i denotes the minimal subset of mappings in Γ which dominate all mappings $\Gamma \setminus \Gamma_i$ w.r.t. resource objectives.

2. All mappings $\mu \in \Gamma_i$ are annotated with rank i and eliminated from Γ.

3. If Γ is not empty, i.e., some mappings are not ranked yet, i is incremented by one, and the procedure returns to step 1.

The Pareto-ranking procedure terminates when all mappings are ranked. Figure 5.3 illustrates an example of Pareto-ranked mappings in the space of two resource objectives o_1^{res} and o_2^{res} where the mappings with the same rank are connected with dashed

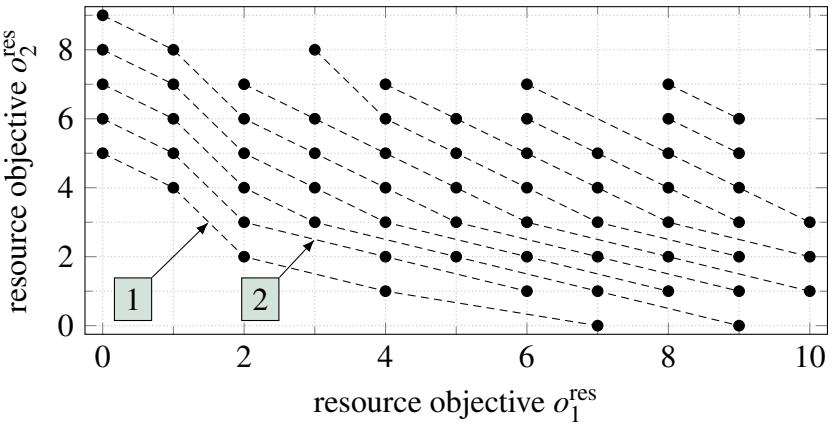


Figure 5.3: Example of Pareto ranking in the space of two resource objectives o_1^{res} and o_2^{res}. Dashed lines connect mappings with the same rank [PGT17].

lines. Note that mappings with lower Pareto ranks generally exhibit a modest demand for resources (efficiency). Moreover, as mappings with the same rank are mutually incomparable w.r.t. resource objectives, they also exhibit diverse resource combinations. In the proposed distillation methodology, mappings with a lower Pareto rank have a higher priority of being distilled as they also exhibit a higher embeddability.

5.2.3 Automatic Distillation Algorithm

The proposed distillation methodology is developed based on the quality-space partitioning scheme elaborated in Section 5.2.1 and the resource-space Pareto-ranking scheme detailed in Section 5.2.2. The proposed methodology embodies a two-step algorithm in which, first, mappings are clustered into regions of comparable trade-offs w.r.t. quality objectives O^{qlt}, and then, representative mappings are distilled from within different quality trade-off regions based on their Pareto rank w.r.t. resource objectives O^{res} to retain mappings with efficient yet diverse resource requirements. The proposed distillation methodology is provided in Algorithm 1 where—for the sake of clarity and without loss of generality—an exemplary problem with two quality objectives (to be minimized[2]) and multiple resource objectives is considered.

In the first step of the algorithm (lines 2–7), quality regions are formed by placing the previously described hyper-grid in the space of the quality objectives O^{qlt} and assigning mappings to their respective regions according to their given quality trade-offs w.r.t. the quality objectives O^{qlt}. This step is realized as follows. First, the granularity of the hyper-grid is determined according to the given distillation capacity k as shown in line 2 and elucidated in the following. Figure 5.4 illustrates an exemplary

[2]A maximization problem or a combined problem of maximization and minimization can be trivially transformed into a minimization problem using the duality principle, see [Deb01] for details.

Algorithm 1: Automatic mapping-set distillation algorithm [PGT17]

Input: Initial set of Pareto-optimal mappings Γ, quality objectives O^{qlt}, resource objectives O^{res}, distillation capacity k
Output: Distilled set of Pareto-optimal mappings Γ^*

function `Distill`$(\Gamma, O^{qlt}, O^{res}, k)$
  `div` $\leftarrow \left\lceil \frac{1+\sqrt{1+8k}}{2} \right\rceil - 1$ `/* quality-space subdivision */`
  for $i \in \{1, \ldots,$ `div`$\}$ **do**
    for $j \in \{1, ...,$ `div`$\}$ **do**
      `cell`$_{i,j} \leftarrow \{\mu \in \Gamma \mid$ `CellID`$(\mu) = (i, j)\}$
    end
  end
  `ParetoRank`(Γ, O^{res}) `/* resource-space Pareto-ranking */`
  `/* distillation of mappings from high-quality cells */`
  for $x \in \{1, \ldots,$ `div` $- 1\}$ **do**
    for $i \in \{1, ..., x\}$ **do**
      $j \leftarrow x - i + 1$
      $\mu \leftarrow$ `Select`$($`cell`$_{i,j})$
      $\Gamma^* \leftarrow \Gamma^* \cup \{\mu\}$
      $\Gamma \leftarrow \Gamma \setminus \{\{\mu\} \cup \{\mu' \in \Gamma \mid$ `cellID`$(\mu) \succ$ `cellID`$(\mu') \wedge \mu \succ_{O^{\text{res}}} \mu'\}\}$
    end
  end
  `/* distillation of additional resource-efficient mappings */`
  while $(|\Gamma^*| < k) \wedge (\Gamma \neq \varnothing)$ **do**
    $\mu \leftarrow$ `Select`(Γ)
    $\Gamma^* \leftarrow \Gamma^* \cup \{\mu\}$
    $\Gamma \leftarrow \Gamma \setminus \{\{\mu\} \cup \{\mu' \in \Gamma \mid$ `cellID`$(\mu) \succ$ `cellID`$(\mu') \wedge \mu \succ_{O^{\text{res}}} \mu'\}\}$
  end
end

partitioned space of two quality objectives where the semi-diagonal line divides the hyper-grid in half. Here, mappings in the lower half excel w.r.t. quality trade-offs but often have a relatively high demand for resources. These mappings are referred to as *quality-dominant* mappings. On the other hand, mappings in the upper half have a modest demand for resources but often exhibit inferior quality trade-offs. These mappings are referred to as *resource-efficient* mappings. To distill an efficient set of quality trade-offs, it is sufficient to retain mappings from quality-dominant regions, i.e., gray-shaded cells in Figure 5.4. Nonetheless, to enhance embeddability in run-time scenarios with low resource availability, resource-efficient mappings must be distilled as well, e.g., the blue mappings in Figure 5.4. To that end, in the proposed

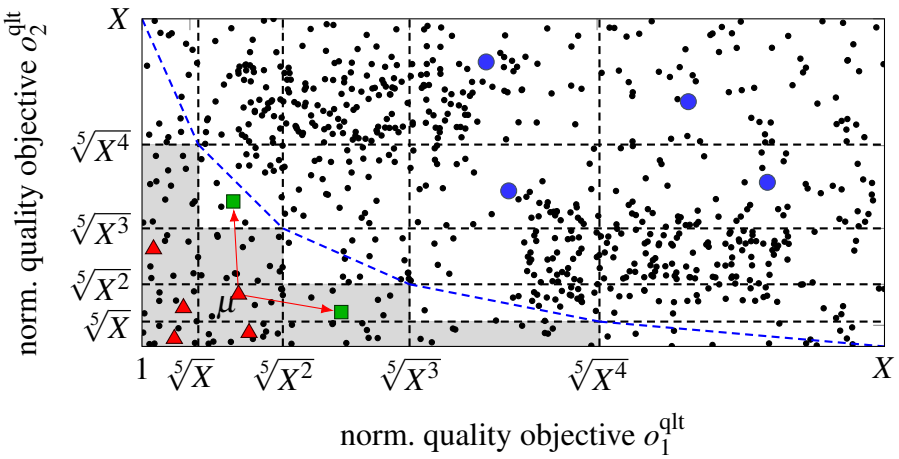


Figure 5.4: Example of a partitioned space of two quality objectives o_1^{qlt} and o_2^{qlt} with $k = 12$ and $div = 5$ in iteration $(x,i) = (3,2)$ of distilling mappings from high-quality cells in Algorithm 1 (lines 9–16). Gray-shaded cells represent quality-dominant regions. Red triangles indicate distilled mappings. Green squares are mappings dominated by mapping μ. Blue circles denote examples of resource-efficient mappings [PGT17].

algorithm, the granularity of the hyper-grid (controlled by *div*) is adjusted in line 2 such that, alongside the mappings distilled from quality-dominant regions, at least one resource-efficient mapping is distilled as well. Here, a space of two quality objectives encompasses a total of $\frac{1}{2}(div-1)(div)$ quality-dominate regions. Thus, the grid granularity shall be set to the maximum value $div \in \mathbb{N}$ satisfying the inequality $\frac{1}{2}(div-1)(div) < k$, resulting in $div = \left\lceil \frac{1+\sqrt{1+8k}}{2} \right\rceil - 1$, see line 2 of the algorithm.

After the grid granularity (parameter *div*) has been determined, each mapping $\mu \in \Gamma$ will be associated with its respective quality region (cell), see lines 3–7 of the algorithm. Here, the function `CellID`(μ) returns the grid coordinates (i,j) of the quality region $\texttt{cell}_{i,j}$ that encompasses mapping μ in the hyper-grid.

In the second step of the algorithm (lines 8–21), the distillation of representative mappings takes place. To that end, first, the Pareto ranking of mappings in the space of resource objectives O^{res} is performed as described in Section 5.2.2 using function `ParetoRank` (line 8). Subsequently, representative mappings are distilled from quality-dominant regions (lines 9–16), and eventually, resource-efficient mappings are distilled (lines 17–21). For the distillation of quality-dominant mappings (lines 9–16), the quality-dominant regions (cells) are explored in the order of dominance $\succ$ specified by Eq. (5.1). From each region (cell), one mapping μ is distilled using function `Select` (line 12) and is added to the initially empty set of distilled mappings Γ^* (line 13). Function `Select` will be discussed later in detail.

$$\texttt{cell}_{i,j} \succ \texttt{cell}_{i',j'} \iff i \leq i' \wedge j \leq j' \tag{5.1}$$

Since quality trade-offs are approximated by the hyper-grid, a distilled mapping μ may dominate further mappings $\mu' \in \Gamma$ based on the approximated quality trade-offs[3]. Here, a distilled mapping μ dominates mapping $\mu' \in \Gamma$ if (i) $\texttt{cell}(\mu) \succ \texttt{cell}(\mu')$ according to Eq. (5.1), and (ii) μ dominates μ' w.r.t. the resource objectives O^{res}, denoted by $\succ_{O^{\text{res}}}$. During the distillation procedure, once a mapping μ has been distilled, the remaining mappings $\mu' \in \Gamma$ are examined, and those that are dominated by μ according to (i) and (ii) are eliminated from Γ together with μ (line 14) before proceeding to the next cells. After the distillation of mappings from quality-dominant regions (lines 9–16) has been performed as detailed above, resource-efficient mappings are distilled from Γ (lines 17–21), in a similar procedure.

Function "`Select`"

In the proposed distillation algorithm, function `Select` is used to distill a mapping from a given set of mappings, see lines 12 and 18 in Algorithm 1. For this purpose, it ranks the given mappings based on an estimate of their embeddability and, then, distills the mapping that has the highest rank, i.e., the highest estimated embeddability. Note that, embeddability denotes the probability of a mapping being embeddable at different run-time resource availability scenarios which can be accurately assessed only at run time when the resource availability scenarios become known. Hence, an exact assessment of embeddability for each mapping is not feasible offline at the time of distillation. As a remedy, we use a heuristic to obtain an estimate of the *relative embeddability* of the given mappings based on which a preference scheme can be established between different mappings w.r.t. embeddability.

For the ranking of mappings, function `Select` compares them according to their (i) Pareto rank w.r.t. resource objectives and (ii) overall resource cost. The overall resource cost of a mapping is calculated as its accumulative utilization rate of different core types, where the utilization rate of a core type by a mapping is defined as the proportion of the number of instances of that core type allocated by the mapping. Here, a mapping with a lower Pareto rank w.r.t. resource objectives (primarily) and a lower resource cost (secondarily) is taken to have a higher relative embeddability and, hence, is prioritized for distillation.

If two mappings offer the same relative embeddability estimate according to (i) and (ii), function `Select` considers two additional criteria to reflect the quality measures of mappings in their ranking: (iii) the distance of the quality region of each mapping from the best quality region ($\texttt{cell}_{1,1}$), calculated as $(i-1)+(j-1)$ for $\texttt{cell}_{i,j}$, and (iv) the number of already distilled mappings from each respective quality region. Here, under identical rankings based on (i) and (ii), a mapping from a region (cell) with a better quality trade-off (primarily) and fewer representative mappings in

[3] Note that any pair of mappings $\mu \in \Gamma$ and $\mu' \in \Gamma$ are Pareto optimal based on their *accurate* quality measures. However, the *approximation* of quality trade-offs (by means of the hyper-grid) may alter the dominance relation between μ and μ' such that μ dominates μ' or vice versa.

the distilled set (secondarily) is prioritized for distillation. Note that, since mappings from the same quality region share the same preference w.r.t. (iii) and (iv), these metrics come into effect only for the distillation of resource-efficient mappings where the set of considered mappings belong to different quality regions.

Computational Complexity

The proposed distillation algorithm is established based on functions `ParetoRank` and `Select` as well as a set of `for` and `while` loops as shown in Algorithm 1. Pareto ranking of a set of mappings Γ w.r.t. a set of resource objectives O^{res} has a computational complexity of $O(|\Gamma|^2|O^{\text{res}}|)$ where $|\Gamma|$ and $|O^{\text{res}}|$ denote the cardinality of sets Γ and O^{res} , respectively, cf. [ZTC+15]. Hence, function `ParetoRank` has a maximum computational complexity of $O(|\Gamma|^2|O^{\text{res}}|)$. Function `Select` embodies a `find-min` procedure with an iteration space of $|\Gamma|$ mapping evaluations, each involving an enumeration of the mapping qualities w.r.t. resource objectives O^{res} and quality objectives O^{qlt}. Thus, function `Select` introduces a maximum computational complexity of $O(|\Gamma|(|O^{\text{res}}|+|O^{\text{qlt}}|))$.

The iteration space of the quality-partitioning loops in Algorithm 1 (lines 3–7) and the overall iteration space of the distillation loops (lines 9–21) each scale with div^2. Given that $div^2 \propto k$ (see line 2), the proposed distillation algorithm introduces an overall computational complexity of $O(k|\Gamma|+|\Gamma|^2|O^{\text{res}}|+k|\Gamma|(|O^{\text{res}}|+|O^{\text{qlt}}|))$ which can be simplified as $O(|\Gamma|^2|O^{\text{res}}|+k|\Gamma|(1+|O^{\text{res}}|+|O^{\text{qlt}}|))$. Given that, in general, $|O^{\text{res}}| \approx |O^{\text{qlt}}|$ and $k \leq |\Gamma|$, the computational complexity of Pareto ranking dominates the other terms, and thus, the overall computational complexity of the proposed distillation algorithm becomes $O(|\Gamma|^2|O^{\text{res}}|)$ which is certainly affordable as the distillation takes place at design time.

5.3 Experimental Results

This section presents the results of a series of experiments that assess the effectiveness of the proposed distillation methodology compared to multi-objective truncation techniques that are typically used for mapping-set distillation in the context of HAM. The experiments entail DSE of several applications to obtain Pareto-optimal mappings for distillation, followed by a series of run-time embedding trials to assess the embeddability of the distilled mappings in view of a dynamic availability of resources.

5.3.1 Experiments Setup

Applications and Architectures As platform, a 36-core heterogeneous many-core architecture with a 6×6 NoC organization and single-core tiles is used. The platform comprises diverse numbers of instances from three types of cores (IBM PowerPC,

AMD K6-2, and AMD K6-III) whose specifications are provided by Embedded System Synthesis Benchmarks Suite (E3S) [Dic10]. As applications, four real-time applications are used from the embedded domains of networking (7 tasks, 9 messages), consumer (11 tasks, 12 messages), telecommunications (14 tasks, 20 messages), and automotive (18 tasks, 21 messages). The applications are provided as benchmarks by E3S [Dic10]. For each application, the benchmarks suite also provides the payload size of each message and the Worst-Case Execution Time (WCET) and the power consumption of each task on each core type.

Design Objectives The offline optimization of mappings for each application is performed w.r.t. two quality objectives, namely, end-to-end latency (makespan) and energy consumption, as well as three resource objectives denoting the number of allocated cores per core type. The worst-case latency of each mapping is derived using the timing analysis from Chapter 3. The energy consumption of each mapping is evaluated based on the Worst-Case Response Time (WCRT) of its tasks, their power consumption, the payload size of each message, and the NoC/bus energy model from [WSK+05] considering a length of 2 mm for NoC links and 5 mm for buses.

Design Space Exploration (DSE) For the optimization of mappings, the DSE approach from [WGW+14] is employed which uses the NSGA-II [DPA+02] multi-objective evolutionary algorithm that is provided by the optimization framework OPT4J [LGR+11]. The DSE is configured to perform 2,000 optimization iterations with a population size of 100 mappings where 25 mappings are generated per iteration. During the optimization, a per se not bounded set of Pareto-optimal mappings (archive) is collected and updated which will be the output of the DSE. After the DSE, the constraint graph of each obtained Pareto-optimal mapping is constructed as detailed in [WGW+14]. The Pareto-optimal mappings serve as input to the investigated mapping-set distillation methodologies introduced in the following.

Investigated Distillation Approaches In the experiments, the proposed distillation methodology is compared with three standard truncation techniques, namely, Crowding Distance (CD) [DPA+02], Nearest Neighbor (NN) [ZLT01], and Adaptive Grid (AG) [KC03], from the domain of multi-objective optimization. For the distillation of mappings, a distillation capacity of $k = 12$ mappings is considered to achieve an affordable RPM overhead according to the overhead measures given in Table 5.1 when scaled to an embedded setting. In addition to the proposed distillation methodology and the multi-objective truncation techniques above, we also incorporate the set of Pareto-optimal mappings w.r.t. quality objectives only, referred to as *quality front*, and the set of Pareto-optimal mappings w.r.t. resource objectives only, referred to as *resource front*, as references of comparison in the experiments.

Run-Time Embedding and Feasibility Check For the run-time embedding of mappings, the backtracking-based algorithm from [WWG+18a] is employed for the embedding feasibility check of the constraint graph of each mapping. To emulate the dynamism in resource availability at run time, various resource availability scenarios are constructed as follows: Starting with an empty platform, i.e., 100% resource availability, cores are preoccupied one by one, iteratively, and at random. At each iteration, a run-time embedding experiment is performed in which the embeddability of the mappings distilled by each approach is assessed for the current availability of resources using the backtracking feasibility check. As part of this assessment, the following data is recorded: (i) the number of embeddable mappings from the set of distilled mappings provided by each distillation approach and the best quality achievable by the embeddable subset of mappings w.r.t. (ii) latency, (iii) energy consumption, and (iv) Energy-Latency Product (ELP). The iterations terminate when all cores are preoccupied.

5.3.2 Results Discussion

The following experimental results provide a comparative investigation of the effectiveness of the considered distillation methodologies in terms of the embeddability of distilled mappings and the diversity of the quality trade-offs offered by them. The provided results represent an average over the 10 runs of DSE per application and the five trials of the embedding experiment per DSE run to incorporate diverse mixes of preoccupied cores for the embedding feasibility checks at each utilization level.

Run-Time Embeddability

The run-time embeddability of the mappings provided by each distillation methodology is evaluated using two parameters: *success rate* and *embeddability rate*. For a given set of mappings and a set of available resources, success rate denotes whether *at least one* mapping can be feasibly embedded on the given set of available resources. Thus, success rate reflects whether the application in question can at all be launched using the provided set of distilled mappings. Embeddability rate, on the other hand, denotes the *proportion* of embeddable mappings and, thus, reflects the flexibility of the RPM in choosing between multiple embeddable mappings, e.g., to enhance energy efficiency.

The success rate and the embeddability rate observed in the run-time embedding experiments are provided in Figure 5.5 and Figure 5.6, respectively, for various distilled and reference sets of mappings. In each figure, each plot corresponds to one benchmark application specified by the plot title. In Figure 5.5, the *original set* denotes the initial set of Pareto-optimal mappings and, hence, serves as the best-case reference for the maximum achievable success rate. In Figure 5.6, the *resource front* denotes the set of Pareto-optimal mappings w.r.t. resource objectives and, thus, serves as a (near-)optimal reference for embeddability rate.

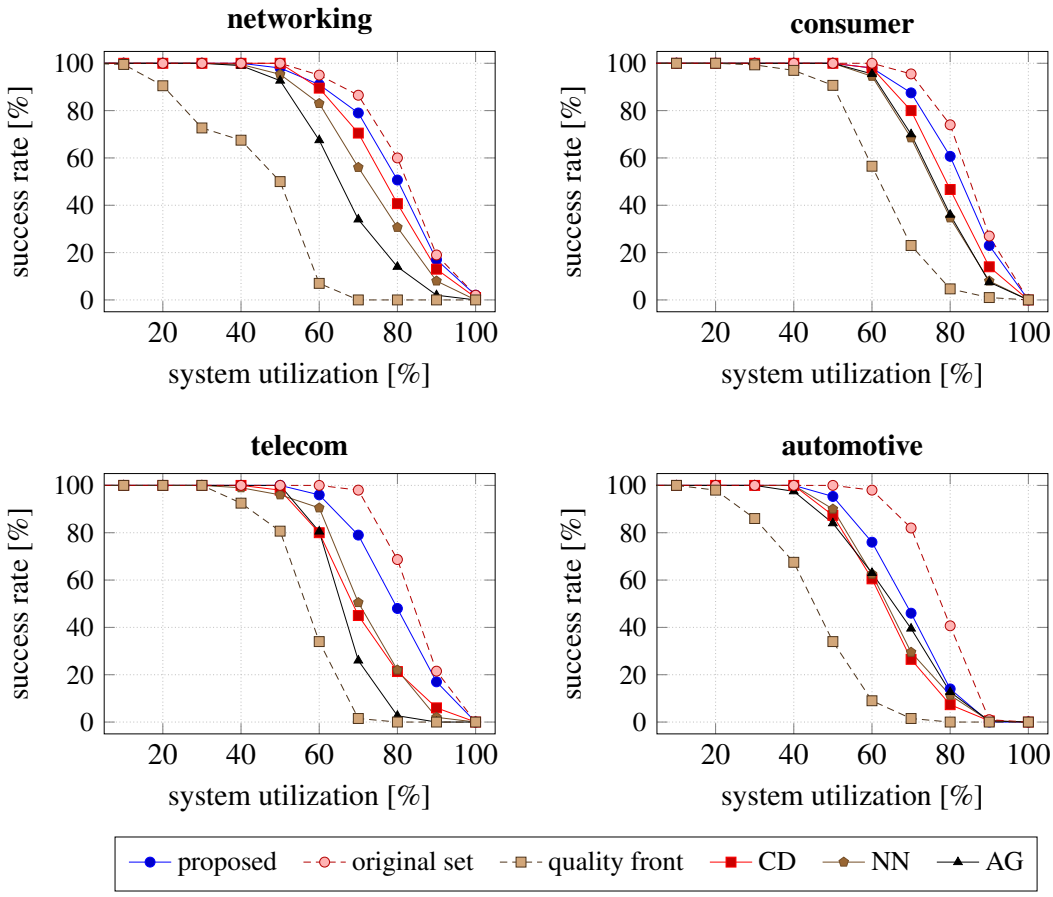


Figure 5.5: Success rate of the investigated distillation approaches for different applications. The *original set* serves as the best-case reference [PGT17].

As illustrated in Figures 5.5 and 5.6, while the ranking of the three standard multi-objective truncation techniques, namely, CD, NN, and AG, varies per application and criterion (success- or embeddability rate), the proposed distillation methodology seamlessly outperforms the standard truncation techniques for all applications w.r.t. both success rate and embeddability rate, especially at higher system utilization levels where only a limited set of resources are available. For each application and criterion, the proposed methodology improves the success rate by 8% on average (up to 45%) and embeddability rate by 16% on average (up to 45%) compared to the standard truncation technique with the *best* average results for the respective application and criterion. Note that while the enhanced success rate is crucial to bring applications to execution, the enhanced embeddability rate extends the options for the RPM to select between different embeddable mappings. Also note that, when merely the quality-dominant mappings are retained (quality front), embeddability is drastically degraded as the retained mappings often impose a high demand for resources.

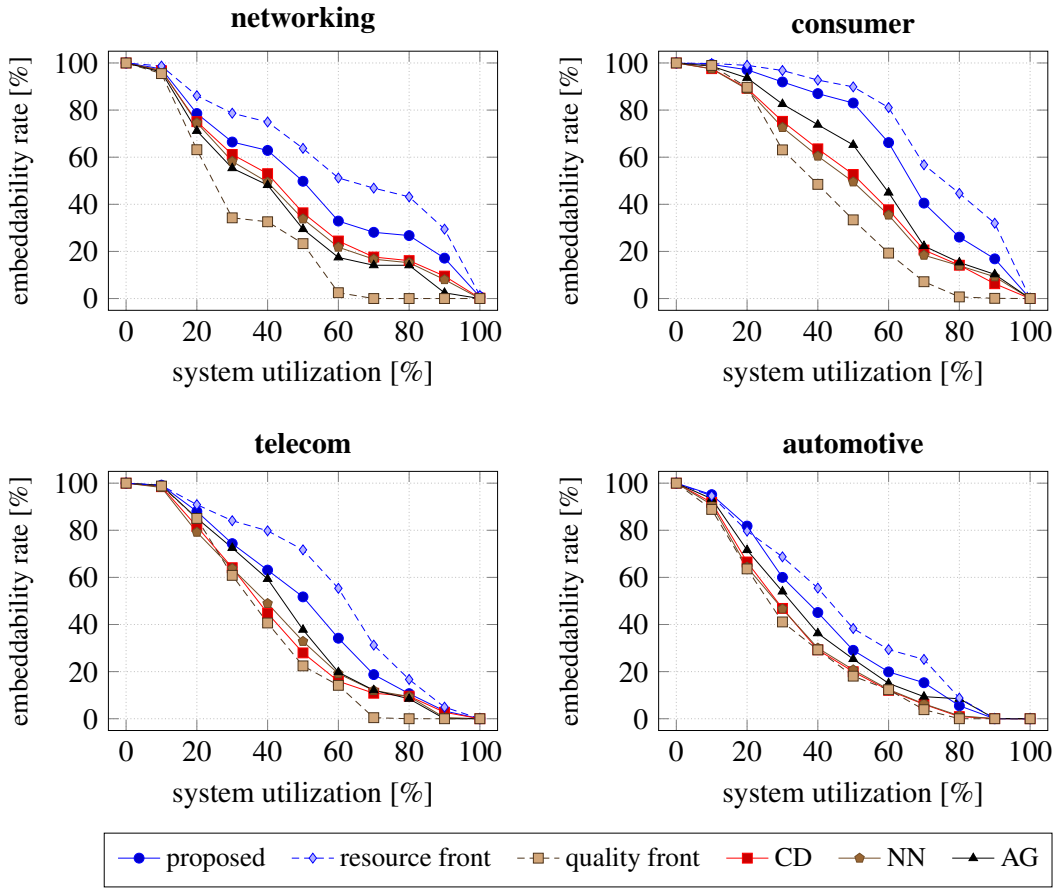


Figure 5.6: Embeddability rate of the distillation approaches for different applications. The *resource front* serves as a (near-)optimal reference [PGT17].

Quality Trade-Offs

To assess the diversity and efficiency of the distilled mappings w.r.t. their quality trade-offs, the well-established ε-*dominance* indicator [LTD+02] from the domain of multi-objective optimization is used. ε-dominance provides a scalar value $\varepsilon \in [0,1)$ which reflects the quality of a set of solutions Γ relative to a given reference set Γ_{ref} of best solutions in a given space of multiple objectives. Formally, the ε-*dominance* of set Γ w.r.t. reference set Γ_{ref} provides the smallest value of ε for which all solutions in Γ_{ref} will be dominated by the solutions in Γ if the latter are scaled at a rate of $(1-\varepsilon)$ w.r.t. all objectives, see Eq. (3.13) for the formal definition. Thus, a lower ε-dominance indicator for Γ denotes a higher resemblance between Γ and Γ_{ref} and, hence, a higher quality of mappings in Γ.

Figure 5.7 provides the ε-dominance of the distilled sets of mappings obtained by each investigated approach w.r.t. the quality objectives. For each application, the size of the original set of mappings is given in parentheses. Recall that each

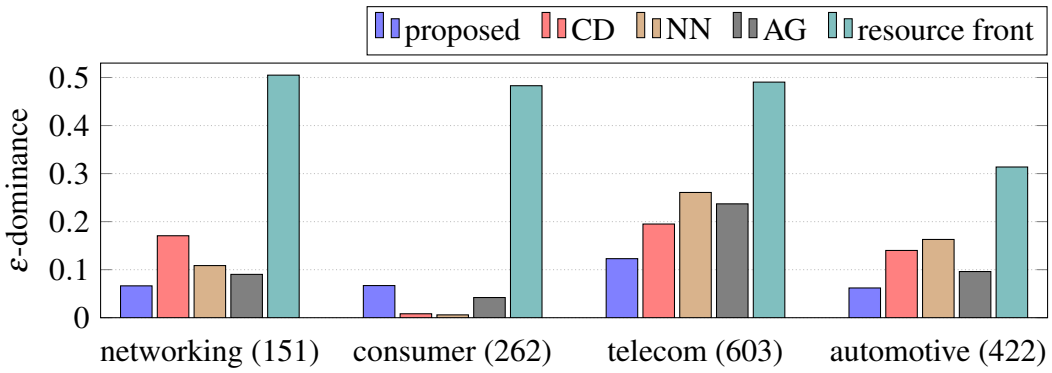


Figure 5.7: ε-Dominance of the distillation approaches (distillation capacity $k = 12$) w.r.t. quality objectives for different applications, where the original number of Pareto-optimal mappings is given in parentheses [PGT17].

distillation approach retains a total of only $k = 12$ mappings. For the evaluation of ε-dominance, the *quality front* is used as the reference set since it comprises the Pareto-optimal mappings w.r.t. quality objectives. Resource-efficient mappings retained in the *resource front* typically exhibit inferior quality measures as reflected by the high ε-dominance of the resource front in Figure 5.7. As illustrated, the proposed distillation methodology outperforms the three standard truncation techniques as it exhibits the lowest ε-dominance for nearly all applications. In comparison to the standard truncation approaches, the proposed methodology exhibits an ε-dominance reduction of 0.04–0.06 on average over all applications, which translates into a relative improvement of 35% on average (up to 42%).

In the case of the *consumer* application, the proposed distillation methodology exhibits a higher ε-dominance compared to standard techniques. For this application, the distribution of mappings in the space of quality objectives is such that a small number of quality regions contain a huge number of mappings while other regions are empty. Hence, the majority of distilled mappings are resource-efficient mappings while only a small portion of the distilled mappings belong to the quality-dominant regions, resulting in a degraded ε-dominance. Note that, for this application, the proposed approach significantly improves the embeddability rate, cf. Figure 5.6.

While Figure 5.7 provides a measure for the distribution of the quality trade-offs of distilled mappings, the quality trade-offs that can be practically achieved by a set of mappings can be assessed only in conjunction with embeddability since a trade-off becomes reachable only if the respective mapping is indeed embeddable. To assess the reachable quality trade-offs for each set of distilled mappings, during the run-time embedding experiment, the best quality achievable by the *embeddable* subset of mappings in terms of (i) end-to-end latency, (ii) energy consumption, and (iii) Energy-Latency Product (ELP) is recorded as well. ELP provides a measure of

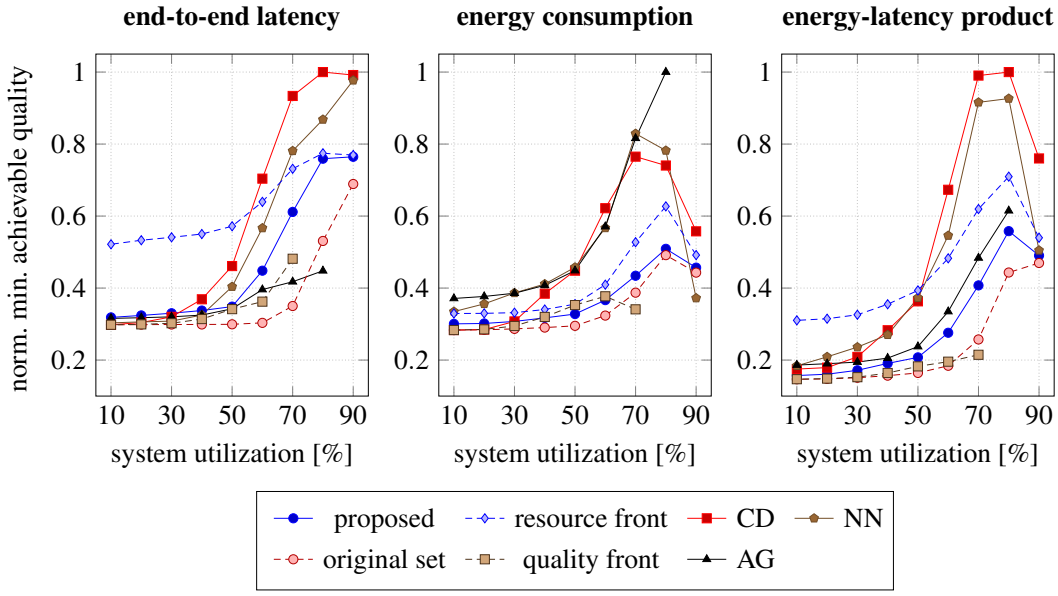


Figure 5.8: Normalized minimum latency (left), energy consumption (middle), and Energy-Latency Product (ELP) (right) achievable by the embeddable subset of mappings distilled by each approach for the telecom application. The *original set* serves as the best-case reference [PGT17].

the *efficiency* of quality trade-offs w.r.t. latency and energy consumption.

Figure 5.8 provides the obtained results for the exemplary application of *telecom*. Similar results are obtained for other applications as well. Here, the *original set* provides the best-case reference in all three aspects. As illustrated, the *quality front* performs similarly to the original set at low system utilization levels since, there, nearly all mappings are embeddable and, hence, the quality trade-offs of the distilled mappings become the decisive factor of the best achievable quality. However, as the system utilization increases, the mappings in the quality front lose their embeddability rapidly due to their high demand for resources. In fact, none of the mappings in the quality front are embeddable at a system utilization level of 70% or higher. On the other hand, the mappings in the *resource front* exhibit low energy consumption but suffer from high latency, resulting in a degraded ELP of the resource front as well.

Note the decrease in the energy consumption and ELP plots at high system utilization levels in Figure 5.8. This decrease contradicts the expected increasing trend observed at low system utilization levels and occurs for the following reason: The results in Figure 5.8 represent an average over 10 runs of the DSE. Mappings from different DSE runs lose their embeddability at different levels of system utilization. At a high system utilization, the—otherwise increasing—minimum energy (and, thus, minimum ELP) measures indeed decrease since, for some DSE runs, none of the mappings is embeddable anymore. This results in a lower average energy consumption

(and, thereby, lower ELP) of embeddable mappings, compare to the case where some mappings are still embeddable but exhibit a high energy demand (resulting in a higher average energy demand). In the case of a single DSE run, however, the measures monotonically increase. The same observation also applies where the minimum quality measure of a distilled set falls below that of the original set which will never occur if the results are investigated per DSE run.

As illustrated in Figure 5.8, the proposed methodology exhibits a high efficiency of achievable quality trade-offs indicated by its low ELP measures. In fact, compared to the standard truncation technique with the *best* ELP, the proposed methodology achieves an average ELP improvement of 7% (up to 18%) for system utilization levels of up to 80%. Taking the quality properties individually into account, the proposed approach achieves efficient measures w.r.t. *both* latency and energy consumption, whereas each standard truncation technique exhibits degraded achievable quality measures in one or even both criteria.

5.4 Related Work

In general, Hybrid Application Mapping (HAM) methodologies which consider mapping-set distillation rely on standard truncation techniques for the selection of representative mappings. In the optimization paradigm, the *multi-objective truncation* problem is well-studied [VDP15; HBK05], and several de facto standard truncation techniques are widely used. These techniques aim at retaining a *diverse* set of solutions by maximizing the *spacing*[4] among the distilled solutions [HBK05]. For instance, the so-called *Adaptive Grid (AG)* truncation technique in [KC03] places a multi-dimensional grid in the objective space and prunes crowded grid cells iteratively to attain a uniform spread of the retained solutions. In [ZLT01], solutions with the smallest distance to their *Nearest Neighbor (NN)* are eliminated iteratively to maximize the distances among the remaining solutions. The approach in [DPA+02] introduces a dispersion metric termed *Crowding Distance (CD)* to identify crowded regions of the objective space which are pruned accordingly for a uniform spread of retained solutions.

In recent years, several truncation techniques have also emerged which prioritize the *efficiency* of retained trade-offs over their diversity. They aim at identifying the so-called *knee-point solutions* which offer the smallest trade-off loss w.r.t. all objectives compared to their neighboring solutions (which often incur a large loss in at least one objective to attain a small improvement in another objective) [CYJ16; BSR+17; YJO18]. As discussed earlier in this chapter, multi-objective truncation techniques indeed fail in preserving *embeddability* when adopted for mapping-set distillation

[4] Spacing provides a measure of how evenly the retained solutions are spread over the objective space covered by the original set where a higher spacing implies a better diversity of retained solutions.

since they regard all objectives independently and, therefore, cannot consider the interdependence between resource objectives and the impact of their collective efficiency on embeddability. Thus, in the context of HAM, novel distillation methodologies are required which are specifically designed to preserve both the diversity of quality trade-offs and embeddability of the original mapping set.

The HAM methodologies in [SKS13; KSK+13] introduce problem-specific distillation schemes customized for an optimization of mappings w.r.t. throughput and energy consumption as quality objectives. In [SKS13], first, the mapping with the highest throughput and the one with the lowest energy consumption are identified for each combination of the number of allocated instances per core type. Then, from the identified mappings, the Pareto-optimal ones in the space of energy, throughput, and resource objectives are distilled. In [KSK+13], for each resource combination, the mapping with the highest ratio of throughput/energy is retained. In spite of their consideration of distillation, both approaches retain representative solutions for every possible resource combination which can grow prohibitively with the application's size (number of tasks) and platform's heterogeneity (number of resource types). To alleviate the RPM's overhead, both approached [SKS13; KSK+13] provide the RPM with a *statically sorted* set of mappings to eliminate the need for thorough feasibility checks. Nonetheless, numerous mappings may still have to be checked before the first embeddable one is found. Furthermore, a static prioritization of mappings for all possible online scenarios may degrade the efficiency of the RPM, cf. [WGT14].

The distillation methodology proposed in this work addresses the shortcomings of the approaches discussed above. It offers a (i) configurable distillation capacity and distills mappings with (ii) a high diversity of quality trade-offs while retaining mappings with (iii) efficient yet diverse resource requirements to enhance run-time embeddability.

5.5 Summary

The offline DSE in HAM incorporates several quality objectives, e.g., latency, throughput, and energy consumption, and several resource objectives, one per core type in the architecture, for the optimization of mappings. As a result, a huge set of Pareto-optimal mappings is typically obtained from the DSE which may impose an intolerable evaluation and decision-making overhead on the RPM and even put real-time constraints at the risk of violation by degrading the responsiveness of the RPM. Thus, only a small subset of the mappings may be practically considered for online use. The process of selecting a representative subset of mappings is referred to as *mapping-set distillation*. State-of-the-art HAM methodologies either disregard distillation altogether or rely on problem-specific distillation schemes or generic multi-objective truncation techniques that suffer from limited applicability and poor distillation performance, respectively.

As a remedy, this chapter presented in Section 5.2 an automatic and configurable

distillation methodology tailored to the distillation of Pareto-optimal mappings in the context of heterogeneous many-core systems. By distinguishing resource-related design objectives from quality-related objectives and proper handling of each set, the proposed methodology retains a configurable number of Pareto-optimal mappings, offering diverse quality trade-offs and, at the same time, modest resource demands to enhance the run-time embeddability of the application. A series of experimental results presented in Section 5.3 demonstrated the effectiveness of the proposed methodology in comparison to well-established multi-objective truncation techniques.

6
Hard Real-Time Online Mapping Adaptation

In line with the principles of Hybrid Application Mapping (HAM), the methodologies presented in Chapters 3 to 5 enable a timing-predictable and deadline-adhering execution of real-time applications in dynamic embedded systems targeting heterogeneous many-core platforms. In light of these methodologies, real-time applications can be launched and terminated dynamically on-demand using per-computed mappings with worst-case timing guarantees and at an affordable online mapping and management overhead. In this context, each application is launched using a mapping that is *feasible* w.r.t. the current situation, i.e., relies only on available resources and satisfies the non-functional requirements of the application and system, e.g., real-time constraints.

In many-core systems, the dynamism created by the state of resources and the workload of the applications puts the feasibility of mappings in use by running applications at a steady risk of annihilation: On the one hand, the relentless shrinking of feature size in semiconductor technologies invested in many-core platforms renders them increasingly susceptible to resource failures, e.g., due to hardware faults or overheating. Such situations annihilate the feasibility of active mappings that depend on the failed resources. On the other hand, changes in the non-functional requirements and/or workload of an application may lead to situations in which the mapping in use can no longer satisfy the given requirements, and hence, its feasibility is obliterated. To preserve the intended execution of applications in spite of such unforeseeable events, it becomes inevitable to adapt their mappings on-the-fly during their execution. *Mapping adaptation* is particularly crucial for hard real-time applications as a violation of their timing constraints is not tolerable and may even lead to system failure.

Mapping adaptation is realized using *task migration* and may involve the relocation of several (if not all) tasks of the application which entails a temporal overhead. To ensure a seamless satisfaction of hard timing constraints, it must be guaranteed that both the temporal overhead of the adaptation process and the timing behavior of the post-adaptation mapping of the application respect the given timing constraints. This, in turn, necessitates a *timing-predictable adaptation mechanism* so that the worst-case latency of the adaptation process can be analyzed. Moreover, timing analysis

is also required to derive the necessary timing guarantees. To that end, this chapter presents two timing-predictable mapping adaptation methodologies for hard real-time applications in heterogeneous many-core systems: a (i) *mapping reconfiguration methodology* and a (ii) *task migration methodology*.

In the proposed *mapping reconfiguration methodology*, online mapping adaptations are realized in the form of a timing-predictable reconfiguration (switch) between the statically computed mappings of applications with worst-case timing guarantees. This methodology consists of a timing-predictable (i) *reconfiguration mechanism* to be used online by the Run-time Platform Manager (RPM) to perform mapping reconfigurations in a timing-predictable manner, a (ii) *reconfiguration analysis* which is performed offline or hybrid (offline/online) to derive the necessary information (including timing guarantees) for each reconfiguration option to be provided to the RPM, and a lightweight (iii) *reconfiguration admission check* to be used by the RPM to examine the admissibility of each reconfiguration option based on its previously derived information and current timing constraints of the application.

The proposed *task migration methodology* enables a finer granularity of mapping adaptation by empowering the RPM to migrate *any* subset of the application's tasks to *any* destination cores with worst-case timing guarantees *without* relying on an offline analysis or being restricted to a precomputed set of options. Thus, compared to mapping reconfiguration, it enhances the flexibility of the RPM in choosing the degree of modification to be exerted on a mapping. The proposed task migration methodology consists of three components used online by the RPM: a timing-predictable (i) *migration mechanism* which enables the RPM to perform task migrations timing-predictably, a lightweight (ii) *migration analysis* to formally bound the worst-case latency of migration processes, and a lightweight (iii) *migration admission check* to examine the admissibility of the migration latency w.r.t. the timing constraints of the application and the fluctuations in its timing behavior during and after the migration processes.

The remainder of this chapter is organized as follows[1]. Section 6.1, elucidates the role of mapping adaptation in many-core systems and outlines its major challenges. In Section 6.2, the proposed mapping reconfiguration methodology is presented and its effectiveness is demonstrated using experimental evaluations. Following that, Section 6.3 presents the proposed task migration methodology and provides an experimental evaluation of it in comparison to mapping reconfiguration. Finally, an overview of related work on mapping adaptation in multi/many-core systems is provided in Section 6.4, followed by a summary of the chapter in Section 6.5.

[1] The contents of this chapter are based on and partly published in [PWG+17; PWG+19; PSW+20]. In particular, the contents of Section 6.2 are based on [PWG+17] which has appeared in the Proceedings of the 25th International Conference on Real-Time Networks and Systems (RTNS) and [PWG+19] published in the Springer Journal of Real-Time Systems (RTS). The contents of Section 6.3 are based on [PSW+20] which has appeared in the Proceedings of the workshop on Next Generation Real-Time Embedded Systems (NG-RES).

6.1 Mapping Adaptation in Many-Core Systems

Dynamic embedded systems rely on adaptive application mapping and management schemes, e.g., HAM, to achieve a high utilization of system resources in view of the dynamism in the mix and workload of the concurrently executed applications. In these systems, the mapping decisions made by the RPM are carried out based on the currently available information about the state of the system resources and demands of applications without prior knowledge about the future events which are typically not known in advance and arrive in an unpredictable manner, e.g., launch request issued by the user. In this context, it may be beneficial (e.g., for load balancing) or even inevitable (e.g., in the case of hardware faults) to revise and adapt the mapping of applications launched at an earlier stage according to an updated view of the state of system resources and active applications.

The online adaptation of the mapping of a running application is subject to several challenges and obstacles. For instance, since the adaptation process modifies the location of tasks in the system, an explicit synchronization between communicating tasks becomes necessary to prevent data-flow inconsistencies consequent to out-of-order delivery or even loss of messages. In fact, the mapping adaptation of an application must happen transparently to the application itself. However, this is hard to achieve since, especially in the case of heterogeneous many-core platforms, mapping adaptation often relies on source code modification (or at least augmentation). Moreover, it entails a temporal overhead which may be threatening for real-time applications. This section provides an overview of the role of mapping adaptation in many-core systems in Section 6.1.1, followed by a summary of major challenges of mapping adaptation in the context of heterogeneous many-core systems and hard real-time applications in Section 6.1.2. This overview provides the fundamentals required for the presentation of the proposed mapping reconfiguration and task migration methodologies in Sections 6.2 and 6.3.

6.1.1 Importance of Mapping Adaptation

The online adaptation of mappings in many-core systems involves the relocation of tasks between cores and, thus, relies on *task migration* support [MDP+00]. The initial practices of task migration date back to nearly three decades ago where it was used in the scope of distributed systems and cluster computers for load balancing, see, e.g., [CO95; LL94]. With the advent of embedded multi/many-core platforms, task migration has also been adopted widely in embedded systems to adapt the mapping of running applications, primarily for load balancing [HIB10] and energy management [HV12]. In recent years, task migration has also been used for power management and temperature balancing in many-core systems [RPM+19; LTH+14]. Moreover, in systems with a distributed memory scheme, task migration is used to improve data locality and, hence, mitigate the load on the interconnection media, e.g.,

the Network-on-Chip (NoC) [KZL+11].

As outlined above, mapping adaptation and task migration have proven effective in enhancing the performance, resource utilization, and energy efficiency of many-core systems. A mapping adaptation process, however, entails an inherent temporal overhead which may become threatening for real-time applications, especially in the case of *hard* timing constraints. Therefore, its practice is principally restricted to best-effort applications. Recent years, however, give evidence of a shift in the role of mapping adaptation in many-core systems such that it is increasingly becoming an inevitable mechanism to preserve the desired execution of applications rather than only a supplementary means for beneficial yet optional improvements. The ongoing reduction in technology feature size brings about an unprecedented increase in the rate of transient/permanent resource failures in many-core systems, e.g., due to hardware faults or overheating. To preserve the execution of applications in view of such mostly unpredictable events, mapping adaptation becomes necessary to evacuate the affected resources and relocate their mapped tasks to other resources, see, e.g., [SPM09].

In modern embedded applications, both the workload and the non-functional requirements may change dynamically and unforeseeably, e.g., upon user requests. Sometimes, the mapping in use by an application can no longer fulfill its requirements in view of a change in its workload and/or requirements. Therefore, mapping adaptation becomes inevitable to achieve a seamless satisfaction of the requirements, e.g., by moving the critical tasks to powerful processing resources. Note that a mapping that satisfies the most stringent requirements under all possible workload scenarios can, in principle, lift the need for mapping adaptation consequent to workload/requirement fluctuations. However, such mappings (if at all available) typically exhibit an immense over-reservation and under-utilization of resources for the vast majority of possible run-time situations. Therefore, in practice, they are typically avoided whenever possible. As a result, mapping adaptation remains the key solution to cope with the dynamism in the workload and/or requirements of applications.

Mapping adaptation is crucial for HAM methodologies in particular: In HAM, applications are launched on-demand by selecting a *feasible* precomputed mapping, i.e., a mapping which relies only on the currently available resources and satisfies the non-functional requirements of the application. In some cases, an incoming application cannot be launched at all, since, for none of its precomputed mappings, all required resources are available. In such situations, mapping adaptation becomes crucial to release the resources required for launching the incoming application by reorganizing the already launched applications without disrupting their execution.

6.1.2 Challenges of Mapping Adaptation

Establishing the support for online mapping adaptation and task migration in many-core systems is subject to several major challenges that are outlined in the following.

Context Relocation

The memory scheme of a system has a strong impact on the complexity of establishing mapping adaptation support in it. In the case of a shared memory scheme, the migration of tasks is often realized implicitly without requiring an actual relocation of their context between different memories. Here, the *context* of a task denotes the minimum set of data required for a seamless resumption of its execution after migration [Smi88]. It consists of the address space of the task (i.e., code, data, and stack) as well as its Process Control Block (PCB). The PCB of a task reflects its state comprising the process identifier, contents of core registers, period, deadline, etc. In the scope of many-core systems, the shared memory scheme within each tile enables task migrations between different cores of the same tile, i.e., *intra-tile migrations*, to be realized *implicitly*. Contrarily, due to the distributed memory scheme between different tiles of a many-core platform, task migrations between different tiles, i.e., *inter-tile migrations*, involves an *explicit relocation* of the context of each migrating task between its source and destination tiles. Here, an *explicit reservation* of a NoC route for the relocation of the context becomes necessary to preserve composability.

Synchronization and Data-Flow Consistency

Inter-tile migrations also necessitate an *explicit synchronization* of data-dependent tasks of the application to preserve its data-flow consistency. For this purpose, the predecessor tasks of a migrating task (which send messages to it) must be notified about the relocation so that new messages are sent to the new location thereafter. Moreover, when migrating a task, a number of unprocessed input messages from its predecessors may already exist or later arrive at the source location. These messages must be made available at the destination location by the time the execution of the migrating task is resumed there. In fact, to preserve data-flow consistency, all input messages of the migrating task must be provided to it in the original order they would have reached it in the absence of the migration. Likewise, output messages of migrating tasks must reach their target tasks in their originally intended order. Improper handling of messages during migration processes may give rise to an inconsistent execution behavior of the application and/or erroneous computation results. When performing task migrations, a synchronization between data-dependent tasks often relies on *message-forwarding* and *buffer-reordering* service routines which are, however, typically not available in embedded systems. This complicates inter-tile migrations and can potentially restrict the practice of mapping adaptation to intra-tile migrations only.

Execution Preemption

Before the relocation process of a migrating task is initiated, its execution must be suspended to ensure that its whole context is available in the memory to be relocated

to the destination location. Execution suspension may be realized *preemptively*, *non-preemptively*, or *cooperatively*. These schemes offer a trade-off between the initiation delay and the context size of the migration and are introduced in the following.

In a *preemptive* suspension scheme, the operating system may stop the execution of the task at *any* point during its execution. This enables an immediate initiation of the migration at the cost of a relatively large context size. Contrarily, in *non-preemptive* suspension, the execution of the task may be suspended only after its current iteration of execution (job) is completed. This often leads to an inflated initiation delay for the migration, but benefits from a relatively small context size to be relocated. Finally, in *cooperative* suspension, the execution may be suspended only at so-called *migration points* specified in the source code of the task [SH98]. Migration points are inserted (by the programmer) at code locations where the context size is particularly small so that a moderate trade-off between migration initiation delay and context size is achieved. The choice of suspension scheme for mapping adaptation strongly depends on the *heterogeneity* of the source and destination cores of a migration as discussed in the following.

Resource Heterogeneity

The complexity of a task migration strongly depends on the heterogeneity of its source and destination cores. In the case of task migrations between homogeneous cores, the architectural and micro-architectural compatibility between the source and destination cores enables all three variants of suspension schemes to be used. In the case of heterogeneous cores, however, an adaptation of the context may become necessary subject to the *Instruction Set Architecture (ISA)* of the source and destination cores.

For task migrations between heterogeneous cores with the same ISA, the micro-architectural differences between the two cores necessitate *state-transformation* service routines under a preemptive or a cooperative suspension scheme to adapt the process state (w.r.t. the contents of core registers) of the task to the micro-architecture of the destination core. Since often state-transformation services are too complex to be provided in an embedded setting, the practice of preemptive or cooperative suspension schemes in many-core systems is typically restricted to homogeneous migrations only. Contrarily, since the process state becomes irrelevant under a non-preemptive suspension scheme, a migration of tasks between heterogeneous cores does not depend on state-transformation services under a non-preemptive suspension scheme.

For task migrations between heterogeneous cores with different ISAs, *binary-translation* service routines become necessary to render the binary (source code) of the task compatible with the ISA of the destination core. Similar to state-transformation services, binary-translation services are too complex to be provided in an embedded system. In this context, *fat binaries* may be used to empower task migrations between heterogeneous-ISA cores. A fat binary consists of a collection of binaries, one for each available ISA, from which the compatible binary can be selected at the

location where the task will be executed [MR14]. Note that, due to the lack of per-instruction correspondence between different binaries of a task, migrations between heterogeneous-ISA cores are typically restricted to non-preemptive or cooperative suspension schemes.

Adaptation Timing Guarantees

Task migration, especially in the case of an inter-tile migration, entails a temporal overhead that must be accounted for in the context of real-time applications. In fact, a migration process may affect the timing behavior of the application in two aspects: (i) *During* the migration process, the timing behavior of the migrating tasks is affected due to their suspension. This also affects the timing of the consumption and production of messages by the migrating tasks. (ii) *After* the migration process, the (worst-case) timing properties of migrating tasks and their input/output messages may differ from their pre-migration measures. For instance, the migration of a task between heterogeneous cores often results in a change in its Worst-Case Response Time (WCRT). Likewise, a change in the length of the NoC route of an input/output message of a migrating task results in a change in its Worst-Case Traversal Time (WCTT) as well.

To preserve a consistent satisfaction of the timing constraints of real-time applications during and after mapping adaptation, it is necessary to verify that both the temporal overhead of the adaptation process and the changes in the timing behavior of the tasks and messages of the application adhere to the given real-time constraints. Such a verification scheme necessitates a (i) worst-case timing analysis to bound the latency of the migration process, (ii) worst-case timing analyses to bound the post-migration WCRT of tasks and WCTT of messages, and a (iii) feasibility check to verify that the bounds derived in (i) and (ii) are guaranteed to respect the timing constraints of the application seamlessly. The adaptation mechanisms employed in many-core systems are, however, typically timing unpredictable which renders a formal worst-case timing analysis of the adaptation process impossible or results in timing guarantees that are too loose to be of any practical use.

Contributions Overview

This work presents two methodologies, namely, a (i) *mapping reconfiguration methodology* and a (ii) *task migration methodology*, which enable the online mapping adaptation of real-time applications in heterogeneous many-core systems while addressing the adaptation challenges outlined above. In the proposed mapping reconfiguration methodology, mapping adaptations are realized in the form of a reconfiguration (switch) between the precomputed mappings of an application. The proposed task migration methodology, on the other hand, enables a finer granularity of adaptation by allowing any subset of an application's tasks to be migrated to any desired locations.

Each of the proposed mapping adaptation methodologies is supported with a (i) *timing-predictable adaptation mechanism* (reconfiguration or migration) that enables the RPM to perform mapping adaptations in a timing-predictable manner. Moreover, for each methodology, a (ii) *worst-case timing analysis* is developed to derive a safe upper bound on the latency of the adaptation process formally. The availability of mappings at design time enables the mapping reconfiguration methodology to benefit from an offline or hybrid (offline/online) analysis to derive all necessary information (including timing guarantees) for the reconfiguration between each pair of source and target mappings. This information is then provided to the RPM to perform informed reconfigurations. On the other hand, the task migration methodology benefits from a lightweight timing analysis that enables the RPM to consider *any* form of mapping adaptation without being restricted to a precomputed set of adaptation options. Each methodology provides the RPM also with an (iii) *adaptation admission check* which examines the admissibility of an adaptation w.r.t. the timing constraints of the application to ensure a seamless satisfaction of the timing requirements during and after the adaptation process. For its examination, the admission check takes into account the timing properties of the application before and after the adaptation, the changes in its timing behavior during the adaptation process, and its given timing constraints.

Note that an investigation of *mapping adaptation policies* used by the RPM, i.e., the identification of conditions triggering a mapping adaptation and the decision criteria of the RPM for selecting a proper form of adaptation, falls out of the scope of this work. Since the proposed adaptation methodologies provide a means to perform a given adaptation timing-predictably with worst-case timing guarantees, they can be employed in conjunction with any mapping adaptation policy that decides on the time, type, and choice of adaptations to be performed.

6.2 Hard Real-Time Mapping Reconfiguration

This section presents the proposed mapping reconfiguration methodology. Figure 6.1 provides an overview of the reconfiguration methodology in the context of Hybrid Application Mapping (HAM) where the reconfiguration-related steps and components are highlighted in blue and outlined in the following.

The proposed methodology enables the RPM to perform mapping adaptations in form of a reconfiguration (switch) between the precomputed mappings of the application. Each mapping reconfiguration involves the migration of several tasks to construct the target mapping from the source mapping. In the proposed methodology, at design time, *mapping reconfiguration analysis* is performed on the (Pareto-optimal) mappings obtained from the Design Space Exploration (DSE) to collect all necessary information (including timing guarantees) for each and every possible reconfiguration. To that end, for each pair of source and target mappings, the analysis first determines migrating tasks and their destination cores. For each inter-tile task migration, it

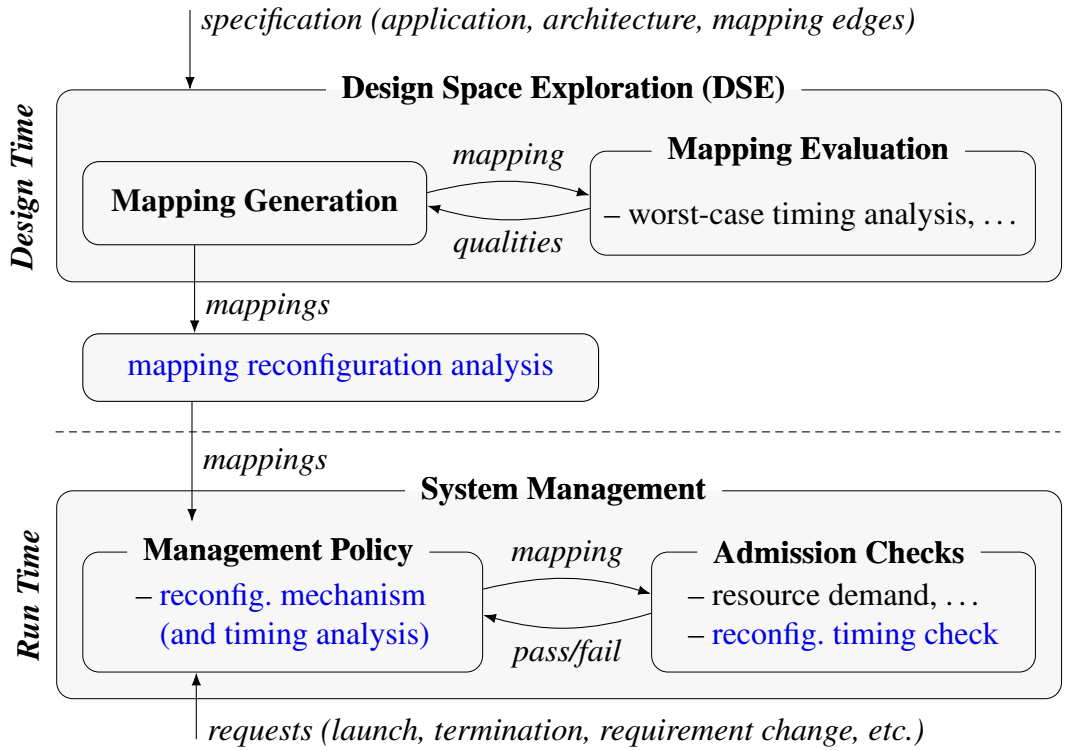


Figure 6.1: An overview of the proposed mapping reconfiguration methodology in the context of HAM. The contributions are highlighted in blue.

identifies a NoC route with minimal allocation overhead (primary goal) and lowest worst-case transfer latency (secondary goal). Finally, using a formal worst-case timing analysis, it bounds the latency of each migration and, subsequently, the end-to-end latency of the mapping reconfiguration process.

The statically derived migration routes and timing guarantees for each reconfiguration option are provided to the RPM to perform deadline-adhering mapping reconfigurations on-demand. For this purpose, the RPM is provided with a *timing-predictable reconfiguration mechanism* that enables it to perform mapping reconfigurations in a timing-predictable manner (for which the statically derived timing bounds are guaranteed to hold). The RPM is also provided with a *reconfiguration admission check* to verify the real-time conformity of each reconfiguration option based on the timing guarantees provided for it w.r.t. the current timing constraints of the application and, thereby, avoid reconfiguration options which may lead to the violation of real-time constraints.

As an alternative to the offline analysis, a *hybrid* reconfiguration analysis may also be established. In such a scheme, the computation of efficient migration routes and parts of the reconfiguration timing analysis are performed offline, while some parts of the timing analysis are performed online by the RPM to obtain tighter timing

guarantees by considering the actual availability of the NoC bandwidth in the analysis.

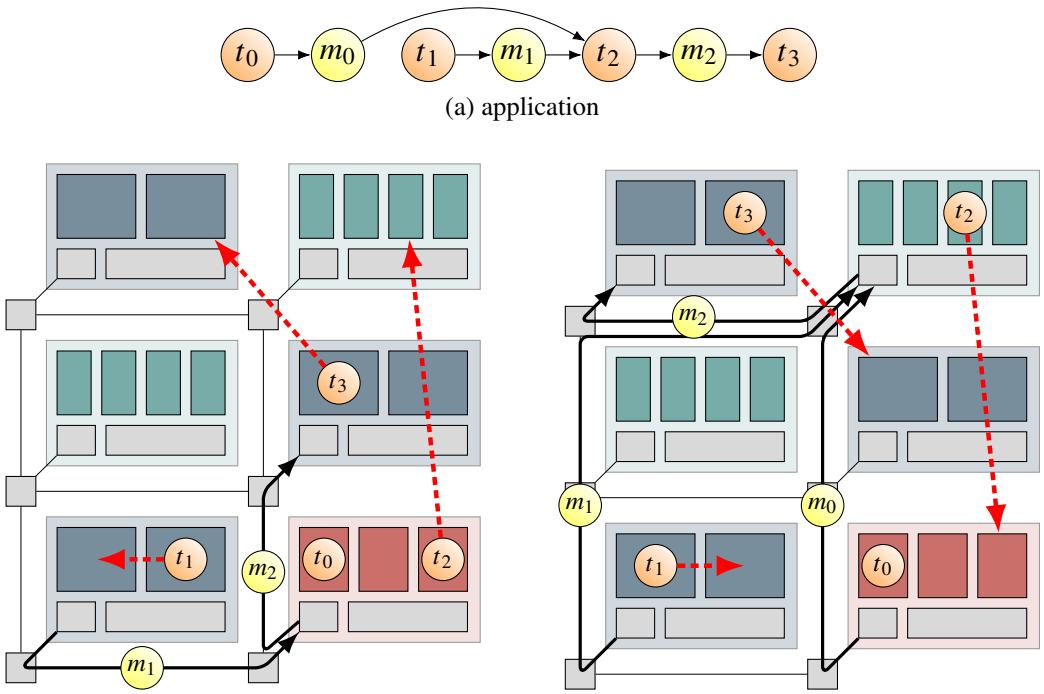
In the following, Section 6.2.1 presents the mapping reconfiguration mechanism which specifies the procedure of performing mapping reconfigurations at run time. Then, Section 6.2.2 presents the offline reconfiguration analysis and details on the optimization of migration routes and reconfiguration worst-case timing analysis. Following that, Section 6.2.3 presents the reconfiguration admission check to be used online by the RPM to examine the real-time conformity of mapping reconfigurations. Finally, the effectiveness of the proposed reconfiguration methodology and the overheads associated with it are experimentally evaluated in Section 6.2.4.

6.2.1 Mapping Reconfiguration Mechanism

A mapping reconfiguration between a source and a target mapping involves the migration of a set of tasks to construct the target mapping from the source mapping. Figure 6.2 illustrates an exemplary application with two mappings of it (mapping μ_0 and mapping μ_1) on a 2×3 heterogeneous many-core architecture. In each mapping, the red dashed arrows indicate the set of migrating tasks and their destination cores for a reconfiguration to the other mapping. For instance, for the mapping reconfiguration $\mu_0 \rightarrow \mu_1$, tasks t_1, t_2, and t_3 must be migrated as indicated by the red dashed arrows in Figure 6.2b.

The shared memory scheme within each tile of a many-core platform enables intra-tile migrations to be realized implicitly, i.e., without the relocation of data between different memories, e.g., for task t_1 in Figure 6.2b. In contrast, the distributed inter-tile memory scheme necessitates an explicit relocation of the task's context between the source and destination tiles in the case of an inter-tile migration, e.g., for tasks t_2 and t_3 in Figure 6.2b. The inter-tile migration of a task requires a so-called *migration route* over the NoC to establish a connection between the source and destination tiles of the migrating task. To preserve composability, the migration route of each task must be reserved prior to the initiation of the reconfiguration process. To that end, given the migration route of a task, one or more arbitration slots must be reserved on each link located on the migration route to establish the necessary connection. In the proposed mapping reconfiguration methodology, given a pair of source and target mappings, (i) the migration route of each task, (ii) the (minimum) number of arbitration slots to be reserved per router for the reconfiguration process, and (iii) the worst-case latency of each migration are analyzed offline (presented in Section 6.2.2) and are provided to the RPM. Therefore, all necessary information about the migrations involved in a mapping reconfiguration process is available to the RPM at run time to perform mapping reconfigurations on-demand.

In the context of hard real-time applications, a mapping reconfiguration must be performed *timing-predictably* so that worst-case timing guarantees can be derived for it to enable examining its conformity with the hard timing constraints of the application. To enable that, the proposed mapping reconfiguration methodology introduces



(a) application

(b) mapping μ_0

(c) mapping μ_1

Figure 6.2: Illustration of mapping reconfiguration between two exemplary mappings, μ_0 and μ_1, of an application. In each mapping, the red dashed arrows indicate the set of migrating tasks and their destination cores for a reconfiguration to the other mapping.

a *timing-predictable reconfiguration mechanism* to be used online by the RPM to perform mapping reconfigurations timing-predictably. For the proposed reconfiguration mechanism, a *reconfiguration timing analysis* is developed as well (presented in Section 6.2.2) which provides worst-case latency guarantees for each mapping reconfiguration performed in line with the proposed reconfiguration mechanism. Moreover, a lightweight *reconfiguration admission check* (presented in Section 6.2.3) is provided to be used by the RPM online to examine the real-time conformity of a mapping reconfiguration based on the timing guarantees provided for it by the reconfiguration timing analysis.

At run time, when a mapping reconfiguration shall take place[2], the RPM first examines the admissibility of the reconfiguration w.r.t. the current availability of resources

[2]As mentioned earlier, an investigation of mapping adaptation *policies*, i.e., the decisions on the time, the type (reconfiguration or migration), and the concrete choice of adaptation to be performed falls out of the scope of this work. The methodologies proposed in this chapter provide timing-predictable mapping adaptation *mechanisms* that enable performing adaptations with hard timing guarantees and can be employed in conjunction with any mapping adaptation policy.

and current timing constraints of the application[3]. A reconfiguration from mapping μ_0 to mapping μ_1 is considered admissible if (i) the required resources for mapping μ_1 are available, (ii) the NoC budgets required to establish the migration routes are available, and (iii) the reconfiguration is verified to respect the timing requirements of the application. Once an admissible reconfiguration option has been identified, the RPM reserves the required resources as outlined in (i) and (ii) and initiates the reconfiguration process which is coordinated by the proposed reconfiguration mechanism as follows.

To perform a mapping reconfiguration, first, the execution of the application on the source mapping is suspended. To that end, the execution of its tasks is suspended in line with the preemption scheme in use, in a procedure similar to the context-switch procedure of the operating system. Also, the injection of its inter-tile messages into the NoC is blocked such that the application comes to a complete standstill. Note that the latter is crucial for preserving the data-flow consistency of the application as it prevents the loss and out-of-order delivery of messages without relying on message-forwarding and buffer-reordering service routines which are often not available in an embedded system. The suspension process entails a delay termed *suspension latency* whose upper bound is analyzed previously at design time. Therefore, once the suspension command has been issued, the RPM waits for the duration of the suspension latency to ensure a complete suspension of the application. After the application has been suspended, the migration of tasks takes place. While intra-tile migrations do not involve the relocation of the context of tasks, for inter-tile migrations, the context of each respective task is relocated to its destination tile over the statically determined migration routes. The unprocessed input messages and blocked output messages of each migrating task which are available at the source location of the migration are relocated together with the task to prevent data-flow inconsistencies, e.g., consequent to the loss of messages or their out-of-order delivery. Once all migrations have been performed, the target mapping is constructed. Thus, the RPM may resume the execution of the application on the target mapping and release those reserved resources which were reserved for the source mapping or the migration routes but are not required by the target mapping.

6.2.2 Mapping Reconfiguration Analysis

Given a set of (Pareto-optimal) mappings, the proposed mapping reconfiguration analysis provides three sets of information for the mapping reconfiguration between each pair of mappings: (i) the migration route of each migrating task, (ii) an upper bound on the worst-case latency of each step of the reconfiguration process as well as the whole reconfiguration, and (iii) an upper bound on the end-to-end latency of the

[3]Note that in the case of a thermally composable HAM methodology as introduced in Chapter 4, the thermal-safety admission check is performed as well to ensure that the target mapping is thermally safe for the current system state.

application in case the reconfiguration is performed. This information empowers the RPM to perform mapping adaptations on-demand with worst-case timing guarantees.

Given the pair of source and target mappings of a reconfiguration, first, efficient migration routes are identified for the migrating tasks that are to be relocated between different tiles. To that end, a migration-route optimization approach is presented which identifies migration routes with minimal allocation cost (primary goal) and minimal worst-case latency (secondary goal). Once the migration routes have been determined, worst-case timing guarantees are derived for different steps of the reconfiguration process and on the end-to-end latency of the application in view of the reconfiguration. The three steps of the reconfiguration analysis are detailed in the following.

Migration Route Optimization

The choice of migration routes for a mapping reconfiguration strongly affects the resource and timing efficiency of the reconfiguration and, hence, its run-time feasibility. Therefore, achieving a minimal allocation cost for establishing the migration routes for a mapping reconfiguration is a primary goal of the proposed reconfiguration methodology to enhance the feasibility of mapping reconfigurations in view of a dynamic availability of resources at run time. The allocation overhead of a reconfiguration can be mitigated by utilizing the NoC budget that is reserved for the transfer of messages in the source and target mappings of the reconfiguration. Recall from Section 6.2.1 that the resources required by the source and target mappings are reserved for the application prior to the initiation of the reconfiguration process. Since the execution of the application is suspended during the reconfiguration process, the NoC budget reserved for the messages in the involved mappings can be used to establish the necessary migration routes. This budget may, however, not suffice for establishing all necessary migration routes. For instance, for the mapping reconfiguration $\mu_0 \rightarrow \mu_1$ in Figure 6.2, the migration route of task t_2 can be established merely using the NoC budget reserved for the transfer of messages m_0, m_1, and m_2 in mappings μ_0 and μ_1. For task t_3, however, a connection between the source and destination tiles cannot be established merely based on the reserved budget. Therefore, additional NoC budget must be reserved to construct its migration route.

In this work, the efficiency of a migration route is quantified in terms of (i) its *allocation overhead*, denoting the number of arbitration time slots that must be reserved additionally on the NoC for its construction, and (ii) its *Worst-Case Relocation Latency (WCRL)*, denoting an upper bound on its worst-case latency for the relocation of the migrating data. To minimize the allocation overhead, the NoC budget reserved for the source/target mappings is utilized for the construction of migration routes so that additional budgets are allocated only if the available budget does not suffice for establishing the required connection. To mitigate the WCRL, those links that offer a higher reserved budget are prioritized for inclusion in the migration route of each task.

For the identification of efficient migration routes, this work introduces a graph-

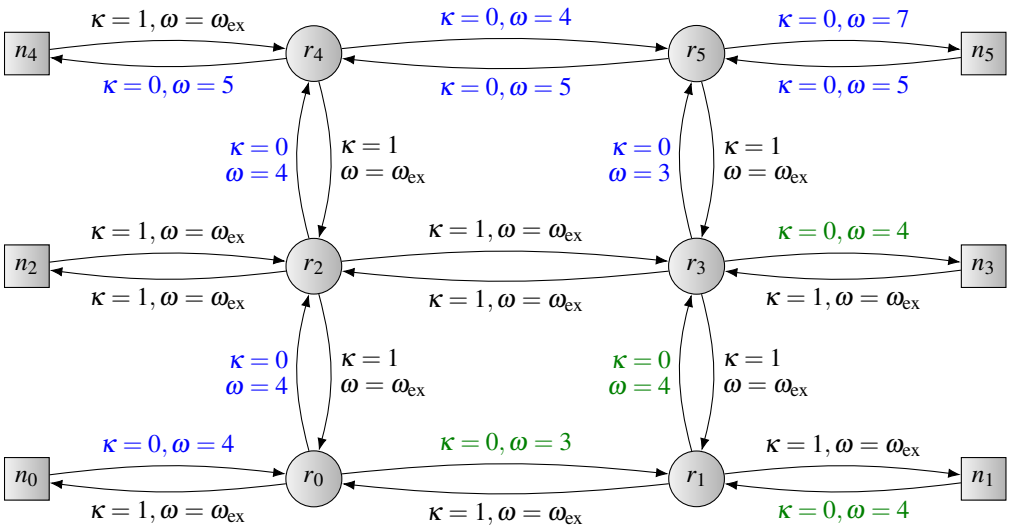


Figure 6.3: Routing graph constructed based on mappings μ_0 and μ_1 in Figure 6.2. It reflects the NoC availability w.r.t. NoC budgets reserved in μ_0 and μ_1 and is used for the identification of low-cost and low-latency NoC routes for the migration of tasks in mapping reconfigurations between μ_0 and μ_1.

based intermediate representation of the NoC availability termed the *routing graph* and denoted as $G_{\text{route}}(V_{\text{route}}, E_{\text{route}})$. The vertices $V_{\text{route}} = R \cup N$ of a routing graph comprise the set of routers R and the set of Network Adapters (NAs) N in the architecture graph of the platform. Edges $E_{\text{route}} \subset E_{\text{arch}}$ represent the NoC links connecting each router to its adjacent routers and its local NA. The routing graph is constructed for each given pair of mappings and is used to identify efficient migration routes for the reconfigurations between the two mappings. To reflect the NoC availability w.r.t. the reserved NoC budget in the source/target mappings, each edge $e \in E_{\text{route}}$ in the routing graph is annotated with two attributes, namely, *allocation cost* κ and *reserved arbitration weight* ω. Figure 6.3 illustrates the routing graph corresponding to mappings μ_0 and μ_1 in Figure 6.2, assuming reserved arbitration weights of $W^{m_1}_{\rho_{m_1}} = 3$ and $W^{m_2}_{\rho_{m_2}} = 4$ for inter-tile messages in mapping μ_0 and $W^{m_0}_{\rho_{m_0}} = 3$, $W^{m_1}_{\rho_{m_1}} = 4$, and $W^{m_2}_{\rho_{m_2}} = 5$ for inter-tile messages in mapping μ_1. In the following, first, the link attributes are introduced, and the cost and WCRL analysis of migration routes is elucidated. Then, the route optimization procedure is presented.

Route Allocation Cost

The allocation overhead of a migration route denotes the overall additional NoC budget (arbitration time slots) required to construct that route. To reflect the allocation overhead per NoC link, each edge $e \in E_{\text{route}}$ in the routing graph is annotated with two attributes: (i) *allocation cost* $\kappa_e \in \{0, 1\}$ which denotes whether the source/target

mappings have reserved *any* budget on the NoC link corresponding to edge e, and (ii) *reserved arbitration weight* $\omega_e \in \mathbb{N}^+_{\leq K_e}$ which denotes the number of arbitration time slots available on the link corresponding to edge e where K_e reflects the arbitration capacity of the respective link. For edges corresponding to already allocated links, the allocation cost is $\kappa = 0$ and the reserved arbitration weight ω is equal to the number of time slots reserved on that link as part of the source/target mappings[4]. For edges corresponding to links that are not allocated in either of the involved mappings, the allocation cost is $\kappa = 1$ and the arbitration weight is equal to an adjustable minimal budget $\omega = \omega_{\text{ex}}$. Here, a total of ω_{ex} time slots must be reserved additionally on the respective link if it is contained in the migration route. In Figure 6.3, the attributes of allocated links (with $\kappa = 0$) are given in color which denotes whether the available arbitration weight is allocated by mapping μ_0 (green) or mapping μ_1 (blue). Based on the link attributes above, the allocation cost of a migration route $\rho_{\text{mig}} = (e \mid e \in E_{\text{route}})$ consisting of a connected sequence of edges $e \in E_{\text{route}}$ is calculated as:

$$\text{cost}(\rho_{\text{mig}}) = \sum_{e \in \rho_{\text{mig}}} \kappa_e \cdot \omega_e \tag{6.1}$$

Worst-Case Relocation Latency (WCRL)

The WCRL of a migrating task t on migration route ρ_{mig} denotes the maximum end-to-end latency of the migration process of t which can be analyzed formally based on the size of the *migration data-set* of t and the amount of budget available on the links involved in the transfer of t. The migration data-set of a migrating task t denotes the set of data that is required at the destination location for a seamless resumption of t's execution after the migration. It consists of t's context (code, data, state, etc.) of size $\Psi_{\text{ctx}}(t)$ and its unprocessed input- and blocked output messages $m \in \text{IO}(t)$ which are available at the source tile of the migration and must be relocated together with t. Thus, the size of the migration data-set of a task t, denoted by $\Psi_{\text{mig}}(t)$, can be bounded using Eq. (6.2) where PLD_m denotes the maximum payload size of message m.

$$\Psi_{\text{mig}}(t) = \Psi_{\text{ctx}}(t) + \sum_{m \in \text{IO}(t)} PLD_m \tag{6.2}$$

Given $\Psi_{\text{mig}}(t)$, the WCRL of task t on migration route ρ_{mig} can be analyzed using Eq. (6.3) where $\rho^{r \to r}_{\text{mig}}$ denotes the sequence of inter-router links on the migration route ρ_{mig} (i.e., excluding the first and last links on ρ_{mig} connected to the NAs on the source/destination tiles), cf. Eq. (3.6). The Transmitter (TX) delay $D^{\text{TX}}(\Psi_{\text{mig}}, \rho_{\text{mig}})$ and the Receiver (RX) delay $D^{\text{RX}}(\Psi_{\text{mig}}, \rho_{\text{mig}})$ can be bounded using the NA latency analysis presented in Section 3.3.2 based on the available budget ω on the first and

[4] If a link is allocated in both source and target mappings, the *maximum* budget reserved on that link among the two mappings is considered to be available. Within one mapping, if a link is contained on the route of multiple messages, the *sum* of the reserved budgets is taken to be available.

last links on ρ_{mig} which represent the arbitration weight of the relocation process on the TX and RX units located on the source and destination tiles of the migration, respectively.

$$WCRL(\Psi_{\text{mig}}, \rho_{\text{mig}}) = D^{\text{TX}}(\Psi_{\text{mig}}, \rho_{\text{mig}}) + D^{\text{NoC}}(\Psi_{\text{mig}}, \rho_{\text{mig}}^{r \to r}) + D^{\text{RX}}(\Psi_{\text{mig}}, \rho_{\text{mig}}) \quad (6.3)$$

The analysis of the NoC latency $D^{\text{NoC}}(\Psi_{\text{mig}}, \rho_{\text{mig}}^{r \to r})$ in Eq. (6.3), however, must be adapted as given in Eq. (6.4) to bound the transfer latency on a NoC route which consists of links with *non-identical* available budgets. In Eq. (6.4), the first summand bounds the basic transfer latency of $F_{\Psi_{\text{mig}}}$ flits of data over the NoC routers on $\rho_{\text{mig}}^{r \to r}$, i.e., the transfer latency in the absence of any NoC interferences. The second summand in Eq. (6.4) bounds the worst-case jitter imposed on the transfer of the head flit in the presence of interfering flows. Here, on each traversed link $e \in \rho_{\text{mig}}^{r \to r}$ with a router arbitration period of P^{router} and a reserved link budget of ω_e available for the migration, a maximum wait time of $(P^{\text{router}} - \omega_e \cdot \tau^{\text{NoC}})$ may be imposed on the head flit where τ^{NoC} denotes the duration of a NoC clock cycle. Finally, the third summand in Eq. (6.4) bounds the jitter imposed between the arrival time of the head and tail flits in the presence of interferences. This jitter is imposed on the link with the lowest reserved budget $\omega_{\min} = \min_{e \in \rho_{\text{mig}}^{r \to r}} \omega_e$. On this link, a maximum of $(\lceil F_{\Psi_{\text{mig}}} / \omega_{\min} \rceil - 1)$ additional arbitration periods may be required for the routing of body and tail flits where, in each period, a maximum wait time of $(P^{\text{router}} - \omega_{\min} \cdot \tau^{\text{NoC}})$ may be imposed.

$$\begin{aligned} D^{\text{NoC}}(\Psi_{\text{mig}}, \rho_{\text{mig}}^{r \to r}) = {} & (F_{\Psi_{\text{mig}}} - 1 + |\rho_{\text{mig}}^{r \to r}| \cdot D^{\text{router}}) \cdot \tau^{\text{NoC}} \\ & + \sum\nolimits_{e \in \rho_{\text{mig}}^{r \to r}} \left(P^{\text{router}} - \omega_e \cdot \tau^{\text{NoC}} \right) \\ & + \left(\left\lceil F_{\Psi_{\text{mig}}} / \min_{e \in \rho_{\text{mig}}^{r \to r}} \omega_e \right\rceil - 1 \right) \cdot \left(P^{\text{router}} - \min_{e \in \rho_{\text{mig}}^{r \to r}} \omega_e \cdot \tau^{\text{NoC}} \right) \end{aligned} \quad (6.4)$$

Migration Route Selection Procedure

The identification of efficient migration routes is realized using an iterative procedure developed based on Dijkstra's shortest path algorithm [Dij+59]. Given a pair of NA vertices in the routing graph (denoting the source and destination tiles of the migration), the proposed procedure identifies an efficient migration route with minimized allocation cost (primary goal) and WCRL (secondary goal) between the two vertices. For this purpose, starting at the source node, partial migration routes are iteratively constructed and extended towards the destination node until the complete migration route with the lowest cost (primarily) and WCRL (secondarily) is obtained. Initially, the partial route consists of the source node (NA) and its immediate router. The cost and WCRL of the partial route are derived using Eqs. (6.1) and (6.3), respectively, and the partial route is stored in an initially empty route queue.

Once the initial partial route has been constructed and evaluated, further partial routes are constructed iteratively by extending the existing partial route that has the lowest cost (primarily) and WCRL (secondarily) by one hop towards the destination node. To that end, we iteratively (i) take out from the route queue the partial route with the lowest cost (primarily) and WCRL (secondarily). The selected partial route is then (ii) extended by one hop to the adjacent router(s) of its tail router towards the destination node. (iii) The cost and the WCRL of the resulting partial route(s) are then calculated using Eqs. (6.1) and (6.3), respectively, and (iv) the partial route(s) are added to the route queue. Steps (i)–(iv) are repeated iteratively until the partial route selected from the route queue in step (i) already ends at the destination node (NA), denoting that the optimal route is found.

The route selection procedure above is performed for all migrating tasks sequentially. After the route selection for a migrating task is finished, the link attributes in the routing graph are updated to reflect the availability of the additionally allocated links for the subsequent migrations. To that end, the cost attribute of the additionally allocated links (initially set to $\kappa = 1$) will be updated to $\kappa = 0$.

Worst-Case Timing Analysis

To guarantee a seamless satisfaction of the timing constraints of a real-time application in spite of its reconfiguration, the worst-case latency of the mapping reconfiguration process must be bounded and its admissibility w.r.t. the timing constraints of the application must be examined. The reconfiguration timing analysis presented in the following derives timing upper bounds (i) on the worst-case latency of each step of the reconfiguration process as well as the whole reconfiguration, and (ii) on the end-to-end latency of the application in case the reconfiguration is performed. The analysis is performed offline and the obtained guarantees will be provided to the RPM for a lightweight admission check of mapping reconfigurations at run time.

Reconfiguration Latency (Δ_{reconfig}) The timeline of a mapping reconfiguration is illustrated in Figure 6.4. As also summarized in Eq. (6.5), the end-to-end latency of a mapping reconfiguration, denoted by Δ_{reconfig}, consists of (i) the worst-case latency of suspending the execution of the application on the source mapping, referred to as the *suspension latency* of the reconfiguration and denoted by Δ_{susp}, and (ii) the worst-case latency of relocating the migrating tasks to construct the target mapping, referred to as the *relocation latency* of the reconfiguration and denoted by Δ_{reloc}. In the following, a safe upper bound is formally derived for each of these latency components.

$$\Delta_{\text{reconfig}} = \Delta_{\text{susp}} + \Delta_{\text{reloc}} \tag{6.5}$$

Suspension Latency (Δ_{susp}) The suspension latency of a mapping reconfiguration, denoted by Δ_{susp}, reflects the time it takes (i) to stop the execution of the tasks of

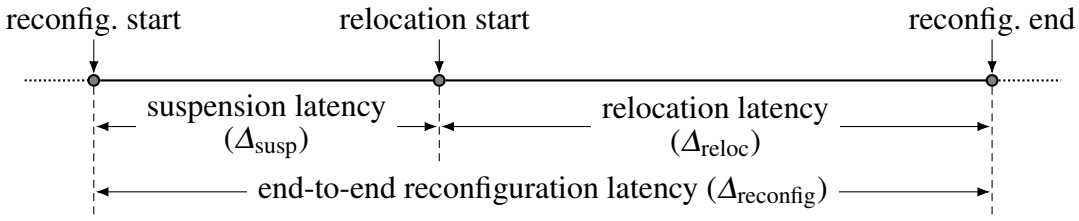


Figure 6.4: Timeline of a mapping reconfiguration.

the application and (ii) to have the messages injected into the NoC delivered to their respective destination tiles such that the application comes to a complete standstill[5]. Suspending the execution of each task is performed according to the preemption scheme in use and entails a worst-case latency termed the *preemption latency* of that task and denoted by $\delta(t)$ for task t. In the case of a *preemptive* suspension scheme, the execution of tasks can be suspended at any time during their execution. Thus, the preemption latency of a task t under a preemptive suspension scheme is guaranteed not to exceed the context-switch latency of the operating system denoted as D_{OS}, hence, $\delta(t) = D_{\text{OS}}$. In the case of a *cooperative* suspension scheme, the execution of tasks can be suspended only at certain points during their execution termed the migration points. Thus, the preemption latency of a task t under a cooperative suspension scheme can be bounded as $\delta(t) = D_{\text{OS}} + D_{\text{mp}}(t)$ where $D_{\text{mp}}(t)$ denotes the maximum temporal distance between any two consecutive migration points in the binary of t which can be derived statically[6]. Finally, in the case of a *non-preemptive* suspension scheme, the execution of tasks can be suspended only after their current iteration of execution (job) is completed. Thus, the preemption latency of a task t under a non-preemptive suspension scheme can be bounded as $\delta(t) = D_{\text{OS}} + WCRT(t)$ where $WCRT(t)$ denotes the Worst-Case Response Time of t. The suspension of an inter-tile message m imposes a worst-case latency equal to the WCTT of that message which is analyzed as discussed in Section 3.3.2. Therefore, given the preemption latency $\delta(t)$ of each task $t \in T$ derived as detailed above, the worst-case suspension latency Δ_{susp} of the application can be bounded as:

$$\Delta_{\text{susp}} = \max_{t \in T} \left\{ \delta(t), \max_{m \in \text{IO}(t)} WCTT(m) \right\} \tag{6.6}$$

Relocation Latency (Δ_{reloc}) The relocation latency of a mapping reconfiguration, denoted by Δ_{reloc}, reflects the time it takes to relocate all migrating tasks to their

[5] The suspension of messages is not only crucial for preserving the application's data-flow consistency as discussed in Section 6.2.1 but also renders the allocated budget on the NoC available to be used for the migration of tasks and, thereby, mitigates the allocation overhead of the reconfiguration.

[6] An investigation of the best locations to insert migration points in the binary of a task and the analysis of the temporal distance between migration points of a task fall out of the scope of this work.

destination location after the application has been suspended on the source mapping. Each migrating task is relocated over the migration route identified for it using the route selection procedure presented before. The WCRL of the migration data-set $\Psi_{\text{mig}}(t)$ of task t over its migration route ρ_{mig} is also analyzed as part of the route selection procedure using Eq. (6.3). In the proposed reconfiguration mechanism, the relocation of migrating tasks is performed sequentially. Thus, the latency of the relocation process for all tasks can be bounded using Eq. (6.7) where $T_{\text{mig}} \subseteq T$ denotes the set of migrating tasks that are relocated between different tiles. Note that, due to the shared memory scheme within each tile, intra-tile migrations (i.e., migrations between cores on the same tile) are realized implicitly without an actual relocation of data between different memories.

$$\Delta_{\text{reloc}} = \sum_{t \in T_{\text{mig}}} WCRL\big(\Psi_{\text{mig}}(t), \rho_{\text{mig}}(t)\big) \tag{6.7}$$

Compound Application Latency To ensure a seamless satisfaction of the deadline of a real-time application in spite of a mapping reconfiguration taking place, it must be guaranteed that the end-to-end processing time of the application never exceeds the given deadline of the application in spite of the suspension of the application's execution during the reconfiguration process and the difference in the timing behavior of its tasks and messages between the source and target mappings of the reconfiguration. In view of a mapping reconfiguration, the processing of some inputs of the application through the tasks of the application will (i) advance partially on the source mapping before the reconfiguration, (ii) be suspended during the reconfiguration process, and (iii) progress to completion on the target mapping after the reconfiguration process. While the reconfiguration latency analysis presented before provides an upper bound on the duration of (ii), an upper bound is also required on the duration of (i) and (iii) to derive a safe upper bound on the end-to-end processing latency of the application considering the reconfiguration process and the changes in the timing behavior of the application before and after the reconfiguration. To that end, we derive the so-called *compound latency* of the application denoted as $\hat{L}_{\text{app}}$ which serves as an upper bound on the processing time of any input of the application that may be partially processed on the source mapping and partially on the target mapping. Thus, $\hat{L}_{\text{app}}$ safely bounds the collective latency of (i) and (iii) above. For a given pair of mappings, the compound latency of the application $\hat{L}_{\text{app}}$ is derived as follows:

- For each task $t \in T$, the so-called compound latency of t, denoted by $\hat{L}(t)$, is determined as the maximum WCRT of t between the source and target mappings.
- For each message $m \in M$, the so-called compound latency of m, denoted by $\hat{L}(m)$, is determined as the maximum WCTT of m between the source and target mappings. The WCTT of an intra-tile message is considered to be zero.

- The compound latency of the application is determined using Eq. (6.8) based on the compound latency of its tasks and messages where Π denotes the set of all end-to-end paths in the application graph, cf. Eq. (3.12).

$$\hat{L}_{\text{app}} = \max_{\pi \in \Pi} \left\{ \sum\nolimits_{t \in (T \cap \pi)} \hat{L}(t) + \sum\nolimits_{m \in (M \cap \pi)} \hat{L}(m) \right\} \tag{6.8}$$

6.2.3 Mapping Reconfiguration Admission Check

The offline reconfiguration analysis presented in Section 6.2.2 provides the RPM with the following set of information for each pair of source and target mappings: (i) the migration route of each migrating task, (ii) the amount of additional NoC budget to be allocated for the construction of migration routes, (iii) an upper bound on the suspension latency Δ_{susp}, relocation latency Δ_{reloc}, and end-to-end latency Δ_{reconfig} of the reconfiguration process, and (iv) the compound application latency $\hat{L}_{\text{app}}$ w.r.t. the source and target mappings in question.

At run time, when selecting a target mapping for performing a mapping reconfiguration for an application, the RPM examines the admissibility of the reconfiguration to each potential target mapping w.r.t. the (i) resource requirements of the target mapping and the reconfiguration process and (ii) the given timing constraints of the application. In this context, any reconfiguration option for which (parts of) the required resources are not available is discarded. For any reconfiguration option which passes the resource check, a lightweight timing admission check is performed which examines the satisfaction of the condition in Eq. (6.9) where D_{app} denotes the current deadline of the application, $\hat{L}_{\text{app}}$ the compound application latency for the reconfiguration in question, and Δ_{reconfig} the end-to-end reconfiguration latency. Any reconfiguration option which satisfies Eq. (6.9) is guaranteed to respect the application's deadline seamlessly and, therefore, is admissible.

$$D_{\text{app}} - \hat{L}_{\text{app}} - \Delta_{\text{reconfig}} \geq 0 \tag{6.9}$$

Hybrid Reconfiguration Analysis

In case a reconfiguration fails in passing the reconfiguration timing admission check, the RPM may optionally repeat the lightweight parts of the reconfiguration timing analysis at run time, taking into account the current availability of the NoC to tighten the reconfiguration latency bound Δ_{reconfig} at the cost of a minimal computational overhead: As will be demonstrated experimentally in Section 6.2.4, the relocation latency Δ_{reloc} of a reconfiguration process is typically far larger than its suspension latency Δ_{susp}. The relocation latency of a reconfiguration is determined based on the WCRL of each migrating task (see Eq. (6.7)) which, in turn, strongly depends on the available budget on the NoC links contained in the migration route (see Eqs. (6.3) and (6.4)). At run time, the available budget on the NoC is often considerably larger

than the budget allocated by the source/target mappings of a reconfiguration process and the minimal additional budget reserved to enable constructing the migration routes. Therefore, a considerably tighter upper bound may be derived on Δ_{reloc} if its analysis is performed based on the actual availability of the NoC. Note that an analysis of the relocation latency Δ_{reloc} is typically computationally affordable at run time as it is performed using Eqs. (6.3), (6.4) and (6.7) which are lightweight and impose an overall computational complexity of $O(|T||R|)$. Here, $|T|$ denotes the size of the application (in number of tasks), and $|R|$ denotes the size of the architecture (in number of routers) where the set of routers in the architecture is denoted by R.

Analyzing the relocation latency at run time establishes a *hybrid reconfiguration analysis* in which the compute-heavy analyses (e.g., the optimization of migration routes) are performed offline, while lightweight parts of the analysis with significant impact on the reconfiguration latency guarantees are performed online, taking into account the actual availability of the NoC. Such a hybrid reconfiguration analysis (i) tightens the reconfiguration latency bound considerably which, in turn, (ii) renders many of the reconfiguration options admissible which did not pass the condition in Eq. (6.9) based on their statically derived reconfiguration latency. This not only (iii) increases the chances of finding at least one feasible reconfiguration option to preserve the execution of the application (e.g., in view of a resource failure) but also (iv) enhances the *flexibility* of the RPM in choosing between a larger number of reachable target mappings, e.g., to improve the system's performance by optimizing secondary goals such as energy efficiency or load balance.

Note that reserving the available NoC budget for a reconfiguration process does not impose a notable utilization overhead compared to the regular load of the NoC since mapping reconfigurations are typically performed infrequently and the duration of each reconfiguration process is far shorter than the typical lifespan of an application. Therefore, the NoC load of mapping reconfigurations is imposed for only a small fraction of time such that their NoC overhead can be considered insignificant.

6.2.4 Experimental Results

This section presents the results of a series of experiments on the applicability, efficiency, and run-time feasibility of the proposed mapping reconfiguration methodology for hard real-time applications in heterogeneous many-core systems.

Experiments Setup

Architectures The experiments are performed based on a heterogeneous many-core architecture with 6×6 tiles. The architecture encompasses 36 cores (one per tile) with diverse numbers of instances from three different core types (IBM PowerPC, AMD K6-2, and AMD K6-III) whose specifications are provided by Embedded System Synthesis Benchmarks Suite (E3S) [Dic10]. Each shared resource is considered to

have a Weighted Round-Robin (WRR) arbitration policy. For each core c, a scheduling capacity of $K_c = 10$ slots, a scheduling slot length of $S_c = 50$ us, and a context-switch overhead of $D_c = D_{\mathrm{OS}} = 10$ us is considered. For each outbound NoC link of each router r, an arbitration capacity of $K_r = 10$ slots and an arbitration slot length of $S_r = \tau^{\mathrm{NoC}} = 10$ ns are considered.

Applications For the experiments, four real-time embedded applications are used from the areas of networking (7 tasks, 9 messages), consumer (11 tasks, 12 messages), telecommunications (14 tasks, 20 messages), and automotive (18 tasks, 21 messages). The applications are provided as benchmarks by E3S [Dic10]. For each application, [Dic10] also provides the Worst-Case Execution Time (WCET) of each task on each core type, its memory demand, the payload size of each message, and the execution/transfer period of each task/message which are required for the timing analysis of mappings.

Design Objectives The optimization of mappings for each application is performed w.r.t. three design objectives: (i) *worst-case application latency (makespan)* derived using the timing analysis presented in Section 3.3, (ii) *resource cost* calculated as the number of required core instances from each type, and (iii) *energy consumption*, calculated based on the WCRT of tasks, the core power-consumption parameters provided by [Dic10], the payload size of each message provided by [Dic10], and the NoC/bus energy model from [WSK+05] considering a length of 2 mm for NoC links and 5 mm for buses.

Design Space Exploration (DSE) The offline optimization of mappings per application is performed using the OpenDSE framework [RLG+18] which uses the NSGA-II [DPA+02] multi-objective evolutionary algorithm provided by the optimization framework OPT4J [LGR+11]. The DSE is configured to perform 2,000 optimization iterations with a population size of 100 mappings where 25 mappings are generated per iteration. Once the DSE has finished, the distillation methodology presented in Chapter 5 is used to distill 20 Pareto-optimal mappings per application. The following experimental results represent an average over five runs of DSE per application.

Mapping Reconfiguration Analysis Given the set of 20 mappings per application, the mapping reconfiguration analysis presented in Section 6.2.2 is performed for the $20 \times 19 = 380$ possible pairs of source and target mappings. For the optimization of migration routes, $\omega_{\mathrm{ex}} = 1$ additional time slot is allocated on those NoC links that must be allocated additionally to establish the required migration routes. We have verified empirically that $\omega_{\mathrm{ex}} = 1$ results in an efficient trade-off between the allocation overhead and the latency of mapping reconfigurations. The suspension of applications

is taken to follow a cooperative preemption scheme. For each task $t \in T$, the maximum temporal distance $D_{\mathrm{mp}}(t)$ between migration points and the context size $\Psi_{\mathrm{ctx}}(t)$ is provided by E3S [Dic10].

Investigated Approaches The experiments investigate mapping reconfiguration based on two variants of the proposed reconfiguration analysis: (i) *offline reconfiguration analysis* in which the reconfiguration analysis is completely performed offline and the results of the analysis are provided to the RPM and (ii) *hybrid reconfiguration analysis* in which Δ_{reloc} is analyzed online by the RPM based on the current availability of the NoC while the other aspects of the reconfiguration analysis are performed offline. In addition to an investigation of the allocation and temporal overheads associated with mapping reconfiguration, the two variants of reconfiguration analysis are evaluated comparatively in a case study on dynamic thermal management of many-core systems.

Reconfiguration Allocation Overhead

Achieving a minimal allocation overhead for mapping reconfiguration is of crucial importance to enhance the feasibility of reconfigurations in view of a dynamic availability of the NoC at run time. For the four benchmark applications, Figure 6.5 illustrates the Cumulative Distribution Function (CDF) of mapping reconfigurations as a function of their relative NoC allocation overhead. Here, the relative NoC allocation overhead of a reconfiguration is defined as the ratio of the additionally allocated NoC budget (periodic time slots) for the construction of the migration routes for it relative to the overall NoC budget reserved as part of its source/target mappings. As illustrated, depending on the application, 7–23% of mapping reconfigurations impose no allocation overhead at all and, thus, depend merely on the NoC budget allocated for the involved mappings of the reconfiguration. For all applications, over 95% of the mapping reconfigurations exhibit an allocation overhead of less than 10%. The proposed methodology achieves such low levels of allocation overhead (i) by utilizing the already allocated NoC budget (of the source/target mappings) for the construction of migration routes and (ii) by performing task migrations sequentially so that different migrations can be performed over a common set of allocated links.

The allocation overhead of a mapping reconfiguration grows proportionally to the number ω_{ex} of time slots reserved on additional links, see Section 6.2.2. The experimental results presented here are derived based on $\omega_{\mathrm{ex}} = 1$ which has proven empirically to offer an efficient trade-off between the allocation overhead and latency of mapping reconfigurations. A higher value of ω_{ex} results in a linear increase in the allocation overhead of mapping reconfigurations, while achieving only a sub-linear reduction in the reconfiguration latency. Note that, in Figure 6.5, the reconfiguration allocation overhead for the larger applications (telecom and automotive) is notably lower than that of the smaller applications (networking and consumer) at each density of enabled reconfigurations (horizontal intersects). This trend is observed because (i) the

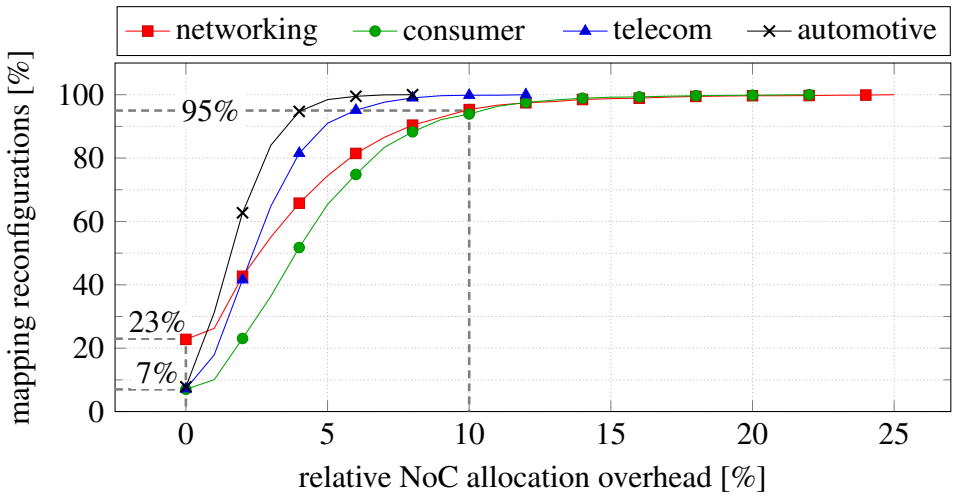


Figure 6.5: Cumulative Distribution Function (CDF) of mapping reconfigurations as a function of their NoC allocation overhead relative to the NoC budget reserved for the respective source/target mappings [PWG+17].

larger number of messages in larger applications typically results in a large amount of NoC budget allocated for their mappings which can be used for the migration of tasks. This alleviates the amount of additional NoC budget required to establish the migration routes. Moreover, (ii) the allocation overhead of a given amount of additional NoC budget is typically lower in larger applications as the allocation overhead of the reconfigurations is calculated relative to the allocated NoC budget of source/target mappings where the latter scales with the number of messages in the application.

Reconfiguration Temporal Overhead (Offline Analysis)

The temporal overhead of a mapping reconfiguration is another aspect with a crucial impact on the feasibility of each reconfiguration option in view of the hard timing constraints of the application at run time. For the four benchmark applications, Figure 6.6 illustrates the distribution of the latency bound of the 380 mapping reconfigurations relative to the hard deadline of the application. The deadline of each application is reported in parentheses below the application's name in Figure 6.6 which corresponds to an overhead of 100% in the plots of each respective application. For the experiments at hand, the deadline of each application is taken to be equal to the sum of the execution periods of tasks on the longest path of the application. Note that, in practice, the deadline of a real-time application may change dynamically at run time, e.g., upon user request.

To investigate the contribution of different steps of a mapping reconfiguration to the overall reconfiguration latency, the distribution of the suspension latency Δ_{susp} and relocation latency Δ_{reloc} of the mapping reconfigurations is also provided

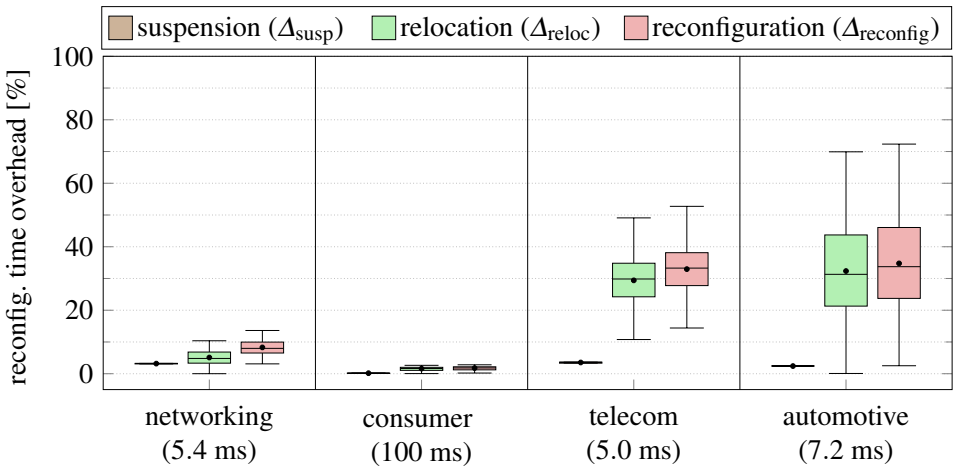


Figure 6.6: Distribution of suspension latency (Δ_{susp}), relocation latency (Δ_{reloc}) and end-to-end latency ($\Delta_{\text{reconfig}} = \Delta_{\text{susp}} + \Delta_{\text{reloc}}$) of mapping reconfigurations relative to the application's deadline (given in parentheses), derived in an offline analysis [PWG+17].

in Figure 6.6 alongside the distribution of the end-to-end reconfiguration latency $\Delta_{\text{reconfig}} = \Delta_{\text{susp}} + \Delta_{\text{reloc}}$. As illustrated, the distribution of the reconfiguration latency varies considerably across different applications. Nonetheless, for all applications, the results demonstrate that suspension latency contributes only marginally to the end-to-end reconfiguration latency, compared to the relocation latency. Recall from Section 6.2.1 that the suspension of the application prior to the reconfiguration plays a crucial role in preserving the data-flow consistency and synchronization of the application. Figure 6.6 demonstrates that the advantages of execution suspension are achieved at the expense of only a marginal latency overhead.

Subject to the size of the application and its given deadline, the reconfiguration latency may account for a large share of the overall time available for the execution and adaptation of the application, see, e.g., Δ_{reconfig} for the telecom and automotive benchmarks in Figure 6.6. Evidently, the time margin left above the Δ_{reconfig} box plots in Figure 6.6 may be too tight for the compound latency $\hat{L}_{\text{app}}$ of the application to guarantee a seamless satisfaction of the given deadline. In this context, if a reconfiguration is verified not to satisfy the admission check in Eq. (6.9), the RPM may benefit from a hybrid timing analysis of the reconfiguration to tighten the derived bound on the reconfiguration latency as investigated in the following experiment.

Reconfiguration Temporal Overhead (Hybrid Analysis)

Hybrid timing analysis of mapping reconfiguration empowers the RPM to reflect the *actual* availability of the NoC in the timing analysis and, thereby, tighten the derived

timing guarantees. As a result, many reconfigurations which fail in the timing feasibility check of Eq. (6.9) will indeed pass this check based on the tightened guarantee on their relocation latency Δ_{reloc} and, thereby, their end-to-end reconfiguration latency $\Delta_{\text{reconfig}} = \Delta_{\text{susp}} + \Delta_{\text{reloc}}$. The following experiment investigates the impact of a hybrid reconfiguration latency analysis on tightening the reconfiguration latency bound in comparison to the offline analysis.

Figure 6.7 illustrates the distribution of the reconfiguration latency overhead relative to the application's deadline, derived at different levels of NoC traffic using a hybrid reconfiguration latency analysis which is partly performed by the RPM online. At each NoC traffic level, for each reconfiguration, first, the resources required by the source/target mappings or the reconfiguration itself are allocated before additional traffic is introduced into the NoC to reach the desired traffic level. This scheme ensures that all reconfigurations are investigated since the additional NoC traffic is introduced on top of the resources reserved for the reconfiguration and, hence, cannot harm its feasibility. For the analysis of the relocation latency Δ_{reloc} and, thereby, the overall reconfiguration latency Δ_{reconfig}, the whole available NoC budget on the migration route of each task is taken into account to tighten its WCRL guarantee. In Figure 6.7, the box plots corresponding to a NoC traffic level of [90–100]% (depicted in red) closely resemble the distribution of Δ_{reconfig} in Figure 6.6 derived in an offline analysis. As illustrated in Figure 6.7, a hybrid analysis of reconfiguration latency provides the RPM with a considerably tighter Δ_{reconfig} bound compared to an offline analysis, especially at system states with relatively low NoC traffic.

The amount of reduction achieved in Δ_{reconfig} not only depends on NoC availability but also the *length* of the migration routes which directly depends on the NoC size of the many-core platform. The impact of the NoC size on the reconfiguration latency overhead is illustrated in Figure 6.8. The results are provided for two representative benchmark applications, namely, networking (small) and automotive (large), on four many-core platforms with NoC sizes of 4×4, 6×6, 8×8, and 12×12. For each application, the provided results correspond to the average reconfiguration latency overhead among all reconfigurations. As illustrated, at each NoC traffic level, the reconfiguration latency overhead increases with the NoC size due to the corresponding increase in the length of migration routes which results in an increase in the WCRL of migrations. Moreover, as the NoC traffic increases, Δ_{reconfig} on larger NoCs grows faster. The larger gap between the Δ_{reconfig} at traffic levels of [0–10]% and [90–100]% for the larger NoCs gives evidence that the mitigation of the reconfiguration latency bound achieved by a hybrid latency analysis increase as the NoC size grows.

Reconfiguration Success and Flexibility

The following experiment investigates the viability of the proposed mapping reconfiguration methodology for dynamic mapping adaptation of hard real-time applications. To that end, a dynamic thermal management case study is considered in which map-

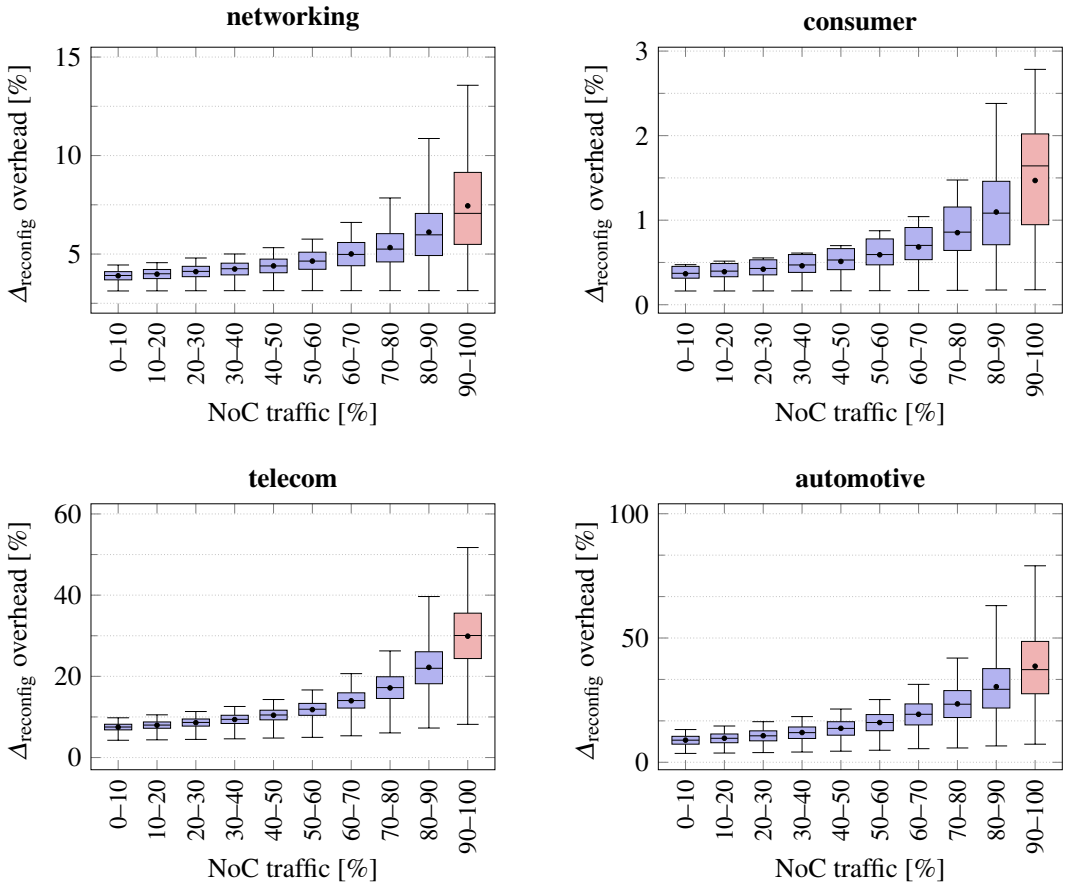


Figure 6.7: Distribution of reconfiguration latency (Δ_{reconfig}) overhead relative to the application's deadline derived in a hybrid reconfiguration latency analysis for different levels of NoC traffic [PWG+19].

ping reconfiguration is used to evacuated hot cores use by the application during its execution in a deadline-adhering manner. In this case study, each application is initially launched on an otherwise empty platform (100% resource availability). Then, available resources of the platform are occupied iteratively to establish various resource availability scenarios until all resources are occupied (100% system utilization). At each system utilization step, one of the cores in use by the application is selected randomly and marked as hot such that a mapping reconfiguration in near future becomes necessary in order to evacuate the hot core and, thereby, preserve the execution of the application. Subsequently, the RPM considers all possible reconfiguration options from the mapping in use to the other precomputed mappings and identifies the feasible reconfiguration options. As detailed in Section 6.2.3, a mapping reconfiguration is feasible if (i) the required resources by the source/target mappings and the reconfiguration itself are available and (ii) it passes the timing admission check

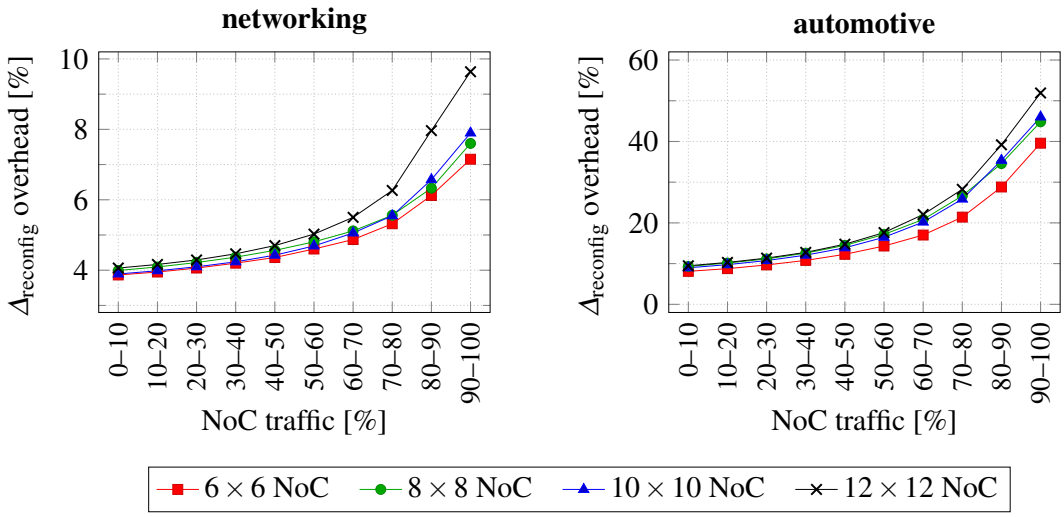


Figure 6.8: Impact of NoC size on reconfiguration latency (Δ_{reconfig}) overhead derived in a hybrid latency analysis. The results are given for two applications (networking and automotive) at different NoC traffic levels [PWG+19].

in Eq. (6.9) ensuring a seamless satisfaction of the application's real-time constraint during and after the reconfiguration process. At each iteration, the *offline* and the *hybrid* variants of the reconfiguration analysis are used to bound the latency Δ_{reconfig} of each reconfiguration, and the proportion of feasible reconfiguration options based on each variant of the analysis is recorded.

For the exemplary application of automotive, Figure 6.9 illustrates the *success rate* (left) and *flexibility rate* (right) of mapping reconfiguration at various system utilization levels. Similar results are obtained for other applications as well. The results represent an average over 100 repetitions of the thermal management experiment to incorporate diverse mixes of hot cores and resource/NoC availability.

Success Rate Success rate denotes the proportion of adaptation attempts in which the RPM has identified *at least one* feasible reconfiguration option such that an adaptation can at all take place to preserve the execution of the application in view of the sudden failure of the resources used by it. As illustrated in Figure 6.9 (left), at low levels of system utilization, the RPM achieves a reconfiguration success rate of nearly 100% using either of the offline or hybrid analyses. The success rate under both analysis approaches diminishes as the system utilization increases (i.e., the availability of the resources decreases). In this context, the hybrid analysis exhibits a higher success rate as it provides a tighter reconfiguration latency bound which enhances the chances of the reconfiguration options in passing the timing admission check in Eq. (6.9). Compared to the offline analysis, the hybrid analysis results in an average improvement of 10% (up to 22%) in the success rate of mapping reconfiguration.

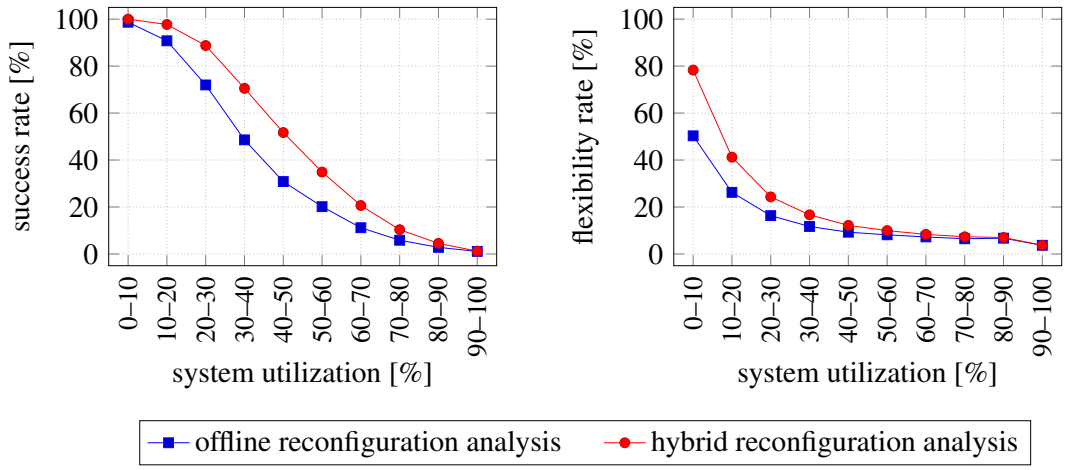


Figure 6.9: Success rate (left) and flexibility rate (right) of hard real-time mapping adaptation of the automotive application using mapping reconfiguration.

Flexibility Rate Complementary to success rate, flexibility rate denotes the *proportion* of feasible reconfigurations at each adaptation attempt and, hence, reflects the flexibility of the RPM in choosing between different reachable target mappings, e.g., to also improve the system's performance w.r.t. energy efficiency or load balance in addition to preserving the execution of the application. As illustrated in Figure 6.9 (right), the reconfiguration flexibility rate achieved by each of the investigated reconfiguration latency analysis approaches, offline and hybrid, diminishes as the availability of the resources decreases. In general, mapping reconfigurations analyzed using the hybrid analysis exhibit a higher flexibility rate. The advantage of the hybrid analysis is best demonstrated in a comparison of the flexibility rate achieved by the two analyses at system utilization of [0–10]%. Here, nearly all reconfiguration options pass the resource availability admission check as nearly all resources are available. Hence, the flexibility rate gap of 28% between the hybrid and offline analysis here is mainly governed by the timing feasibility of the reconfigurations. Compared to the offline analysis, the hybrid analysis results in an average improvement of 6% (up to 28%) in the flexibility rate of mapping reconfiguration.

6.3 Hard Real-Time Task Migration

The mapping reconfiguration methodology presented in Section 6.2 is the first methodology known to us which enables individual and on-demand mapping adaptation of hard real-time applications with worst-case adaptation timing guarantees in the context of heterogeneous many-core systems with a distributed memory scheme. Mapping adaptations in the form of a reconfiguration between precomputed mappings benefit from an offline/hybrid analysis of different reconfiguration options such that the RPM

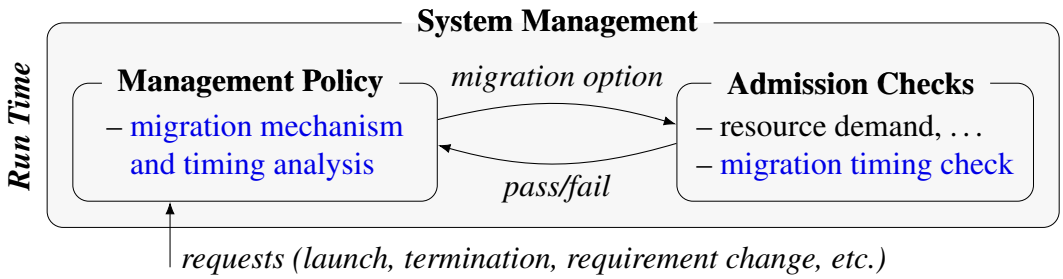


Figure 6.10: An overview of the proposed task migration methodology in the context of HAM. The contributions are highlighted in blue.

is provided with all necessary information for performing timely mapping reconfigurations at run time with negligible run-time computational overhead. However, mapping reconfiguration enables the dynamic adaptation of real-time applications only to a limited extent: The adaptation flexibility of mapping reconfiguration is restricted to a *precomputed set of adaptation options* which may not be able to address all possible online situations necessitating an adaptation. Moreover, mapping reconfiguration typically involves a coarse-grained adaptation of the application where most or even all tasks of the application must be rearranged on the platform in order to construct the target mapping. Such adaptations typically entail a *significant temporal overhead* that may not be affordable in many cases as discussed in Section 6.2.

To address these restrictions of mapping reconfiguration, this section presents a *task migration methodology* that empowers a finer granularity of hard real-time mapping adaptations compared to mapping reconfiguration. Figure 6.10 provides an overview of the proposed task migration methodology in the context of HAM where the migration-related steps and components are highlighted in blue and outlined in the following. The proposed task migration methodology provides the RPM with a (i) *timing-predictable migration mechanism* which enables the RPM to timing-predictably migrate a given set of tasks between given source and destination locations on the platform. Moreover, the RPM is provided with a lightweight (ii) *migration timing analysis* that formally bounds the worst-case end-to-end latency of the migration processes, and a lightweight (iii) *migration admission check* that examines the admissibility of the migration latency w.r.t. the given timing constraints of the application and the fluctuations in its timing behavior during and after the migration processes. The proposed task migration methodology, therefore, enables the RPM to perform fine-grained mapping adaptations on-demand in the form of the migration of *any* subset of an application's tasks to *any* destination cores with worst-case timing guarantees and *without* relying on an offline timing analysis or being restricted to a precomputed set of adaptation options[7].

The proposed task migration methodology is presented in the following. First,

[7] See Footnote 2 on Page 137.

Section 6.3.1 presents the task migration mechanism which specifies the procedure of performing task migrations at run time. Then, Section 6.3.2 presents the worst-case timing analysis of migrations followed by Section 6.3.3 which presents a migration admission check to examine the real-time conformity of migration options. Finally, experimental results assessing the effectiveness of the proposed migration methodology in comparison to mapping reconfiguration are presented in Section 6.3.4.

6.3.1 Task Migration Mechanism

The migration mechanism in the proposed task migration methodology enables the RPM to perform a given set of task migrations in a timing-predictable manner so that worst-case timing guarantees can be derived formally on the latency of different steps of the migration process as well as on its end-to-end latency. In general, the migration of a given set of tasks may entail a set of *intra-tile migrations* where the respective tasks shall be migrated between different cores within a tile as well as a set of *inter-tile migrations* where the respective tasks shall be migrated between different tiles. The shared memory scheme within each tile of a many-core platform enables intra-tile migrations to be realized *implicitly*, i.e., without entailing the relocation of data between different memories. However, due to the distributed memory scheme between tiles, an explicit relocation of tasks becomes necessary for inter-tile migrations. The inter-tile relocation of a migrating task requires a migration route over the NoC to establish a connection between the source and destination tiles of the migration. The migration route of a migrating task must be reserved prior to the initiation of the migration to preserve system composability.

The migration of a set of tasks may be realized sequentially or in parallel, subject to the availability of the NoC and the amount of time available for the migration process. Given a subset $T_{\text{mig}} \subseteq T$ of tasks of an application selected for migration and the migration scheme (sequential or in parallel) specified by the RPM, the RPM must first derive an upper bound on the latency of the migrations and, based on that, on the end-to-end latency of the overall mapping adaptation process. This can be realized using the lightweight migration timing analysis that will be presented in Section 6.3.2. Subsequently, the RPM must examine the admissibility of the migrations in question in terms of (i) the availability of the required resources and (ii) the real-time conformity of the migration process using the lightweight admission check that will be presented in Section 6.3.3. If the migrations in question pass the admission check, the RPM must reserve the required resources, namely, (i) the (required budget on the) destination core of each migrating task, (ii) the post-migration NoC routes for inter-tile messages to/from each migrating task, and (iii) the migration route of the migrating tasks that are to be relocated between different tiles. The migration process of each task can be initiated after the required resources have been reserved and consists of a *suspension phase* followed by a *relocation phase* as elucidated in the following.

Suspension Phase The migration of tasks begins with the *suspension* of their execution on their source location according to the suspension scheme in use (preemptive, cooperative, or non-preemptive). To preserve the data-flow consistency of the application, in addition to the migrating tasks, the input/output messages $m \in \text{IO}(t)$ of each task t to be migrated between different tiles are suspended as well by blocking the injection of new instances of these messages into the NoC. The suspension of messages prevents their loss or out-of-order delivery without relying on message-forwarding and buffer-reordering system services which are typically not available in an embedded system. Once the suspension phase has been initiated, the RPM waits for the duration of the suspension process (whose upper bound is derived by the proposed migration latency analysis presented in Section 6.3.2 and known to the RPM) to ensure that the execution of tasks and the transfer of messages affected by the migration process have come to a complete standstill.

Relocation Phase Once the suspension process has been completed, the inter-tile relocation of the migrating task may be performed according to the migration scheme selected by the RPM (sequential or parallel). During the relocation phase, the migration data-set of each migrating task is relocated from its current tile to the destination tile over the migration route specified by the RPM. The migration data-set of a task consists of its (i) context (code, data, stack, etc.) and its (ii) unprocessed input messages and blocked output messages available at the source tile of the migration. The execution of a migrating task may be resumed after its relocation is completed. To preserve the data-flow consistency, however, the injection of suspended messages into the NoC is resumed only after the relocation of all migrating tasks is completed.

6.3.2 Task Migration Timing Analysis

The migration process of a task $t \in T$ over a given migration route ρ_{mig} entails a (i) *suspension latency* and a (ii) *relocation latency*. Give the migrating subset of tasks $T_{\text{mig}} \subseteq T$ of an application, for each migrating task $t \in T_{\text{mig}}$, the following analysis derives an upper bound on its worst-case suspension latency denoted by $\delta_{\text{susp}}(t)$ and its worst-case relocation latency denoted by $\delta_{\text{reloc}}(t)$. The bounds derived for the migrating tasks are subsequently used to derive a safe upper bound on the end-to-end latency of the multi-task migration process subject to the migration scheme in use (parallel or sequential migrations).

Suspension Latency

The suspension of a migrating task $t \in T_{\text{mig}}$ involves two parallel processes: (i) suspension of t's execution on the source core and (ii) suspension of its input/output messages $m \in \text{IO}(t)$. The suspension of t's execution can be realized preemptively, cooperatively, or non-preemptively. In the case of a *preemptive* suspension scheme,

the execution of t may be suspended at any time during its execution. Therefore, the latency of the preemption process, denoted by $\delta(t)$, is guaranteed not to exceed the context-switch latency of the operating system denoted as D_{OS}, thus, $\delta(t) = D_{\mathrm{OS}}$. In the case of a *cooperative* suspension scheme, the execution of t can be suspended only at the migration points augmented to its source code (binary). Thus, the preemption latency of t can be bounded as $\delta(t) = D_{\mathrm{OS}} + D_{\mathrm{mp}}(t)$ where $D_{\mathrm{mp}}(t)$ denotes the maximum temporal distance between any two consecutive migration points in the binary of t which can be derived statically. Finally, in the case of a *non-preemptive* suspension scheme, the execution of a task can be suspended only after its current iteration of execution (job) is completed. Thus, the preemption latency of task t under a non-preemptive suspension scheme can be bounded as $\delta(t) = D_{\mathrm{OS}} + WCRT(t)$ where $WCRT(t)$ denotes the Worst-Case Response Time of t.

The suspension of input/output messages $m \in \mathrm{IO}(t)$ of a task t is realized by blocking the injection of new instances of these messages into the NoC while allowing the already-injected messages to reach their destination. Since the transfer latency of each inter-tile message is guaranteed not to exceed its Worst-Case Traversal Time (WCTT), and given the parallel transfer of messages, the worst-case suspension latency of the input/output messages of a migrating task t is guaranteed not to exceed the largest WCTT among all input/output messages of t. Thus, the worst-case suspension latency of a migrating task $t \in T_{\mathrm{mig}}$, denoted by $\delta_{\mathrm{susp}}(t)$, can be bounded as:

$$\delta_{\mathrm{susp}}(t) = \max \left\{ \delta(t), \max_{m \in \mathrm{IO}(t)} WCTT(m) \right\} \tag{6.10}$$

Relocation Latency

The relocation process of a migrating task $t \in T_{\mathrm{mig}}$ involves the transfer of t's migration data-set between the source and destination tiles of its migration. The maximum size of the migration data-set, denoted as $\Psi_{\mathrm{mig}}(t)$ for task t, can be bounded using Eq. (6.2). Subsequently, given the migration route $\rho_{\mathrm{mig}}(t)$ of a migrating task t and the size of its migration data-set $\Psi_{\mathrm{mig}}(t)$, the Worst-Case Relocation Latency (WCRL) of t denoted by $\delta_{\mathrm{reloc}}(t)$ can be bounded using Eq. (6.11) where $WCRL(\Psi_{\mathrm{mig}}(t), \rho_{\mathrm{mig}}(t))$ is derived using Eq. (6.3).

$$\delta_{\mathrm{reloc}}(t) = WCRL\big(\Psi_{\mathrm{mig}}(t), \rho_{\mathrm{mig}}(t)\big) \tag{6.11}$$

End-To-End Migration Latency

The end-to-end migration latency of a given set of migrating tasks $T_{\mathrm{mig}} \subseteq T$ reflects the overall temporal overhead imposed by the migration process of T_{mig} on the regular execution of the application. Thus, it denotes the time interval between the moment at which the preemption of the migrating tasks begins until the relocation of all

migrating tasks is completed. In the case of multiple migrating tasks ($|T_{\text{mig}}| > 1$), the RPM may decide to perform the relocation of tasks (i) sequentially or (ii) in parallel. The following analysis bounds the worst-case end-to-end migration latency of the migrating tasks $T_{\text{mig}} \subseteq T$, denoted by $\delta_{\text{mig}}(T_{\text{mig}})$, for the cases of sequential and parallel relocation of tasks.

Sequential Relocation In the case of a sequential relocation of tasks, the end-to-end migration latency will depend on the order in which the migrating tasks will be relocated. Since the relocation of each migrating task may begin only after its suspension is completed, once the migration process has been initiated, the relocation of the first migrating task is guaranteed to begin no later than the maximum suspension latency among all migrating tasks. Thus, the end-to-end migration latency for a sequential relocation of migrating tasks can be bounded as:

$$\delta_{\text{mig}}^{\text{seq}}(T_{\text{mig}}) = \max_{t \in T_{\text{mig}}} \left\{ \delta_{\text{susp}}(t) \right\} + \sum_{t \in T_{\text{mig}}} \left\{ \delta_{\text{reloc}}(t) \right\} \tag{6.12}$$

Parallel Relocation In the case of a parallel relocation of tasks, the relocation of each migrating task begins immediately after the completion of its suspension. Here, the end-to-end migration latency of each migrating task $t \in T_{\text{mig}}$ can be bounded as $\delta_{\text{susp}}(t) + \delta_{\text{reloc}}(t)$. Thus, the end-to-end migration latency for all migrating tasks is guaranteed not to exceed the longest migration latency among all migrating tasks and, therefore, can be bounded as:

$$\delta_{\text{mig}}^{\text{par}}(T_{\text{mig}}) = \max_{t \in T_{\text{mig}}} \left\{ \delta_{\text{susp}}(t) + \delta_{\text{reloc}}(t) \right\} \tag{6.13}$$

Computational Complexity Any timing analysis to be used by the RPM must be lightweight such that its computational overhead is acceptable for online use. The following analysis derives the computational complexity of the proposed migration timing analysis. The analysis of a task's suspension latency using Eq. (6.10) introduces a computational complexity of $O(|M|)$. The relocation latency analysis of each task introduces a computational complexity of $O(|R|)$ where R denotes the set of NoC routers and, thus, $|R|$ reflects the longest possible hop-length for a migration route, see Eqs. (6.3), (6.4) and (6.11). Therefore, the analysis of the end-to-end migration latency according to Eqs. (6.12) and (6.13) imposes an overall computational complexity of $O(|T_{\text{mig}}||M| + |T_{\text{mig}}||R|)$ in the general case. However, since each message is unicast (i.e., has only one consumer task), the total number of calls for $WCTT(m)$ when deriving $\delta_{\text{susp}}(t)$ in Eqs. (6.12) and (6.13) will never exceed $2|M|$ which reduces the overall computational complexity of the migration timing analysis to $O(|T_{\text{mig}}| + 2|M| + |T_{\text{mig}}||R|)$ which can be simplified as $O(|M| + |T_{\text{mig}}||R|)$.

6.3.3 Task Migration Admission Check

At run time, when selecting a set of tasks $T_{\text{mig}} \subseteq T$ for migration, the RPM examines the admissibility of the migrations in question in terms of their (i) resource requirements and (ii) real-time conformity. For the examination of resource requirements, the RPM checks the availability of (i) the (required budget on the) destination core of each migrating task $t \in T_{\text{mig}}$. In case t must be relocated between different tiles, the RPM also examines the availability of (ii) the migration route $\rho_{\text{mig}}(t)$ of t and (iii) the post-migration NoC routes for inter-tile input/output messages $m \in \text{IO}(t)$ of t. Any migration option (i.e., set of migrating tasks T_{mig} to be relocated to given destination locations) for which all required resources are not available is discarded.

For any migration option that passes the resource check, a lightweight timing admission check is performed subsequently to examine whether the timing constraint (deadline) of the application is guaranteed to be seamlessly satisfied during and after the migration(s) in spite of the changes in the timing behavior of the application. The timing admission check of migration options is realized similarly to the timing admission check of mapping reconfigurations in which the *compound application latency* denoted by $\hat{L}_{\text{app}}$ and the end-to-end adaptation latency (here, $\delta_{\text{mig}}(T_{\text{mig}})$) are examined against the application's deadline D_{app} as given in Eq. (6.14).

$$D_{\text{app}} - \hat{L}_{\text{app}} - \delta_{\text{mig}}(T_{\text{mig}}) \geq 0 \tag{6.14}$$

The compound application latency $\hat{L}_{\text{app}}$ provides a safe upper bound on the end-to-end processing time of the application, taking into account the changes in the WCRT of migrating tasks and the WCTT of their input/output messages. The post-migration WCRT of each task and the WCTT of each message can be bounded using the analyses presented in Sections 3.3.1 and 3.3.2, respectively. Based on the pre- and post-migration WCRT of tasks and WCTT of messages, the compound application latency $\hat{L}_{\text{app}}$ is derived as follows:

- For each task $t \in T$, the compound latency of t denoted by $\hat{L}(t)$ is determined as the maximum of the pre- and post-migration WCRT of t. Note that the WCRT of any non-migrating task $t \in T \setminus T_{\text{mig}}$ remains intact.
- For each message $m \in M$, the compound latency of m denoted by $\hat{L}(m)$ is determined as the maximum of the pre- and post-migration WCTT of m. The WCTT of messages communicated between non-migrating tasks remains intact. The WCTT of an intra-tile message is considered to be zero.
- The compound latency of the application, denoted by $\hat{L}_{\text{app}}$, is determined using Eq. (6.8) based on the compound latency of its tasks and messages where Π denotes the set of all end-to-end paths in the application graph, cf. Eq. (3.12).

Computational Complexity Since the migration timing admission check is to be used by the RPM, it must be lightweight so that its computational overhead is acceptable for online use. The following analysis derives the computational complexity of the proposed migration timing admission check. Given the set $T_{\text{mig}} \subseteq T$ of migrating tasks, the analysis of the end-to-end migration latency $\delta_{\text{mig}}(T_{\text{mig}})$ introduces a computational complexity of $O(|M| + |T_{\text{mig}}||R|)$ as detailed in Section 6.3.2. For the analysis of the compound application latency $\hat{L}_{\text{app}}$, the post-migration WCRT of each migrating task $t \in T_{\text{mig}}$ is derived using the constant-time non-iterative WCRT analysis presented in Section 3.3.1 with a computational complexity of $O(1)$. Therefore, the WCRT analysis for all migrating tasks $t \in T_{\text{mig}}$ imposes a computation complexity of $O(|T_{\text{mig}}|)$. Similarly, the post-migration WCTT of each message $m \in M$ is derived using the WCTT analysis presented in Section 3.3.2 with a computational complexity of $O(1)$. Therefore, the WCTT analysis of all messages affected by the migration imposes a computation complexity of $O(|M|)$. Having the application graph provided as an adjacency list, the calculation of the compound application latency $\hat{L}_{\text{app}}$ can be realized using a Depth-First Search (DFS) algorithm with a computational complexity of $O(|T| + |M|)$. Hence, the overall computational complexity of the migration timing analysis and feasibility check will be $O((|M| + |T_{\text{mig}}||R|) + (|T_{\text{mig}}|) + (|M|) + (|T| + |M|))$ which can be simplified as $O(|T_{\text{mig}}||R| + |T| + |M|)$ given that $T_{\text{mig}} \subseteq T$. Thus, the migration timing analysis and feasibility check introduce a computational overhead with a linear dependence on the application's size which renders it scalable for online use.

6.3.4 Experimental Results

This section presents the results of a series of experiments which assess the effectiveness and the efficiency of the proposed task migration methodology in comparison to mapping reconfiguration for hard real-time application adaptations in heterogeneous many-core systems.

Experiments Setup

Architectures The experiments are performed based on two heterogeneous many-core architectures with 6×6 and 8×8 tiles, respectively. Each architecture comprises processing cores of three different types (IBM PowerPC, AMD K6-2, and AMD K6-III) whose specifications are provided by Embedded System Synthesis Benchmarks Suite (E3S) [Dic10]. Each tile encompasses four homogeneous cores as well as peripherals, e.g., memories and a NA. Each shared resource is considered to have a WRR arbitration policy. For each core c, a scheduling capacity of $K_c = 10$ slots, a scheduling slot length of $S_c = 50\,\text{us}$, and a context-switch overhead of $D_c = D_{\text{OS}} = 10\,\text{us}$ is considered. Each router r is considered to have an arbitration capacity of $K_r = 10$ slots and an arbitration slot length of $S_r = \tau^{\text{NoC}} = 10\,\text{ns}$.

Applications For the experiments, four real-time embedded applications are used from the areas of networking (7 tasks, 9 messages), consumer (11 tasks, 12 messages), telecommunications (14 tasks, 20 messages), and automotive (18 tasks, 21 messages). The applications are provided as benchmarks by E3S [Dic10]. For each application, [Dic10] also provides the WCET of each task on each core type, its memory demand, the payload size of each message, and the execution/transfer period of each task/message which are required for the timing analysis of mappings. The deadline D_{app} of each application is considered to be equal to 80% of the sum of the execution period of tasks and production period of messages on the longest path in the application graph.

Design Objectives The optimization of mappings for each application is performed w.r.t. three design objectives: (i) *worst-case application latency (makespan)* derived using the timing analysis presented in Section 3.3, (ii) *resource cost* calculated as the number of required instances from each core type, and (iii) *energy consumption*, calculated based on the WCRT of tasks, the core power-consumption parameters provided by [Dic10], the payload size of each message provided by [Dic10], and the NoC/bus energy model from [WSK+05] considering a length of 2 mm for NoC links and 5 mm for buses.

Design Space Exploration (DSE) The offline optimization of mappings for each application is performed using the OpenDSE framework [RLG+18] which uses the NSGA-II [DPA+02] multi-objective evolutionary algorithm provided by the optimization framework OPT4J [LGR+11]. The DSE is configured to perform 1,000 optimization iterations with a population size of 100 mappings where 25 mappings are generated per iteration. The following experimental results represent an average over five runs of DSE per application on each architecture.

Investigated Approaches The experiments investigate mapping adaptations based on the proposed task migration methodology in comparison to the mapping reconfiguration methodology presented in Section 6.2. For the migration methodology, the RPM uses the XY-routing algorithm [NM93] for the construction of migration routes and routing of messages among tasks. For fairness of comparison, the relocation of tasks is realized sequentially, and the mapping reconfiguration methodology employs a hybrid reconfiguration analysis where the WCRL of each migrating task is analyzed online considering the actual availability of the NoC. The suspension of applications is taken to follow a non-preemptive scheme. For each task $t \in T$, the context size $\Psi_{\text{ctx}}(t)$ is provided by the benchmarks suite [Dic10].

Results Discussion

The effectiveness and the efficiency of the proposed task migration methodology have been investigated in a case study on the thermal management of many-core systems using mapping adaptation. The case study provides a comparative investigation of the proposed task migration methodology and the mapping reconfiguration methodology presented in Section 6.2 as possible schemes for on-demand mapping adaptation for hard real-time applications.

Similarly to the thermal-management case study presented in Section 6.2.4, the experiment begins with the application in question begin mapped on an otherwise empty platform. Then, through a series of iterations, additional load is introduced gradually into the system to realize various system utilization states. At each iteration, one of the cores in use by the application is selected randomly and marked as hot which necessitates an online adaptation of the application in near future. Here, (i) *task migration* and (ii) *mapping reconfiguration* are considered as mapping adaptation schemes. Using task migration, the RPM relocates all tasks running on the affected core to a core on another tile subject to the availability of required resources and the real-time conformity of the adaptation examined as detailed in Section 6.3.3. Using mapping reconfiguration, the RPM adapts the application's mapping in the form of a reconfiguration to another precomputed mapping for which the resource availability and real-time conformity admission criteria detailed in Section 6.2.3 are satisfied. At each iteration of the experiments, it has been recorded whether each adaptation approach has been able to evacuate the hot core successfully. The iterations of introducing additional load into the system and evaluating the adaptation of the application are continued until the platform is fully occupied. The results presented in the following are an average over 20 repetitions of the experiment per application on each many-core architecture to incorporate diverse mixes of preoccupied resources for each system utilization level.

Success Rate The success rate of the investigated adaptation approaches, task migration and mapping reconfiguration, in evacuating the hot core at various system utilization levels is illustrated in Figure 6.11 for the networking and consumer applications and in Figure 6.12 for the telecom and automotive applications. In each figure, each column (row) of plots corresponds to one application (architecture). The high success rate of task migration in all plots confirms the effectiveness of the proposed task migration methodology as an online mapping adaptation scheme for hard real-time applications. Using task migration, the RPM achieves a significantly higher adaptation success rate, compared to mapping reconfiguration. The substantial gap between the success rate of the two adaptation approaches roots in the following three advantages of task migration over mapping reconfiguration: Since task migration empowers a fine granularity of adaptation, mapping adaptations based on task migration often involve the relocation of only a subset of the application's tasks. Thus, a mapping adaptation

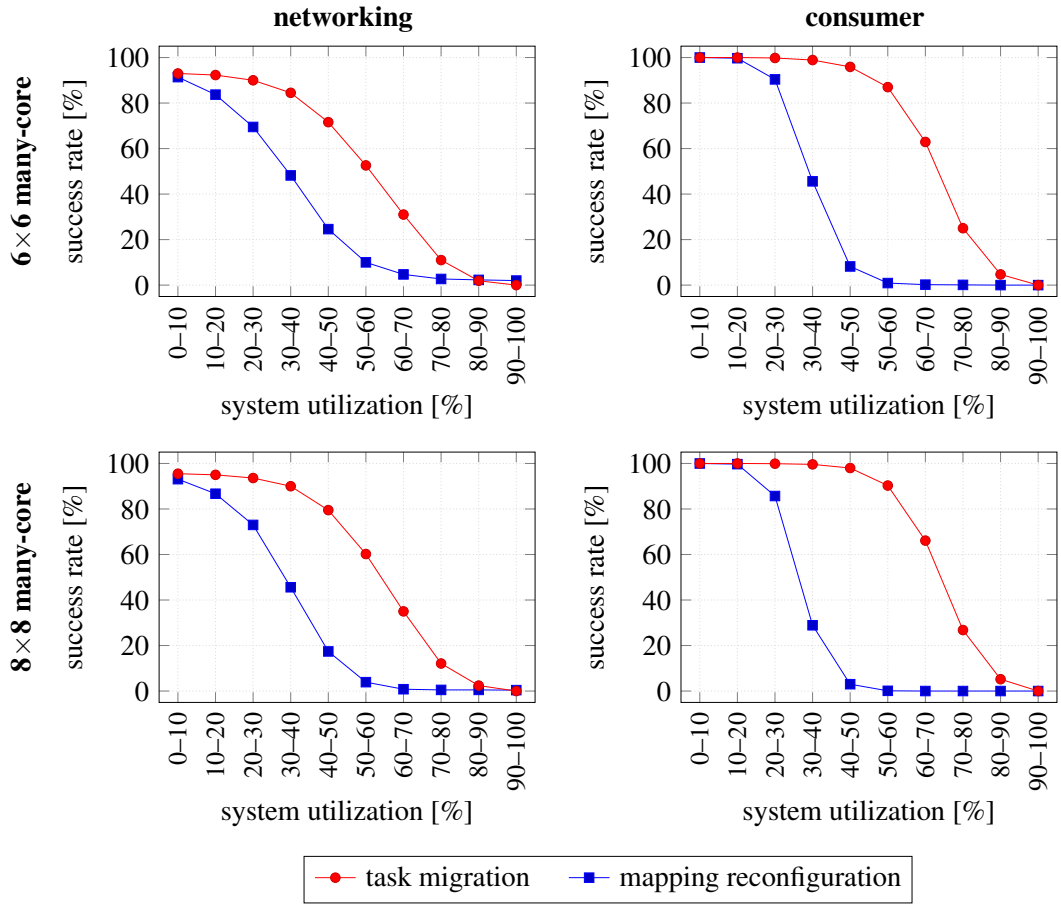


Figure 6.11: Success rate of task migration and mapping reconfiguration at different system utilization levels for the networking and consumer applications (plot columns) on the two many-core architectures (plot rows) [PSW+20].

based on task migration (i) requires a smaller set of resources which enhances its admissibility w.r.t. the availability of resources, (ii) entails a lower temporal overhead which enhances its admissibility w.r.t. the timing constraints of the application, and (iii) empowers the RPM to consider *all* possible adaptation options instead of restricting it to a set of statically computed options. These advantages enable the proposed task migration methodology to achieve an average improvement of 35% (up to 95%) in the adaptation success rate compared to mapping reconfiguration among all considered applications on both architectures. Note that the success rate gap between the task migration and mapping reconfiguration approaches scales with the NoC size. This renders the proposed task migration methodology an attractive approach for on-demand mapping adaptation, particularly for large scale many-core systems.

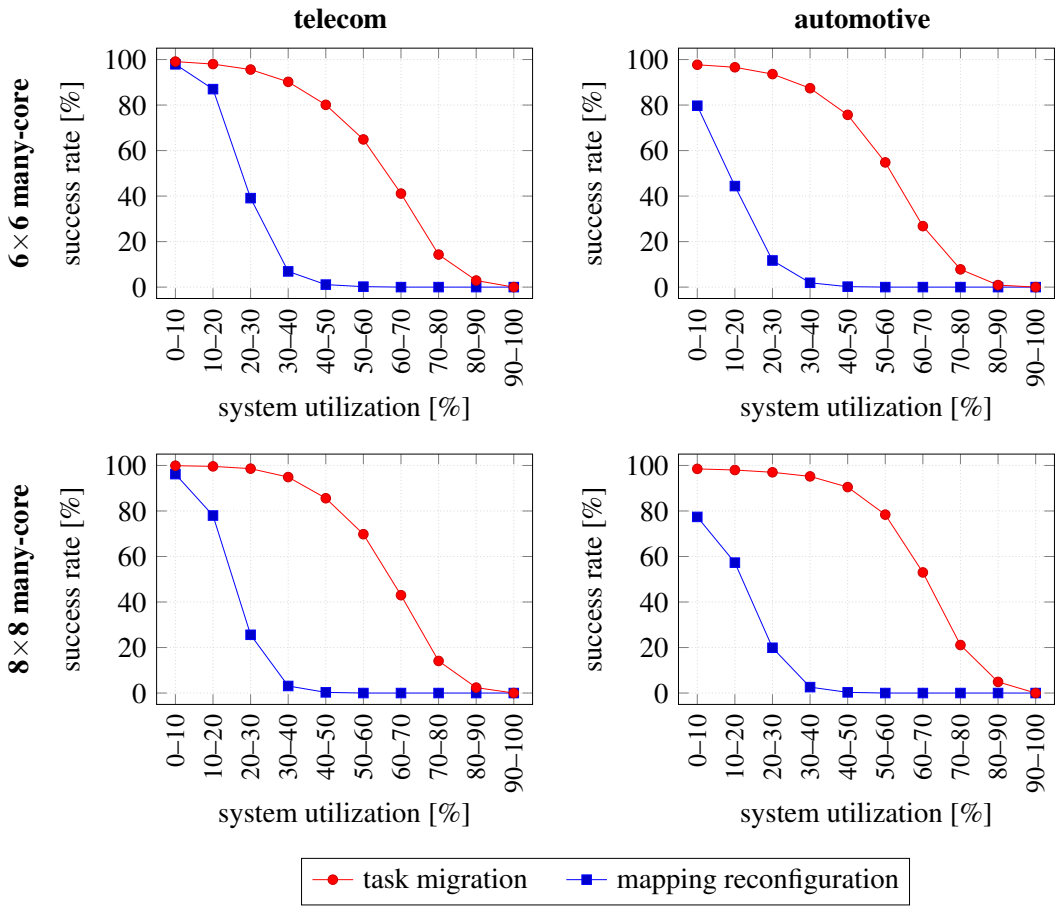


Figure 6.12: Success rate of task migration and mapping reconfiguration at different system utilization levels for the telecom and automotive applications (plot columns) on the two many-core architectures (plot rows) [PSW+20].

Online Computational Overhead At run time, once a core has been identified as hot and, hence, the adaptation of the application that uses it becomes necessary, the RPM begins the search for a suitable target location for the migration of the tasks that are currently running on that core. During the search process, the application continues its regular execution. Therefore, the overhead of the search process is not critical w.r.t. the real-time constraints of the application. However, given the limited computational power available to the RPM for the search, the overhead of the search process (which is mainly due to the resource- and timing admission checks) must be acceptable for online use. In Section 6.3.3 the scalability of the proposed migration timing analysis and feasibility check was demonstrated by proving their linear computational complexity w.r.t. the size of the application. To investigate the imposed overhead in absolute time, during the thermal-management experiment, the time spent by the RPM searching for a suitable destination location for the migrations

has been recorded as well. This time span begins with the emergence of the hot spot and terminates when the RPM finds the first migration destination for which both the resource- and timing admission conditions are satisfied. The records denote an average overhead of 1.08 ms (with a standard deviation of 0.16 ms) for the resource check and 0.57 ms (with a standard deviation of 0.06 ms) for the timing check. These measures were derived on an Intel i7-4770 CPU operating at 3.4 GHz with 32 GiB of RAM. According to these results, the overhead of the proposed migration timing analysis and feasibility check is on average 47% lower than that of the resource check which verifies their lightness to be used at run time.

6.4 Related Work

There exists a large body of work on online mapping adaptation in multi/many-core systems. These works employ mapping adaptation for dynamic load balancing [HIB10; AVB+10; KBL+11; FCW+14], thermal management [RPM+19; LTH+14; GMQ10; MAA+09], energy management [HV12], or fault resilience [AMZ16; SPM09]. Similarly to the mapping reconfiguration and task migration methodologies proposed in this work, existing mapping adaptation approaches employ task migration for modifying the arrangement of tasks of running applications on the platform. The majority of existing mapping adaptation approaches, however, either (i) rely on certain assumptions about the target platform which do not necessarily hold for many-core platforms in general, or (ii) disregard the temporal overhead imposed by the adaptation process which renders them inapplicable for hard real-time applications. In the following, an overview of current practices in mapping adaptation in multi/many-core systems is provided, and their applicability in the context of real-time applications and systems targeting many-core platforms is elucidated.

The *memory scheme* of a system has a significant impact on the choice of migration mechanisms applicable to that system. The vast majority of existing task migration mechanisms assume a globally shared memory scheme, e.g., [AAC+08; BAB+06; HV11; HLL+13; PAC+07; SPM09]. In a many-core context, such mechanisms can be employed for an *intra-tile migration* of tasks. However, due to the distributed memory scheme across tiles of a many-core system, *inter-tile migrations* can be realized merely by an explicit relocation of the migration data-set between the source and destination tiles of the migrating tasks as employed in [EGR+15; FWL+13; BBW+07; VSK+15] and in the adaptation methodologies proposed in this work.

Existing works on task migration use three schemes to provide the migration data-set to the destination location: A large share of works rely on a static *replication* of the code of tasks on every memory across the platform so that only the dynamic part of the migration data-set is relocated in the case of a migration [PAC+07; EGR+15; GLB+09; AAC+08; MAA+09]. Other approaches terminate the migrating task at the source location and *recreate* it from scratch at the destination location [NMA+05;

VSK+15; BAB+06]. Both strategies, replication and recreation, mitigate the temporal overhead of task migration by reducing the size of data to be relocated. However, recreation relies on a shared memory scheme, and replication lacks storage scalability such that its storage overhead is often intolerable for embedded many-core systems. Alternatively, in [BBW+07; MJD+11], similarly to the adaptation methodologies proposed in this work, the whole migration data-set is *relocated* between the source and destination locations which is practical for many-core systems due to its storage scalability and conformity to a distributed memory scheme.

In an application, processing tasks communicate with each other via messages. To preserve the *data-flow consistency* of the application during a migration process, it must be ensured that (i) each and every instance of messages communicated between the tasks of the application are indeed provided to their respective target tasks which must also happen (ii) in the original order they would have reached the target task in the absence of the adaptation process. To avoid the loss of messages consequent to a migration process, [NMA+05] employs a *buffer-and-forward* scheme in which input messages of migrating tasks, which arrive at the old location after the initiation of the migration process, are buffered in the old location and, after the relocation of the task, are forwarded to the new location. In [WRW+14], *dual-casting* and *data forwarding* schemes are employed as alternatives to the buffer-and-forward scheme, while [EGR+15] uses a *drop-and-resend* scheme in which unprocessed input messages are dropped at the old location and are re-transmitted by the producer to the new location after the migration process. These approaches may lead to an out-of-order delivery of messages which harms the application's data-flow consistency. Moreover, they either rely on message-forwarding services which are typically not available in embedded systems or necessitate the involvement of the application in the migration process which annihilates the transparency of the migration to the application. Noteworthy, the majority of existing migration mechanisms do not address the data-flow consistency concerns at all. The adaptation methodologies proposed in this work are transparent to the application and employ *communication suspension* to prevent the loss or out-of-order delivery of messages *without* relying on message-forwarding or buffer-reordering services.

Mapping adaptation of hard real-time applications requires a (i) *timing-predictable adaptation mechanism* supported with a (ii) *worst-case timing analysis* to bound the temporal overhead of the adaptation process. In the literature, only a few works investigate the temporal overhead of task migration, e.g. [AAC+08; BBW+07; HV11; SMR11]. Nonetheless, they either focus on *soft real-time* constraints and, hence, do not provide timing *guarantees* [AAC+08; BBW+07], or investigate hard real-time task migrations but rely on certain assumptions, e.g., a globally shared memory scheme, which renders them inapplicable for inter-tile migrations in many-core systems [HV11; SMR11]. Contrarily, the mapping reconfiguration and task migration methodologies proposed in this work employ a timing-predictable adaptation mechanism supported with a formal worst-case timing analysis and temporal admission check of the adapta-

tion processes. Authors in [DSI17] investigate hard real-time *system reconfigurations* between statically known system modes, each corresponding to a unique choice and deployment of running applications. This approach is, however, not a viable solution for dynamic systems with a large number of independent applications as the number of system modes and involved migrations per mode transition grows exponentially with the number of applications. The mapping reconfiguration methodology proposed in this work offers a better adaptation scalability as it empowers a *per-application* adaptation scheme in which each running application can be reconfigured between a set of precomputed mappings independently of the other applications. The proposed task migration methodology further lifts the restriction of precomputed adaptation options and enables the finest granularity of mapping adaptation for hard real-time applications in many-core systems.

6.5 Summary

Online and on-demand adaptation of mappings is an indispensable utility in emerging embedded systems to guarantee the timely execution of hard real-time applications in view of the steady and unforeseeable changes in the workload of applications and the availability of resources. This chapter presented two mapping adaptation methodologies that empower a timing-predictable adaptation of hard real-time applications in heterogeneous many-core systems with worst-case adaptation timing guarantees.

The *mapping reconfiguration methodology* presented in Section 6.2 enables online mapping adaptations in the form of a reconfiguration between the precomputed mappings of each application. The *task migration methodology* presented in Section 6.3 enables a finer granularity of adaptation by empowering the migration of any subset of an application's tasks to any desired destination locations. Both proposed adaptation methodologies employ a (i) *timing-predictable adaptation mechanism* and are supported with a (ii) *worst-case timing analysis* to formally bound the worst-case latency of the adaptation process and an (iii) *adaptation admission check* which examines the real-time conformity of the adaptation temporal overhead and the changes in the timing behavior of the application during and after the adaptation process. While the mapping reconfiguration methodology benefits from a computationally heavy analysis of reconfiguration options at design time and, thereby, imposes a negligible online computational overhead, the task migration methodology provides the RPM with a lightweight migration timing analysis to enable mapping adaptations *without* being restricted to a statically computed set of adaptation options. The effectiveness of the proposed methodologies for an online adaptation of mappings on-demand with hard timing guarantees was demonstrated experimentally in Sections 6.2.4 and 6.3.4.

7 Conclusion and Future Directions

Embedded systems in various areas, e.g., automotive, telecommunications, and industrial automation, are increasingly encompassing applications with hard real-time constraints. A seamless satisfaction of the timing requirements of such applications throughout their lifespan is crucial as any violation of their timing constraints may lead to the failure of the whole system. Recent years give evidence of a rapid increase in the number and workload dynamism of applications in emerging embedded systems. The resulting massive computational-power demand of such systems is projected to render (heterogeneous) many-core architectures their de facto standard target platform in the near future. In this context, novel design methodologies are required to empower an adaptive mapping and management of applications on many-core platforms to achieve high utilization of the underlying resources, while satisfying the individual requirements of the applications, e.g., real-time constraints, in view of the steady, unforeseeable fluctuations in the mix of concurrently executed applications, the state and the availability of system resources, and the workload of each running application during its lifespan. To that end, this work proposes a collection of methodologies, analyses, and techniques that address the major design challenges of hard real-time systems targeting heterogeneous many-core platforms. Together, the contributions of this work enable an automated mapping and adaptive management of hard real-time applications on many-core platforms with verified timing guarantees in line with their real-time constraints. In the following, the key contributions of this work are outlined, followed by a perspective on possible directions for future work.

7.1 Summary

Given the computational complexity of timing verification procedures, adaptive mapping and management of hard real-time applications on many-core platforms necessarily relies on a Hybrid Application Mapping (HAM) scheme consisting of a design-time application characterization phase followed by a run-time system management phase: At design time, mapping optimization, analysis, and evaluation is performed in an automatic Design Space Exploration (DSE) of each application to compute a set of high-quality mapping alternatives with verified timing guarantees

and diverse resource requirements. At run time, the computed mappings for each application are provided to a Run-time Platform Manager (RPM) to launch each application on-demand using a precomputed mapping that adheres to the current timing constraints of the application and the availability of resources. By performing the compute-intensive optimization and evaluation of mappings offline, HAM enables adaptive mapping and management of hard real-time applications on many-core platforms with an affordable run-time computational overhead. The contributions of this work outlined in the following are developed in line with this hybrid design scheme.

Many-core timing analysis procedures are fundamental prerequisites for the adoption of many-core platforms for real-time applications and systems. The timing analyses used for many-core systems often cannot analyze all aspects of timing interferences that may arise during the execution of applications, in the general case. Therefore, to provide safe timing guarantees, they rely on several restrictions on the mapping of applications and the resource-reservation policy they follow which often result in massive under-utilization of resources or unsatisfactory timing guarantees. As a remedy, in Chapter 3, this work proposes a novel methodology for *mapping optimization and timing analysis* of hard real-time applications on many-core platforms. This methodology provides a (i) *worst-case timing analysis* that is specifically developed for many-core systems and, thus, can provide formal timing guarantees for applications, without depending on the aforementioned restrictions imposed by existing analyses. In light of this analysis, a novel (ii) *mapping optimization* approach is introduced which enables a fine-grained adjustment of the amount of resources allocated per mapping during DSE. Together, (i) and (ii) lift the aforementioned restrictions imposed by existing analyses and enable the DSE to deliver resource-efficient mappings with satisfactory timing guarantees.

Many-core systems are susceptible to critically high temperatures on the chip. To preserve the thermal integrity of the system, they employ autonomous Dynamic Thermal Management (DTM) mechanisms that continuously monitor the thermal state of the chip and, upon the emergence of hot spots, suspend or decelerate the execution of applications in the thermally affected region to reduce its temperature. This practice, however, may lead to an unacceptable violation of hard timing constraints. As a remedy, in Chapter 4, this work proposes a *thermally composable HAM methodology* that enforces thermal safety in the system proactively at all times and, thereby, eliminates the exposure of hard real-time applications to DTM mechanisms. The proposed methodology introduces a (i) *thermal-safety analysis* into the DSE to derive an abstract thermal-safety index for each mapping and provides a lightweight (ii) *thermal-safety admission check* to the RPM to be used when launching an application to examine the thermal safety of each mapping candidate for the current system state based on its precomputed thermal-safety index. Together, (i) and (ii) enable the RPM to preserve the thermal safety of the system proactively and, thereby, eliminate DTM interrupts on the execution of hard real-time applications in many-core systems.

During the DSE in HAM, mappings are often optimized w.r.t. a set of design

objectives assessing their quality, e.g., in terms of timeliness and energy consumption, and a set of objectives assessing the number of required resources of each type. This relatively large number of design objectives often results in a huge set of Pareto-optimal mappings which may impose an immense overhead for the RPM if considered in its entirety at run time. Particularly, as this overhead degrades the responsiveness of the RPM, it can become fatal for real-time applications due to their dependence on prompt reactions from the RPM, e.g., upon a change in their timing constraints. As a remedy, in Chapter 5, this work proposes a *mapping-set distillation methodology* that selects a representative subset of the Pareto-optimal mappings whose cardinality can be adjusted according to the computational power available to the RPM. The proposed methodology distinguishes quality objectives from resource objectives and treats each properly. Thereby, it distills a subset of mappings such that diverse quality trade-offs are retained to meet the varying quality requirements of the application at run time while retaining diverse yet efficient resource combinations to enhance the application's embeddability w.r.t. the dynamic availability of resources at run time.

Real-time applications in many-core systems are often subject to unforeseeable online events, e.g., resource failures due to overheating or on-demand changes in their timing constraints. Such events may invalidate the mapping currently in use by them. To preserve a seamless satisfaction of hard real-time constraints despite such events, adapting the mapping of applications on-the-fly during their execution becomes inevitable. To that end, in Chapter 6, this work proposes two mapping adaptation methodologies for hard real-time applications in many-core systems: a (i) *mapping reconfiguration methodology* that enables mapping adaptations in the form of a reconfiguration (switch) between precomputed mappings of an application, and a (ii) *task migration methodology* that enables a finer granularity of adaptation by allowing the migration of any subset of an application's tasks to any desired destination locations. Each proposed methodology provides a (i) *timing-predictable adaptation mechanism*, a (ii) *worst-case timing analysis* to formally bound the latency of the adaptation process, and an (iii) *admission check* to examine the real-time conformity of each adaptation option w.r.t. the real-time constraints of the application. While the mapping reconfiguration methodology benefits from a computationally intensive analysis of reconfiguration options at design time and, thereby, imposes a negligible online computational overhead, the task migration methodology provides the RPM with a lightweight migration timing analysis to enable fine-grained mapping adaptations without being restricted to a precomputed set of adaptation options.

7.2 Future Work

The methodologies proposed in this work provide the fundamental utilities and mechanisms necessary for the adoption of many-core platforms for real-time applications and systems. These contributions can serve as the groundwork for several directions

of future research, some of which are outlined in the following.

The timing analysis presented in Chapter 3 serves as the backbone for the majority of the methodologies proposed in this work. Currently, this analysis does not reflect the impact of local caches of cores on alleviating the worst-case timing properties of applications. Extending the timing analysis by an investigation of Cache-Related Preemption Delays (CRPDs) is, hence, a promising future direction for improving the tightness of the derived timing guarantees. Given that the proposed timing analysis is developed in a compositional manner, an extension of it will introduce only minimal modifications to its existing parts.

The thermally composable HAM methodology presented in Chapter 4 relies on an offline thermal-safety analysis of mappings. An offline thermal safety analysis entails a certain degree of pessimism in the provided thermal safety guarantees which manifests itself in the form of a system utilization penalty at run time as elaborated in Chapter 4. In this line, future research can be invested in postponing parts of the offline thermal analysis to run time to empower the RPM to mitigate this penalty by reflecting the actual system state into the analysis of the thermally safe power-density constraint and, thereby, alleviate the pessimism entailed in thermal composability. Moreover, with such a shift, the uniform power-density constraint imposed on all active cores can further be substituted with an adaptive power-density constraint in which the thermally safe power budget of each core is adjusted according to its peak power consumption under nominal frequency and the power profile of the tasks executed on it.

The mapping adaptation methodologies presented in Chapter 6 provide *mechanisms* for an on-demand mapping adaptation of hard real-time applications in many-core systems. These mechanisms determine *how* a given mapping adaptation must be performed. In light of these mechanisms, future research is required to investigate adaptation *policies* and *strategies* to support the RPM in deciding *when* adaptations must take place, *which* applications must be adapted, *which* adaptation mechanism (mapping reconfiguration or task migration) to be used, and *which* adaptation option (target mapping or post-adaptation deployment) to opt for.

In general, the methodologies proposed in this work enable an on-demand mapping and adaptation of real-time applications in many-core systems based on a set of precomputed mapping options and subject to several online admission checks in terms of resource demand, timing, thermal safety, etc. These methodologies are guaranteed to meet the timing requirements of a hard real-time application as long as at least one of its precomputed mappings fulfills each and every admission criterion when launching the application or adapting its mapping due to the failure of its current mapping, e.g., consequent to hardware faults. In this context, a promising and crucial direction for future research lies in the investigation of *fallback solutions* in case none of the precomputed mappings fulfills all admission criteria. An investigation of strategies for terminating or reconfiguring low-priority applications to enhance the system state in favor of high-priority applications or to release the resources required by them can serve as initial steps in that direction.

German Part

Methoden zur Abbildung, Analyse und Verwaltung von Echtzeitanwendungen in Mehrkernsystemen

Zusammenfassung

Eingebettete Systeme in verschiedenen Bereichen, z. B. Automobil, Telekommunikation und Industrieautomatisierung, beinhalten zunehmend Anwendungen mit harten Echtzeitanforderungen. Bei Anwendungen dieser Art ist die Einhaltung der applikationsspezifischen zeitlichen Beschränkungen von entscheidender Bedeutung, da eine Verletzung dieser zum Systemausfall oder im schlimmsten Fall sogar zur Gefährdung von Menschen führen kann. In den letzten Jahren ist eine rasche Zunahme der Anzahl und der Dynamik von Anwendungen in eingebetteten Systemen zu beobachten. Der daraus resultierende massive Bedarf an Rechenleistung wird voraussichtlich dazu führen, dass (heterogene) Vielkernarchitekturen in naher Zukunft zu der De-Facto-Standard-Zielplattform in diesem Bereich werden.

In diesem Zusammenhang sind neuartige Entwurfsmethoden erforderlich, die eine adaptive Abbildung und Verwaltung von Anwendungen auf Vielkernplattformen ermöglichen, um eine hohe Auslastung der zugrundeliegenden Rechenressourcen zu erreichen und gleichzeitig die individuellen Anforderungen der Anwendungen zu erfüllen. Die Einhaltung dieser Anforderungen, die sowohl funktionale (z. B. Echtzeit) als auch nichtfunktionale Anforderungen beinhalten, muss über die gesamte Lebensdauer der Anwendung hinweg garantiert werden. Dies stellt besonders im Hinblick auf die sich stetig und unvorhersehbar ändernden Ausführungsbedingungen (z. B. Art und Anzahl der ausgeführten Anwendungen, die Menge der Arbeitslast oder die Verfügbarkeit der Systemressourcen) eine signifikante Herausforderung dar. Zu diesem Zweck schlägt diese Arbeit eine Reihe von Methoden, Analysen und Techniken vor, die sich mit den wichtigsten Entwurfsherausforderungen harter Echtzeitsysteme für heterogene Vielkernplattformen befassen. Zusammen ermöglichen die Beiträge dieser Arbeit eine automatisierte Abbildung und adaptive Verwaltung von Anwendungen mit harten Echtzeitanforderungen auf Vielkernplattformen mit verifizierten Laufzeitgarantien entsprechend ihrer Echtzeitanforderungen. Im Folgenden werden die wichtigsten Beiträge dieser Arbeit zusammengefasst.

Angesichts des hohen Rechenaufwands von Zeitanalyseverfahren beruht die adaptive Abbildung und Verwaltung von harten Echtzeitanwendungen auf Vielkernplattformen notwendigerweise auf einem hybriden Anwendungsabbildungsschema (englisch: Hybrid Application Mapping (HAM)). Die HAM-Methodik besteht aus einer Phase zur Charakterisierung der Anwendungen zur Entwurfszeit. Dieser folgt ein dynamisches Laufzeitmanagemant während ihrer Ausführung, die dabei die zur Entwurfszeit gewonnen Charakteristika ausnutzt. Zur Entwurfszeit wird die Optimierung, Analyse und Bewertung der Abbildungsmöglichkeiten jeder Anwendung auf die

Zielplattform in einer automatischen Entwurfsraumexploration (englisch: Design Space Exploration (DSE)) durchgeführt, um eine Menge Pareto-optimaler Abbildungsalternativen mit verifizierten Zeitgarantien und unterschiedlichen Ressourcenanforderungen zu berechnen. Zur Laufzeit werden die berechneten Abbildungsalternativen für jede Anwendung einem Laufzeit-Plattformmanager (englisch: Run-time Platform Manager (RPM)) zur Verfügung gestellt, um jede Anwendung bei Bedarf entsprechend einer vorberechneten Abbildung zu starten, die zu den Echtzeitanforderungen der Anwendung und der Verfügbarkeit von Ressourcen passt. Da die rechenintensive Optimierung und Auswertung der relavanten Charakteristiken der betrachteten Abbildungen bereits zur Entwurfzeit stattgefunden hat, ermöglicht HAM ein skalierbares Laufzeitmanagement adaptiver Echtzeitanwendungen mit geringem Rechenaufwand. Im Folgenden sind die Beiträge dieser Arbeit zusammengefasst.

Als ersten Beitrag schlägt diese Arbeit eine formale Zeitanalyse vor, die verschiedene Arten von zeitlichen Garantien für Echtzeitanwendungen in Mehrkernsystemen ableiten kann. Basierend auf dieser Analyse wird ein neuartiger Ansatz zur Optimierung der Abbildungen auf Vielkernplattformen vorgeschlagen, bei dem die Menge der pro Abbildung zugewiesenen Ressourcen entsprechend den Anforderungen der Anwendung bezüglich Zeit und Rechenleistung fein abgestimmt wird, um qualitativ hochwertige Abbildungen mit zufriedenstellenden Laufzeitgarantien und verbesserter Ressourceneffizienz zu erhalten.

In Mehrkernsystemen sind Anwendungen mit harten Echtzeitanforderungen besonders anfällig für die autonomen eingebauten Mechanismen des dynamischen Temperaturmanagements (englisch: Dynamic Thermal Management (DTM)), die für die Einhaltung der thermischen Sicherheit des Systems verantwortlich sind. Beim Auftreten von Überhitzung unterbrechen oder verlangsamen die DTM-Mechanismen die Ausführung der Anwendungen, die in den thermisch betroffenen Regionen laufen, was zur Verletzung ihrer Echtzeitanforderungen führen kann. Um solche fatalen Interferenzen zu eliminieren, schlägt diese Arbeit eine neuartige HAM-Methodik vor, die die thermische Sicherheit des Systems jederzeit proaktiv bewahrt und dadurch verhindert, dass Echtzeitanwendungen den DTM-Mechanismen ausgesetzt werden, ohne die thermische Integrität des Systems zu gefährden.

Die DSE in HAM-Methodiken liefert in der Regel eine riesige Menge an Pareto-optimalen Abbildungsalternativen, die dem RPM einen erheblichen Managementaufwand aufbürden und seine Reaktionsfähigkeit verschlechtern. Dies kann insbesondere für Echtzeitanwendungen fatal sein, da sie—beispielsweise bei Ausfall einer der von ihnen genutzten Ressourcen—auf eine schnelle Reaktion des RPM angewiesen sind. Als Abhilfemaßnahme schlägt dies Arbeit eine automatische Destillationsmethode vor. Ihre Aufgabe ist es, eine Teilmenge der Pareto-optimalen Abbildungsalternativen zu bestimmen, die dann dem Laufzeitmanagement zur Verfügung stehen. Das Ziel dabei ist, den Overhead des RPM zu verringern, während gleichzeitig die beibehaltenen Abbildungsalternativen im Hinblick auf Qualität und Ressourcenbedarf sehr divers sind, um verschiedenen Laufzeitszenarien und -anforderungen gerecht zu werden.

Um zu garantieren, dass Echtzeitanforderungen auch bei unerwarteten Laufzeitereignissen, z. B. beim Ausfall von Ressourcen, erfüllt werden können, ist es notwendig, die Abbildungen von Echtzeitanwendungen zur Laufzeit dynamisch anpassen zu können. Zu diesem Zweck schlägt diese Arbeit zwei Methoden zur dynamischen Anpassung der Abbildung von Anwendungen mit harten Echtzeitanfoderungen in Vielkernsystemen vor. Im ersten Ansatz werden Anpassungen in Form einer Rekonfiguration zwischen vorberechneten Abbildungen realisiert, während der zweite Ansatz feingranulare Anpassungen der Abbildungen in Form der Migration einer beliebigen Teilmenge der Tasks der Anwendung zwischen beliebigen Kernen im System ermöglicht. Beide Methoden bieten formale Zeitanalysen und leichtgewichtige Zulassungsprüfungen, um die Echtzeit-Konformität jeder Anpassungsoption entsprechend den Zeitbeschränkungen der Anwendung zur Laufzeit zu überprüfen.

Bibliography

[AAB+00] Bryan Ackland, Alex Anesko, Douglas Brinthaupt, Steven J Daubert, Asawaree Kalavade, Joseph Knobloch, E Micca, Mallik Moturi, Chris J Nicol, Jay H O'Neill, et al. A single-chip, 1.6-billion, 16-b MAC/s multiprocessor DSP. *IEEE Journal of Solid-State Circuits (JSSC)*, 35(3):412–424, 2000.

[AAC+03] Alain Artieri, Viviana D Alto, Richard Chesson, Mark Hopkins, and Marco C Rossi. Nomadik–open multimedia platform for next-generation mobile devices. *STMicroelectronics Technical Article TA305*, 2003.

[AAC+08] Andrea Acquaviva, Andrea Alimonda, Salvatore Carta, and Michele Pittau. Assessing task migration impact on embedded soft real-time streaming multimedia applications. *EURASIP Journal on Embedded Systems*, 2008(9):1–15, 2008.

[ADI+15] Sebastian Altmeyer, Robert I Davis, Leandro Indrusiak, Claire Maiza, Vincent Nelis, and Jan Reineke. A generic and compositional framework for multicore response time analysis. In *Proceedings of the International Conference on Real-Time Networks and Systems (RTNS)*, pages 129–138. ACM, 2015.

[AMH+11] Benny Akesson, Anca Molnos, Andreas Hansson, Jude Ambrose Angelo, and Kees Goossens. Composability and predictability for independent application development, verification, and execution. In *Multiprocessor System-on-Chip*, pages 25–56. Springer, 2011.

[AMZ16] Zaid Al-bayati, Brett H Meyer, and Haibo Zeng. Fault-tolerant scheduling of multicore mixed-criticality systems under permanent failures. In *Proceedings of the International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, pages 57–62. IEEE, 2016.

[ATB+13] Iraklis Anagnostopoulos, Vasileios Tsoutsouras, Alexandros Bartzas, and Dimitrios Soudris. Distributed run-time resource management for malleable applications on many-core platforms. In *Proceedings of the Design Automation Conference (DAC)*, page 168. ACM/IEEE, 2013.

[AVB+10] Gabriel Marchesan Almeida, Sameer Varyani, Rémi Busseuil, Gilles Sassatelli, Pascal Benoit, Lionel Torres, Everton Alceu Carara, and Fernando Gehm Moraes. Evaluating the impact of task migration in multi-processor systems-on-chip. In *Proceedings of the Symposium on Integrated Circuits and System Design (SBCCI)*, pages 73–78, 2010.

[AZ18] Athena Abdi and Hamid R Zarandi. HYSTERY: a hybrid scheduling and mapping approach to optimize temperature, energy consumption and lifetime reliability of heterogeneous multiprocessor systems. *Journal of Supercomputing*:1–26, 2018.

[BAB+06] Stefano Bertozzi, Andrea Acquaviva, Davide Bertozzi, and Antonio Poggiali. Supporting task migration in multi-processor systems-on-chip: a feasibility study. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 15–20. ACM/IEEE, 2006.

[BBE+11] Enrico Bini, Giorgio Buttazzo, Johan Eker, Stefan Schorr, Raphael Guerra, Gerhard Fohler, Karl-Erik Arzen, Vanessa Romero, and Claudio Scordino. Resource management on multicore systems: the ACTORS approach. *IEEE Micro*, 31(3):72–81, 2011.

[BBJ14] R Borkar, M Bohr, and S Jourdan. Advancing Moore's law in 2014. *Intel*, 2014.

[BBL+10] Alessio Bonfietti, Luca Benini, Michele Lombardi, and Michela Milano. An efficient and complete approach for throughput-maximal SDF allocation and scheduling on multi-core platforms. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 897–902. ACM/IEEE, 2010.

[BBW+07] Eduardo Wenzel Brião, Daniel Barcelos, Fabio Wronski, and Flávio Rech Wagner. Impact of task migration in NoC-based MPSoCs for soft real-time applications. In *IFIP International Conference on Very Large Scale Integration (VLSI-SoC)*, pages 296–299. IEEE, 2007.

[BD02] Luca Benini and Giovanni De Micheli. Networks on chip: a new paradigm for systems on chip design. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 418–419. ACM/IEEE, 2002.

[BDY06] Luca Benini, Giovanni De Micheli, and T Tao Ye. *Networks on chips*, volume 1. Morgan Kaufmann Burlington, 2006.

[BLA+05] Giorgio Buttazzo, Giuseppe Lipari, Luca Abeni, and Marco Caccamo. *Soft real-time systems*. Springer, 2005.

[BM06] Tobias Bjerregaard and Shankar Mahadevan. A survey of research and practices of network-on-chip. *ACM Computing Surveys (CSUR)*, 38(1):1–es, 2006.

[BSP+17] Brent Bohnenstiehl, Aaron Stillmaker, Jon J Pimentel, Timothy Andreas, Bin Liu, Anh T Tran, Emmanuel Adeagbo, and Bevan M Baas. KiloCore: a 32-nm 1000-processor computational array. *IEEE Journal of Solid-State Circuits (JSSC)*, 52(4):891–902, 2017.

[BSR+17] Kalyan Shankar Bhattacharjee, Hemant Kumar Singh, Michael Ryan, and Tapabrata Ray. Bridging the gap: many-objective optimization and informed decision-making. *IEEE Transactions on Evolutionary Computation (TEVC)*, 21(5):813–820, 2017.

[BTT98] Tobias Blickle, Jürgen Teich, and Lothar Thiele. System-level synthesis using evolutionary algorithms. *Design Automation for Embedded Systems*, 3(1):23–58, 1998.

[But11] Giorgio C Buttazzo. *Hard real-time computing systems: Predictable scheduling algorithms and applications*, volume 24. Springer Science & Business Media, 2011.

[CGS+05] Philippe Charles, Christian Grothoff, Vijay Saraswat, Christopher Donawa, Allan Kielstra, Kemal Ebcioglu, Christoph Von Praun, and Vivek Sarkar. X10: an object-oriented approach to non-uniform cluster computing. *Acm Sigplan Notices*, 40(10):519–538, 2005.

[CHD10] Thidapat Chantem, X Sharon Hu, and Robert P Dick. Temperature-aware scheduling and assignment for hard real-time applications on MPSoCs. *IEEE Transactions on Very Large Scale Integrated Systems (TVLSI)*, 19(10):1884–1897, 2010.

[CMM+13] Guilherme Castilhos, Marcelo Mandelli, Guilherme Madalozzo, and Fernando Moraes. Distributed resource management in NoC-based MPSoCs with dynamic cluster sizes. In *Proceedings of the IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 153–158. IEEE, 2013.

[CO95] Hua Wu David Chang and William JB Oldham. Dynamic task allocation models for large distributed computing systems. *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 6(12):1301–1315, 1995.

[COM08] Chen-Ling Chou, Umit Y Ogras, and Radu Marculescu. Energy-and performance-aware incremental mapping for networks on chip with multiple voltage levels. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 27(10):1866–1879, 2008.

[Cor01] C-Port Corporation. C-5 network processor architecture guide. 2001. URL: https://www.nxp.com/docs/en/reference-manual/C5NPD0-AG.pdf.

[CRR+14] Sudipta Chattopadhyay, Abhik Roychoudhury, Jakob Rosén, Petru Eles, and Zebo Peng. Time-predictable embedded software on multi-core platforms: analysis and optimization. *Foundations and Trends in Electronic Design Automation*, 8(3-4):199–356, 2014.

[CYJ16] Wei-Yu Chiu, Gary G Yen, and Teng-Kuei Juan. Minimum manhattan distance approach to multiple criteria decision making in multiobjective optimization problems. *IEEE Transactions on Evolutionary Computation (TEVC)*, 20(6):972–985, 2016.

[DAI+17] Robert I Davis, Sebastian Altmeyer, Leandro S Indrusiak, Claire Maiza, Vincent Nelis, and Jan Reineke. An extensible framework for multicore response time analysis. *Real-Time Systems (RTS)*:1–55, 2017.

[Dal92] William J Dally. Virtual-channel flow control. *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 3(2):194–205, 1992.

[Dav91] Lawrence Davis. *Handbook of genetic algorithms*. VNR computer library. Van Nostrand Reinhold, 1991.

[DB11] Robert I Davis and Alan Burns. A survey of hard real-time scheduling for multiprocessor systems. *ACM computing surveys (CSUR)*, 43(4):1–44, 2011.

[dDAB+13] Benoît Dupont de Dinechin, Renaud Ayrignac, Pierre-Edouard Beaucamps, Patrice Couvert, Benoît Ganne, Pierre Guironnet de Massas, François Jacquet, Samuel Jones, Nicolas Morey Chaisemartin, Frédéric Riss, et al. A clustered manycore processor architecture for embedded and accelerated applications. In *Proceedings of the IEEE High-Performance Extreme Computing Conference (HPEC)*, pages 1–6. IEEE, 2013.

[Deb01] Kalyanmoy Deb. *Multi-objective optimization using evolutionary algorithms*, volume 16. John Wiley & Sons, 2001.

[DGR+74] Robert H Dennard, Fritz H Gaensslen, V Leo Rideout, Ernest Bassous, and Andre R LeBlanc. Design of ion-implanted MOSFET's with very small physical dimensions. *IEEE Journal of Solid-State Circuits (JSSC)*, 9(5):256–268, 1974.

[Dic10] Robert Dick. Embedded system synthesis benchmarks suite (E3S), 2010. URL: http://ziyang.eecs.umich.edu/~dickrp/e3sdd/.

[Dij+59] Edsger W Dijkstra et al. A note on two problems in connexion with graphs. *Numerische mathematik*, 1(1):269–271, 1959.

[DJR01] Santanu Dutta, Rune Jensen, and Alf Rieckmann. Viper: a multiprocessor SOC for advanced set-top box and digital TV systems. *IEEE Design & Test of Computers*, 18(5):21–31, 2001.

[DKV14] Anup Das, Akash Kumar, and Bharadwaj Veeravalli. Temperature aware energy-reliability trade-offs for mapping of throughput-constrained applications on multimedia MPSoCs. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 1–6. ACM/IEEE, 2014.

[DNA16] Dakshina Dasari, Vincent Nelis, and Benny Akesson. A framework for memory contention analysis in multi-core platforms. *Real-Time Systems (RTS)*, 52(3):272–322, 2016.

[DPA+02] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation (TEVC)*, 6(2):182–197, 2002.

[DRW98] Robert P Dick, David L Rhodes, and Wayne Wolf. TGFF: task graphs for free. In *Proceedings of the International Workshop on Hardware/-Software Codesign.(CODES/CASHE'98)*, pages 97–101. IEEE, 1998.

[DSI17] Piotr Dziurzanski, Amit Kumar Singh, and Leandro Soares Indrusiak. Multi-criteria resource allocation in modal hard real-time systems. *EURASIP Journal on Embedded Systems*, 2017(1):30, 2017.

[DT04] William James Dally and Brian Patrick Towles. *Principles and practices of interconnection networks*. Elsevier, 2004.

[EBA+11] Hadi Esmaeilzadeh, Emily Blem, Renee St Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pages 365–376. IEEE, 2011.

[EGR+15] Ashaf El-Antably, Olivier Gruber, Frederic Rousseau, and Nicolas Fournel. Transparent and portable agent based task migration for dataflow applications on multi-tiled architectures. In *Proceedings of the Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, pages 183–192. ACM/IEEE, 2015.

[FCW+14] Weiwei Fu, Tianzhou Chen, Chao Wang, and Li Liu. Optimizing memory access traffic via runtime thread migration for on-chip distributed memory systems. *The Journal of Supercomputing*, 69(3):1491–1516, 2014.

[FF95] Carlos M Fonseca and Peter J Fleming. An overview of evolutionary algorithms in multiobjective optimization. *Evolutionary computation*, 3(1):1–16, 1995.

[FHT06] Joachim Falk, Christian Haubelt, and Jürgen Teich. Efficient representation and simulation of model-based designs in SystemC. In *Proceedings of the Forum on Design Languages (FDL)*, volume 6, 2006.

[FWL+13] FangFa Fu, Liang Wang, Yu Lu, and Jinxiang Wang. Low overhead task migration mechanism in NoC-based MPSoC. In *International Conference on ASIC*, pages 1–4. IEEE, 2013.

[GAC+13] Kees Goossens, Arnaldo Azevedo, Karthik Chandrasekar, Manil Dev Gomony, Sven Goossens, Martijn Koedam, Yonghui Li, Davit Mirzoyan, Anca Molnos, Ashkan Beyranvand Nejad, et al. Virtual execution platforms for mixed-time-criticality systems: the CompSOC architecture and design flow. *ACM SIGBED Review*, 10(3):23–34, 2013.

[GJ79] Michael R Garey and David S Johnson. *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman and Co., 1979.

[GLB+09] Laurent Gantel, Salah Layouni, Mohamed El Amine Benkhelifa, François Verdier, and Stéphanie Chauvet. Multiprocessor task migration implementation in a reconfigurable platform. In *Proceedings of the International Conference on Reconfigurable Computing and FPGAs (ReConFig)*, pages 362–367. IEEE, 2009.

[GLS+12] Georgia Giannopoulou, Kai Lampka, Nikolay Stoimenov, and Lothar Thiele. Timed model checking with abstractions: towards worst-case response time analysis in resource-sharing manycore systems. In *Proceedings of the International Conference on Embedded Software (EMSOFT)*, pages 63–72. ACM, 2012.

[GMQ10] Yang Ge, Parth Malani, and Qinru Qiu. Distributed task migration for thermal management in many-core systems. In *Proceedings of the Design Automation Conference (DAC)*, pages 579–584. ACM/IEEE, 2010.

[GSH+16] Georgia Giannopoulou, Nikolay Stoimenov, Pengcheng Huang, Lothar Thiele, and Benoît Dupont de Dinechin. Mixed-criticality scheduling on cluster-based manycores with shared communication and storage resources. *Real-Time Systems (RTS)*, 52(4):399–449, 2016.

[GTL+17] Michael Glaß, Jürgen Teich, Martin Lukasiewycz, and Felix Reimann. Hybrid optimization techniques for system-level design space exploration. In *Handbook of Hardware/Software Codesign*. Volume 1, pages 217–246. Springer, 2017.

[HAE+17] Robin Hofmann, Leonie Ahrendts, Rolf Ernst, Soonhoi Ha, and Jürgen Teich. CPA: compositional performance analysis. In Soonhoi Ha and Jürgen Teich, editors, *Handbook of Hardware/Software Codesign*, pages 721–751. Springer, 2017. ISBN: 978-94-017-7267-9. DOI: 10.1007/978-94-017-7267-9_24.

[HBK05] Nasreddine Hallam, Peter Blanchfield, and Graham Kendall. Handling diversity in evolutionary multiobjective optimization. In *Proceedings of the Congress on Evolutionary Computation (CEC)*, volume 3, pages 2233–2240. IEEE, 2005.

[HC17] Can Hankendi and Ayse Kivilcim Coskun. Scale & Cap: scaling-aware resource management for consolidated multi-threaded applications. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 22(2):30, 2017.

[HDH+10] Jason Howard, Saurabh Dighe, Yatin Hoskote, Sriram Vangal, David Finan, Gregory Ruhl, David Jenkins, Howard Wilson, Nitin Borkar, Gerhard Schrom, et al. A 48-core IA-32 message-passing processor with DVFS in 45nm CMOS. In *Proceedings of the IEEE International Conference on Solid-State Circuits (ISSCC)*, pages 108–109. IEEE, 2010.

[HGB+09] Andreas Hansson, Kees Goossens, Marco Bekooij, and Jos Huisken. CoMPSoC: a template for composable and predictable multi-processor system on chips. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 14(1):2, 2009.

[HGV+06] Wei Huang, Shougata Ghosh, Sivakumar Velusamy, Karthik Sankaranarayanan, Kevin Skadron, and Mircea R Stan. HotSpot: a compact thermal modeling methodology for early-stage VLSI design. *IEEE Transactions on Very Large Scale Integrated Systems (TVLSI)*, 14(5):501–513, 2006.

[HIB10] Steven Hofmeyr, Costin Iancu, and Filip Blagojević. Load balancing on speed. *ACM Sigplan Notices*, 45(5):147–158, 2010.

[HKK+13] Jan Heisswolf, Ralf König, Martin Kupper, and Jürgen Becker. Providing multiple hard latency and throughput guarantees for packet switching networks on chip. *Computers & Electrical Engineering*, 39(8):2603–2622, 2013.

[HLL+13] Simon Holmbacka, Wictor Lund, Sebastien Lafond, and Johan Lilius. Task migration for dynamic power and performance characteristics on many-core distributed operating systems. In *Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pages 310–317. IEEE, 2013.

[HM04] Jingcao Hu and Radu Marculescu. Energy-aware communication and task scheduling for network-on-chip architectures under real-time constraints. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 234–239. ACM/IEEE, 2004.

[HM05] Jingcao Hu and Radu Marculescu. Energy-and performance-aware mapping for regular NoC architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 24(4):551–562, 2005.

[HRG+16] Salma Hesham, Jens Rettkowski, Diana Goehringer, and Mohamed A Abd El Ghany. Survey on real-time networks-on-chip. *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 28(5):1500–1517, 2016.

[HRS+11] Frank Hannig, Sascha Roloff, Gregor Snelting, Jürgen Teich, and Andreas Zwinkau. Resource-aware programming and simulation of MPSoC architectures through extension of X10. In *Proceedings of the Conference on Languages, Compilers and Tools for Embedded Systems (SCOPES)*, pages 48–55. ACM, 2011.

[HRW15] Sebastian Hahn, Jan Reineke, and Reinhard Wilhelm. Towards compositionality in execution time analysis: definition and challenges. *ACM SIGBED Review*, 12(1):28–36, 2015.

[HV11] Robert Hilbrich and J Reinier Van Kampenhout. Partitioning and task transfer on NoC-based many-core processors in the avionics domain. *Journal Softwaretechnik-Trends*, 30(3):6, 2011.

[HV12] Vinay Hanumaiah and Sarma Vrudhula. Energy-efficient operation of multicore processors by DVFS, task migration, and active cooling. *IEEE Transactions on Computers (TC)*, 63(2):349–360, 2012.

[HWZ+15] Jan Heisswolf, Andreas Weichslgartner, Aurang Zaib, Stephanie Friederich, Leonard Masing, Carsten Stein, Marco Duden, Roman Klöpfer, Jürgen Teich, Thomas Wild, et al. Fault-tolerant communication in invasive networks on chip. In *NASA/ESA Conference on Adaptive Hardware and Systems (AHS)*, pages 1–8. IEEE, 2015.

[ITR15] ITRS. International technology roadmap for semiconductors (ITRS) 2.0, 2015. URL: http://www.itrs2.net/.

[JW04] Ahmed Jerraya and Wayne Wolf. *Multiprocessor systems-on-chips*. Elsevier, 2004.

[KAH18] Heba Khdr, Hussam Amrouch, and Jörg Henkel. Aging-constrained performance optimization for multi cores. In *Proceedings of the Design Automation Conference (DAC)*, pages 1–6. ACM/IEEE, 2018.

[KBL+11] Sebastian Kobbe, Lars Bauer, Daniel Lohmann, Wolfgang Schröder-Preikschat, and Jörg Henkel. DistRM: distributed resource management for on-chip many-core systems. In *Proceedings of the Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, pages 119–128. ACM/IEEE, 2011.

[KC03] Joshua Knowles and David Corne. Properties of an adaptive archiving algorithm for storing nondominated vectors. *IEEE Transactions on Evolutionary Computation (TEVC)*, 7(2):100–116, 2003.

[Ken10] James Kennedy. Particle swarm optimization. *Encyclopedia of Machine Learning*:760–766, 2010.

[KGV83] Scott Kirkpatrick, C Daniel Gelatt, and Mario P Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[KHM+13] Timon Kelter, Tim Harde, Peter Marwedel, and Heiko Falk. Evaluation of resource arbitration methods for multi-core real-time systems. In *Proceedings of the International Workshop on Worst-Case Execution Time Analysis (WCET)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2013.

[Kni02] John C Knight. Safety critical systems: challenges and directions. In *Proceedings of the International Conference on Software Engineering (ICSE)*, pages 547–550. IEEE, 2002.

[Kop11] Hermann Kopetz. *Real-time systems: Design principles for distributed embedded applications*. Springer, 2nd edition, 2011.

[KP08] Raimund Kirner and Peter Puschner. Obstacles in worst-case execution time analysis. In *International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*, pages 333–339. IEEE, 2008.

[KPS+17] Heba Khdr, Santiago Pagani, Ericles Sousa, Vahid Lari, Anuj Pathania, Frank Hannig, Muhammad Shafique, Jürgen Teich, and Jörg Henkel. Power density-aware resource management for heterogeneous tiled multicores. *IEEE Transactions on Computers (TC)*, 66(3):488–501, 2017.

[KSK+13] Pham Nam Khanh, Amit Kumar Singh, Akash Kumar, and Khin Mi Mi Aung. Incorporating energy and throughput awareness in design space exploration and run-time mapping for heterogeneous MPSoCs. In *Proceedings of the Euromicro Conference on Digital System Design Architectures, Methods and Tools (DSD)*, pages 513–521. IEEE, 2013.

[KZL+11] Mahmut Kandemir, Yuanrui Zhang, Jun Liu, and Taylan Yemliha. Neighborhood-aware data locality optimization for NoC-based multicores. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 191–200. IEEE, 2011.

[LGH+07] Martin Lukasiewycz, Michael Glaß, Christian Haubelt, and Jürgen Teich. SAT-decoding in evolutionary algorithms for discrete constrained optimization problems. In *Proceedings of the Congress on Evolutionary Computation (CEC)*, pages 935–942. IEEE, 2007.

[LGR+11] Martin Lukasiewycz, Michael Glaß, Felix Reimann, and Jürgen Teich. OPT4J: a modular framework for meta-heuristic optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 1723–1730. ACM, 2011.

[LL94] Chin Lu and Sau-Ming Lau. A performance study on load balancing algorithms with task migration. In *Proceedings of the International Conference on Frontiers of Computer Technology (TENCON)*, pages 357–364. IEEE, 1994.

[LS99] Thomas Lundqvist and Per Stenstrom. Timing anomalies in dynamically scheduled microprocessors. In *Proceedings of the Real-Time Systems Symposium (RTSS)*, pages 12–21. IEEE, 1999.

[LSF14] Martin Lukasiewycz, Shanker Shreejith, and Suhaib A Fahmy. System simulation and optimization using reconfigurable hardware. In *Proceeedings of the International Symposium on Integrated Circuits (ISIC)*, pages 468–471, 2014.

[LSG+09] Martin Lukasiewycz, Martin Streubuhr, Michael Glaß, Christian Haubelt, and Jürgen Teich. Combined system synthesis and communication architecture exploration for MPSoCs. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 472–477. ACM/IEEE, 2009.

[LTD+02] Marco Laumanns, Lothar Thiele, Kalyanmoy Deb, and Eckart Zitzler. Combining convergence and diversity in evolutionary multiobjective optimization. *Evolutionary Computation*, 10(3):263–282, 2002.

[LTH+14] Zao Liu, Sheldon X-D Tan, Xin Huang, and Hai Wang. Task migrations for distributed thermal management considering transient effects. *IEEE Transactions on Very Large Scale Integrated Systems (TVLSI)*, 23(2):397–401, 2014.

[LWH+05] Liang-Yu Lin, Cheng-Yeh Wang, Pao-Jui Huang, Chih-Chieh Chou, and Jing-Yang Jou. Communication-driven task binding for multiprocessor with latency insensitive network-on-chip. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 39–44. ACM/IEEE, 2005.

[MAA+09] Fabrizio Mulas, David Atienza, Andrea Acquaviva, Salvatore Carta, Luca Benini, and Giovanni De Micheli. Thermal balancing policy for multiprocessor stream computing platforms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 28(12):1870–1882, 2009.

[Mar06] Grant Martin. Overview of the MPSoC design challenge. In *Proceedings of the Design Automation Conference (DAC)*, pages 274–279. ACM/IEEE, 2006.

[MAV+10] Giovanni Mariani, Prabhat Avasare, Geert Vanmeerbeeck, Chantal Ykman-Couvreur, Gianluca Palermo, Cristina Silvano, and Vittorio Zaccaria. An industrial design space exploration framework for supporting run-time resource management on multi-core systems. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 196–201. ACM/IEEE, 2010.

[MBS+05] César Marcon, André Borin, Altamiro Susin, Luigi Carro, and Flávio Wagner. Time and energy efficient mapping of embedded applications onto NoCs. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 33–38. ACM/IEEE, 2005.

[MDA+16] Guilherme Madalozzo, Liana Duenha, Rodolfo Azevedo, and Fernando G Moraes. Scalability evaluation in many-core systems due to the memory organization. In *Proceedings of the International Conference on Electronics, Circuits and Systems (ICECS)*, pages 396–399. IEEE, 2016.

[MDP+00] Dejan S Milojičić, Fred Douglis, Yves Paindaveine, Richard Wheeler, and Songnian Zhou. Process migration. *ACM Computing Surveys (CSUR)*, 32(3):241–299, 2000.

[MJD+11] Thomas Megel, Mathieu Jan, Vincent David, and Christian Fraboul. Evaluation of task migration mechanisms for hard real-rime distributed systems. In *Proceedings of the International Conference on Real-Time Networks and Systems (RTNS)*, pages 159–168. ACM, 2011.

[MMC+17] Kanchan Manna, Priyajit Mukherjee, Santanu Chattopadhyay, and Indranil Sengupta. Thermal-aware application mapping strategy for network-on-chip based system design. *IEEE Transactions on Computers (TC)*, 67(4):528–542, 2017.

[Moo65] Gordon E Moore. Cramming more components onto integrated circuits, 1965.

[MOP+09] Radu Marculescu, Umit Y Ogras, Li-Shiuan Peh, Natalie Enright Jerger, and Yatin Hoskote. Outstanding research problems in NoC design: system, microarchitecture, and circuit perspectives. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 28(1):3–21, 2009.

[Moy13] Bryon Moyer. *Real world multicore embedded systems*. Newnes, 2013.

[MPV+13] Thannirmalai Somu Muthukaruppan, Mihai Pricopi, Vanchinathan Venkataramani, Tulika Mitra, and Sanjay Vishin. Hierarchical power management for asymmetric multi-core in dark silicon era. In *Proceedings of the Design Automation Conference (DAC)*, page 174. ACM/IEEE, 2013.

[MR14] Peter Munk and Jan Richling. Migration-aware WCET estimation for heterogeneous multi-cores. *ACM SIGBED Review*, 11(3):22–25, 2014.

[MRR+19] Claire Maiza, Hamza Rihani, Juan M Rivas, Joël Goossens, Sebastian Altmeyer, and Robert I Davis. A survey of timing verification techniques for multi-core real-time systems. *ACM Computing Surveys (CSUR)*, 52(3):1–38, 2019.

[MTT18] Tulika Mitra, Jürgen Teich, and Lothar Thiele. Time-critical systems design: a survey. *IEEE Design & Test*, 35(2):8–26, 2018.

[NFM+12] George P Nychis, Chris Fallin, Thomas Moscibroda, Onur Mutlu, and Srinivasan Seshan. On-chip networks from a networking perspective: congestion and scalability in many-core interconnects. *ACM SIGCOMM computer communication review*, 42(4):407–418, 2012.

[NM93] Lionel M Ni and Philip K McKinley. A survey of wormhole routing techniques in direct networks. *Computer*, 26(2):62–76, 1993.

[NMA+05] Vincent Nollet, Théodore Marescaux, Prabhat Avasare, Diederik Verkest, and J-Y Mignolet. Centralized run-time resource management in a network-on-chip containing reconfigurable hardware tiles. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 234–239. ACM/IEEE, 2005.

[PAC+07] Michele Pittau, Andrea Alimonda, Salvatore Carta, and Andrea Acquaviva. Impact of task migration on streaming multimedia for embedded multiprocessors: a quantitative evaluation. In *Workshop on Embedded Systems for Real-Time Multimedia (ESTIMedia)*, pages 59–64. IEEE, 2007.

[PCS+18] Santiago Pagani, Jian-Jia Chen, Muhammad Shafique, and Jörg Henkel. *Advanced techniques for power, energy, and thermal management for clustered manycores*. Springer, 2018.

[PKC+17] Santiago Pagani, Heba Khdr, Jian-Jia Chen, Muhammad Shafique, Minming Li, and Jörg Henkel. Thermal safe power (TSP): efficient power budgeting for heterogeneous manycore systems in dark silicon. *IEEE Transactions on Computers (TC)*, 66(1):147–162, 2017.

[PP12] Roberta Piscitelli and Andy D Pimentel. Design space pruning through hybrid analysis in system-level design space exploration. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 781–786. ACM/IEEE, 2012.

[QLD10] Yue Qian, Zhonghai Lu, and Qiang Dou. QoS scheduling for NoCs: strict priority queueing versus weighted round robin. In *Proceedings of the International Conference on Computer Design (ICCD)*, pages 52–59. IEEE, 2010.

[RDP17] Benjamin Rouxel, Steven Derrien, and Isabelle Puaut. Tightening contention delays while scheduling parallel applications on multi-core architectures. *ACM Transactions on Embedded Computing Systems (TECS)*, 16(5s):164, 2017.

[RLG+18] Felix Reimann, Martin Lukasiewycz, Michael Glaß, and Fedor Smirnov. OpenDSE - Open design space exploration framework, 2018. URL: http://opendse.sourceforge.net/.

[RMM+16] Hamza Rihani, Matthieu Moy, Claire Maiza, Robert I Davis, and Sebastian Altmeyer. Response time analysis of synchronous data flow programs on a many-core processor. In *Proceedings of the International Conference on Real-Time Networks and Systems (RTNS)*, pages 67–76. ACM, 2016.

[RPM+19] Martin Rapp, Anuj Pathania, Tulika Mitra, and Jörg Henkel. Prediction-based task migration on S-NUCA many-cores. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 1579–1582. ACM/IEEE, 2019.

[RWT+06] Jan Reineke, Björn Wachter, Stefan Thesing, Reinhard Wilhelm, Ilia Polian, Jochen Eisinger, and Bernd Becker. A definition and classification of timing anomalies. In *Proceedings of the International Workshop on Worst-Case Execution Time Analysis (WCET)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2006.

[Sam19] Samsung. Exynos 5 Octa (5422), 2019. URL: http://www.samsung.com/exynos/.

[SDM+17] Amit Kumar Singh, Piotr Dziurzanski, Hashan Roshantha Mendis, and Leandro Soares Indrusiak. A survey and comparative study of hard and soft real-time dynamic resource allocation strategies for multi-/many-core systems. *ACM Computing Surveys (CSUR)*, 50(2):1–40, 2017.

[SEU+15] Selma Saidi, Rolf Ernst, Sascha Uhrig, Henrik Theiling, and Benoît Dupont de Dinechin. The shift to multicores in real-time and safety-critical systems. In *Proceedings of the Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, pages 220–229. ACM/IEEE, 2015.

[SGH+14] Muhammad Shafique, Siddharth Garg, Jörg Henkel, and Diana Marculescu. The EDA challenges in the dark silicon era: temperature, reliability, and variability perspectives. In *Proceedings of the Design Automation Conference (DAC)*, pages 1–6. ACM/IEEE, 2014.

[SH98] Peter Smith and Norman C Hutchinson. Heterogeneous process migration: the Tui system. *Software-Practice and Experience*, 28(6):611–640, 1998.

[SKS13] Amit Kumar Singh, Akash Kumar, and Thambipillai Srikanthan. Accelerating throughput-aware runtime mapping for heterogeneous MPSoCs. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 18(1):9:1–9:29, 2013.

[Smi88] Jonathan M Smith. A survey of process migration mechanisms. *ACM SIGOPS Operating Systems Review*, 22(3):28–40, 1988.

[SMR11] Abhik Sarkar, Frank Mueller, and Harini Ramaprasad. Predictable task migration for locked caches in multi-core systems. *ACM SIGPLAN Notices*, 46(5):131–140, 2011.

[Sod15] Avinash Sodani. Knights Landing (KNL): 2nd generation Intel® Xeon Phi processor. In *IEEE Hot Chips 27 Symposium (HCS)*, pages 1–24. IEEE, 2015.

[SPM09] Prabhat Kumar Saraswat, Paul Pop, and Jan Madsen. Task migration for fault-tolerance in mixed-criticality embedded systems. *ACM Special Interest Group on Embedded Systems (SIGBED) Review*, 6(3):6, 2009.

[SRR+18] Tobias Schwarzer, Sascha Roloff, Valentina Richthammer, Rami Khaldi, Stefan Wildermann, Michael Glaß, and Jürgen Teich. On the complexity of mapping feasibility in many-core architectures. In *International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSOC)*, pages 176–183. IEEE, 2018.

[SS16] Stefanos Skalistis and Alena Simalatsar. Worst-case execution time analysis for many-core architectures with NoC. In *International Conference on Formal Modeling and Analysis of Timed Systems*, pages 211–227. Springer, 2016.

[SS17] Stefanos Skalistis and Alena Simalatsar. Near-optimal deployment of dataflow applications on many-core platforms with real-time guarantees. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 752–757. ACM/IEEE, 2017.

[SSK+13] Amit Kumar Singh, Muhammad Shafique, Akash Kumar, and Jörg Henkel. Mapping on multi/many-core systems: survey of current and emerging trends. In *Proceedings of the Design Automation Conference (DAC)*, pages 1–10. ACM/IEEE, 2013.

[SWG+17] Tobias Schwarzer, Andreas Weichslgartner, Michael Glaß, Stefan Wildermann, Peter Brand, and Jürgen Teich. Symmetry-eliminating design space exploration for hybrid application mapping on many-core architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 37(2):297–310, 2017.

[Tei08] Jürgen Teich. Invasive algorithms and architectures. *it-Information Technology*, 50(5):300–310, 2008.

[TGR+16] Jürgen Teich, Michael Glaß, Sascha Roloff, Wolfgang Schröder-Preikschat, Gregor Snelting, Andreas Weichslgartner, and Stefan Wildermann. Language and compilation of parallel programs for *-predictable MPSoC execution using invasive computing. In *International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSOC)*, pages 313–320. IEEE, 2016.

[THH+11] Jürgen Teich, Jörg Henkel, Andreas Herkersdorf, Doris Schmitt-Landsiedel, Wolfgang Schröder-Preikschat, and Gregor Snelting. Invasive computing: an overview. In *Multiprocessor System-on-Chip*, pages 241–268. Springer, 2011.

[Til12] Tilera Corporation. Tile processor architecture overview for the TILE-Gx series, 2012.

[UBE+12] Ivan Ukhov, Min Bao, Petru Eles, and Zebo Peng. Steady-state dynamic temperature analysis and reliability optimization for embedded multiprocessor systems. In *Proceedings of the Design Automation Conference (DAC)*, pages 197–204. ACM/IEEE, 2012.

[VDP15] Vimal L Vachhani, Vipul K Dabhi, and Harshadkumar B Prajapati. Survey of multi objective evolutionary algorithms. In *Proceedings of the International Conference on Circuit, Power and Computing Technologies (ICCPCT)*, pages 1–9, 2015.

[VSK+15] Shyamsundar Venkataraman, Rui Santos, Akash Kumar, and Jasper Kuijsten. Hardware task migration module for improved fault tolerance and predictability. In *Proceedings of the International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, pages 197–202. IEEE, 2015.

[WEE+08] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, et al. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Transactions on Embedded Computing Systems (TECS)*, 7(3):1–53, 2008.

[WGT14] Stefan Wildermann, Michael Glaß, and Jürgen Teich. Multi-objective distributed run-time resource management for many-cores. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 1–6. ACM/IEEE, 2014.

[WGW+14] Andreas Weichslgartner, Deepak Gangadharan, Stefan Wildermann, Michael Glaß, and Jürgen Teich. DAARM: design-time application analysis and run-time mapping for predictable execution in many-core systems. In *Proceedings of the Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, pages 1–10. ACM/IEEE, 2014.

[WJM08] Wayne Wolf, Ahmed Amine Jerraya, and Grant Martin. Multiprocessor system-on-chip (MPSoC) technology. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 27(10):1701–1713, 2008.

[WLC+14] Yan Wang, Kenli Li, Hao Chen, Ligang He, and Keqin Li. Energy-aware data allocation and task scheduling on heterogeneous multiprocessor systems with time constraints. *IEEE Transactions on Emerging Topics in Computing (TETC)*, 2(2):134–148, 2014.

[WLC16] Yue Wu, Chao Lu, and Yunji Chen. A survey of routing algorithm for mesh network-on-chip. *Frontiers of Computer Science*, 10(4):591–601, 2016.

[WRW+14] Stefan Wallentowitz, Stefan Rosch, Thomas Wild, Andreas Herkersdorf, Volker Wenzel, and Jörg Henkel. Dependable task and communication migration in tiled manycore system-on-chip. In *Forum on Specification and Design Languages (FDL)*, volume 978, pages 1–8. IEEE, 2014.

[WSK+05] Pascal T Wolkotte, Gerard JM Smit, Nikolay Kavaldjiev, Jens E Becker, and Jürgen Becker. Energy model of networks-on-chip and a bus. In *Proceedings of the International Symposium on System-on-Chip (SoC)*, pages 82–85. IEEE, 2005.

[WWG+16] Andreas Weichslgartner, Stefan Wildermann, Johannes Götzfried, Felix Freiling, Michael Glaß, and Jürgen Teich. Design-time/run-time mapping of security-critical applications in heterogeneous MPSoCs. In *Proceedings of the Conference on Languages, Compilers and Tools for Embedded Systems (SCOPES)*, pages 153–162. ACM, 2016.

[WWG+18a] Andreas Weichslgartner, Stefan Wildermann, Deepak Gangadharan, Michael Glaß, and Jürgen Teich. A design-time/run-time application mapping methodology for predictable execution time in MPSoCs. *ACM Transactions on Embedded Computing Systems (TECS)*, 17(5):1–25, 2018.

[WWG+18b] Andreas Weichslgartner, Stefan Wildermann, Michael Glaß, and Jürgen Teich. *Invasive computing for mapping parallel programs to many-core architectures*, volume 1. Springer, 2018.

[WZM+14] Xiaohang Wang, Baoxin Zhao, Terrence Mak, Mei Yang, Yingtao Jiang, Masoud Daneshtalab, and Maurizio Palesi. Adaptive power allocation for many-core systems inspired from multiagent auction model. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, page 333. ACM/IEEE, 2014.

[YAM+11] Chantal Ykman-Couvreur, Prabhat Avasare, Giovanni Mariani, Gianluca Palermo, Cristina Silvano, and Vittorio Zaccaria. Linking run-time resource management of embedded multi-core platforms with automated design-time exploration. *IET Computers and Digital Techniques*, 5(2):123–135, 2011.

[YJO18] Guo Yu, Yaochu Jin, and Markus Olhofer. A method for a posteriori identification of knee points based on solution density. In *Proceedings of the Congress on Evolutionary Computation (CEC)*, pages 1–8. IEEE, 2018.

[ZB08] Shi Zheng and Alan Burns. Real-time communication analysis for on-chip networks with wormhole switching. In *Proceedings of the International Symposium on Networks-on-Chip (NOCS)*, pages 161–170, 2008.

[ZLT01] Eckart Zitzler, Marco Laumanns, and Lothar Thiele. SPEA2: improving the strength pareto evolutionary algorithm. In *Proceedings of the Conference on Evolutionary Methods for Design, Optimisation*

and Control with Application to Industrial Problems (EUROGEN), pages 95–100, 2001.

[ZSO+13] Jia Zhan, Nikolay Stoimenov, Jin Ouyang, Lothar Thiele, Vijaykrishnan Narayanan, and Yuan Xie. Designing energy-efficient NoC for real-time embedded systems through slack optimization. In *Proceedings of the Design Automation Conference (DAC)*, pages 1–6. ACM/IEEE, 2013.

[ZTC+15] Xingyi Zhang, Ye Tian, Ran Cheng, and Yaochu Jin. An efficient approach to non-dominated sorting for evolutionary multi-objective optimization. *IEEE Transactions on Evolutionary Computation (TEVC)*, 19(2):201–213, 2015.

Author's Own Publications

[PGT17] Behnaz Pourmohseni, Michael Glaß, and Jürgen Teich. Automatic operating point distillation for hybrid mapping methodologies. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 1135–1140. ACM/IEEE, 2017. DOI: 10.23919/DATE.2017.7927160.
Contributions: *This work proposes an automatic mapping-set distillation methodology that, given a set of Pareto-optimal mappings in the space multiple quality- and resource objectives, distills a configurable subset of mappings such that diverse trade-offs w.r.t. quality objectives and efficient trade-offs w.r.t. resource objectives are retained.*

[PWG+17] Behnaz Pourmohseni, Stefan Wildermann, Michael Glaß, and Jürgen Teich. Predictable run-time mapping reconfiguration for real-time applications on many-core systems. In *Proceedings of the International Conference on Real-Time Networks and Systems (RTNS)*, pages 148–157. ACM, 2017. DOI: 10.1145/3139258.3139278. *Outstanding Paper Award. Best Presentation Award.*
Contributions: *This work proposes a mapping reconfiguration methodology consisting of a predictable reconfiguration mechanism and an offline reconfiguration analysis that enable on-the-fly mapping adaptation of hard real-time applications in many-core systems with worst-case timing guarantees and minimal run-time compute overhead.*

[PWG+19] Behnaz Pourmohseni, Stefan Wildermann, Michael Glaß, and Jürgen Teich. Hard real-time application mapping reconfiguration for NoC-based many-core systems. *Real-Time Systems (RTS)*, 55(2):433–469, 2019. DOI: 10.1007/s11241-019-09326-y.
Contributions: *This work proposes a mapping reconfiguration methodology consisting of a predictable reconfiguration mechanism and an offline/hybrid reconfiguration analysis that enable on-the-fly mapping adaptation of real-time applications in many-core systems with worst-case timing guarantees and affordable run-time compute overhead.*

[PSW+19] Behnaz Pourmohseni, Fedor Smirnov, Stefan Wildermann, and Jürgen Teich. Isolation-aware timing analysis and design space exploration for predictable and composable many-core systems. In *Proceedings of the Euromicro Conference on Real-Time Systems (ECRTS)*, volume 133,

12:1–12:24. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019. DOI: 10.4230/LIPIcs.ECRTS.2019.12.
Contributions: *This work proposes a formal and compositional worst-case timing analysis for composable many-core systems and, in light of that, proposes a novel DSE approach that is able to deliver high-quality mappings by exploring the combinations of inter-application isolation schemes on the allocated resources of each mapping.*

[PSK+19] Behnaz Pourmohseni, Fedor Smirnov, Heba Khdr, Stefan Wildermann, Jürgen Teich, and Jörg Henkel. Thermally composable hybrid application mapping for real-time applications in heterogeneous many-core systems. In *Proceedings of the Real-Time Systems Symposium (RTSS)*, pages 220–232. IEEE, 2019. DOI: 10.1109/RTSS46320.2019.00029.
Contributions: *This work proposes a thermally composable Hybrid Application Mapping (HAM) methodology that enforces the thermal safety of many-core systems proactively and, thereby, preserves the system composability which is crucial for hard real-time applications.*

[PSW+20] Behnaz Pourmohseni, Fedor Smirnov, Stefan Wildermann, and Jürgen Teich. Real-time task migration for dynamic resource management in many-core systems. In *Workshop on Next Generation Real-Time Embedded Systems (NG-RES)*, volume 77, 5:1–5:14. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2020. DOI: 10.4230/OASIcs.NG-RES.2020.5.
Contributions: *This work proposes a task migration methodology consisting of a predictable migration mechanism and a migration timing analysis and admission check that enable on-the-fly task migrations for hard real-time applications in many-core systems with worst-case timing guarantees and tolerable run-time compute overhead.*

[PGH+20] Behnaz Pourmohseni, Michael Glaß, Jörg Henkel, Heba Khdr, Martin Rapp, Valentina Richthammer, Tobias Schwarzer, Fedor Smirnov, Jan Spieck, Jürgen Teich, Andreas Weichslgartner, and Stefan Wildermann. Hybrid application mapping for composable many-core systems: overview and future perspective. *Journal of Low Power Electronics and Applications*, 10(4):1–37, 2020. DOI: 10.3390/jlpea10040038.
Contributions: *This work presents an overview of the state of the art for Hybrid Application Mapping (HAM) in composable many-core systems and outlines possible directions of future research in Hybrid Application Mapping (HAM) with focus on the role of machine learning for DSE and run-time system management.*

[SPG+19a] Fedor Smirnov, Behnaz Pourmohseni, Michael Glaß, and Jürgen Teich. IGOR, get me the optimum! prioritizing important design decisions

during the DSE of embedded systems. *ACM Transactions on Embedded Computing Systems (TECS)*, 18(5s):78:1–78:22, 2019. DOI: 10.1145/3358204.
Contributions: *This work proposes a DSE approach employing a machine-learning-inspired technique to dynamically analyze the importance of design decisions during the optimization and, thereby, guide the optimizer towards promising regions of the design space.*

[SPG+19b] Fedor Smirnov, Behnaz Pourmohseni, Michael Glaß, and Jürgen Teich. Variety-aware routing encoding for efficient design space exploration of automotive communication networks. In *International Conference on Vehicle Technology and Intelligent Transport Systems (VEHITS)*, volume 1, pages 242–253. SCITEPRESS, 2019. DOI: 10.5220/0007797802420253.
Contributions: *This work proposes a routing optimization approach with a novel routing encoding scheme which employs a pre-processing algorithm to gathers knowledge about the topology of the network to reduce the optimization run time and provide results of higher quality.*

[SPT20] Fedor Smirnov, Behnaz Pourmohseni, and Jürgen Teich. Using learning classifier systems for the DSE of adaptive embedded systems. In *Proceedings of the Design, Automation and Test in Europe (DATE)*, pages 957–962. ACM/IEEE, 2020. DOI: 10.23919/DATE48585.2020.9116383.
Contributions: *This work proposes an optimization framework for the DSE of dynamic embedded systems in which the learning process of the system manager is transferred to design time where a collection of conditionally applicable solutions are optimized to be used online.*

[TPK+20] Jürgen Teich, Behnaz Pourmohseni, Oliver Keszöcze, Jan Spieck, and Stefan Wildermann. Run-time enforcement of non-functional application requirements in heterogeneous many-core systems. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 629–636. ACM/IEEE, 2020. DOI: 10.1109/ASP-DAC47756.2020.9045536.
Contributions: *This work presents the definitions and taxonomy of Run-Time Requirement Enforcement (RRE) techniques for many-core systems and provides a discussion on their advantages, drawbacks, and challenges in a case study on the enforcement of timing and reliability requirements for a soft real-time image processing application.*

[TMP+20] Jürgen Teich, Pouya Mahmoody, Behnaz Pourmohseni, Sascha Roloff, Wolfgang Schröder-Preikschat, and Stefan Wildermann. Run-time enforcement of non-functional program properties on MPSoCs. In

Jian-Jia Chen, editor, *A Journey of Embedded and Cyber-Physical Systems*. Springer, 2020. DOI: 10.1007/978-3-030-47487-4_9.
Contributions: *This work presents a formalization and classification of Run-Time Requirement Enforcement (RRE) techniques for MPSoCs and exemplifies the practice of centralized and distributed RRE for the enforcement of timing requirements of a soft real-time application.*

List of Symbols

a	switching activity of a core
A	die area of a core
α	allocation
b	a bus in an architecture graph
B	set of all buses in an architecture graph
β	binding
β^{msg}	binding of messages to memories
β^{task}	binding of tasks to cores
c	a core (processing resource) in an architecture graph
C	set of all cores in an architecture graph
$\hat{c}$	effective capacitance of a core
cost	allocation cost of a migration route
D	arbitration delay
D_{app}	application deadline
D_{OS}	context-switch latency of the operating system
D^{router}	routing delay of a router
div	per-objective granularity of the distillation hyper-grid
D_{mp}	maximum temporal distance between consecutive migration points
D^{NoC}	transfer delay on the Network-on-Chip (NoC)
D^{RX}	transfer delay of an Receiver (RX) unit
D^{TX}	transfer delay of a Transmitter (TX) unit
δ	preemption latency of a task
δ_{mig}	end-to-end migration latency
$\delta_{\text{mig}}^{\text{par}}$	end-to-end latency of parallel migrations
$\delta_{\text{mig}}^{\text{seq}}$	end-to-end latency of sequential migrations

δ_{reloc}	relocation latency of a task
δ_{susp}	suspension latency of a task
Δ_{reconfig}	end-to-end latency of a mapping reconfiguration
Δ_{reloc}	relocation latency of a mapping reconfiguration
Δ_{susp}	suspension latency of a mapping reconfiguration
E_{app}	edges of an application graph
E_{arch}	edges of an architecture graph
E_{route}	edges of a r routing graph
f	operational frequency
F	flow control digits (flits)
G_{app}	application graph
G_{arch}	architecture graph
G_{route}	routing graph
Γ	set of mappings
I^{bus}	timing interference on bus
I^{core}	timing interference on core
I_q	leakage current
k	distillation capacity
K	arbitration capacity
κ	cost attribute of a link in a routing graph
L_{app}	end-to-end latency (makespan) of an application
$\hat{L}_{\text{app}}$	compound latency of an application
$\hat{L}$	compound latency of a task or message
m	a message in an application graph
M	set of all messages in an application graph
MD	memory demand of a task
μ	a mapping

n	a Network Adapter (NA) in an architecture graph
N	set of all Network Adapters (NAs) in an architecture graph
ν^{CR}	core-reservation decision variable
ν^{TR}	tile-reservation decision variable
o	an optimization objective
O	set of all optimization objectives
ω	arbitration weight of a link in a routing graph
ω_{ex}	additionally allocated weight on a link in a routing graph
p	power consumption
P	arbitration period
PD	density of power consumption
PRD	period or (minimum) inter-arrival time of a task or message
π	an end-to-end path in an application graph
Π	set of all end-to-end paths in an application graph
PLD	payload size of a message
Ψ_{ctx}	maximum context size of a task
Ψ_{mig}	maximum size of migration data-set of a task
q	a memory in an architecture graph
Q	set of all memories in an architecture graph
r	a router in an architecture graph
R	set of all routers in an architecture graph
rx	Receiver (RX) unit of a Network Adapter (NA)
ρ	routing
ρ_{mig}	migration route
$\rho_{\mathrm{mig}}^{r \to r}$	inter-router links in a migration route
S	arbitration interval (time slot) duration
ST	service time of a memory
σ	schedule
σ^{msg}	schedule of messages

σ^{task}	schedule of tasks
t	a (processing) task in an application graph
T	set of all (processing) tasks in an application graph
TH_{app}	throughput of an application
T_{mig}	set of all migrating tasks
$TSPD$	Thermally Safe Power-density Dictionary (TSPD)
TSU	Thermally Safe Utilization (TSU)
tx	Transmitter (TX) unit of a Network Adapter (NA)
τ^{NoC}	clock cycle duration of the Network-on-Chip (NoC)
u	a tile in an architecture graph
U	set of all tiles in an architecture graph
v	a mapping edge in the specification
V	set of all mapping edges in the specification
V_{app}	vertices of an application graph
V_{arch}	vertices of an architecture graph
V^{dd}	supply voltage
V_{route}	vertices of a routing graph
W	arbitration weight
$WCET$	worst-case execution time of a task
$WCRL$	worst-case relocation latency of a migrating task
$WCRT$	worst-case response time of a task
$WCTT$	worst-case traversal time of a message
X	distillation normalization factor

Acronyms

AG	Adaptive Grid
CD	Crowding Distance
CDF	Cumulative Distribution Function
CR	Core-Reservation
CRPD	Cache-Related Preemption Delay
CS	Core-Sharing
DFS	Depth-First Search
DSE	Design Space Exploration
DTM	Dynamic Thermal Management
DVFS	Dynamic Voltage and Frequency Scaling
E3S	Embedded System Synthesis Benchmarks Suite
ELP	Energy-Latency Product
FCFS	First-Come-First-Serve
FP	Fixed-Priority
HAM	Hybrid Application Mapping
ILP	Integer Linear Programming
ISA	Instruction Set Architecture
MPSoC	Multi-Processor System-on-Chip
NA	Network Adapter
NN	Nearest Neighbor
NoC	Network-on-Chip
NORMA	No Remote Memory Access

OS	Operating System
PB	Pseudo-Boolean
PCB	Process Control Block
RPM	Run-time Platform Manager
RR	Round-Robin
RX	Receiver
SLDL	Systems-Level Design Language
TDMA	Time-Division Multiple Access
TDP	Thermal Design Power
TR	Tile-Reservation
TSPD	Thermally Safe Power-density Dictionary
TSU	Thermally Safe Utilization
TX	Transmitter
WCET	Worst-Case Execution Time
WCRL	Worst-Case Relocation Latency
WCRT	Worst-Case Response Time
WCTT	Worst-Case Traversal Time
WRR	Weighted Round-Robin

Index

A

allocation 35
application deadline 30
application graph 29
arbitration
    parameters of 25, 26, 32
    wheel size of 25
arbitration tuple 56, 58
arbitration weight 26, 56
architecture graph 31

B

binding 34

C

communication link 31
composability 24
    thermal 82
    timing 24
compositional analysis 22
compund latency 145, 161
constraint graph 41, 106
    embedding of 42, 107

D

data dependency 30
Design Space Exploration (DSE) 40
distillation 109, 110
distillation algorithm 113
distillation capacity 110

E

embeddability rate 97, 119
epsilon-dominance 69, 121

F

fat binary 132
flexibility rate 155
flit 16

H

Hybrid Application Mapping (HAM) . 40

I

Instruction Set Architecture (ISA) . . 132
invasive computing 27
isolation 24
    spatial 24, 47
    temporal 24, 47
isolation scheme 47
    Core-Reservation (CR) 49
    Core-Sharing (CS) 48
    Tile-Reservation (TR) 49

M

many-core 12
    architecture of 12
    communication scheme of 15
    memory scheme of 15
    tile of 13
mapping 34
    feasability of 35
    generation of 34, 54
mapping edge 32
message 29
migration allocation overhead 139
migration data-set 141
migration route 136
multi-core 11

N

Network Adapter (NA) 14
    Receiver (RX) of 14
    Transmitter (TX) of 14
Network-on-Chip (NoC) 13
    flow control of 17
    Network Adapter (NA) of 14
    router of 14

routing algorithm of 16
switching strategy of 16
topology of 13
virtual channel of 17

P

Pareto ranking 112
Pareto-optimal 4
predictability
*-predictability 19
timing behavior 23
preemption scheme 132
cooperative 132, 144, 159
non-preemptive 132, 144, 159
preemptive 132, 144, 158
Process Control Block (PCB) 131

R

real-time constraint 18
hard 18
soft 18
requirement 2, 28
routing 34
Run-time Platform Manager (RPM) . . 41

S

SAT-Decoding 55
schedulability 21, 57
schedule 35
specification 29
success rate 119, 164

T

task 29
task context 131
thermally safe power constraint 85
Thermally Safe Power-density Dictionary (TSPD) 86
Thermally Safe Utilization (TSU) 87
Time-Division Multiple Access (TDMA) 25
timing anomaly 23

W

Weighted Round-Robin (WRR) 26
Worst-Case Execution Time (WCET) . 19
worst-case network jitter 22
Worst-Case Response Time (WCRT) . 20, 62
Worst-Case Traversal Time (WCTT) . 21, 65

X

X10 28